

Failure Taxonomy and Case Analysis of IPL-to-ZKP Circuit Translation

This document serves as the detailed failure case appendix and the core reference for knowledge refinement in our study *TransZKEval: Unraveling LLM Behaviors in ZKP Circuit Translation*. The primary purpose of this appendix is twofold: it presents a systematic taxonomy and representative instantiations of diverse error patterns observed in large language model-driven translation from IPL logic to constraint-based zero-knowledge circuit code, and further provides empirical, case-based knowledge support for refining domain-specific translation rules, error awareness, and cross-domain coding knowledge.

Our empirical analysis demonstrates that domain-specific circuit translation introduces unique, underexplored challenges distinct from conventional general-purpose code translation tasks. While existing code translation benchmarks predominantly evaluate generic syntactic correctness and basic semantic consistency, they largely neglect domain rigid constraints, static verification rules, and context-sensitive syntax limitations inherent in zero-knowledge proof circuit development. It is worth clarifying that considering the structural complexity and redundant implementation details of raw benchmark samples, the cases compiled in this appendix are refined and organized for intuitive presentation and academic illustration. All displayed examples faithfully preserve the core error characteristics and failure mechanisms summarized from our full dataset, serving as standardized, readable, and knowledge-enriched instantiations for both experimental evaluation and knowledge refinement.

This appendix systematically classifies and elaborates on IPL-to-ZKP circuit translation failure modes, aiming to provide researchers and practitioners with three core contributions: (1) a fine-grained hierarchical classification framework for domain-specific circuit translation errors that summarizes generalized failure rules for knowledge induction, (2) concrete, annotated failure cases covering all major error categories for intuitive validation, comparative analysis, and targeted knowledge refinement, and (3) in-depth qualitative analysis of the essential root causes behind syntax conflicts, semantic mismatches, and constraint violations in LLM cross-domain code translation, which delivers actionable domain knowledge for future LLM-based circuit translation optimization and capability enhancement.

Contents

A	Syntax and Parse Errors	4
A-1	Anonymous Component Access (P1008 & P1012)	4
A-2	Structural Collapse (P1008)	6
A-3	Bitwise Operation (P1008 & P1012)	10
A-4	Python-Style Division (P1008)	18
A-5	Boolean-Ternary Logic Misuse (P1008 & P1012)	22
A-6	Nested Constraint Conflict (P1008 & P1012)	26
A-7	Invalid Signal Type Modifier (P1008)	30
A-8	Numeric Suffix Hallucination (P1008 & P1012)	34
A-9	Namespace-Method Hallucination (P1008 & P1012)	41
A-10	Decl-Constraint Conflict (P1008 & P1012)	43
A-11	Foreign Language Declaration (P1008)	47
A-12	Component Public Modifier Hallucination (P1008)	49
A-13	Incomplete Statement (P1008)	52
A-14	Invalid Type Declaration (P1008 & P1012)	57
A-15	Declaration-Initialization Fusion (P1008 & P1012)	59
A-16	Floating-Point Hallucination (P1008 & P1012)	62
A-17	Component Block Declaration Hallucination (P1008)	64
A-18	Python-Style Division (P1008 & P1012)	68
A-19	Type-Casting Hallucination (P1008 & P1012)	70
A-20	Invalid Assignment Operator (P1008 & P1012)	73
A-21	Bitwise Operation Hallucination (P1008)	76
A-22	Chained Assignment Hallucination (P1008)	80
A-23	Keyword Hallucination (P1008 & P1012)	83
A-24	Functional Logic Hallucination (P1012)	86
A-25	Illegal Context Declaration (P1012)	90
A-26	Illegal Operator in Expression (P1012)	92
A-27	Bare Illegal Expression (P1012)	95
A-28	Illegal Index Expression (P1012)	97
A-29	Illegal Comparison Expression (P1012)	100
B	Undefined Symbol Calls	111
B-1	Undefined Addition/Subtraction Symbol (T2021)	111
B-2	Undefined Bit/Data Operation Symbol (T2021)	117
B-3	Undefined Comparison Symbol (T2021)	122
B-4	Undefined Division/Modulo Symbol (T2021)	124
B-5	Undefined Multiplication Symbol (T2021)	127
B-6	Undeclared Symbol (T2021)	132
B-7	T3001 Other (T3001)	134
C	Non-Quadratic Constraints	136
C-1	Topological Sequence Violation (T3001)	136
C-2	Forbidden Bitwise Operation (T3001)	137
C-3	Constant/Linear (T3001)	138
C-4	Parallel Sum (Needs Intermediate Signal) (T3001)	140

C-5	Comparison Logic (Non-Algebraic) (T3001)	142
C-6	High-Order Polynomial (T3001)	144
C-7	Forbidden Division (T3001)	146
C-8	Multiple Definition/Re-assignment (T3001)	148
C-9	Parallel Sum (Needs Simplification) (T3001)	150
C-10	Forbidden Non-Polynomial Operator (T3001)	152
C-11	Other Initialization Gap (T3001)	154
C-12	Mixed Quadratic-Linear (T3001)	155
C-13	Imperative Self-Update (T3001)	157
C-14	Constraint-Only Missing Assignment (T3001)	158
D	Dynamic Signal in Static Context	160
D-1	Signal-Dependent Declaration (T2050)	160
D-2	Dynamic Array Length (T20460)	161
D-3	Dynamic Index (T20462)	163
D-4	Dynamic Template Parameter (T20461)	173
D-5	Signal-Dependent Condition (T2005)	175
D-6	Declaration in While Scope (T2011)	178
E	Project/Module Errors	179
E-1	Duplicate Symbol (T2008)	179
E-2	Bad Include/Import (P1009)	182
E-3	Template Not Exist (TAC01)	183
E-4	Bad Assign Target (TAC02)	186
E-5	Missing Main (P1001)	187
F	Wrong Argument Count	190
F-1	Wrong Argument Count (T2012)	190
G	Type Mismatch	191
G-1	Dimension Mismatch (T2061)	191
G-2	Tag on Signal (T2048)	194
G-3	= on Signal (T2054)	197
G-4	Operator Type Error (T2028)	201
G-5	Component vs Value (T2057)	205
G-6	Template as Signal (T2060)	209
G-7	Unknown Type (T2003)	211
G-8	Array Access Mismatch (T2032)	215
G-9	<== on Non-Signal (T2055)	216
H	Component Initialization/Interface	219
H-1	Uninitialized Component (T20466)	219
H-2	Mixed Templates in Array (T2056)	224
H-3	Undefined Port (T2046)	232
H-4	Partial Component Access (T2043)	236
H-5	Inline Component Array (T2034)	238

A Syntax and Parse Errors

A-1 Anonymous Component Access (P1008 & P1012)

Description: This error class happens when Circom code tries to use a template like a normal inline function call or access the output of an unnamed temporary component directly, such as ‘Template(...).out’. Circom components must be explicitly instantiated, their input signals assigned, and only then can their outputs be referenced; otherwise the parser reports syntax errors like "missing semicolon" and "illegal expression." The fix is to create a named component instance, wire its inputs with ‘<==’, and read ‘instance.out’ in a separate step.

Original Error Tag: P1008+P1012-FEAT-Anonymous-Component-Access

Example:

```
1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12            arr[i + 1], arr[high] = arr[high], arr[i + 1]
13            return int((i + 1) * 100 + swaps)
```

Listing 1: Source Python quicksort_partition_logic implementation

```
1 // Import necessary components
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/arithmetic/binary.circom";
4
5 // Template for quicksort partition logic
6 template quicksort_partition_logic() {
7     // Signal declarations
8     signal input a0, a1, a2, a3, a4, a5, a6, low, high;
9     signal pivot, i, swaps, arr[7], j, cond, swap;
10    signal output out;
11
12    // Component templates
13    component GreaterEqThan = GreaterEqThan(2);
14    component Binary = Binary();
15
16    // Initialize pivot as the middle element
17    pivot <== (a0 + a1 + a2 + a3 + a4 + a5 + a6) / 7;
18
19    // Initialize i and swaps
20    i <== low - 1;
21    swaps <== 0;
22
23    // Loop logic (unrolled for 7 iterations)
24    for (var j = 0; j < 7; j++) {
25        // Calculate cond as a binary value (0 or 1)
26        cond <== GreaterEqThan(arr[j], pivot).out;
27    }
```

```

28     // Update i and swaps
29     i <== i + cond;
30     swaps <== swaps + cond;
31
32     // Swap arr[i] and arr[j] if cond is 1
33     arr[j] <== (1 - cond) * arr[j] + cond * arr[i];
34     arr[i] <== (1 - cond) * arr[i] + cond * arr[j];
35 }
36
37 // Final swap
38 swap <== Binary(arr[i + 1], arr[high]).out;
39 arr[i + 1] <== arr[i + 1] * (1 - swap) + swap * arr[high];
40 arr[high] <== arr[high] * (1 - swap) + swap * arr[i + 1];
41
42 // Calculate out
43 out <== (i + 1) * 100 + swaps;
44 }
45
46 // Component main
47 component main = quicksort_partition_logic();

```

Listing 2: Faulty translated Circom code with Anonymous Component Access (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":30:46
3
4 30         cond <== GreaterEqThan(arr[j], pivot).out;
5                                     ^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8   "circuit.circom":30:47
9
10 30         cond <== GreaterEqThan(arr[j], pivot).out;
11                                     ^^^^^ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":42:43
15
16 42         swap <== Binary(arr[i + 1], arr[high]).out;
17                                     ^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":42:44
21
22 42         swap <== Binary(arr[i + 1], arr[high]).out;
23                                     ^^^^^ here
24
25 previous errors were found

```

Listing 3: Circom compiler error log

Explanation: In this example, ‘GreaterEqThan(arr[j], pivot).out’ and ‘Binary(arr[i + 1], arr[high]).out’ are being written as if ‘GreaterEqThan’ and ‘Binary’ were regular functions returning values. Circom does not allow that syntax, so those lines must be rewritten by creating actual component instances inside the loop/final-swap logic, assigning their inputs, and then using their ‘out’ signals.

A-2 Structural Collapse (P1008)

Description: This error class usually happens when the Circom parser loses the structure of the file after an earlier syntax violation, so it later reports a misleading "missing semicolon" near the end of the circuit. The fix is to look just before the reported line for malformed structure such as declarations placed in the wrong region, broken braces/brackets, or statements that violate Circom's ordering rules, then restore a valid block layout so the parser can recover.

Original Error Tag: P1008-FEAT-Structural-Collapse

Example:

```
1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)
```

Listing 4: Source Python shuffle_integrity_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template shuffle_integrity_logic() {
6     // Public inputs
7     signal input c0;
8     signal input c1;
9     signal input c2;
10    signal input c3;
11    signal input c4;
12
13    signal input s0;
14    signal input s1;
15    signal input s2;
16    signal input s3;
17    signal input s4;
18
19    signal input sd;
20
21    // Private witnesses
22    signal input diff_sum_inv;
23    signal input diff_sq_inv;
24
25    signal input d0_inv;
26    signal input d1_inv;
27    signal input d2_inv;
28    signal input d3_inv;
29    signal input d4_inv;
30
31    // Intermediate signals
32    signal sum_c;
33    signal sum_s;
```

```

34     signal diff_sum;
35     signal eq_sum;
36
37     signal c0_sq;
38     signal c1_sq;
39     signal c2_sq;
40     signal c3_sq;
41     signal c4_sq;
42
43     signal s0_sq;
44     signal s1_sq;
45     signal s2_sq;
46     signal s3_sq;
47     signal s4_sq;
48
49     signal sumsq_c;
50     signal sumsq_s;
51     signal diff_sq;
52     signal eq_sq;
53
54     signal precheck;
55
56     signal d0;
57     signal d1;
58     signal d2;
59     signal d3;
60     signal d4;
61
62     signal m0;
63     signal m1;
64     signal m2;
65     signal m3;
66     signal m4;
67
68     signal any_match;
69     signal iv;
70
71     signal gt01;
72     signal gt02;
73     signal gt03;
74     signal gt04;
75     signal gt12;
76     signal gt13;
77     signal gt14;
78     signal gt23;
79     signal gt24;
80     signal gt34;
81
82     signal inv;
83     signal total;
84
85     signal output out;
86
87     // Sum checks
88     sum_c <= c0 + c1 + c2 + c3 + c4;
89     sum_s <= s0 + s1 + s2 + s3 + s4;
90     diff_sum <= sum_c - sum_s;
91

```

```

92 diff_sum * eq_sum === 0;
93 diff_sum * diff_sum_inv === 1 - eq_sum;
94 eq_sum * (eq_sum - 1) === 0;
95
96 // Square checks
97 c0_sq <== c0 * c0;
98 c1_sq <== c1 * c1;
99 c2_sq <== c2 * c2;
100 c3_sq <== c3 * c3;
101 c4_sq <== c4 * c4;
102
103 s0_sq <== s0 * s0;
104 s1_sq <== s1 * s1;
105 s2_sq <== s2 * s2;
106 s3_sq <== s3 * s3;
107 s4_sq <== s4 * s4;
108
109 sumsq_c <== c0_sq + c1_sq + c2_sq + c3_sq + c4_sq;
110 sumsq_s <== s0_sq + s1_sq + s2_sq + s3_sq + s4_sq;
111 diff_sq <== sumsq_c - sumsq_s;
112
113 diff_sq * eq_sq === 0;
114 diff_sq * diff_sq_inv === 1 - eq_sq;
115 eq_sq * (eq_sq - 1) === 0;
116
117 precheck <== eq_sum * eq_sq;
118
119 // Match checks for s0 against c[0..4]
120 d0 <== s0 - c0;
121 d1 <== s0 - c1;
122 d2 <== s0 - c2;
123 d3 <== s0 - c3;
124 d4 <== s0 - c4;
125
126 d0 * m0 === 0;
127 d0 * d0_inv === 1 - m0;
128 m0 * (m0 - 1) === 0;
129
130 d1 * m1 === 0;
131 d1 * d1_inv === 1 - m1;
132 m1 * (m1 - 1) === 0;
133
134 d2 * m2 === 0;
135 d2 * d2_inv === 1 - m2;
136 m2 * (m2 - 1) === 0;
137
138 d3 * m3 === 0;
139 d3 * d3_inv === 1 - m3;
140 m3 * (m3 - 1) === 0;
141
142 d4 * m4 === 0;
143 d4 * d4_inv === 1 - m4;
144 m4 * (m4 - 1) === 0;
145
146 // any_match = 1 - (1 - m0)(1 - m1)(1 - m2)(1 - m3)(1 - m4)
147 signal am_t0;
148 signal am_t1;
149 signal am_t2;

```



```

150  signal am_t3;
151
152  am_t0 <= (1 - m0) * (1 - m1);
153  am_t1 <= am_t0 * (1 - m2);
154  am_t2 <= am_t1 * (1 - m3);
155  am_t3 <= am_t2 * (1 - m4);
156  any_match <= 1 - am_t3;
157
158  any_match * (any_match - 1) === 0;
159  iv <= 100 * any_match;
160
161  // Greater-than comparisons
162  component cmp01 = GreaterThan(64);
163  cmp01.in[0] <= s0;
164  cmp01.in[1] <= s1;
165  gt01 <= cmp01.out;
166  gt01 * (gt01 - 1) === 0;
167
168  component cmp02 = GreaterThan(64);
169  cmp02.in[0] <= s0;
170  cmp02.in[1] <= s2;
171  gt02 <= cmp02.out;
172  gt02 * (gt02 - 1) === 0;
173
174  component cmp03 = GreaterThan(64);
175  cmp03.in[0] <= s0;
176  cmp03.in[1] <= s3;
177  gt03 <= cmp03.out;
178  gt03 * (gt03 - 1) === 0;
179
180  component cmp04 = GreaterThan(64);
181  cmp04.in[0] <= s0;
182  cmp04.in[1] <= s4;
183  gt04 <= cmp04.out;
184  gt04 * (gt04 - 1) === 0;
185
186  component cmp12 = GreaterThan(64);
187  cmp12.in[0] <= s1;
188  cmp12.in[1] <= s2;
189  gt12 <= cmp12.out;
190  gt12 * (gt12 - 1) === 0;
191
192  component cmp13 = GreaterThan(64);
193  cmp13.in[0] <= s1;
194  cmp13.in[1] <= s3;
195  gt13 <= cmp13.out;
196  gt13 * (gt13 - 1) === 0;
197
198  component cmp14 = GreaterThan(64);
199  cmp14.in[0] <= s1;
200  cmp14.in[1] <= s4;
201  gt14 <= cmp14.out;
202  gt14 * (gt14 - 1) === 0;
203
204  component cmp23 = GreaterThan(64);
205  cmp23.in[0] <= s2;
206  cmp23.in[1] <= s3;
207  gt23 <= cmp23.out;

```

```

208 gt23 * (gt23 - 1) === 0;
209
210 component cmp24 = GreaterThan(64);
211 cmp24.in[0] <== s2;
212 cmp24.in[1] <== s4;
213 gt24 <== cmp24.out;
214 gt24 * (gt24 - 1) === 0;
215
216 component cmp34 = GreaterThan(64);
217 cmp34.in[0] <== s3;
218 cmp34.in[1] <== s4;
219 gt34 <== cmp34.out;
220 gt34 * (gt34 - 1) === 0;
221
222 inv <== gt01 + gt02 + gt03 + gt04 + gt12 + gt13 + gt14 + gt23 + gt24 + gt34;
223 total <== iv + inv;
224 out <== precheck * total;
225 }
226
227 component main {public [
228     c0, c1, c2, c3, c4,
229     s0, s1, s2, s3, s4,
230     sd
231 ]} = shuffle_integrity_logic();

```

Listing 5: Faulty translated Circom code with Structural Collapse (P1008)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":231:31
3
4 231   ]} = shuffle_integrity_logic();
5   ^
6
7 232
8 233   component main = shuffle_integrity_logic();
9   ^ A semicolon is needed here
10
previous errors were found

```

Listing 6: Circom compiler error log

Explanation: In this example, the real problem is that ‘signal am_t0;’, ‘signal am_t1;’, ‘signal am_t2;’, and ‘signal am_t3;’ are declared in the middle of the template after constraints have already started. Move those signal declarations up into the template’s declaration section with the other ‘signal’ declarations.

A-3 Bitwise Operation (P1008 & P1012)

Description: This error class usually appears when Circom code uses non-Circom expression syntax inside signal declarations or tries to write high-level boolean/bitwise/comparison logic directly as ordinary expressions. In Circom, ‘signal x = ...;’ is not valid for computed signals: you must declare the signal first and then assign/constrain it with ‘<==’ (or use the appropriate assignment form), and operators like ‘==’, ‘||’, ‘>’, and similar logic/comparison constructs generally need dedicated components such as ‘IsEqual’, ‘LessThan’, ‘OR’, ‘AND’, etc. Fixes usually involve splitting declaration from assignment, replacing unsupported operators with circuit gadgets, and ensuring outputs are constrained with ‘<==’ rather than plain ‘=’.

Original Error Tag: P1008+P1012-FEAT-Bitwise-Operation
Example:

```
1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)
```

Listing 7: Source Python shuffle_integrity_logic implementation

```
1 template shuffle_integrity_logic() {
2     signal input c0;
3     signal input c1;
4     signal input c2;
5     signal input c3;
6     signal input c4;
7     signal input s0;
8     signal input s1;
9     signal input s2;
10    signal input s3;
11    signal input s4;
12    signal input sd;
13    signal output out;
14
15    // Check if sum of c equals sum of s
16    signal sum_c = c0 + c1 + c2 + c3 + c4;
17    signal sum_s = s0 + s1 + s2 + s3 + s4;
18    component eq_sum = Equal(64);
19    eq_sum.in[0] <== sum_c;
20    eq_sum.in[1] <== sum_s;
21    signal eq_sum_out <== eq_sum.out;
22
23    // Check if sum of squares of c equals sum of squares of s
24    signal square_c0 = c0 * c0;
25    signal square_c1 = c1 * c1;
26    signal square_c2 = c2 * c2;
27    signal square_c3 = c3 * c3;
28    signal square_c4 = c4 * c4;
29    signal sum_square_c = square_c0 + square_c1 + square_c2 + square_c3 + square_c4;
30    signal square_s0 = s0 * s0;
31    signal square_s1 = s1 * s1;
32    signal square_s2 = s2 * s2;
33    signal square_s3 = s3 * s3;
34    signal square_s4 = s4 * s4;
35    signal sum_square_s = square_s0 + square_s1 + square_s2 + square_s3 + square_s4;
36    component eq_square_sum = Equal(64);
37    eq_square_sum.in[0] <== sum_square_c;
38    eq_square_sum.in[1] <== sum_square_s;
39    signal eq_square_sum_out <== eq_square_sum.out;
40
41    // Check if s0 is in c
```

```

42  signal is_s0_in_c = (s0 == c0) || (s0 == c1) || (s0 == c2) || (s0 == c3) || (s0 ==
    c4);
43  signal iv = is_s0_in_c * 100;
44
45  // Calculate inversions
46  signal inv_s0_s1 = (s0 > s1) * 1;
47  signal inv_s0_s2 = (s0 > s2) * 1;
48  signal inv_s0_s3 = (s0 > s3) * 1;
49  signal inv_s0_s4 = (s0 > s4) * 1;
50  signal inv_s1_s2 = (s1 > s2) * 1;
51  signal inv_s1_s3 = (s1 > s3) * 1;
52  signal inv_s1_s4 = (s1 > s4) * 1;
53  signal inv_s2_s3 = (s2 > s3) * 1;
54  signal inv_s2_s4 = (s2 > s4) * 1;
55  signal inv_s3_s4 = (s3 > s4) * 1;
56  signal inversions = inv_s0_s1 + inv_s0_s2 + inv_s0_s3 + inv_s0_s4 + inv_s1_s2 +
    inv_s1_s3 + inv_s1_s4 + inv_s2_s3 + inv_s2_s4 + inv_s3_s4;
57
58  // Output
59  component and = And(2);
60  and.in[0] <= eq_sum_out;
61  and.in[1] <= eq_square_sum_out;
62  signal and_out = and.out;
63  out = and_out * (inversions + iv);
64 }
65
66 component main = shuffle_integrity_logic();

```

Listing 8: Faulty translated Circom code with Bitwise Operation (P1008 & P1012)

```

1  error[P1008]: Missing semicolon
2    "circuit.circom":20:16
3
4  20    signal sum_c = c0 + c1 + c2 + c3 + c4;
5         ^ A semicolon is needed here
6
7  error[P1012]: illegal expression
8    "circuit.circom":20:18
9
10  20    signal sum_c = c0 + c1 + c2 + c3 + c4;
11         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
12
13  error[P1008]: Missing semicolon
14    "circuit.circom":21:16
15
16  21    signal sum_s = s0 + s1 + s2 + s3 + s4;
17         ^ A semicolon is needed here
18
19  error[P1012]: illegal expression
20    "circuit.circom":21:18
21
22  21    signal sum_s = s0 + s1 + s2 + s3 + s4;
23         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
24
25  error[P1008]: Missing semicolon
26    "circuit.circom":28:20
27
28  28    signal square_c0 = c0 * c0;

```

```

29      ^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":28:22
33
34 28   signal square_c0 = c0 * c0;
35      ^^^^^^^^^ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":29:20
39
40 29   signal square_c1 = c1 * c1;
41      ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44   "circuit.circom":29:22
45
46 29   signal square_c1 = c1 * c1;
47      ^^^^^^^^^ here
48
49 error[P1008]: Missing semicolon
50   "circuit.circom":30:20
51
52 30   signal square_c2 = c2 * c2;
53      ^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56   "circuit.circom":30:22
57
58 30   signal square_c2 = c2 * c2;
59      ^^^^^^^^^ here
60
61 error[P1008]: Missing semicolon
62   "circuit.circom":31:20
63
64 31   signal square_c3 = c3 * c3;
65      ^ A semicolon is needed here
66
67 error[P1012]: illegal expression
68   "circuit.circom":31:22
69
70 31   signal square_c3 = c3 * c3;
71      ^^^^^^^^^ here
72
73 error[P1008]: Missing semicolon
74   "circuit.circom":32:20
75
76 32   signal square_c4 = c4 * c4;
77      ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80   "circuit.circom":32:22
81
82 32   signal square_c4 = c4 * c4;
83      ^^^^^^^^^ here
84
85 error[P1008]: Missing semicolon
86   "circuit.circom":33:23

```



```

198
199 error[P1012]: illegal expression
200     "circuit.circom":50:22
201
202 50     signal inv_s0_s1 = (s0 > s1) * 1;
203         ~~~~~ here
204
205 error[P1008]: Missing semicolon
206     "circuit.circom":51:20
207
208 51     signal inv_s0_s2 = (s0 > s2) * 1;
209         ^ A semicolon is needed here
210
211 error[P1012]: illegal expression
212     "circuit.circom":51:22
213
214 51     signal inv_s0_s2 = (s0 > s2) * 1;
215         ~~~~~ here
216
217 error[P1008]: Missing semicolon
218     "circuit.circom":52:20
219
220 52     signal inv_s0_s3 = (s0 > s3) * 1;
221         ^ A semicolon is needed here
222
223 error[P1012]: illegal expression
224     "circuit.circom":52:22
225
226 52     signal inv_s0_s3 = (s0 > s3) * 1;
227         ~~~~~ here
228
229 error[P1008]: Missing semicolon
230     "circuit.circom":53:20
231
232 53     signal inv_s0_s4 = (s0 > s4) * 1;
233         ^ A semicolon is needed here
234
235 error[P1012]: illegal expression
236     "circuit.circom":53:22
237
238 53     signal inv_s0_s4 = (s0 > s4) * 1;
239         ~~~~~ here
240
241 error[P1008]: Missing semicolon
242     "circuit.circom":54:20
243
244 54     signal inv_s1_s2 = (s1 > s2) * 1;
245         ^ A semicolon is needed here
246
247 error[P1012]: illegal expression
248     "circuit.circom":54:22
249
250 54     signal inv_s1_s2 = (s1 > s2) * 1;
251         ~~~~~ here
252
253 error[P1008]: Missing semicolon
254     "circuit.circom":55:20
255

```



```

256 55    signal inv_s1_s3 = (s1 > s3) * 1;
257      ^ A semicolon is needed here
258
259 error[P1012]: illegal expression
260    "circuit.circom":55:22
261
262 55    signal inv_s1_s3 = (s1 > s3) * 1;
263      ^^^^^^^^^^^^^^^^^ here
264
265 error[P1008]: Missing semicolon
266    "circuit.circom":56:20
267
268 56    signal inv_s1_s4 = (s1 > s4) * 1;
269      ^ A semicolon is needed here
270
271 error[P1012]: illegal expression
272    "circuit.circom":56:22
273
274 56    signal inv_s1_s4 = (s1 > s4) * 1;
275      ^^^^^^^^^^^^^^^^^ here
276
277 error[P1008]: Missing semicolon
278    "circuit.circom":57:20
279
280 57    signal inv_s2_s3 = (s2 > s3) * 1;
281      ^ A semicolon is needed here
282
283 error[P1012]: illegal expression
284    "circuit.circom":57:22
285
286 57    signal inv_s2_s3 = (s2 > s3) * 1;
287      ^^^^^^^^^^^^^^^^^ here
288
289 error[P1008]: Missing semicolon
290    "circuit.circom":58:20
291
292 58    signal inv_s2_s4 = (s2 > s4) * 1;
293      ^ A semicolon is needed here
294
295 error[P1012]: illegal expression
296    "circuit.circom":58:22
297
298 58    signal inv_s2_s4 = (s2 > s4) * 1;
299      ^^^^^^^^^^^^^^^^^ here
300
301 error[P1008]: Missing semicolon
302    "circuit.circom":59:20
303
304 59    signal inv_s3_s4 = (s3 > s4) * 1;
305      ^ A semicolon is needed here
306
307 error[P1012]: illegal expression
308    "circuit.circom":59:22
309
310 59    signal inv_s3_s4 = (s3 > s4) * 1;
311      ^^^^^^^^^^^^^^^^^ here
312
313 error[P1008]: Missing semicolon

```

```

314 "circuit.circom":60:21
315
316 60    signal inversions = inv_s0_s1 + inv_s0_s2 + inv_s0_s3 + inv_s0_s4 + inv_s1_s2 +
      inv_s1_s3 + inv_s1_s4 + inv_s2_s3 + inv_s2_s4 + inv_s3_s4;
317      ^ A semicolon is needed here
318
319 error[P1012]: illegal expression
320 "circuit.circom":60:23
321
322 60    signal inversions = inv_s0_s1 + inv_s0_s2 + inv_s0_s3 + inv_s0_s4 + inv_s1_s2 +
      inv_s1_s3 + inv_s1_s4 + inv_s2_s3 + inv_s2_s4 + inv_s3_s4;
323
      ~~~~~~
      here
324
325 error[P1008]: Missing semicolon
326 "circuit.circom":66:18
327
328 66    signal and_out = and.out;
329      ^ A semicolon is needed here
330
331 error[P1012]: illegal expression
332 "circuit.circom":66:20
333
334 66    signal and_out = and.out;
335      ~~~~~~ here
336
337 previous errors were found

```

Listing 9: Circom compiler error log

Explanation: In this example, lines like ‘signal sum_c = c0 + ...;’ and ‘signal square_c0 = c0 * c0;’ are being parsed as invalid signal declarations, which is why the compiler reports a "missing semicolon" followed by "illegal expression." The later membership and inversion checks are also invalid because they use direct ‘==’, ‘||’, and ‘>’ expressions inside signals, so they must be rewritten using equality/comparison/logic components, and ‘out = ...’ should also be changed to a proper Circom assignment.

A-4 Python-Style Division (P1008)

Description: This error class is usually caused by writing syntax that looks valid in another language or in an older mental model of Circom, but is not valid Circom syntax. The fix is to rewrite those statements using supported Circom constructs and statement terminators: constants should generally be expressed with ‘var’/template parameters or used as literal values, and every declaration or assignment must follow valid Circom grammar with proper semicolons.

Original Error Tag: P1008-FEAT-Python-Style-Division

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3

```

```

8  pe = 1 if (p1 == 0 and r > 5) else 0
9  fs = (bp * 100) + (ts * 20) - (pe * 50)
10 return int(fs if fs > 0 else 0)

```

Listing 10: Source Python iterated_game_memory_score implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4 include "node_modules/circomlib/circuits/bitify.circom";
5
6 template iterated_game_memory_score() {
7     signal input p1;
8     signal input p2;
9     signal input h1;
10    signal input h2;
11    signal input h3;
12    signal input r;
13    signal output out;
14
15    // Constants
16    signal constant ZERO = 0;
17    signal constant ONE = 1;
18    signal constant THREE = 3;
19    signal constant FIVE = 5;
20    signal constant HUNDRED = 100;
21    signal constant TWENTY = 20;
22    signal constant FIFTY = 50;
23    signal constant FIVE_COMP = 5; // For comparison
24
25    // Range checks (Rule 14)
26    component p1_range = Num2Bits(2);
27    p1_range.in <== p1;
28
29    component p2_range = Num2Bits(2);
30    p2_range.in <== p2;
31
32    // bp calculation using priority chain (Rule 9)
33    // Condition 1: p1 == 1 && p2 == 1
34    component eq_p1_1 = IsEqual();
35    eq_p1_1.in[0] <== p1;
36    eq_p1_1.in[1] <== ONE;
37
38    component eq_p2_1 = IsEqual();
39    eq_p2_1.in[0] <== p2;
40    eq_p2_1.in[1] <== ONE;
41
42    signal cond1 <== eq_p1_1.out * eq_p2_1.out; // AND gate (Rule 8)
43
44    // Condition 2: p1 == 1 && p2 == 0
45    component eq_p2_0 = IsEqual();
46    eq_p2_0.in[0] <== p2;
47    eq_p2_0.in[1] <== ZERO;
48
49    signal cond2 <== eq_p1_1.out * eq_p2_0.out;
50
51    // Condition 3: p1 == 0 && p2 == 1
52    component eq_p1_0 = IsEqual();

```

```

53 eq_p1_0.in[0] <= p1;
54 eq_p1_0.in[1] <= ZERO;
55
56 signal cond3 <= eq_p1_0.out * eq_p2_1.out;
57
58 // Priority chain for bp (Rule 9)
59 signal isCase1 <= cond1;
60 signal isCase2 <= (1 - isCase1) * cond2;
61 signal isCase3 <= (1 - isCase1) * (1 - isCase2) * cond3;
62 signal isDefault <= (1 - isCase1) * (1 - isCase2) * (1 - isCase3);
63
64 // bp value selection (Rule 15)
65 signal bp_case1 <= isCase1 * THREE;
66 signal bp_case2 <= isCase2 * ZERO;
67 signal bp_case3 <= isCase3 * FIVE;
68 signal bp_default <= isDefault * ONE;
69
70 signal bp <= bp_case1 + bp_case2 + bp_case3 + bp_default;
71
72 // ts calculation
73 signal ts <= h1 + h2 + h3;
74
75 // pe calculation: (p1 == 0 && r > 5) ? 1 : 0
76 component gt_r_5 = GreaterThan(64);
77 gt_r_5.in[0] <= r;
78 gt_r_5.in[1] <= FIVE_COMP;
79
80 signal cond_pe <= eq_p1_0.out * gt_r_5.out;
81
82 // Mux for pe (Rule 15)
83 signal pe <= cond_pe * (ONE - ZERO) + ZERO;
84
85 // fs calculation
86 signal bp_term <= bp * HUNDRED;
87 signal ts_term <= ts * TWENTY;
88 signal pe_term <= pe * FIFTY;
89
90 signal fs_raw <= bp_term + ts_term - pe_term;
91
92 // Clamp to 0 if negative (Rule 4, Rule 15)
93 component is_negative = LessThan(64);
94 is_negative.in[0] <= fs_raw;
95 is_negative.in[1] <= ZERO;
96
97 // Note: In field arithmetic, we need to check if fs_raw < 0
98 // Since we can't directly compare to 0, we use LessThan
99 // But actually fs_raw < 0 means fs_raw > P/2 in field terms
100 // Let's use a safer approach with Num2Bits
101
102 component fs_bits = Num2Bits(64);
103 fs_bits.in <= fs_raw;
104
105 // Check if fs_raw is negative by looking at the highest bit
106 // For 64-bit numbers, if the 63rd bit is 1, it's negative
107 // But in field arithmetic, we need a different approach
108
109 // Alternative: Use LessThan to check if fs_raw < 0
110 // Actually, we want to check if fs_raw < 0, but in field terms

```

```

111 // we can check if fs_raw > P/2
112
113 // Let's use a simpler approach: if fs_raw < 0, return 0, else return fs_raw
114 // We'll use the comparator approach
115
116 component lt_zero = LessThan(64);
117 lt_zero.in[0] <= fs_raw;
118 lt_zero.in[1] <= ZERO;
119
120 // Since we're in a prime field, we need to be careful with negative numbers
121 // Let's use the absolute value shielding approach (Rule 5)
122
123 // Extract sign (simplified for this use case)
124 // We'll use a different approach: check if subtraction would underflow
125
126 // Actually, for this specific case, we can use:
127 // out = (fs_raw < 0) ? 0 : fs_raw
128 // But we need to implement the comparison properly
129
130 // Let's use the standard approach with LessThan
131 // signal is_negative <= LessThan(64)(fs_raw, 0);
132
133 // Since we can't pass 0 directly to LessThan (it's a signal),
134 // we need to create a zero signal
135
136 signal zero_signal <= 0;
137
138 component is_fs_negative = LessThan(64);
139 is_fs_negative.in[0] <= fs_raw;
140 is_fs_negative.in[1] <= zero_signal;
141
142 // Now apply the clamp: if negative, return 0, else return fs_raw
143 // Using the mux formula (Rule 15)
144 out <= (1 - is_fs_negative.out) * (fs_raw - zero_signal) + zero_signal;
145 }
146
147 component main = iterated_game_memory_score();

```

Listing 11: Faulty translated Circom code with Python-Style Division (P1008)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":15:20
3
4   15      signal constant ZERO = 0;
5           ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8   "circuit.circom":16:20
9
10  16      signal constant ONE = 1;
11         ^ A semicolon is needed here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":17:20
15
16  17      signal constant THREE = 3;
17         ^ A semicolon is needed here
18

```

```

19 error[P1008]: Missing semicolon
20     "circuit.circom":18:20
21
22     18         signal constant FIVE = 5;
23                 ^ A semicolon is needed here
24
25 error[P1008]: Missing semicolon
26     "circuit.circom":19:20
27
28     19         signal constant HUNDRED = 100;
29                 ^ A semicolon is needed here
30
31 error[P1008]: Missing semicolon
32     "circuit.circom":20:20
33
34     20         signal constant TWENTY = 20;
35                 ^ A semicolon is needed here
36
37 error[P1008]: Missing semicolon
38     "circuit.circom":21:20
39
40     21         signal constant FIFTY = 50;
41                 ^ A semicolon is needed here
42
43 error[P1008]: Missing semicolon
44     "circuit.circom":22:20
45
46     22         signal constant FIVE_COMP = 5; // For comparison
47                 ^ A semicolon is needed here
48
49 previous errors were found

```

Listing 12: Circom compiler error log

Explanation: In this example, ‘signal constant ZERO = 0;’ and the similar lines are not valid Circom declarations, so the parser gets confused and reports missing semicolons at ‘constant’. The immediate fix is to replace those constant declarations with valid Circom code, such as ‘var ZERO = 0;’, or inline the numeric literals directly where they are used.

A-5 Boolean-Ternary Logic Misuse (P1008 & P1012)

Description: This error class happens when Circom code mixes constraint syntax with ordinary expressions in a way the parser does not allow, especially trying to use ‘===’ inside a value-producing assignment or ternary expression. In Circom, ‘===’ creates a constraint statement, not a boolean value, so it cannot be used as the condition of ‘?’; instead, compute boolean-like signals with valid gadgets/components such as ‘IsEqual’, ‘LessThan’, ‘GreaterThan’, or algebraic boolean constraints, then use those resulting 0/1 signals in arithmetic selections.

Original Error Tag: P1008+P1012-FEAT-Boolean-Ternary-Logic

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5

```

```

6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 13: Source Python iterated_game_memory_score implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template iterated_game_memory_score() {
6     // Public inputs
7     signal input p1;
8     signal input p2;
9     signal input h1;
10    signal input h2;
11    signal input h3;
12    signal input r;
13
14    // Output
15    signal output out;
16
17    // Private witnesses
18    signal bp;
19    signal ts;
20    signal pe;
21    signal fs;
22    signal fs_lt_zero;
23    signal fs_sel;
24
25    // Intermediate signals for conditions
26    signal cond_p1_eq_1;
27    signal cond_p2_eq_1;
28    signal cond_p1_eq_0;
29    signal cond_p2_eq_0;
30    signal cond_r_gt_5;
31    signal cond_p1_eq_1_and_p2_eq_1;
32    signal cond_p1_eq_1_and_p2_eq_0;
33    signal cond_p1_eq_0_and_p2_eq_1;
34    signal cond_p1_eq_0_and_r_gt_5;
35
36    // Condition computations
37    // p1 == 1
38    signal p1_minus_1 <== p1 - 1;
39    cond_p1_eq_1 <== p1_minus_1 * p1_minus_1 === 0 ? 1 : 0;
40
41    // p2 == 1
42    signal p2_minus_1 <== p2 - 1;
43    cond_p2_eq_1 <== p2_minus_1 * p2_minus_1 === 0 ? 1 : 0;
44
45    // p1 == 0
46    cond_p1_eq_0 <== p1 * p1 === 0 ? 1 : 0;
47
48    // p2 == 0
49    cond_p2_eq_0 <== p2 * p2 === 0 ? 1 : 0;
50

```

```

51 // r > 5
52 component r_gt_5 = GreaterThan(64);
53 r_gt_5.in[0] <== r;
54 r_gt_5.in[1] <== 5;
55 cond_r_gt_5 <== r_gt_5.out;
56
57 // Combined conditions
58 cond_p1_eq_1_and_p2_eq_1 <== cond_p1_eq_1 * cond_p2_eq_1;
59 cond_p1_eq_1_and_p2_eq_0 <== cond_p1_eq_1 * cond_p2_eq_0;
60 cond_p1_eq_0_and_p2_eq_1 <== cond_p1_eq_0 * cond_p2_eq_1;
61 cond_p1_eq_0_and_r_gt_5 <== cond_p1_eq_0 * cond_r_gt_5;
62
63 // Base points calculation
64 bp <== cond_p1_eq_1_and_p2_eq_1 * 3
65      + cond_p1_eq_1_and_p2_eq_0 * 0
66      + cond_p1_eq_0_and_p2_eq_1 * 5
67      + (1 - cond_p1_eq_1_and_p2_eq_1 - cond_p1_eq_1_and_p2_eq_0 -
68      cond_p1_eq_0_and_p2_eq_1) * 1;
69
70 // Total score
71 ts <== h1 + h2 + h3;
72
73 // Penalty
74 pe <== cond_p1_eq_0_and_r_gt_5;
75
76 // Final score
77 fs <== (bp * 100) + (ts * 20) - (pe * 50);
78
79 // Check if final score is less than zero
80 component fs_lt_zero_check = LessThan(64);
81 fs_lt_zero_check.in[0] <== fs;
82 fs_lt_zero_check.in[1] <== 0;
83 fs_lt_zero <== fs_lt_zero_check.out;
84
85 // Selector for final output
86 fs_sel <== fs_lt_zero;
87
88 // Final output (max of 0 and fs)
89 out <== fs_sel * (0 - fs) + fs;
90
91 // Constraints
92 // Binary constraints for condition signals
93 cond_p1_eq_1 * (cond_p1_eq_1 - 1) === 0;
94 cond_p2_eq_1 * (cond_p2_eq_1 - 1) === 0;
95 cond_p1_eq_0 * (cond_p1_eq_0 - 1) === 0;
96 cond_p2_eq_0 * (cond_p2_eq_0 - 1) === 0;
97 cond_r_gt_5 * (cond_r_gt_5 - 1) === 0;
98
99 // Combined condition constraints
100 cond_p1_eq_1_and_p2_eq_1 === cond_p1_eq_1 * cond_p2_eq_1;
101 cond_p1_eq_1_and_p2_eq_0 === cond_p1_eq_1 * cond_p2_eq_0;
102 cond_p1_eq_0_and_p2_eq_1 === cond_p1_eq_0 * cond_p2_eq_1;
103 cond_p1_eq_0_and_r_gt_5 === cond_p1_eq_0 * cond_r_gt_5;
104
105 // Main computation constraints
106 bp === cond_p1_eq_1_and_p2_eq_1 * 3
107      + cond_p1_eq_1_and_p2_eq_0 * 0
108      + cond_p1_eq_0_and_p2_eq_1 * 5

```



```

108     + (1 - cond_p1_eq_1_and_p2_eq_1 - cond_p1_eq_1_and_p2_eq_0 -
109     cond_p1_eq_0_and_p2_eq_1) * 1;
110
111     ts === h1 + h2 + h3;
112     pe === cond_p1_eq_0_and_r_gt_5;
113     fs === (bp * 100) + (ts * 20) - (pe * 50);
114
115     // Binary constraint for fs_lt_zero
116     fs_lt_zero * (fs_lt_zero - 1) === 0;
117     fs_sel === fs_lt_zero;
118     out === fs_sel * (0 - fs) + fs;
119 }
120 component main = iterated_game_memory_score();

```

Listing 14: Faulty translated Circom code with Boolean-Ternary Logic Misuse (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":39:46
3
4 39     cond_p1_eq_1 <== p1_minus_1 * p1_minus_1 === 0 ? 1 : 0;
5                                     ^^^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8   "circuit.circom":39:50
9
10 39     cond_p1_eq_1 <== p1_minus_1 * p1_minus_1 === 0 ? 1 : 0;
11                                     ^^^^^^^^^^^ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":43:46
15
16 43     cond_p2_eq_1 <== p2_minus_1 * p2_minus_1 === 0 ? 1 : 0;
17                                     ^^^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":43:50
21
22 43     cond_p2_eq_1 <== p2_minus_1 * p2_minus_1 === 0 ? 1 : 0;
23                                     ^^^^^^^^^^^ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":46:30
27
28 46     cond_p1_eq_0 <== p1 * p1 === 0 ? 1 : 0;
29                                     ^^^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":46:34
33
34 46     cond_p1_eq_0 <== p1 * p1 === 0 ? 1 : 0;
35                                     ^^^^^^^^^^^ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":49:30
39
40 49     cond_p2_eq_0 <== p2 * p2 === 0 ? 1 : 0;
41                                     ^^^ A semicolon is needed here

```

```

42
43 error[P1012]: illegal expression
44     "circuit.circom":49:34
45
46 49         cond_p2_eq_0 <== p2 * p2 === 0 ? 1 : 0;
47                               ~~~~~ here
48
49 previous errors were found

```

Listing 15: Circom compiler error log

Explanation: In this example, lines like ‘cond_p1_eq_1 <== p1_minus_1 * p1_minus_1 === 0 ? 1 : 0;’ are parsed as if the statement should end at ‘=== 0’, which causes the "missing semicolon" error, and the trailing ‘? 1 : 0’ becomes an illegal expression. The fix is to replace these comparisons with proper equality-check signals, e.g. using an ‘IsEqual’ component or explicit boolean constraints to derive ‘cond_p1_eq_1’, ‘cond_p2_eq_1’, ‘cond_p1_eq_0’, and ‘cond_p2_eq_0’.

A-6 Nested Constraint Conflict (P1008 & P1012)

Description: This error class happens when Circom constraint syntax is mixed incorrectly, especially by nesting a constraint operator like ‘===’ inside an assignment or signal declaration using ‘<==’. In Circom, ‘<==’ is for assigning a signal from an expression, while ‘===’ creates a separate constraint and is not itself a value-producing expression, so the parser reports a missing semicolon and then flags the remaining text as illegal. The fix is to split these into separate statements: declare or assign the signal normally, and write the equality constraint on its own line, or use an ‘Eq’ component if you want a boolean output signal.

Original Error Tag: P1008+P1012-FEAT-Nested-Constraint-Conflict

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)

```

Listing 16: Source Python shuffle_integrity_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2 include "node_modules/circomlib/circuits/arith.circom";
3
4 template shuffle_integrity_logic() {
5     signal input c0;
6     signal input c1;
7     signal input c2;
8     signal input c3;
9     signal input c4;
10    signal input s0;
11    signal input s1;
12    signal input s2;

```

```

13  signal input s3;
14  signal input s4;
15  signal input sd;
16
17  // Check if c0+c1+c2+c3+c4 equals s0+s1+s2+s3+s4
18  component add = Add(64);
19  add.in[0] <== c0;
20  add.in[1] <== c1;
21  add.in[2] <== c2;
22  add.in[3] <== c3;
23  add.in[4] <== c4;
24  signal sum_c <== add.out;
25  component add_s = Add(64);
26  add_s.in[0] <== s0;
27  add_s.in[1] <== s1;
28  add_s.in[2] <== s2;
29  add_s.in[3] <== s3;
30  add_s.in[4] <== s4;
31  signal sum_s <== add_s.out;
32  signal eq_sum <== sum_c == sum_s;
33
34  // Check if c0*c0+c1*c1+c2*c2+c3*c3+c4*c4 equals s0*s0+s1*s1+s2*s2+s3*s3+s4*s4
35  component mul = Mul(64);
36  mul.in[0] <== c0;
37  mul.in[1] <== c0;
38  signal sum_c2 <== mul.out;
39  component mul_s = Mul(64);
40  mul_s.in[0] <== s0;
41  mul_s.in[1] <== s0;
42  signal sum_s2 <== mul_s.out;
43  component add2 = Add(64);
44  add2.in[0] <== sum_c2;
45  add2.in[1] <== c1;
46  add2.in[2] <== c1;
47  signal sum_c2_2 <== add2.out;
48  component add_s2 = Add(64);
49  add_s2.in[0] <== sum_s2;
50  add_s2.in[1] <== s1;
51  add_s2.in[2] <== s1;
52  signal sum_s2_2 <== add_s2.out;
53  component add3 = Add(64);
54  add3.in[0] <== sum_c2_2;
55  add3.in[1] <== c2;
56  add3.in[2] <== c2;
57  signal sum_c2_3 <== add3.out;
58  component add_s2_3 = Add(64);
59  add_s2_3.in[0] <== sum_s2_2;
60  add_s2_3.in[1] <== s2;
61  add_s2_3.in[2] <== s2;
62  signal sum_s2_3 <== add_s2_3.out;
63  component add4 = Add(64);
64  add4.in[0] <== sum_c2_3;
65  add4.in[1] <== c3;
66  add4.in[2] <== c3;
67  signal sum_c2_4 <== add4.out;
68  component add_s2_4 = Add(64);
69  add_s2_4.in[0] <== sum_s2_3;
70  add_s2_4.in[1] <== s3;

```

```

71 add_s2_4.in[2] <= s3;
72 signal sum_s2_4 <= add_s2_4.out;
73 component add5 = Add(64);
74 add5.in[0] <= sum_c2_4;
75 add5.in[1] <= c4;
76 add5.in[2] <= c4;
77 signal sum_c2_5 <= add5.out;
78 component add_s2_5 = Add(64);
79 add_s2_5.in[0] <= sum_s2_4;
80 add_s2_5.in[1] <= s4;
81 add_s2_5.in[2] <= s4;
82 signal sum_s2_5 <= add_s2_5.out;
83 signal eq_sum2 <= sum_c2_5 == sum_s2_5;
84
85 // Check if c0 equals s0
86 component eq = Eq(64);
87 eq.in[0] <= c0;
88 eq.in[1] <= s0;
89 signal eq_c_s <= eq.out;
90 signal iv <= eq_c_s * 100;
91
92 // Check if s0 is greater than any other element in sh
93 component gt = GreaterThan(64);
94 gt.in[0] <= s0;
95 gt.in[1] <= s1;
96 signal gt_s0_s1 <= gt.out;
97 component add_gt = Add(64);
98 add_gt.in[0] <= gt_s0_s1;
99 add_gt.in[1] <= 1;
100 signal inv <= add_gt.out;
101 component gt_s0_s2 = GreaterThan(64);
102 gt_s0_s2.in[0] <= s0;
103 gt_s0_s2.in[1] <= s2;
104 signal gt_s0_s2_out <= gt_s0_s2.out;
105 component add_gt2 = Add(64);
106 add_gt2.in[0] <= gt_s0_s2_out;
107 add_gt2.in[1] <= inv;
108 signal inv2 <= add_gt2.out;
109 component gt_s0_s3 = GreaterThan(64);
110 gt_s0_s3.in[0] <= s0;
111 gt_s0_s3.in[1] <= s3;
112 signal gt_s0_s3_out <= gt_s0_s3.out;
113 component add_gt3 = Add(64);
114 add_gt3.in[0] <= gt_s0_s3_out;
115 add_gt3.in[1] <= inv2;
116 signal inv3 <= add_gt3.out;
117 component gt_s0_s4 = GreaterThan(64);
118 gt_s0_s4.in[0] <= s0;
119 gt_s0_s4.in[1] <= s4;
120 signal gt_s0_s4_out <= gt_s0_s4.out;
121 component add_gt4 = Add(64);
122 add_gt4.in[0] <= gt_s0_s4_out;
123 add_gt4.in[1] <= inv3;
124 signal inv4 <= add_gt4.out;
125 component gt_s1_s2 = GreaterThan(64);
126 gt_s1_s2.in[0] <= s1;
127 gt_s1_s2.in[1] <= s2;
128 signal gt_s1_s2_out <= gt_s1_s2.out;

```

```

129     component add_gt5 = Add(64);
130     add_gt5.in[0] <== gt_s1_s2_out;
131     add_gt5.in[1] <== inv4;
132     signal inv5 <== add_gt5.out;
133     component gt_s1_s3 = GreaterThan(64);
134     gt_s1_s3.in[0] <== s1;
135     gt_s1_s3.in[1] <== s3;
136     signal gt_s1_s3_out <== gt_s1_s3.out;
137     component add_gt6 = Add(64);
138     add_gt6.in[0] <== gt_s1_s3_out;
139     add_gt6.in[1] <== inv5;
140     signal inv6 <== add_gt6.out;
141     component gt_s1_s4 = GreaterThan(64);
142     gt_s1_s4.in[0] <== s1;
143     gt_s1_s4.in[1] <== s4;
144     signal gt_s1_s4_out <== gt_s1_s4.out;
145     component add_gt7 = Add(64);
146     add_gt7.in[0] <== gt_s1_s4_out;
147     add_gt7.in[1] <== inv6;
148     signal inv7 <== add_gt7.out;
149     component gt_s2_s3 = GreaterThan(64);
150     gt_s2_s3.in[0] <== s2;
151     gt_s2_s3.in[1] <== s3;
152     signal gt_s2_s3_out <== gt_s2_s3.out;
153     component add_gt8 = Add(64);
154     add_gt8.in[0] <== gt_s2_s3_out;
155     add_gt8.in[1] <== inv7;
156     signal inv8 <== add_gt8.out;
157     component gt_s2_s4 = GreaterThan(64);
158     gt_s2_s4.in[0] <== s2;
159     gt_s2_s4.in[1] <== s4;
160     signal gt_s2_s4_out <== gt_s2_s4.out;
161     component add_gt9 = Add(64);
162     add_gt9.in[0] <== gt_s2_s4_out;
163     add_gt9.in[1] <== inv8;
164     signal inv9 <== add_gt9.out;
165     component gt_s3_s4 = GreaterThan(64);
166     gt_s3_s4.in[0] <== s3;
167     gt_s3_s4.in[1] <== s4;
168     signal gt_s3_s4_out <== gt_s3_s4.out;
169     component add_gt10 = Add(64);
170     add_gt10.in[0] <== gt_s3_s4_out;
171     add_gt10.in[1] <== inv9;
172     signal inv10 <== add_gt10.out;
173     signal inv11 <== inv10 + iv;
174
175     // Output the result
176     signal out <== inv11;
177 }
178
179 component main = shuffle_integrity_logic();

```

Listing 17: Faulty translated Circom code with Nested Constraint Conflict (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2     "circuit.circom":33:29
3
4 33     signal eq_sum <== sum_c === sum_s;

```

```

5          ^^^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8   "circuit.circom":33:33
9
10 33      signal eq_sum <== sum_c === sum_s;
11          ^^^^^^ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":84:33
15
16 84      signal eq_sum2 <== sum_c2_5 === sum_s2_5;
17          ^^^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":84:37
21
22 84      signal eq_sum2 <== sum_c2_5 === sum_s2_5;
23          ^^^^^^^^^^ here
24
25 previous errors were found

```

Listing 18: Circom compiler error log

Explanation: In this example, ‘signal eq_sum <== sum_c === sum_s;’ and ‘signal eq_sum2 <== sum_c2_5 === sum_s2_5;’ try to assign the result of a constraint as though it were an expression value. Replace them with standalone constraints like ‘sum_c === sum_s;’ and ‘sum_c2_5 === sum_s2_5;’, or use an equality comparator component if ‘eq_sum’ and ‘eq_sum2’ are meant to hold 0/1 results.

A-7 Invalid Signal Type Modifier (P1008)

Description: This error class happens when a Circom ‘signal’ declaration uses an unsupported type-/modifier keyword, so the parser no longer recognizes the declaration grammar and starts reporting misleading syntax errors such as "missing semicolon" or "illegal expression." In Circom, signals are typically declared as ‘signal’, ‘signal input’, or ‘signal output’ without extra type modifiers like ‘long’; fix it by removing the invalid modifier and, if needed, expressing size/bit-width through components such as comparators rather than through the signal declaration itself.

Original Error Tag: P1008-FEAT-Invalid-Signal-Type-Modifier

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 19: Source Python iterated_game_memory_score implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template iteratedGameMemoryScore() {
4     signal input long p1, p2, h1, h2, h3, r;
5     signal output long out;
6     component GreaterEqThan = GreaterEqThan(64);
7     GreaterEqThan.in[0] <== p1;
8     GreaterEqThan.in[1] <== 1;
9     signal long gte_out1 <== GreaterEqThan.out;
10    GreaterEqThan.in[0] <== p2;
11    GreaterEqThan.in[1] <== 1;
12    signal long gte_out2 <== GreaterEqThan.out;
13    signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1 ==
14    0 && p2 == 1) ? 5 : 1;
15    signal long ts <== h1 + h2 + h3;
16    GreaterEqThan.in[0] <== r;
17    GreaterEqThan.in[1] <== 5;
18    signal long gte_out3 <== GreaterEqThan.out;
19    signal long pe <== (p1 == 0 && gte_out3) ? 1 : 0;
20    signal long fs <== (bp * 100) + (ts * 20) - (pe * 50);
21    out <== sel ? 0 : fs;
22 }
23 component main = iteratedGameMemoryScore();

```

Listing 20: Faulty translated Circom code with Invalid Signal Type Modifier (P1008)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":6:22
3
4 6     signal input long p1, p2, h1, h2, h3, r;
5       ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8   "circuit.circom":6:25
9
10 6     signal input long p1, p2, h1, h2, h3, r;
11       ^ A semicolon is needed here
12
13 error[P1012]: illegal expression
14   "circuit.circom":6:23
15
16 6     signal input long p1, p2, h1, h2, h3, r;
17       ^^^ here
18
19 error[P1008]: Missing semicolon
20   "circuit.circom":6:29
21
22 6     signal input long p1, p2, h1, h2, h3, r;
23       ^ A semicolon is needed here
24
25 error[P1012]: illegal expression
26   "circuit.circom":6:27
27
28 6     signal input long p1, p2, h1, h2, h3, r;
29       ^^^ here
30

```

```

31 error[P1008]: Missing semicolon
32     "circuit.circom":6:33
33
34 6     signal input long p1, p2, h1, h2, h3, r;
35                                     ^ A semicolon is needed here
36
37 error[P1012]: illegal expression
38     "circuit.circom":6:31
39
40 6     signal input long p1, p2, h1, h2, h3, r;
41                                     ^^^ here
42
43 error[P1008]: Missing semicolon
44     "circuit.circom":6:37
45
46 6     signal input long p1, p2, h1, h2, h3, r;
47                                     ^ A semicolon is needed here
48
49 error[P1012]: illegal expression
50     "circuit.circom":6:35
51
52 6     signal input long p1, p2, h1, h2, h3, r;
53                                     ^^^ here
54
55 error[P1008]: Missing semicolon
56     "circuit.circom":6:41
57
58 6     signal input long p1, p2, h1, h2, h3, r;
59                                     ^ A semicolon is needed here
60
61 error[P1012]: illegal expression
62     "circuit.circom":6:39
63
64 6     signal input long p1, p2, h1, h2, h3, r;
65                                     ^^^ here
66
67 error[P1012]: illegal expression
68     "circuit.circom":6:43
69
70 6     signal input long p1, p2, h1, h2, h3, r;
71                                     ^^ here
72
73 error[P1008]: Missing semicolon
74     "circuit.circom":7:23
75
76 7     signal output long out;
77                                     ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80     "circuit.circom":7:24
81
82 7     signal output long out;
83                                     ^^^^^ here
84
85 error[P1008]: Missing semicolon
86     "circuit.circom":11:16
87
88 11    signal long gte_out1 <== GreaterEqThan.out;

```



```

89         ^ A semicolon is needed here
90
91 error[P1008]: Missing semicolon
92     "circuit.circom":14:16
93
94 14         signal long gte_out2 <== GreaterEqThan.out;
95         ^ A semicolon is needed here
96
97 error[P1008]: Missing semicolon
98     "circuit.circom":15:16
99
100 15         signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
101         == 0 && p2 == 1) ? 5 : 1;
102         ^ A semicolon is needed here
103
104 error[P1008]: Missing semicolon
105     "circuit.circom":15:74
106
107 15         signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
108         == 0 && p2 == 1) ? 5 : 1;
109
110         semicolon is needed here
111         ^ A
112
113 error[P1008]: Missing semicolon
114     "circuit.circom":15:78
115
116 15         signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
117         == 0 && p2 == 1) ? 5 : 1;
118
119         semicolon is needed here
120         ^ A
121
122 error[P1012]: illegal expression
123     "circuit.circom":15:76
124
125 15         signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
126         == 0 && p2 == 1) ? 5 : 1;
127
128         here
129
130 error[P1012]: illegal expression
131     "circuit.circom":15:80
132
133 15         signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
134         == 0 && p2 == 1) ? 5 : 1;
135
136         ~~~~~ here
137
138 error[P1008]: Missing semicolon
139     "circuit.circom":16:16
140
141 16         signal long ts <== h1 + h2 + h3;
142         ^ A semicolon is needed here
143
144 error[P1008]: Missing semicolon
145     "circuit.circom":19:16
146
147 19         signal long gte_out3 <== GreaterEqThan.out;
148         ^ A semicolon is needed here

```

```

138
139 error[P1008]: Missing semicolon
140     "circuit.circom":20:16
141
142 20     signal long pe <== (p1 == 0 && gte_out3) ? 1 : 0;
143     ^ A semicolon is needed here
144
145 error[P1008]: Missing semicolon
146     "circuit.circom":21:16
147
148 21     signal long fs <== (bp * 100) + (ts * 20) - (pe * 50);
149     ^ A semicolon is needed here
150
151 error[P1008]: Missing semicolon
152     "circuit.circom":22:16
153
154 22     signal long sel <== fs < 0;
155     ^ A semicolon is needed here
156
157 previous errors were found

```

Listing 21: Circom compiler error log

Explanation: In this example, ‘long’ is being used in declarations like ‘signal input long p1’ and ‘signal long gte_out1’, but Circom does not allow that modifier there. Once ‘long’ is removed, many of the reported semicolon and expression errors will disappear because they are just follow-on parse errors caused by the invalid declaration syntax.

A-8 Numeric Suffix Hallucination (P1008 & P1012)

Description: This error class is usually caused by writing Circom as if it were a general-purpose language: using raw boolean/comparison expressions like ‘a < b’ or ‘a >= 0’ as standalone constraints, or declaring signals with inline assignments like ‘signal x = y + z;’. In Circom, comparisons must be enforced through comparator components or rewritten as valid arithmetic constraints, and signals must generally be declared first and then assigned with ‘<==’ (or constrained with ‘===’). The fix is to replace illegal expressions with Circom-native constraints, split ‘signal x = ...’ into ‘signal x; x <== ...;’, and ensure component instantiations and the ‘main’ declaration follow valid syntax.

Original Error Tag: P1008+P1012-FEAT-Numeric-Suffix-Hallucination

Example:

```

1 def anova_f_test_logic(g1_s, g1_n, g2_s, g2_n, total_ss):
2     m1, m2 = g1_s // g1_n, g2_s // g2_n
3     gm = (g1_s + g2_s) // (g1_n + g2_n)
4     ssb = g1_n * (m1 - gm)**2 + g2_n * (m2 - gm)**2
5     ssw = total_ss - ssb
6     if ssw <= 0: return int(gm)
7     return int((ssb * 100) // ssw + gm)

```

Listing 22: Source Python anova_f_test_logic implementation

```

1 pragma circom 2.1.4;
2
3 include "circomlib/poseidon.circom";
4 include "circomlib/comparators.circom";

```

```

5 include "circomlib/bitify.circom";
6
7 // Integer division template with proper constraints
8 template IntegerDiv() {
9     signal input numerator;
10    signal input denominator;
11    signal output quotient;
12    signal output remainder;
13
14    // Compute quotient and remainder (witness generation)
15    quotient <-- numerator \ denominator;
16    remainder <-- numerator % denominator;
17
18    // Constraints
19    denominator * quotient + remainder === numerator;
20    remainder < denominator;
21    remainder >= 0;
22 }
23
24 // Check if a signal is non-positive (<= 0)
25 template IsNonPositive(n) {
26     signal input in;
27     signal output isNonPositive;
28
29     // Use comparator to check if in <= 0
30     component lt = LessEqThan(n);
31     lt.in[0] <== in;
32     lt.in[1] <== 0;
33     isNonPositive <== lt.out;
34 }
35
36 // Main ANOVA F-test template
37 template ANOVA_F_Test(n) {
38     // n is the bit width for comparisons (e.g., 64 for i64)
39     signal input g1s;
40     signal input g1n;
41     signal input g2s;
42     signal input g2n;
43     signal input tss;
44
45     signal output result;
46
47     // 1. Calculate means with integer division
48     // Mean of group 1: m1 = g1s / g1n
49     component div1 = IntegerDiv();
50     div1.numerator <== g1s;
51     div1.denominator <== g1n;
52     signal m1;
53     m1 <== div1.quotient;
54
55     // Mean of group 2: m2 = g2s / g2n
56     component div2 = IntegerDiv();
57     div2.numerator <== g2s;
58     div2.denominator <== g2n;
59     signal m2;
60     m2 <== div2.quotient;
61
62     // Grand mean: gm = (g1s + g2s) / (g1n + g2n)

```

```

63  signal total_sum = g1s + g2s;
64  signal total_count = g1n + g2n;
65
66  component div3 = IntegerDiv();
67  div3.numerator <= total_sum;
68  div3.denominator <= total_count;
69  signal gm;
70  gm <= div3.quotient;
71
72  // 2. Calculate differences from grand mean
73  signal m1_minus_gm = m1 - gm;
74  signal m2_minus_gm = m2 - gm;
75
76  // 3. Square differences
77  signal m1_minus_gm_sq = m1_minus_gm * m1_minus_gm;
78  signal m2_minus_gm_sq = m2_minus_gm * m2_minus_gm;
79
80  // 4. Weighted sum of squares (Between-group SS)
81  signal g1n_times_sq1 = g1n * m1_minus_gm_sq;
82  signal g2n_times_sq2 = g2n * m2_minus_gm_sq;
83  signal ssb = g1n_times_sq1 + g2n_times_sq2;
84
85  // 5. Within-group sum of squares
86  signal ssw = tss - ssb;
87
88  // 6. Check if ssw <= 0
89  component check_nonpos = IsNonPositive(n);
90  check_nonpos.in <= ssw;
91  signal ssw_nonpos = check_nonpos.isNonPositive;
92
93  // Binary constraint for ssw_nonpos
94  ssw_nonpos * (1 - ssw_nonpos) == 0;
95
96  // Additional constraints for ssw_nonpos
97  // If ssw_nonpos = 1, then ssw <= 0
98  // If ssw_nonpos = 0, then ssw > 0
99  component check_positive = LessThan(n);
100  check_positive.in[0] <= 0;
101  check_positive.in[1] <= ssw;
102  signal ssw_positive = check_positive.out;
103
104  // Constraint: ssw_nonpos = 1 - ssw_positive
105  ssw_nonpos == 1 - ssw_positive;
106
107  // 7. Calculate quotient when ssw > 0
108  signal ssb_times_100 = ssb * 100;
109  signal quotient;
110
111  // Only perform division if ssw > 0
112  // We'll use a conditional constraint
113  signal denominator_safe = ssw + ssw_nonpos; // Add 1 if ssw = 0 to avoid division
by zero
114
115  component div4 = IntegerDiv();
116  div4.numerator <= ssb_times_100;
117  div4.denominator <= denominator_safe;
118  quotient <= div4.quotient;
119

```

```

120 // Constraint: if ssw_nonpos = 1, quotient should be 0
121 // if ssw_nonpos = 0, quotient = (ssb * 100) / ssw
122 quotient * (1 - ssw_nonpos) == (ssb_times_100 - div4.remainder) * (1 - ssw_nonpos
) / denominator_safe;
123
124 // 8. Conditional result
125 // result = ssw_nonpos * gm + (1 - ssw_nonpos) * (quotient + gm)
126 signal adjusted = quotient + gm;
127 result <= ssw_nonpos * gm + (1 - ssw_nonpos) * adjusted;
128
129 // Alternative implementation using selector
130 // result = gm + (1 - ssw_nonpos) * quotient
131 // This is equivalent to the above but simpler
132 // result <= gm + (1 - ssw_nonpos) * quotient;
133 }
134
135 // Example main component for testing
136 component main {public [g1s, g1n, g2s, g2n, tss]} = ANOVA_F_Test(64);

```

Listing 23: Faulty translated Circom code with Numeric Suffix Hallucination (P1008 & P1012)

```

1 error[P1012]: illegal expression
2   "circuit.circom":18:5
3
4 18     remainder < denominator;
5     ~~~~~ here
6
7 error[P1012]: illegal expression
8   "circuit.circom":19:5
9
10 19     remainder >= 0;
11     ~~~~~ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":61:22
15
16 61     signal total_sum = g1s + g2s;
17     ~ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":61:24
21
22 61     signal total_sum = g1s + g2s;
23     ~~~~~ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":62:24
27
28 62     signal total_count = g1n + g2n;
29     ~ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":62:26
33
34 62     signal total_count = g1n + g2n;
35     ~~~~~ here
36
37 error[P1008]: Missing semicolon

```

```

38 "circuit.circom":71:24
39
40 71      signal m1_minus_gm = m1 - gm;
41           ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44 "circuit.circom":71:26
45
46 71      signal m1_minus_gm = m1 - gm;
47           ^^^^^^^^^ here
48
49 error[P1008]: Missing semicolon
50 "circuit.circom":72:24
51
52 72      signal m2_minus_gm = m2 - gm;
53           ^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56 "circuit.circom":72:26
57
58 72      signal m2_minus_gm = m2 - gm;
59           ^^^^^^^^^ here
60
61 error[P1008]: Missing semicolon
62 "circuit.circom":75:27
63
64 75      signal m1_minus_gm_sq = m1_minus_gm * m1_minus_gm;
65           ^ A semicolon is needed here
66
67 error[P1012]: illegal expression
68 "circuit.circom":75:29
69
70 75      signal m1_minus_gm_sq = m1_minus_gm * m1_minus_gm;
71           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
72
73 error[P1008]: Missing semicolon
74 "circuit.circom":76:27
75
76 76      signal m2_minus_gm_sq = m2_minus_gm * m2_minus_gm;
77           ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80 "circuit.circom":76:29
81
82 76      signal m2_minus_gm_sq = m2_minus_gm * m2_minus_gm;
83           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
84
85 error[P1008]: Missing semicolon
86 "circuit.circom":79:26
87
88 79      signal g1n_times_sq1 = g1n * m1_minus_gm_sq;
89           ^ A semicolon is needed here
90
91 error[P1012]: illegal expression
92 "circuit.circom":79:28
93
94 79      signal g1n_times_sq1 = g1n * m1_minus_gm_sq;
95           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here

```

```

96
97 error[P1008]: Missing semicolon
98     "circuit.circom":80:26
99
100 80     signal g2n_times_sq2 = g2n * m2_minus_gm_sq;
101     ^ A semicolon is needed here
102
103 error[P1012]: illegal expression
104     "circuit.circom":80:28
105
106 80     signal g2n_times_sq2 = g2n * m2_minus_gm_sq;
107     ~~~~~ here
108
109 error[P1008]: Missing semicolon
110     "circuit.circom":81:16
111
112 81     signal ssb = g1n_times_sq1 + g2n_times_sq2;
113     ^ A semicolon is needed here
114
115 error[P1012]: illegal expression
116     "circuit.circom":81:18
117
118 81     signal ssb = g1n_times_sq1 + g2n_times_sq2;
119     ~~~~~ here
120
121 error[P1008]: Missing semicolon
122     "circuit.circom":84:16
123
124 84     signal ssw = tss - ssb;
125     ^ A semicolon is needed here
126
127 error[P1012]: illegal expression
128     "circuit.circom":84:18
129
130 84     signal ssw = tss - ssb;
131     ~~~~~ here
132
133 error[P1008]: Missing semicolon
134     "circuit.circom":89:23
135
136 89     signal ssw_nonpos = check_nonpos.isNonPositive;
137     ^ A semicolon is needed here
138
139 error[P1012]: illegal expression
140     "circuit.circom":89:25
141
142 89     signal ssw_nonpos = check_nonpos.isNonPositive;
143     ~~~~~ here
144
145 error[P1008]: Missing semicolon
146     "circuit.circom":100:25
147
148 100    signal ssw_positive = check_positive.out;
149     ^ A semicolon is needed here
150
151 error[P1012]: illegal expression
152     "circuit.circom":100:27
153

```

```

154 100      signal ssw_positive = check_positive.out;
155                                ~~~~~ here
156
157 error[P1008]: Missing semicolon
158   "circuit.circom":106:26
159
160 106      signal ssb_times_100 = ssb * 100;
161                                ^ A semicolon is needed here
162
163 error[P1012]: illegal expression
164   "circuit.circom":106:28
165
166 106      signal ssb_times_100 = ssb * 100;
167                                ~~~~~ here
168
169 error[P1008]: Missing semicolon
170   "circuit.circom":111:29
171
172 111      signal denominator_safe = ssw + ssw_nonpos; // Add 1 if ssw = 0 to avoid
173          division by zero
174                                ^ A semicolon is needed here
175
176 error[P1012]: illegal expression
177   "circuit.circom":111:31
178
179 111      signal denominator_safe = ssw + ssw_nonpos; // Add 1 if ssw = 0 to avoid
180          division by zero
181                                ~~~~~ here
182
183 error[P1008]: Missing semicolon
184   "circuit.circom":124:21
185
186 124      signal adjusted = quotient + gm;
187                                ^ A semicolon is needed here
188
189 error[P1012]: illegal expression
190   "circuit.circom":124:23
191
192 124      signal adjusted = quotient + gm;
193                                ~~~~~ here
194
195 error[P1008]: Missing semicolon
196   "circuit.circom":134:69
197
198 134      component main {public [g1s, g1n, g2s, g2n, tss]} = ANOVA_F_Test(64);
199
200 135
201 136      component main = ANOVA_F_Test();
202          ^ A semicolon is needed here
203
204 previous errors were found

```

Listing 24: Circom compiler error log

Explanation: In this example, ‘remainder < denominator;’ and ‘remainder >= 0;’ are illegal because they are plain comparison expressions, not Circom constraints. Most of the later ‘P1008’/‘P1012’ errors come from lines like ‘signal total_sum = g1s + g2s;’, which should be written as ‘signal total_sum; total_sum <== g1s + g2s;’; there is also an extra duplicate ‘main’ declaration at the end.

A-9 Namespace-Method Hallucination (P1008 & P1012)

Description: This error class happens when Circom code uses features that look like method calls or direct comparison/range syntax from other languages, but are not valid in Circom's grammar. A common fix is to rewrite the code using actual Circom constructs: instantiate components explicitly, wire values through their input/output signals instead of calling 'component.output(...)' like a function, and express checks such as bounds with valid arithmetic/constraint gadgets rather than raw operators like '<'.

Original Error Tag: P1008+P1012-FEAT-Namespace-Method-Hallucination

Example:

```
1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)
```

Listing 25: Source Python ema_signal_logic implementation

```
1 pragma circom 2.1.4;
2
3 template EMA() {
4     signal input p0;
5     signal input p1;
6     signal input p2;
7     signal input p3;
8     signal input p4;
9     signal output out;
10
11     // Constants
12     var ALPHA = 40;
13     var ONE_HUNDRED = 100;
14     var SIXTY = 60;
15
16     // Helper function for EMA calculation
17     component emaCalc = EMAIteration();
18
19     // Initial EMA (p0 * 100)
20     signal ema0;
21     ema0 <== p0 * ONE_HUNDRED;
22
23     // EMA iterations
24     signal ema1 <== emaCalc.ema_out(p1, ema0);
25     signal ema2 <== emaCalc.ema_out(p2, ema1);
26     signal ema3 <== emaCalc.ema_out(p3, ema2);
27     signal ema4 <== emaCalc.ema_out(p4, ema3);
28
29     // Output
30     out <== ema4;
31 }
32
33 // Helper template for a single EMA iteration
34 template EMAIteration() {
35     signal input price;
```

```

36  signal input prev_ema;
37  signal output ema_out;
38
39  var ALPHA = 40;
40  var ONE_HUNDRED = 100;
41  var SIXTY = 60;
42
43  signal temp1;
44  temp1 <== price * ALPHA;
45
46  // ema_{i-1} * (100 - alpha) = ema_{i-1} * 60
47  signal temp2;
48  temp2 <== prev_ema * SIXTY;
49
50  // Integer division: temp2 100
51  // Represent as: temp2 = 100 * q + r, where 0 <= r < 100
52  signal q;
53  signal r;
54
55  // Division constraint: temp2 = 100 * q + r
56  temp2 == q * ONE_HUNDRED + r;
57
58  // Range constraint: 0 <= r < 100
59  r < ONE_HUNDRED;
60
61  // Final EMA calculation
62  ema_out <== temp1 + q;
63 }
64
65 component main = EMA();

```

Listing 26: Faulty translated Circom code with Namespace-Method Hallucination (P1008 & P1012)

```

1  error[P1008]: Missing semicolon
2    "circuit.circom":26:36
3
4  26      signal ema1 <== emaCalc.ema_out(p1, ema0);
5          ~ A semicolon is needed here
6
7  error[P1012]: illegal expression
8    "circuit.circom":26:36
9
10  26      signal ema1 <== emaCalc.ema_out(p1, ema0);
11          ~~~~~ here
12
13  error[P1008]: Missing semicolon
14    "circuit.circom":27:36
15
16  27      signal ema2 <== emaCalc.ema_out(p2, ema1);
17          ~ A semicolon is needed here
18
19  error[P1012]: illegal expression
20    "circuit.circom":27:36
21
22  27      signal ema2 <== emaCalc.ema_out(p2, ema1);
23          ~~~~~ here
24
25  error[P1008]: Missing semicolon

```

```

26 "circuit.circom":28:36
27
28     signal ema3 <== emaCalc.ema_out(p3, ema2);
29         ^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32     "circuit.circom":28:36
33
34     signal ema3 <== emaCalc.ema_out(p3, ema2);
35         ^^^^^^^^^^^^^^ here
36
37 error[P1008]: Missing semicolon
38     "circuit.circom":29:36
39
40     signal ema4 <== emaCalc.ema_out(p4, ema3);
41         ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44     "circuit.circom":29:36
45
46     signal ema4 <== emaCalc.ema_out(p4, ema3);
47         ^^^^^^^^^^^^^^ here
48
49 error[P1012]: illegal expression
50     "circuit.circom":62:5
51
52     62     r < ONE_HUNDRED;
53         ^^^^^^^^^^^^^^^^^ here
54
55 previous errors were found

```

Listing 27: Circom compiler error log

Explanation: In this example, ‘emaCalc.ema_out(p1, ema0)’ is invalid because ‘ema_out’ is an output signal, not a callable method; ‘EMAIteration’ must be instantiated per step and connected via ‘price’, ‘prev_ema’, and ‘ema_out’. The line ‘r < ONE_HUNDRED;’ is also illegal in Circom because ‘<’ is not a standalone constraint statement, so the remainder bound must be enforced with a proper range-check construction.

A-10 Decl-Constraint Conflict (P1008 & P1012)

Description: This error class happens when Circom code mixes up declarations, assignments, and constraints. In Circom, ‘signal’ declarations must end after the name, and you cannot initialize a signal with ‘=’ the way you would in a general-purpose language; instead, declare the signal first and then connect it with ‘<==’ or constrain it with ‘===’ in a separate statement. A related cause is trying to use constraint syntax like ‘===’ inside a normal arithmetic expression, which is illegal; if you need an equality result as a value, use a dedicated boolean/equality gadget such as ‘IsEqual’ and then combine its output.

Original Error Tag: P1008+P1012-FEAT-Decl-Constraint-Conflict

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)

```

```

4  if sc != ss or qc != qs: return 0
5  iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6  sh, inv = [s0, s1, s2, s3, s4], 0
7  for i in range(5):
8      for j in range(i+1, 5):
9          if sh[i] > sh[j]: inv += 1
10 return int(inv + iv)

```

Listing 28: Source Python shuffle_integrity_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template shuffle_integrity_logic() {
4     signal input c0, c1, c2, c3, c4;
5     signal input s0, s1, s2, s3, s4;
6     signal output out;
7
8     // Compute sum of original and shuffled
9     signal sc = c0 + c1 + c2 + c3 + c4;
10    signal ss = s0 + s1 + s2 + s3 + s4;
11
12    // Compute sum of squares of original and shuffled
13    signal qc = c0*c0 + c1*c1 + c2*c2 + c3*c3 + c4*c4;
14    signal qs = s0*s0 + s1*s1 + s2*s2 + s3*s3 + s4*s4;
15
16    // Check if sums and sum of squares are equal (permutation check)
17    signal is_permutation = (sc == ss) * (qc == qs);
18
19    // Since permutation check passed implies s0 is in original list,
20    // bonus is 100 if valid permutation, 0 otherwise
21    signal bonus = is_permutation * 100;
22
23    // Compute number of inversions in shuffled sequence
24    signal inv_count = 0;
25
26    component comp01 = GreaterThan(64);
27    comp01.in[0] <= s0;
28    comp01.in[1] <= s1;
29    signal inv01 <= comp01.out;
30
31    component comp02 = GreaterThan(64);
32    comp02.in[0] <= s0;
33    comp02.in[1] <= s2;
34    signal inv02 <= comp02.out;
35
36    component comp03 = GreaterThan(64);
37    comp03.in[0] <= s0;
38    comp03.in[1] <= s3;
39    signal inv03 <= comp03.out;
40
41    component comp04 = GreaterThan(64);
42    comp04.in[0] <= s0;
43    comp04.in[1] <= s4;
44    signal inv04 <= comp04.out;
45
46    component comp12 = GreaterThan(64);
47    comp12.in[0] <= s1;
48    comp12.in[1] <= s2;

```

```

49  signal inv12 <== comp12.out;
50
51  component comp13 = GreaterThan(64);
52  comp13.in[0] <== s1;
53  comp13.in[1] <== s3;
54  signal inv13 <== comp13.out;
55
56  component comp14 = GreaterThan(64);
57  comp14.in[0] <== s1;
58  comp14.in[1] <== s4;
59  signal inv14 <== comp14.out;
60
61  component comp23 = GreaterThan(64);
62  comp23.in[0] <== s2;
63  comp23.in[1] <== s3;
64  signal inv23 <== comp23.out;
65
66  component comp24 = GreaterThan(64);
67  comp24.in[0] <== s2;
68  comp24.in[1] <== s4;
69  signal inv24 <== comp24.out;
70
71  component comp34 = GreaterThan(64);
72  comp34.in[0] <== s3;
73  comp34.in[1] <== s4;
74  signal inv34 <== comp34.out;
75
76  signal total_inversions = inv01 + inv02 + inv03 + inv04 +
77                          inv12 + inv13 + inv14 +
78                          inv23 + inv24 + inv34;
79
80  // Final output: if permutation is valid, output = bonus + inversions
81  // If not valid, output = 0
82  out <== is_permutation * (bonus + total_inversions);
83 }
84
85 component main = shuffle_integrity_logic();

```

Listing 29: Faulty translated Circom code with Decl-Constraint Conflict (P1008 & P1012)

```

1  error[P1008]: Missing semicolon
2    "circuit.circom":11:15
3
4  11      signal sc = c0 + c1 + c2 + c3 + c4;
5          ^ A semicolon is needed here
6
7  error[P1012]: illegal expression
8    "circuit.circom":11:17
9
10 11      signal sc = c0 + c1 + c2 + c3 + c4;
11          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
12
13 error[P1008]: Missing semicolon
14    "circuit.circom":12:15
15
16 12      signal ss = s0 + s1 + s2 + s3 + s4;
17          ^ A semicolon is needed here
18

```

```

19 error[P1012]: illegal expression
20     "circuit.circom":12:17
21
22 12     signal ss = s0 + s1 + s2 + s3 + s4;
23     ~~~~~ here
24
25 error[P1008]: Missing semicolon
26     "circuit.circom":15:15
27
28 15     signal qc = c0*c0 + c1*c1 + c2*c2 + c3*c3 + c4*c4;
29     ^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32     "circuit.circom":15:17
33
34 15     signal qc = c0*c0 + c1*c1 + c2*c2 + c3*c3 + c4*c4;
35     ~~~~~ here
36
37 error[P1008]: Missing semicolon
38     "circuit.circom":16:15
39
40 16     signal qs = s0*s0 + s1*s1 + s2*s2 + s3*s3 + s4*s4;
41     ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44     "circuit.circom":16:17
45
46 16     signal qs = s0*s0 + s1*s1 + s2*s2 + s3*s3 + s4*s4;
47     ~~~~~ here
48
49 error[P1008]: Missing semicolon
50     "circuit.circom":19:27
51
52 19     signal is_permutation = (sc === ss) * (qc === qs);
53     ^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56     "circuit.circom":19:29
57
58 19     signal is_permutation = (sc === ss) * (qc === qs);
59     ~~~~~ here
60
61 error[P1008]: Missing semicolon
62     "circuit.circom":23:18
63
64 23     signal bonus = is_permutation * 100;
65     ^ A semicolon is needed here
66
67 error[P1012]: illegal expression
68     "circuit.circom":23:20
69
70 23     signal bonus = is_permutation * 100;
71     ~~~~~ here
72
73 error[P1008]: Missing semicolon
74     "circuit.circom":26:22
75
76 26     signal inv_count = 0;

```

```

77         ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80     "circuit.circom":26:24
81
82 26         signal inv_count = 0;
83             ^^ here
84
85 error[P1008]: Missing semicolon
86     "circuit.circom":78:29
87
88 78         signal total_inversions = inv01 + inv02 + inv03 + inv04 +
89             ^ A semicolon is needed here
90
91 error[P1012]: illegal expression
92     "circuit.circom":78:31
93
94 78         signal total_inversions = inv01 + inv02 + inv03 + inv04 +
95     ^
96 79                                     inv12 + inv13 + inv14 +
97 80                                     inv23 + inv24 + inv34;
98     ^ here
99
100 previous errors were found

```

Listing 30: Circom compiler error log

Explanation: In this example, lines such as ‘signal sc = ...’, ‘signal qc = ...’, ‘signal bonus = ...’, and ‘signal total_inversions = ...’ are invalid because they attempt to assign expressions at declaration time instead of declaring first and assigning afterward. The permutation check is also malformed because ‘(sc === ss)’ and ‘(qc === qs)’ are constraints, not values, so ‘signal is_permutation = (sc === ss) * (qc === qs);’ must be rewritten using separate constraints or equality components whose outputs can be multiplied.

A-11 Foreign Language Declaration (P1008)

Description: This error class usually happens when Circom code uses declaration syntax borrowed from another language, such as typed variable forms like ‘const uint ...’, which Circom does not support. The parser then misinterprets the statement and often reports a misleading "missing semicolon" error; the fix is to rewrite the declaration using valid Circom syntax, such as ‘var name = value;’, or otherwise use Circom-native constants and expressions.

Original Error Tag: P1008-FEAT-Foreign-Language-Declaration

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 31: Source Python ema_signal_logic implementation

```

1 template ema_signal_logic() {
2     signal input p0;
3     signal input p1;
4     signal input p2;
5     signal input p3;
6     signal input p4;
7     signal output out;
8     const uint alpha = 40;
9     const uint scale = 100;
10
11     // Initialization
12     signal init_ema <== p0 * scale;
13
14     // Weighted Averaging
15     signal p1_weighted <== p1 * alpha;
16     signal prev_ema_weighted <== (init_ema / scale) * (scale - alpha);
17     signal new_ema <== p1_weighted + prev_ema_weighted;
18
19     // Update EMA for subsequent prices
20     signal p2_weighted <== p2 * alpha;
21     signal prev_ema_weighted_2 <== (new_ema / scale) * (scale - alpha);
22     signal new_ema_2 <== p2_weighted + prev_ema_weighted_2;
23
24     signal p3_weighted <== p3 * alpha;
25     signal prev_ema_weighted_3 <== (new_ema_2 / scale) * (scale - alpha);
26     signal new_ema_3 <== p3_weighted + prev_ema_weighted_3;
27
28     signal p4_weighted <== p4 * alpha;
29     signal prev_ema_weighted_4 <== (new_ema_3 / scale) * (scale - alpha);
30     signal new_ema_4 <== p4_weighted + prev_ema_weighted_4;
31
32     // Output
33     out <== new_ema_4;
34 }
35 component main = ema_signal_logic();

```

Listing 32: Faulty translated Circom code with Foreign Language Declaration (P1008)

```

1 error[P1008]: Missing semicolon
2     "circuit.circom":12:15
3
4     12         const uint alpha = 40;
5             ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8     "circuit.circom":13:15
9
10    13         const uint scale = 100;
11             ^ A semicolon is needed here
12
13 previous errors were found

```

Listing 33: Circom compiler error log

Explanation: In this example, ‘const uint alpha = 40;’ and ‘const uint scale = 100;’ are invalid in Circom, so the compiler flags ‘uint’ as if a semicolon were missing before it. Replace those lines with something like ‘var alpha = 40;’ and ‘var scale = 100;’ to match Circom syntax.

A-12 Component Public Modifier Hallucination (P1008)

Description: This error class happens when a Circom component declaration uses an invalid or hallucinated ‘public’ annotation, or when ‘main’ is declared in a form the parser does not accept in the current context/version. The fix is to use a valid ‘component main = Template();’ declaration, and only add ‘{public [...]}’ if it is supported and written exactly correctly for your Circom version; also make sure you do not accidentally declare ‘main’ twice.

Original Error Tag: P1008-FEAT-Component-Public-Hallucination

Example:

```
1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)
```

Listing 34: Source Python quicksort_partition_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template quicksort_partition_logic() {
6     // Public inputs
7     signal input a0;
8     signal input a1;
9     signal input a2;
10    signal input a3;
11    signal input a4;
12    signal input a5;
13    signal input a6;
14    signal input low;
15    signal input high;
16
17    // Output
18    signal output out;
19
20    // Intermediate signals
21    signal h[7];
22    signal pivot;
23
24    // arr[t][k] corresponds to arrk_t in the pseudo-logic
25    signal arr[8][7];
26
27    // i[t] corresponds to i0..i7
28    signal i[8];
29
30    // s[t] corresponds to s0..s7
```

```

31  signal s[8];
32
33  // Per-iteration condition bits for j = 0..6
34  signal ge_low[7];
35  signal lt_high[7];
36  signal le_pivot[7];
37  signal active[7];
38  signal take[7];
39
40  // target[j][k] corresponds to target_j_k
41  signal target[7][7];
42
43  // picked[j] corresponds to picked_j
44  signal picked[7];
45
46  signal final_pos;
47
48  // Initial array state
49  arr[0][0] <== a0;
50  arr[0][1] <== a1;
51  arr[0][2] <== a2;
52  arr[0][3] <== a3;
53  arr[0][4] <== a4;
54  arr[0][5] <== a5;
55  arr[0][6] <== a6;
56
57  // High selector bits h0..h6
58  component eqHigh[7];
59  for (var k = 0; k < 7; k++) {
60      eqHigh[k] = IsEqual();
61      eqHigh[k].in[0] <== high;
62      eqHigh[k].in[1] <== k;
63      h[k] <== eqHigh[k].out;
64  }
65
66  // Pivot selection
67  pivot <== h[0] * a0 + h[1] * a1 + h[2] * a2 + h[3] * a3 + h[4] * a4 + h[5] * a5 +
h[6] * a6;
68
69  // Initial indices
70  i[0] <== low - 1;
71  s[0] <== 0;
72
73  // Comparator components
74  component geLowCmp[7];
75  component ltHighCmp[7];
76  component lePivotCmp[7];
77  component targetEqCmp[7][7];
78
79  for (var j = 0; j < 7; j++) {
80      // ge_low_j = compare(j >= low)
81      geLowCmp[j] = GreaterEqThan(64);
82      geLowCmp[j].in[0] <== j;
83      geLowCmp[j].in[1] <== low;
84      ge_low[j] <== geLowCmp[j].out;
85
86      // lt_high_j = compare(j < high)
87      ltHighCmp[j] = LessThan(64);

```

```

88     ltHighCmp[j].in[0] <= j;
89     ltHighCmp[j].in[1] <= high;
90     lt_high[j] <= ltHighCmp[j].out;
91
92     // le_pivot_j = compare(arr[j] <= pivot)
93     lePivotCmp[j] = LessEqThan(64);
94     lePivotCmp[j].in[0] <= arr[j][j];
95     lePivotCmp[j].in[1] <= pivot;
96     le_pivot[j] <= lePivotCmp[j].out;
97
98     active[j] <= ge_low[j] * lt_high[j];
99     take[j] <= active[j] * le_pivot[j];
100
101     // target_j_k = compare(i_j + 1 == k)
102     for (var k = 0; k < 7; k++) {
103         targetEqCmp[j][k] = IsEqual();
104         targetEqCmp[j][k].in[0] <= i[j] + 1;
105         targetEqCmp[j][k].in[1] <= k;
106         target[j][k] <= targetEqCmp[j][k].out;
107     }
108
109     // picked_j
110     picked[j] <=
111         target[j][0] * arr[j][0] +
112         target[j][1] * arr[j][1] +
113         target[j][2] * arr[j][2] +
114         target[j][3] * arr[j][3] +
115         target[j][4] * arr[j][4] +
116         target[j][5] * arr[j][5] +
117         target[j][6] * arr[j][6];
118
119     // Array transition arr[*]_{j+1}
120     for (var k = 0; k < 7; k++) {
121         if (k == j) {
122             arr[j + 1][k] <= take[j] * picked[j] + (1 - take[j]) * arr[j][k];
123         } else {
124             arr[j + 1][k] <=
125                 take[j] * (target[j][k] * arr[j][j] + (1 - target[j][k]) * arr[j][
126 k]) +
127                 (1 - take[j]) * arr[j][k];
128         }
129     }
130
131     // Index / swap count transitions
132     i[j + 1] <= i[j] + take[j];
133     s[j + 1] <= s[j] + take[j];
134
135     final_pos <= i[7] + 1;
136     out <= final_pos * 100 + s[7];
137 }
138
139 component main {public [a0, a1, a2, a3, a4, a5, a6, low, high]} =
    quicksort_partition_logic();

```

Listing 35: Faulty translated Circom code with Component Public Modifier Hallucination (P1008)

```

1 error[P1008]: Missing semicolon

```

```

2      "circuit.circom":139:94
3
4 139     component main {public [a0, a1, a2, a3, a4, a5, a6, low, high]} =
      quicksort_partition_logic();
5      ^
6 140
7 141     component main = quicksort_partition_logic();
8      ^ A semicolon is needed here
9
10 previous errors were found

```

Listing 36: Circom compiler error log

Explanation: In this example, the compiler rejects the ‘component main {public [a0, a1, a2, a3, a4, a5, a6, low, high]} = quicksort_partition_logic();’ line and interprets it like a malformed component declaration, so it reports a missing semicolon. The practical fix is to remove the problematic public annotation or rewrite the endpoint using the correct ‘main’ syntax for your toolchain, with only one ‘component main’ declaration.

A-13 Incomplete Statement (P1008)

Description: This error class is usually caused by Circom statements that do not follow the language’s required grammar, so the parser reaches a token where it expects the statement to have already ended with ‘;’. Common causes are using unsupported multi-line declaration syntax, writing comma-separated constructs that Circom does not allow in that context, or trying tuple/multiple assignment syntax that exists in other languages but not in Circom. To fix it, rewrite each declaration and constraint as a valid standalone Circom statement, terminate each one with a semicolon, and replace unsupported shorthand with explicit Circom-compatible code.

Original Error Tag: P1008-FEAT-Incomplete-Statement

Example:

```

1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)

```

Listing 37: Source Python quicksort_partition_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template quicksort_partition_logic() {
4     signal input a0, a1, a2, a3, a4, a5, a6, low, high;
5     signal output out;
6     signal intermediate:
7         pivot,

```

```

8      i,
9      swaps,
10     j,
11     cond,
12     swap_out,
13     arr[7],
14     arr_out[7];
15
16     component arr = Array(7);
17     arr.in <== [a0, a1, a2, a3, a4, a5, a6];
18     arr_out <== arr.out;
19
20     pivot <== a6;
21     i <== low - 1;
22     swaps <== 0;
23     j <== 0;
24
25     cond <== (j >= low) * (j < high);
26     swap_out <== cond * (arr[j] <= pivot);
27     i <== i + swap_out;
28     arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];
29     swaps <== swaps + swap_out;
30     j <== j + 1;
31
32     loop cond <== (j < 7) {
33         cond <== (j < 7);
34         arr_out <== arr;
35         arr <== arr_out;
36         cond <== (j >= low) * (j < high);
37         swap_out <== cond * (arr[j] <= pivot);
38         i <== i + swap_out;
39         arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];
40         swaps <== swaps + swap_out;
41         j <== j + 1;
42     }
43
44     arr_out[i+1], arr_out[high] <== arr_out[high], arr_out[i+1];
45     out <== (i+1)*100 + swaps;
46 }
47 component main = quicksort_partition_logic();

```

Listing 38: Faulty translated Circom code with Incomplete Statement (P1008)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":8:24
3
4 8      signal intermediate:
5         ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8   "circuit.circom":9:14
9
10 9      pivot,
11         ^ A semicolon is needed here
12
13 error[P1012]: illegal expression
14   "circuit.circom":9:9
15

```

```

16 9          pivot,
17     ~~~~~ here
18
19 error[P1008]: Missing semicolon
20 "circuit.circom":10:10
21
22 10          i,
23     ^ A semicolon is needed here
24
25 error[P1012]: illegal expression
26 "circuit.circom":10:9
27
28 10          i,
29     ^^ here
30
31 error[P1008]: Missing semicolon
32 "circuit.circom":11:14
33
34 11          swaps,
35     ^ A semicolon is needed here
36
37 error[P1012]: illegal expression
38 "circuit.circom":11:9
39
40 11          swaps,
41     ~~~~~ here
42
43 error[P1008]: Missing semicolon
44 "circuit.circom":12:10
45
46 12          j,
47     ^ A semicolon is needed here
48
49 error[P1012]: illegal expression
50 "circuit.circom":12:9
51
52 12          j,
53     ^^ here
54
55 error[P1008]: Missing semicolon
56 "circuit.circom":13:13
57
58 13          cond,
59     ^ A semicolon is needed here
60
61 error[P1012]: illegal expression
62 "circuit.circom":13:9
63
64 13          cond,
65     ~~~~~ here
66
67 error[P1008]: Missing semicolon
68 "circuit.circom":14:17
69
70 14          swap_out,
71     ^ A semicolon is needed here
72
73 error[P1012]: illegal expression

```

```

74 "circuit.circom":14:9
75
76 14      swap_out,
77      ~~~~~ here
78
79 error[P1008]: Missing semicolon
80 "circuit.circom":15:15
81
82 15      arr[7],
83      ~ A semicolon is needed here
84
85 error[P1012]: illegal expression
86 "circuit.circom":15:9
87
88 15      arr[7],
89      ~~~~~ here
90
91 error[P1012]: illegal expression
92 "circuit.circom":16:9
93
94 16      arr_out[7];
95      ~~~~~ here
96
97 error[P1008]: Missing semicolon
98 "circuit.circom":30:15
99
100 30      arr_out[i], arr_out[j] <= arr_out[j], arr_out[i];
101      ~ A semicolon is needed here
102
103 error[P1012]: illegal expression
104 "circuit.circom":30:5
105
106 30      arr_out[i], arr_out[j] <= arr_out[j], arr_out[i];
107      ~~~~~ here
108
109 error[P1008]: Missing semicolon
110 "circuit.circom":30:42
111
112 30      arr_out[i], arr_out[j] <= arr_out[j], arr_out[i];
113      ~ A semicolon is needed here
114
115 error[P1012]: illegal expression
116 "circuit.circom":30:44
117
118 30      arr_out[i], arr_out[j] <= arr_out[j], arr_out[i];
119      ~~~~~ here
120
121 error[P1008]: Missing semicolon
122 "circuit.circom":34:26
123
124 34      loop cond <= (j < 7) {
125      ~ A semicolon is needed here
126
127 error[P1008]: Missing semicolon
128 "circuit.circom":41:19
129
130 41      arr_out[i], arr_out[j] <= arr_out[j], arr_out[i];
131      ~ A semicolon is needed here

```

```

132
133 error[P1012]: illegal expression
134     "circuit.circom":41:9
135
136 41         arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];
137         ~~~~~ here
138
139 error[P1008]: Missing semicolon
140     "circuit.circom":41:46
141
142 41         arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];
143                                     ^ A semicolon is needed here
144
145 error[P1012]: illegal expression
146     "circuit.circom":41:48
147
148 41         arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];
149                                     ~~~~~ here
150
151 error[P1008]: Missing semicolon
152     "circuit.circom":46:17
153
154 46         arr_out[i+1], arr_out[high] <== arr_out[high], arr_out[i+1];
155                                     ^ A semicolon is needed here
156
157 error[P1012]: illegal expression
158     "circuit.circom":46:5
159
160 46         arr_out[i+1], arr_out[high] <== arr_out[high], arr_out[i+1];
161         ~~~~~ here
162
163 error[P1008]: Missing semicolon
164     "circuit.circom":46:50
165
166 46         arr_out[i+1], arr_out[high] <== arr_out[high], arr_out[i+1];
167                                     ^ A semicolon is needed here
168
169 error[P1012]: illegal expression
170     "circuit.circom":46:52
171
172 46         arr_out[i+1], arr_out[high] <== arr_out[high], arr_out[i+1];
173                                     ~~~~~ here
174
175 previous errors were found

```

Listing 39: Circom compiler error log

Explanation: In this example, ‘signal intermediate:’ followed by a comma-separated list is not valid Circom syntax, so the compiler keeps reporting missing semicolons and illegal expressions on the following lines. The swap lines like ‘arr_out[i], arr_out[j] <== arr_out[j], arr_out[i];’ and the ‘loop cond <== ... {‘ form are also invalid in Circom, because Circom does not support tuple assignment or that loop syntax; they must be rewritten using valid per-signal assignments and proper control structure syntax.

A-14 Invalid Type Declaration (P1008 & P1012)

Description: This error class happens when a declaration uses syntax that is not valid for the language grammar, so the parser misreads the statement and then reports follow-up errors like "missing semicolon" and "illegal expression." In Circom, signal declarations must use supported forms such as 'signal input', 'signal output', or plain internal 'signal'; modifiers like 'private' are not valid in that position. To fix this, rewrite the declaration using a legal signal form and, if privacy is intended, remember that non-input internal signals are already not public outputs.

Original Error Tag: P1008+P1012-FEAT-Invalid-Type-Declaration

Example:

```
1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)
```

Listing 40: Source Python ema_signal_logic implementation

```
1 template ema_signal_logic() {
2     signal input p0;
3     signal input p1;
4     signal input p2;
5     signal input p3;
6     signal input p4;
7     signal output ema;
8
9     signal private alpha;
10    signal private ema_0;
11    signal private ema_1;
12    signal private ema_2;
13    signal private ema_3;
14    signal private ema_4;
15
16    signal alpha_scaled;
17    signal ema_0_scaled;
18    signal ema_1_scaled;
19    signal ema_2_scaled;
20    signal ema_3_scaled;
21    signal ema_4_scaled;
22
23    alpha <== 40;
24    alpha_scaled <== alpha / 100;
25
26    ema_0 <== p0 * 100;
27    ema_0_scaled <== ema_0;
28
29    ema_1 <== (p1 * alpha_scaled) + (ema_0_scaled * (1 - alpha_scaled));
30    ema_1_scaled <== ema_1;
31
32    ema_2 <== (p2 * alpha_scaled) + (ema_1_scaled * (1 - alpha_scaled));
33    ema_2_scaled <== ema_2;
34
35    ema_3 <== (p3 * alpha_scaled) + (ema_2_scaled * (1 - alpha_scaled));
```

```

36     ema_3_scaled <== ema_3;
37
38     ema_4 <== (p4 * alpha_scaled) + (ema_3_scaled * (1 - alpha_scaled));
39     ema_4_scaled <== ema_4;
40
41     ema <== ema_4;
42 }
43
44 component main = ema_signal_logic();

```

Listing 41: Faulty translated Circom code with Invalid Type Declaration (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":13:19
3
4 13     signal private alpha;
5         ^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8   "circuit.circom":13:20
9
10 13     signal private alpha;
11         ^^^^^^ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":14:19
15
16 14     signal private ema_0;
17         ^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":14:20
21
22 14     signal private ema_0;
23         ^^^^^^ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":15:19
27
28 15     signal private ema_1;
29         ^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":15:20
33
34 15     signal private ema_1;
35         ^^^^^^ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":16:19
39
40 16     signal private ema_2;
41         ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44   "circuit.circom":16:20
45
46 16     signal private ema_2;

```

```

47         ~~~~~ here
48
49 error[P1008]: Missing semicolon
50 "circuit.circom":17:19
51
52 17      signal private ema_3;
53         ~ A semicolon is needed here
54
55 error[P1012]: illegal expression
56 "circuit.circom":17:20
57
58 17      signal private ema_3;
59         ~~~~~ here
60
61 error[P1008]: Missing semicolon
62 "circuit.circom":18:19
63
64 18      signal private ema_4;
65         ~ A semicolon is needed here
66
67 error[P1012]: illegal expression
68 "circuit.circom":18:20
69
70 18      signal private ema_4;
71         ~~~~~ here
72
73 previous errors were found

```

Listing 42: Circom compiler error log

Explanation: In this example, lines like ‘signal private alpha;’ and ‘signal private ema_0;’ are invalid because ‘private’ is not allowed in a Circom signal declaration. The compiler expects the declaration to end after ‘signal’, so it treats ‘private’ as unexpected text, which leads to the semicolon and illegal-expression errors on each of those lines.

A-15 Declaration-Initialization Fusion (P1008 & P1012)

Description: This error class happens when Circom code tries to fuse a ‘signal’ declaration with an initialization expression, such as ‘signal x = ...;’. In Circom, ‘signal’ declarations and value assignment/constraining must be written as separate statements, so the parser reports a missing semicolon at the declaration point and then flags the right-hand side as an illegal expression. To fix it, declare the signal first and then assign or constrain it on a new line with the proper operator, or use ‘var’ instead if the value is just a local compile-time variable.

Original Error Tag: P1008+P1012-FEAT-Decl-Init-Fusion

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 43: Source Python ema_signal_logic implementation

```

1 template EmaSignalLogic() {
2     signal input p0;
3     signal input p1;
4     signal input p2;
5     signal input p3;
6     signal input p4;
7     signal output out;
8
9     signal alpha = 40;
10    signal ema = p0 * 100;
11
12    signal ema1 = (p1 * alpha) + (ema * (100 - alpha));
13    ema1 /= 100;
14
15    signal ema2 = (p2 * alpha) + (ema1 * (100 - alpha));
16    ema2 /= 100;
17
18    signal ema3 = (p3 * alpha) + (ema2 * (100 - alpha));
19    ema3 /= 100;
20
21    signal ema4 = (p4 * alpha) + (ema3 * (100 - alpha));
22    ema4 /= 100;
23
24    out <== ema4;
25 }
26
27 component main = EmaSignalLogic();

```

Listing 44: Faulty translated Circom code with Declaration-Initialization Fusion (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2     "circuit.circom":13:18
3
4 13     signal alpha = 40;
5         ^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8     "circuit.circom":13:20
9
10 13     signal alpha = 40;
11         ^^^ here
12
13 error[P1008]: Missing semicolon
14     "circuit.circom":14:16
15
16 14     signal ema = p0 * 100;
17         ^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20     "circuit.circom":14:18
21
22 14     signal ema = p0 * 100;
23         ^^^^^^^^^ here
24
25 error[P1008]: Missing semicolon
26     "circuit.circom":16:17
27

```

```

28 16      signal ema1 = (p1 * alpha) + (ema * (100 - alpha));
29      ^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":16:19
33
34 16      signal ema1 = (p1 * alpha) + (ema * (100 - alpha));
35      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":19:17
39
40 19      signal ema2 = (p2 * alpha) + (ema1 * (100 - alpha));
41      ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44   "circuit.circom":19:19
45
46 19      signal ema2 = (p2 * alpha) + (ema1 * (100 - alpha));
47      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
48
49 error[P1008]: Missing semicolon
50   "circuit.circom":22:17
51
52 22      signal ema3 = (p3 * alpha) + (ema2 * (100 - alpha));
53      ^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56   "circuit.circom":22:19
57
58 22      signal ema3 = (p3 * alpha) + (ema2 * (100 - alpha));
59      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
60
61 error[P1008]: Missing semicolon
62   "circuit.circom":25:17
63
64 25      signal ema4 = (p4 * alpha) + (ema3 * (100 - alpha));
65      ^ A semicolon is needed here
66
67 error[P1012]: illegal expression
68   "circuit.circom":25:19
69
70 25      signal ema4 = (p4 * alpha) + (ema3 * (100 - alpha));
71      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
72
73 previous errors were found

```

Listing 45: Circom compiler error log

Explanation: In this example, lines like ‘signal alpha = 40;’ and ‘signal ema1 = ...;’ are invalid because ‘signal’ cannot be initialized inline. ‘alpha’ should likely be a ‘var alpha = 40;’, while ‘ema’, ‘ema1’, ‘ema2’, ‘ema3’, and ‘ema4’ should be declared first and then set in separate statements such as ‘signal ema1;’ followed by ‘ema1 <== ...;’.

A-16 Floating-Point Hallucination (P1008 & P1012)

Description: This error class usually happens when Circom code uses unsupported language features or syntax that look valid in general programming languages but are not valid in Circom, especially floating-point literals like '0.5'. Circom works over finite fields and expects integer constants or explicitly encoded fixed-point values, so the parser gets confused by the decimal point and reports syntax errors such as a missing semicolon and illegal expression. To fix it, replace floating-point numbers with integer or fixed-point representations and adjust the surrounding logic accordingly.

Original Error Tag: P1008+P1012-FEAT-Floating-Point-Hallucination

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 46: Source Python aave_v3_interest_logic implementation

```
1 // Import necessary libraries
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/arithmetic/arithmetic.circom";
4
5 // Define the aave_v3_interest_logic template
6 template aave_v3_interest_logic() {
7     // Input signals
8     signal input d; // Debt amount
9     signal input r; // Reserve factor
10    signal input b; // Base interest rate
11    signal input s1; // Interest rate multiplier 1
12    signal input s2; // Interest rate multiplier 2
13
14    // Output signal
15    signal output out;
16
17    // Check if reserve factor is zero
18    component is_zero = IsZero(64);
19    is_zero.in <== r;
20
21    // Calculate utilization ratio
22    component u = Div(64);
23    u.in[0] <== d;
24    u.in[1] <== r;
25
26    // Determine interest rate based on utilization ratio
27    component ir = Mul(64);
28    ir.in[0] <== u.out;
29    ir.in[1] <== s1;
30}
```

```

31 component ir2 = Mul(64);
32 ir2.in[0] <== u.out;
33 ir2.in[1] <== s2;
34
35 component ir_selector = GreaterEqThan(64);
36 ir_selector.in[0] <== u.out;
37 ir_selector.in[1] <== 0.5;
38
39 component ir_final = Mux(64);
40 ir_final.in[0] <== ir.out;
41 ir_final.in[1] <== ir2.out;
42 ir_final.sel <== ir_selector.out;
43
44 // Calculate interest accrued
45 component br = Mul(64);
46 br.in[0] <== ir_final.out;
47 br.in[1] <== b;
48
49 // Calculate supply rate
50 component sr = Mul(64);
51 sr.in[0] <== br.out;
52 sr.in[1] <== u.out;
53
54 // Calculate reserve premium
55 component rp = Mul(64);
56 rp.in[0] <== u.out;
57 rp.in[1] <== s1;
58
59 // Compute total interest
60 component res = Add(64);
61 res.in[0] <== br.out;
62 res.in[1] <== sr.out;
63 res.in[2] <== rp.out;
64
65 // Cap total interest at 1,000,000
66 component cap = GreaterEqThan(64);
67 cap.in[0] <== res.out;
68 cap.in[1] <== 1000000;
69
70 component capped_res = Mux(64);
71 capped_res.in[0] <== res.out;
72 capped_res.in[1] <== 1000000;
73 capped_res.sel <== cap.out;
74
75 // Output the total interest
76 out <== is_zero.out ? b : capped_res.out;
77 }
78
79 // Create an instance of the aave_v3_interest_logic template
80 component main = aave_v3_interest_logic();

```

Listing 47: Faulty translated Circom code with Floating-Point Hallucination (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":41:28
3
4 41     ir_selector.in[1] <== 0.5;
5                                ^ A semicolon is needed here

```

```

6
7 error[P1012]: illegal expression
8   "circuit.circom":41:29
9
10 41      ir_selector.in[1] <== 0.5;
11                                ^^ here
12
13 previous errors were found

```

Listing 48: Circom compiler error log

Explanation: In this example, ‘`ir_selector.in[1] <== 0.5;`’ is the problem: Circom does not accept ‘0.5’ as a valid constant, so it flags the ‘`<==`’ as an illegal expression and then also reports a missing semicolon. Use an integer threshold instead, or represent ‘0.5’ via fixed-point scaling, such as comparing scaled values rather than using a decimal literal.

A-17 Component Block Declaration Hallucination (P1008)

Description: This error class usually happens when a Circom component declaration uses invalid or unsupported syntax, especially around the ‘main’ component and ‘{public [...]}’ blocks, so the parser gets confused and reports a misleading “missing semicolon” error. To fix it, use the exact component declaration syntax supported by your Circom version, avoid inventing new declaration forms, and make sure there is only one valid ‘component main = ...;’ declaration at the end of the file.

Original Error Tag: P1008-FEAT-Component-Block-Decl-Hallucination

Example:

```

1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000

```

Listing 49: Source Python aave_v3_interest_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template aave_v3_interest_logic() {
4     signal input d;
5     signal input r;
6     signal input b;
7     signal input s1;
8     signal input s2;
9     signal input opt;
10
11     signal output out;
12
13     // Check if r == 0
14     component isRZero = IsZero();

```



```

15  isRZero.in <== r;
16  signal br;
17  signal u;
18
19  // Compute u = (d * 10000) / r, but only if r != 0
20  // We'll use conditional logic via multiplexers based on isRZero.out
21
22  // First compute d_times_10000 = d * 10000
23  signal d_times_10000;
24  d_times_10000 <== d * 10000;
25
26  // Safe division: if r == 0, we don't care about u because we return b directly.
27  // But Circom requires all signals to be defined, so we define u conditionally.
28  // We'll compute u_div = d_times_10000 / r using IntegerDivide
29  component divU = IntegerDivide(64);
30  divU.dividend <== d_times_10000;
31  divU.divisor <== r + isRZero.out; // avoid division by zero; when r=0, divisor=1,
    but result unused
32  u <== divU.quotient;
33
34  // Now compute br
35  // Condition: u <= opt ?
36  component cmp_u_opt = LessEqThan(64);
37  cmp_u_opt.in[0] <== u;
38  cmp_u_opt.in[1] <== opt;
39
40  // Compute term1 = (u * s1) / opt
41  signal term1;
42  component divTerm1 = IntegerDivide(64);
43  divTerm1.dividend <== u * s1;
44  divTerm1.divisor <== opt + (1 - cmp_u_opt.out); // avoid div by zero; if not used,
    add 1
45  term1 <== divTerm1.quotient;
46
47  // Compute branch1: b + term1
48  signal branch1;
49  branch1 <== b + term1;
50
51  // Compute term2a = u - opt
52  signal term2a;
53  term2a <== u - opt;
54
55  // Compute term2b = (term2a * s2) / (10000 - opt)
56  signal denom2;
57  denom2 <== 10000 - opt;
58  signal term2b;
59  component divTerm2b = IntegerDivide(64);
60  divTerm2b.dividend <== term2a * s2;
61  divTerm2b.divisor <== denom2 + cmp_u_opt.out; // if u<=opt, this branch unused;
    avoid div by zero
62  term2b <== divTerm2b.quotient;
63
64  // Compute branch2: b + s1 + term2b
65  signal branch2;
66  branch2 <== b + s1 + term2b;
67
68  // Select br = cmp_u_opt.out ? branch1 : branch2
69  br <== cmp_u_opt.out * branch1 + (1 - cmp_u_opt.out) * branch2;

```

```

70
71 // Compute sr = ((br * u * 9000) / 10000) / 10000
72 signal temp_sr1;
73 temp_sr1 <= br * u * 9000;
74
75 signal temp_sr2;
76 component divSr1 = IntegerDivide(64);
77 divSr1.dividend <= temp_sr1;
78 divSr1.divisor <= 10000;
79 temp_sr2 <= divSr1.quotient;
80
81 signal sr;
82 component divSr2 = IntegerDivide(64);
83 divSr2.dividend <= temp_sr2;
84 divSr2.divisor <= 10000;
85 sr <= divSr2.quotient;
86
87 // Compute rp
88 signal rp;
89
90 // Check u > 9500
91 component cmp_u_9500 = GreaterThan(64);
92 cmp_u_9500.in[0] <= u;
93 cmp_u_9500.in[1] <= 9500;
94
95 // Check u > 8000
96 component cmp_u_8000 = GreaterThan(64);
97 cmp_u_8000.in[0] <= u;
98 cmp_u_8000.in[1] <= 8000;
99
100 // Compute rp_high = (u - 9500) * 100
101 signal rp_high;
102 rp_high <= (u - 9500) * 100;
103
104 // Compute rp_mid = (u - 8000) * 20
105 signal rp_mid;
106 rp_mid <= (u - 8000) * 20;
107
108 // rp =
109 //   if u > 9500: rp_high
110 //   elif u > 8000: rp_mid
111 //   else: 0
112 signal cond_high;
113 cond_high <= cmp_u_9500.out;
114
115 signal cond_mid;
116 // u > 8000 and not (u > 9500) => u in (8000, 9500]
117 cond_mid <= cmp_u_8000.out * (1 - cmp_u_9500.out);
118
119 rp <= cond_high * rp_high + cond_mid * rp_mid;
120
121 // res = br + sr + rp
122 signal res_unclamped;
123 res_unclamped <= br + sr + rp;
124
125 // Clamp to 1000000
126 component cmp_res_limit = LessThan(64);
127 cmp_res_limit.in[0] <= res_unclamped;

```

```

128     cmp_res_limit.in[1] <= 1000000;
129
130     out <= cmp_res_limit.out * res_unclamped + (1 - cmp_res_limit.out) * 1000000;
131
132     // If r == 0, output should be b
133     // So override out if r == 0
134     out <= isRZero.out * b + (1 - isRZero.out) * out;
135 }
136
137 // Helper templates
138
139 template IsZero() {
140     signal input in;
141     signal output out;
142     component cmp = EqualsZero(64);
143     cmp.in <= in;
144     out <= cmp.out;
145 }
146
147 template EqualsZero(n) {
148     signal input in;
149     signal output out;
150     component lt = LessThan(n);
151     lt.in[0] <= in;
152     lt.in[1] <= 1;
153     out <= lt.out;
154 }
155
156 template IntegerDivide(n) {
157     signal input dividend;
158     signal input divisor;
159     signal output quotient;
160     signal output remainder;
161
162     quotient <-- dividend / divisor;
163     remainder <-- dividend % divisor;
164
165     dividend == quotient * divisor + remainder;
166     // Constrain remainder to be non-negative and less than divisor
167     // For simplicity, assume inputs are non-negative and divisor > 0
168     component lt = LessThan(n);
169     lt.in[0] <= remainder;
170     lt.in[1] <= divisor;
171     lt.out == 1;
172
173     component ge = GreaterEqThan(n);
174     ge.in[0] <= remainder;
175     ge.in[1] <= 0;
176     ge.out == 1;
177 }
178
179 component main {public [d, r, b, s1, s2, opt]} = aave_v3_interest_logic();

```

Listing 50: Faulty translated Circom code with Component Block Declaration Hallucination (P1008)

```

1 error[P1008]: Missing semicolon
2     "circuit.circom":181:74
3

```

```

4 181     component main {public [d, r, b, s1, s2, opt]} = aave_v3_interest_logic();
5     ^
6 182
7 183     component main = aave_v3_interest_logic();
8     ^ A semicolon is needed here
9
10 previous errors were found

```

Listing 51: Circom compiler error log

Explanation: In this example, the file declares ‘main’ twice, and the first declaration uses a ‘main {public [...]}’ form that the compiler is not parsing correctly in this context. Because of that, the parser reaches the next ‘component main = aave_v3_interest_logic();’ line and reports it as if the previous statement had never properly ended.

A-18 Python-Style Division (P1008 & P1012)

Description: This error class usually happens when Circom code uses syntax from another language that the Circom parser does not support, especially Python-style floor division (‘//’). Once the parser encounters an unsupported token, it often reports follow-up errors like "missing semicolon" and "illegal expression" on the same or nearby lines. To fix it, replace unsupported operators with valid Circom arithmetic patterns, and make sure declarations and assignments use proper Circom syntax.

Original Error Tag: P1008+P1012-FEAT-Python-Style-Division

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 52: Source Python ema_signal_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2 template ema_signal_logic() {
3     signal input p0;
4     signal input p1;
5     signal input p2;
6     signal input p3;
7     signal input p4;
8     signal output out;
9     constant alpha = 40;
10    signal ema = p0 * 100;
11    component gte1 = GreaterEqThan(64);
12    gte1.in[0] <== p0;
13    gte1.in[1] <== p1;
14    signal sel1 <== gte1.out;
15    ema <== sel1 * (p1 * alpha) + (ema * (100 - alpha) // 100);
16    component gte2 = GreaterEqThan(64);
17    gte2.in[0] <== p1;
18    gte2.in[1] <== p2;
19    signal sel2 <== gte2.out;
20    ema <== sel2 * (p2 * alpha) + (ema * (100 - alpha) // 100);

```

```

21     component gte3 = GreaterEqThan(64);
22     gte3.in[0] <== p2;
23     gte3.in[1] <== p3;
24     signal sel3 <== gte3.out;
25     ema <== sel3 * (p3 * alpha) + (ema * (100 - alpha) // 100);
26     component gte4 = GreaterEqThan(64);
27     gte4.in[0] <== p3;
28     gte4.in[1] <== p4;
29     signal sel4 <== gte4.out;
30     ema <== sel4 * (p4 * alpha) + (ema * (100 - alpha) // 100);
31     out <== int(ema);
32 }
33 component main = ema_signal_logic();

```

Listing 53: Faulty translated Circom code with Python-Style Division (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":12:20
3
4   12         constant alpha = 40;
5             ^ A semicolon is needed here
6
7 error[P1012]: illegal expression
8   "circuit.circom":12:22
9
10  12         constant alpha = 40;
11             ^^^ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":13:16
15
16  13         signal ema = p0 * 100;
17             ^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":13:18
21
22  13         signal ema = p0 * 100;
23             ^^^^^^^^^ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":18:9
27
28  18         ema <== sel1 * (p1 * alpha) + (ema * (100 - alpha) // 100);
29             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":18:5
33
34  18         ema <== sel1 * (p1 * alpha) + (ema * (100 - alpha) // 100);
35             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":23:9
39
40  23         ema <== sel2 * (p2 * alpha) + (ema * (100 - alpha) // 100);
41             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ A semicolon is needed here
42

```

```

43 error[P1012]: illegal expression
44     "circuit.circom":23:5
45
46 23     ema <== sel2 * (p2 * alpha) + (ema * (100 - alpha) // 100);
47     ~~~~~ here
48
49 error[P1008]: Missing semicolon
50     "circuit.circom":28:9
51
52 28     ema <== sel3 * (p3 * alpha) + (ema * (100 - alpha) // 100);
53     ~~~~~ A semicolon is needed here
54
55 error[P1012]: illegal expression
56     "circuit.circom":28:5
57
58 28     ema <== sel3 * (p3 * alpha) + (ema * (100 - alpha) // 100);
59     ~~~~~ here
60
61 error[P1008]: Missing semicolon
62     "circuit.circom":33:9
63
64 33     ema <== sel4 * (p4 * alpha) + (ema * (100 - alpha) // 100);
65     ~~~~~ A semicolon is needed here
66
67 error[P1012]: illegal expression
68     "circuit.circom":33:5
69
70 33     ema <== sel4 * (p4 * alpha) + (ema * (100 - alpha) // 100);
71     ~~~~~ here
72
73 previous errors were found

```

Listing 54: Circom compiler error log

Explanation: In this example, the parser is choking on ‘// 100’ inside the EMA update expressions, so the surrounding ‘<==’ statements get misread and trigger the P1008/P1012 errors. The code also uses non-Circom-style declarations like ‘signal ema = ...’, which should be split into a declaration and a separate assignment/constraint.

A-19 Type-Casting Hallucination (P1008 & P1012)

Description: This error class happens when code uses type-casting or runtime-style expressions that Circom does not support, often borrowed from languages like C/C++/Java. Circom has no ‘(int)’ or ‘(long)’ casts, and many values such as array indices, loop bounds, and structural control flow must be compile-time constants or expressed through valid constraint logic instead. The fix is to remove fake casts, rewrite the code using plain Circom expressions, and replace dynamic indexing or imperative updates with static wiring, selector logic, mux patterns, or other constraint-friendly constructions.

Original Error Tag: P1008+P1012-FEAT-Type-Casting-Hallucination

Example:

```

1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0

```

```

6   for j in range(7):
7       if j >= low and j < high:
8           if arr[j] <= pivot:
9               i += 1
10              arr[i], arr[j] = arr[j], arr[i]
11              swaps += 1
12      arr[i + 1], arr[high] = arr[high], arr[i + 1]
13      return int((i + 1) * 100 + swaps)

```

Listing 55: Source Python quicksort_partition_logic implementation

```

1  include "node_modules/circomlib/circuits/comparators.circom";
2
3  template quicksort_partition_logic() {
4      signal input a0, a1, a2, a3, a4, a5, a6, low, high;
5      signal output out;
6      component arr = Array(7);
7      arr.in <== [a0, a1, a2, a3, a4, a5, a6];
8      signal pivot <== arr.in[(int)high];
9      signal i <== (int)low - 1;
10     signal swaps <== 0;
11     signal j <== 0;
12     signal cond;
13     signal arr_idx <== j;
14     signal tmp;
15     signal tmp2;
16     while (j < 7) {
17         cond <== (j >= (int)low) * (j < (int)high);
18         if (cond == 1) {
19             if (arr.in[arr_idx] <= pivot) {
20                 i <== i + 1;
21                 tmp <== arr.in[i];
22                 arr.in[i] <== arr.in[arr_idx];
23                 arr.in[arr_idx] <== tmp;
24                 swaps <== swaps + 1;
25             }
26         }
27         j <== j + 1;
28         arr_idx <== j;
29     }
30     tmp2 <== arr.in[i+1];
31     arr.in[i+1] <== arr.in[(int)high];
32     arr.in[(int)high] <== tmp2;
33     out <== (long)(i+1)*100 + swaps;
34 }
35 component main = quicksort_partition_logic();

```

Listing 56: Faulty translated Circom code with Type-Casting Hallucination (P1008 & P1012)

```

1  error[P1012]: illegal expression
2      "circuit.circom":10:5
3
4  10      signal pivot <== arr.in[(int)high];
5      ~~~~~ here
6
7  error[P1008]: Missing semicolon
8      "circuit.circom":11:23

```

```

9
10 11      signal i <== (int)low - 1;
11      ^ A semicolon is needed here
12
13 error[P1012]: illegal expression
14   "circuit.circom":11:23
15
16 11      signal i <== (int)low - 1;
17      ^^^^^^^^^ here
18
19 error[P1008]: Missing semicolon
20   "circuit.circom":19:14
21
22 19      cond <== (j >= (int)low) * (j < (int)high);
23      ^^^^^^^^^^^^^^^^^ A semicolon is needed here
24
25 error[P1012]: illegal expression
26   "circuit.circom":19:9
27
28 19      cond <== (j >= (int)low) * (j < (int)high);
29      ^^^^^^^^^^^^^^^^^ here
30
31 error[P1008]: Missing semicolon
32   "circuit.circom":19:32
33
34 19      cond <== (j >= (int)low) * (j < (int)high);
35      ^^^ A semicolon is needed here
36
37 error[P1012]: illegal expression
38   "circuit.circom":19:29
39
40 19      cond <== (j >= (int)low) * (j < (int)high);
41      ^^^^^^ here
42
43 error[P1012]: illegal expression
44   "circuit.circom":19:36
45
46 19      cond <== (j >= (int)low) * (j < (int)high);
47      ^^^^^^^^^^^^^^^^^ here
48
49 error[P1008]: Missing semicolon
50   "circuit.circom":33:17
51
52 33      arr.in[i+1] <== arr.in[(int)high];
53      ^^^^^^^^^^^^^^^^^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56   "circuit.circom":33:5
57
58 33      arr.in[i+1] <== arr.in[(int)high];
59      ^^^^^^^^^^^^^^^^^ here
60
61 error[P1012]: illegal expression
62   "circuit.circom":33:33
63
64 33      arr.in[i+1] <== arr.in[(int)high];
65      ^^^^^^ here
66

```



```

67 error[P1012]: illegal expression
68     "circuit.circom":34:5
69
70 34     arr.in[(int)high] <== tmp2;
71     ~~~~~ here
72
73 error[P1008]: Missing semicolon
74     "circuit.circom":35:19
75
76 35     out <== (long)(i+1)*100 + swaps;
77         ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80     "circuit.circom":35:19
81
82 35     out <== (long)(i+1)*100 + swaps;
83         ~~~~~ here
84
85 previous errors were found

```

Listing 57: Circom compiler error log

Explanation: In this example, ‘(int)high’, ‘(int)low’, and ‘(long)(i+1)’ are invalid Circom syntax, so the parser reports ‘illegal expression’ and then cascades into misleading ‘Missing semicolon’ errors. The code also tries to index ‘arr.in[...]’ with signal values like ‘high’ and ‘i’, which is not valid in this form in Circom and must be rewritten using static indices or conditional selection logic.

A-20 Invalid Assignment Operator (P1008 & P1012)

Description: This error class happens when Circom syntax mixes up declaration/instantiation with constraint assignment, or uses expressions that are not legal on the right-hand side of ‘<==’. In Circom, ‘component’ instances must be created with ‘=’, while ‘<==’ is only for assigning/constraining signals, and parser confusion often shows up as a "missing semicolon" followed by "illegal expression." To fix it, use the proper operator for each construct, and replace unsupported inline expressions-such as ad hoc indexed list literals-with valid Circom patterns like declared arrays, intermediate variables, or explicit conditional selection.

Original Error Tag: P1008+P1012-FEAT-Invalid-Assignment-Operator

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)

```

Listing 58: Source Python shuffle_integrity_logic implementation

```

1 template ShuffleIntegrityLogic() {
2     signal input c0;

```

```

3  signal input c1;
4  signal input c2;
5  signal input c3;
6  signal input c4;
7  signal input s0;
8  signal input s1;
9  signal input s2;
10 signal input s3;
11 signal input s4;
12 signal input sd;
13
14 // Check if sum of c values equals sum of s values
15 component eqSum = Equal(64);
16 eqSum.in[0] <== c0 + c1 + c2 + c3 + c4;
17 eqSum.in[1] <== s0 + s1 + s2 + s3 + s4;
18 signal sumEq <== eqSum.out;
19
20 // Check if sum of squares of c values equals sum of squares of s values
21 component eqSquares = Equal(64);
22 eqSquares.in[0] <== c0 * c0 + c1 * c1 + c2 * c2 + c3 * c3 + c4 * c4;
23 eqSquares.in[1] <== s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3 + s4 * s4;
24 signal squaresEq <== eqSquares.out;
25
26 // Check if any of the c values match s0
27 signal iv <== 0;
28 for (var i = 0; i < 5; i++) {
29     component eqC <== Equal(64);
30     eqC.in[0] <== [c0, c1, c2, c3, c4][i];
31     eqC.in[1] <== s0;
32     signal eq <== eqC.out;
33     iv <== iv + eq * 100;
34 }
35
36 // Calculate the number of inversions in the s array
37 signal inv <== 0;
38 for (var i = 0; i < 5; i++) {
39     for (var j = i + 1; j < 5; j++) {
40         component gt <== GreaterThan(64);
41         gt.in[0] <== [s0, s1, s2, s3, s4][i];
42         gt.in[1] <== [s0, s1, s2, s3, s4][j];
43         signal gtOut <== gt.out;
44         inv <== inv + gtOut;
45     }
46 }
47
48 // Return the sum of inversions and iv
49 signal output out <== (sumEq * squaresEq) * (inv + iv);
50 }
51
52 component main = ShuffleIntegrityLogic();

```

Listing 59: Faulty translated Circom code with Invalid Assignment Operator (P1008 & P1012)

```

1  error[P1008]: Missing semicolon
2  "circuit.circom":33:23
3
4  33          component eqC <== Equal(64);
5              ^^^ A semicolon is needed here

```

```

6
7 error[P1012]: illegal expression
8   "circuit.circom":33:27
9
10 33          component eqC <== Equal(64);
11                                ~~~~~~ here
12
13 error[P1008]: Missing semicolon
14   "circuit.circom":34:43
15
16 34          eqC.in[0] <== [c0, c1, c2, c3, c4][i];
17                                ^ A semicolon is needed here
18
19 error[P1012]: illegal expression
20   "circuit.circom":34:43
21
22 34          eqC.in[0] <== [c0, c1, c2, c3, c4][i];
23                                ~~~~ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":44:26
27
28 44          component gt <== GreaterThan(64);
29                                ~~~ A semicolon is needed here
30
31 error[P1012]: illegal expression
32   "circuit.circom":44:30
33
34 44          component gt <== GreaterThan(64);
35                                ~~~~~~ here
36
37 error[P1008]: Missing semicolon
38   "circuit.circom":45:46
39
40 45          gt.in[0] <== [s0, s1, s2, s3, s4][i];
41                                ^ A semicolon is needed here
42
43 error[P1012]: illegal expression
44   "circuit.circom":45:46
45
46 45          gt.in[0] <== [s0, s1, s2, s3, s4][i];
47                                ~~~~ here
48
49 error[P1008]: Missing semicolon
50   "circuit.circom":46:46
51
52 46          gt.in[1] <== [s0, s1, s2, s3, s4][j];
53                                ^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56   "circuit.circom":46:46
57
58 46          gt.in[1] <== [s0, s1, s2, s3, s4][j];
59                                ~~~~ here
60
61 previous errors were found

```

Listing 60: Circom compiler error log

Explanation: In this example, ‘component eqC <== Equal(64);’ and ‘component gt <== GreaterThan(64);’ are invalid because components must be instantiated with ‘=’ instead of ‘<==’. The expressions ‘[c0, c1, c2, c3, c4][i]’ and ‘[s0, s1, s2, s3, s4][i/j]’ are also being rejected as illegal, so those values need to be accessed through a proper Circom array or another valid selection mechanism.

A-21 Bitwise Operation Hallucination (P1008)

Description: This error class happens when code uses syntax or operators that do not exist in Circom, often because it was written as if Circom supported higher-level typed declarations or normal programming-language boolean/conditional expressions. Common causes include invented types like ‘long’, inline signal initialization in declarations, reuse of a component instance in unsupported ways, and operators such as ‘&&’, ‘==’, ‘<’, or ternary ‘?:’ where Circom requires arithmetic/constraint-based constructions instead. To fix it, declare signals with plain ‘signal input/output name;’, instantiate components with valid names, wire outputs in separate statements, and replace conventional boolean/branching logic with comparator components, arithmetic selectors, and explicit constraints.

Original Error Tag: P1008-FEAT-Bitwise-Operation-Hallucination

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 61: Source Python iterated_game_memory_score implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template iteratedGameMemoryScore() {
4     signal input long p1, p2, h1, h2, h3, r;
5     signal output long out;
6     component GreaterEqThan = GreaterEqThan(64);
7     GreaterEqThan.in[0] <== p1;
8     GreaterEqThan.in[1] <== 1;
9     signal long gte_out1 <== GreaterEqThan.out;
10    GreaterEqThan.in[0] <== p2;
11    GreaterEqThan.in[1] <== 1;
12    signal long gte_out2 <== GreaterEqThan.out;
13    signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1 ==
14    0 && p2 == 1) ? 5 : 1;
15    signal long ts <== h1 + h2 + h3;
16    GreaterEqThan.in[0] <== r;
17    GreaterEqThan.in[1] <== 5;
18    signal long gte_out3 <== GreaterEqThan.out;
19    signal long pe <== (p1 == 0 && gte_out3) ? 1 : 0;
20    signal long fs <== (bp * 100) + (ts * 20) - (pe * 50);
21    signal long sel <== fs < 0;
22    out <== sel ? 0 : fs;
}

```

```
23 component main = iteratedGameMemoryScore();
```

Listing 62: Faulty translated Circom code with Bitwise Operation Hallucination (P1008)

```
1 error[P1008]: Missing semicolon
2   "circuit.circom":6:22
3
4   6       signal input long p1, p2, h1, h2, h3, r;
5           ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8   "circuit.circom":6:25
9
10  6       signal input long p1, p2, h1, h2, h3, r;
11          ^ A semicolon is needed here
12
13 error[P1012]: illegal expression
14   "circuit.circom":6:23
15
16  6       signal input long p1, p2, h1, h2, h3, r;
17          ^^^ here
18
19 error[P1008]: Missing semicolon
20   "circuit.circom":6:29
21
22  6       signal input long p1, p2, h1, h2, h3, r;
23          ^ A semicolon is needed here
24
25 error[P1012]: illegal expression
26   "circuit.circom":6:27
27
28  6       signal input long p1, p2, h1, h2, h3, r;
29          ^^^ here
30
31 error[P1008]: Missing semicolon
32   "circuit.circom":6:33
33
34  6       signal input long p1, p2, h1, h2, h3, r;
35          ^ A semicolon is needed here
36
37 error[P1012]: illegal expression
38   "circuit.circom":6:31
39
40  6       signal input long p1, p2, h1, h2, h3, r;
41          ^^^ here
42
43 error[P1008]: Missing semicolon
44   "circuit.circom":6:37
45
46  6       signal input long p1, p2, h1, h2, h3, r;
47          ^ A semicolon is needed here
48
49 error[P1012]: illegal expression
50   "circuit.circom":6:35
51
52  6       signal input long p1, p2, h1, h2, h3, r;
53          ^^^ here
54
```

```

55 error[P1008]: Missing semicolon
56     "circuit.circom":6:41
57
58     6         signal input long p1, p2, h1, h2, h3, r;
59                                     ^ A semicolon is needed here
60
61 error[P1012]: illegal expression
62     "circuit.circom":6:39
63
64     6         signal input long p1, p2, h1, h2, h3, r;
65                                     ^^^ here
66
67 error[P1012]: illegal expression
68     "circuit.circom":6:43
69
70     6         signal input long p1, p2, h1, h2, h3, r;
71                                     ^^ here
72
73 error[P1008]: Missing semicolon
74     "circuit.circom":7:23
75
76     7         signal output long out;
77                                     ^ A semicolon is needed here
78
79 error[P1012]: illegal expression
80     "circuit.circom":7:24
81
82     7         signal output long out;
83                                     ^^^^ here
84
85 error[P1008]: Missing semicolon
86     "circuit.circom":11:16
87
88     11        signal long gte_out1 <== GreaterEqThan.out;
89                                     ^ A semicolon is needed here
90
91 error[P1008]: Missing semicolon
92     "circuit.circom":14:16
93
94     14        signal long gte_out2 <== GreaterEqThan.out;
95                                     ^ A semicolon is needed here
96
97 error[P1008]: Missing semicolon
98     "circuit.circom":15:16
99
100    15        signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
101              == 0 && p2 == 1) ? 5 : 1;
102              ^ A semicolon is needed here
103
104 error[P1008]: Missing semicolon
105     "circuit.circom":15:74
106
107    15        signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
108              == 0 && p2 == 1) ? 5 : 1;
109
110              semicolon is needed here
111
112 error[P1008]: Missing semicolon

```

```

110 "circuit.circom":15:78
111
112 15      signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
    == 0 && p2 == 1) ? 5 : 1;
113
    ^ A
    semicolon is needed here
114
115 error[P1012]: illegal expression
116 "circuit.circom":15:76
117
118 15      signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
    == 0 && p2 == 1) ? 5 : 1;
119
    ^^^
    here
120
121 error[P1012]: illegal expression
122 "circuit.circom":15:80
123
124 15      signal long bp <== (gte_out1 && gte_out2) ? 3 : (p1 == 1 && p2 == 0) ? 0 : (p1
    == 0 && p2 == 1) ? 5 : 1;
125
    ~~~~~ here
126
127 error[P1008]: Missing semicolon
128 "circuit.circom":16:16
129
130 16      signal long ts <== h1 + h2 + h3;
131
    ^ A semicolon is needed here
132
133 error[P1008]: Missing semicolon
134 "circuit.circom":19:16
135
136 19      signal long gte_out3 <== GreaterEqThan.out;
137
    ^ A semicolon is needed here
138
139 error[P1008]: Missing semicolon
140 "circuit.circom":20:16
141
142 20      signal long pe <== (p1 == 0 && gte_out3) ? 1 : 0;
143
    ^ A semicolon is needed here
144
145 error[P1008]: Missing semicolon
146 "circuit.circom":21:16
147
148 21      signal long fs <== (bp * 100) + (ts * 20) - (pe * 50);
149
    ^ A semicolon is needed here
150
151 error[P1008]: Missing semicolon
152 "circuit.circom":22:16
153
154 22      signal long sel <== fs < 0;
155
    ^ A semicolon is needed here
156
157 previous errors were found

```

Listing 63: Circom compiler error log

Explanation: In this example, the parser starts failing at ‘signal input long ...’ and ‘signal output

long ...' because Circom has no 'long' type, so it misreads the rest of the line and reports many missing semicolons and illegal expressions. The code also uses unsupported C-like logic such as '&&', nested ternaries, and 'fs < 0', so the intended score logic must be rewritten using valid Circom signals, comparator components, and arithmetic selection.

A-22 Chained Assignment Hallucination (P1008)

Description: This error class happens when Circom code uses invalid chained component wiring or tries to assign into a subcomponent inline as if expressions could be nested together. In Circom, component instantiation and signal connections must be written as separate statements: first declare a component instance, then assign each input with its own '<==' line, and finally read its 'out' if needed. The fix is to break one compact "chain" into explicit component declarations and individual semicolon-terminated assignments.

Original Error Tag: P1008-FEAT-Chained-Assignment-Hallucination

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)

```

Listing 64: Source Python shuffle_integrity_logic implementation

```

1 // Import necessary libraries
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/arithmetic.circom";
4
5 // Define the template
6 template shuffle_integrity_logic() {
7     // Define input signals
8     signal input c0;
9     signal input c1;
10    signal input c2;
11    signal input c3;
12    signal input c4;
13    signal input s0;
14    signal input s1;
15    signal input s2;
16    signal input s3;
17    signal input s4;
18
19    // Define output signals
20    signal output out;
21
22    // Calculate sum of original array
23    signal sc <== c0 + c1 + c2 + c3 + c4;
24
25    // Calculate sum of shuffled array

```



```

26 signal ss <== s0 + s1 + s2 + s3 + s4;
27
28 // Check sum equality
29 component eq1 = Equal(64);
30 eq1.in[0] <== sc;
31 eq1.in[1] <== ss;
32
33 // Calculate sum of squares of original array
34 signal qc <== c0 * c0 + c1 * c1 + c2 * c2 + c3 * c3 + c4 * c4;
35
36 // Calculate sum of squares of shuffled array
37 signal qs <== s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3 + s4 * s4;
38
39 // Check quadratic sum equality
40 component eq2 = Equal(64);
41 eq2.in[0] <== qc;
42 eq2.in[1] <== qs;
43
44 // Initialize inversion count
45 signal inv <== 0;
46
47 // Count inversions in shuffled array
48 for (var i = 0; i < 5; i++) {
49     for (var j = i + 1; j < 5; j++) {
50         component lt = LessThan(64);
51         lt.in[0] <== s[i];
52         lt.in[1] <== s[j];
53         inv <== inv + lt.out;
54     }
55 }
56
57 // Check if first element of shuffled array is equal to any element in original
58 // array
59 component iv = Or(5);
60 iv.in[0] <== Equal(64).in[0] <== s0, in[1] <== c0;
61 iv.in[1] <== Equal(64).in[0] <== s0, in[1] <== c1;
62 iv.in[2] <== Equal(64).in[0] <== s0, in[1] <== c2;
63 iv.in[3] <== Equal(64).in[0] <== s0, in[1] <== c3;
64 iv.in[4] <== Equal(64).in[0] <== s0, in[1] <== c4;
65
66 // Output sum of inversion count and special flag
67 out <== inv + iv.out;
68
69 // Define the main component
70 component main = shuffle_integrity_logic();

```

Listing 65: Faulty translated Circom code with Chained Assignment Hallucination (P1008)

```

1 error[P1008]: Missing semicolon
2 "circuit.circom":63:27
3
4 63     iv.in[0] <== Equal(64).in[0] <== s0, in[1] <== c0;
5         ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8 "circuit.circom":63:40
9

```

```

10 63      iv.in[0] <== Equal(64).in[0] <== s0, in[1] <== c0;
11                                     ^ A semicolon is needed here
12
13 error[P1008]: Missing semicolon
14 "circuit.circom":64:27
15
16 64      iv.in[1] <== Equal(64).in[0] <== s0, in[1] <== c1;
17                                     ^ A semicolon is needed here
18
19 error[P1008]: Missing semicolon
20 "circuit.circom":64:40
21
22 64      iv.in[1] <== Equal(64).in[0] <== s0, in[1] <== c1;
23                                     ^ A semicolon is needed here
24
25 error[P1008]: Missing semicolon
26 "circuit.circom":65:27
27
28 65      iv.in[2] <== Equal(64).in[0] <== s0, in[1] <== c2;
29                                     ^ A semicolon is needed here
30
31 error[P1008]: Missing semicolon
32 "circuit.circom":65:40
33
34 65      iv.in[2] <== Equal(64).in[0] <== s0, in[1] <== c2;
35                                     ^ A semicolon is needed here
36
37 error[P1008]: Missing semicolon
38 "circuit.circom":66:27
39
40 66      iv.in[3] <== Equal(64).in[0] <== s0, in[1] <== c3;
41                                     ^ A semicolon is needed here
42
43 error[P1008]: Missing semicolon
44 "circuit.circom":66:40
45
46 66      iv.in[3] <== Equal(64).in[0] <== s0, in[1] <== c3;
47                                     ^ A semicolon is needed here
48
49 error[P1008]: Missing semicolon
50 "circuit.circom":67:27
51
52 67      iv.in[4] <== Equal(64).in[0] <== s0, in[1] <== c4;
53                                     ^ A semicolon is needed here
54
55 error[P1008]: Missing semicolon
56 "circuit.circom":67:40
57
58 67      iv.in[4] <== Equal(64).in[0] <== s0, in[1] <== c4;
59                                     ^ A semicolon is needed here
60
61 previous errors were found

```

Listing 66: Circom compiler error log

Explanation: In this example, lines like ‘iv.in[0] <== Equal(64).in[0] <== s0, in[1] <== c0;’ are not valid Circom syntax, so the parser reports missing semicolons where it encounters the illegal chain. You need to create five separate ‘Equal(64)’ components, wire ‘s0’ and each ‘cN’ into them on

separate lines, and then connect each equality component's output into 'iv.in[N]'.

A-23 Keyword Hallucination (P1008 & P1012)

Description: This error class happens when code uses unsupported or hallucinated language features or keywords that look valid in other languages but are not part of Circom's actual grammar. In Circom, 'if ... then ... else' is not a valid inline expression for assignments like '<==', so the parser gets confused and emits cascading 'Missing semicolon' and 'illegal expression' errors. To fix it, rewrite the logic using valid Circom constructs, such as arithmetic selector expressions, separate 'if' statement blocks where allowed, or explicit boolean constraints and intermediate signals instead of inline conditional syntax.

Original Error Tag: P1008+P1012-FEAT-Keyword-Hallucination

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 67: Source Python iterated_game_memory_score implementation

```

1 template iterated_game_memory_score() {
2     signal input p1;
3     signal input p2;
4     signal input h1;
5     signal input h2;
6     signal input h3;
7     signal input r;
8     signal output out;
9
10    signal bp;
11    signal ts;
12    signal pe;
13    signal fs;
14
15    bp <== if (p1 == 1 && p2 == 1) then 3; else if (p1 == 1 && p2 == 0) then 0; else
16    if (p1 == 0 && p2 == 1) then 5; else 0;
17    ts <== h1 + h2 + h3;
18    pe <== if (p1 == 0 && r > 5) then 1; else 0;
19    fs <== (bp * 100) + (ts * 20) - (pe * 50);
20    out <== if (fs < 0) then 0; else fs;
21 }
22 component main = iterated_game_memory_score();

```

Listing 68: Faulty translated Circom code with Keyword Hallucination (P1008 & P1012)

```

1 error[P1008]: Missing semicolon
2     "circuit.circom":19:8
3

```



```

51
52 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
53      ^^^ A semicolon is needed here
54
55 error[P1012]: illegal expression
56   "circuit.circom":21:5
57
58 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
59      ^^^^^^ here
60
61 error[P1008]: Missing semicolon
62   "circuit.circom":21:38
63
64 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
65      ^ A semicolon is needed here
66
67 error[P1012]: illegal expression
68   "circuit.circom":21:34
69
70 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
71      ^^^^^^ here
72
73 error[P1012]: illegal expression
74   "circuit.circom":21:39
75
76 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
77      ^^ here
78
79 error[P1012]: illegal expression
80   "circuit.circom":21:42
81
82 21      pe <== if (p1 == 0 && r > 5) then 1; else 0;
83      ^^^^^^^^ here
84
85 error[P1008]: Missing semicolon
86   "circuit.circom":23:9
87
88 23      out <== if (fs < 0) then 0; else fs;
89      ^^^ A semicolon is needed here
90
91 error[P1012]: illegal expression
92   "circuit.circom":23:5
93
94 23      out <== if (fs < 0) then 0; else fs;
95      ^^^^^^^^ here
96
97 error[P1008]: Missing semicolon
98   "circuit.circom":23:29
99
100 23      out <== if (fs < 0) then 0; else fs;
101      ^ A semicolon is needed here
102
103 error[P1012]: illegal expression
104   "circuit.circom":23:25
105
106 23      out <== if (fs < 0) then 0; else fs;
107      ^^^^^^ here
108

```

```

109 error[P1012]: illegal expression
110     "circuit.circom":23:30
111
112 23      out <== if (fs < 0) then 0; else fs;
113           ~~~~~ here
114
115 error[P1012]: illegal expression
116     "circuit.circom":23:33
117
118 23      out <== if (fs < 0) then 0; else fs;
119           ~~~~~ here
120
121 previous errors were found

```

Listing 69: Circom compiler error log

Explanation: In this example, ‘bp’, ‘pe’, and ‘out’ are assigned with Python-like / pseudocode-style conditional expressions such as ‘bp <== if (...) then ...; else ...;’, which Circom does not support. The compiler therefore misinterprets ‘if’, ‘then’, and ‘else’ as invalid tokens inside an expression and reports both fake semicolon issues and illegal-expression errors on those lines.

A-24 Functional Logic Hallucination (P1012)

Description: This error class happens when code uses programming-language features or dynamic behaviors that Circom does not support, even though they may look natural from JavaScript or general algorithm pseudocode. Typical fixes are to rewrite the logic using valid Circom constructs such as arithmetic selector masks, explicit comparator components, and statically indexed wiring, while avoiding unsupported conditional expressions, invalid equality syntax in expressions, and dynamic signal indexing.

Original Error Tag: P1012-FEAT-Functional-Logic-Hallucination

Example:

```

1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)

```

Listing 70: Source Python quicksort_partition_logic implementation

```

1 include "circomlib/circuits/comparators.circom";
2
3 // Helper: Conditional swap of two signals based on a selector
4 template CondSwap() {
5     signal input in[2];
6     signal input sel; // 1 means swap, 0 means keep

```

```

7   signal output out[2];
8
9   // out = sel ? [in[1], in[0]] : [in[0], in[1]]
10  out[0] <== sel * (in[1] - in[0]) + in[0];
11  out[1] <== sel * (in[0] - in[1]) + in[1];
12 }
13
14 template QuicksortPartitionLogic() {
15     signal input a0;
16     signal input a1;
17     signal input a2;
18     signal input a3;
19     signal input a4;
20     signal input a5;
21     signal input a6;
22     signal input low;
23     signal input high;
24
25     signal output out;
26
27     // Load array into signals
28     signal arr[7];
29     arr[0] <== a0;
30     arr[1] <== a1;
31     arr[2] <== a2;
32     arr[3] <== a3;
33     arr[4] <== a4;
34     arr[5] <== a5;
35     arr[6] <== a6;
36
37     // Ensure low and high are within [0,6]
38     // (Optional: add range checks if needed)
39
40     // pivot = arr[high]
41     signal pivot;
42     // Since high is an index (0..6), we use a mux
43     pivot <==
44         (high === 0 ? arr[0] :
45         (high === 1 ? arr[1] :
46         (high === 2 ? arr[2] :
47         (high === 3 ? arr[3] :
48         (high === 4 ? arr[4] :
49         (high === 5 ? arr[5] : arr[6])))))));
50
51     // i = low - 1
52     signal i;
53     i <== low - 1;
54
55     // We'll simulate the loop for j = 0 to 6
56     signal current_arr[7];
57     for (var k = 0; k < 7; k++) {
58         current_arr[k] <== arr[k];
59     }
60
61     signal current_i;
62     current_i <== i;
63
64     signal swaps;

```

```

65 swaps <== 0;
66
67 // Unroll loop for j = 0 to 6
68 for (var j = 0; j < 7; j++) {
69     // Check if j >= low && j < high
70     component ge_low = GreaterEqThan(3); // indices are small
71     ge_low.in[0] <== j;
72     ge_low.in[1] <== low;
73
74     component lt_high = LessThan(3);
75     lt_high.in[0] <== j;
76     lt_high.in[1] <== high;
77
78     signal in_range;
79     in_range <== ge_low.out * lt_high.out; // logical AND
80
81     // Check if current_arr[j] <= pivot
82     component le_pivot = LessEqThan(64);
83     le_pivot.in[0] <== current_arr[j];
84     le_pivot.in[1] <== pivot;
85
86     signal should_swap;
87     should_swap <== in_range * le_pivot.out;
88
89     // If should_swap, then i = i + 1, and swap arr[i] and arr[j]
90     signal new_i;
91     new_i <== current_i + should_swap;
92
93     // Prepare inputs for conditional swap between current_arr[current_i+1] and
94     current_arr[j]
95     signal temp_in[2];
96     temp_in[0] <== current_arr[new_i]; // note: this is safe because new_i =
97     current_i or current_i+1
98     temp_in[1] <== current_arr[j];
99
100     component swapper = CondSwap();
101     swapper.in[0] <== temp_in[0];
102     swapper.in[1] <== temp_in[1];
103     swapper.sel <== should_swap;
104
105     // Update array
106     signal updated_arr[7];
107     for (var idx = 0; idx < 7; idx++) {
108         updated_arr[idx] <== current_arr[idx];
109     }
110     updated_arr[new_i] <== swapper.out[0];
111     updated_arr[j] <== swapper.out[1];
112
113     // Update state for next iteration
114     for (var idx = 0; idx < 7; idx++) {
115         current_arr[idx] <== updated_arr[idx];
116     }
117     current_i <== new_i;
118     swaps <== swaps + should_swap;
119 }
120
121 // Final swap: arr[i+1] <-> arr[high]
122 signal final_i_plus_1;

```



```

121     final_i_plus_1 <== current_i + 1;
122
123     signal final_swap_in[2];
124     final_swap_in[0] <== current_arr[final_i_plus_1];
125     final_swap_in[1] <== current_arr[high];
126
127     component final_swapper = CondSwap();
128     final_swapper.in[0] <== final_swap_in[0];
129     final_swapper.in[1] <== final_swap_in[1];
130     final_swapper.sel <== 1; // always swap at end
131
132     // Update final array (not needed for output, but for correctness)
133     // We don't need to store it; only care about final_i_plus_1 and swaps
134
135     // Output = (i+1)*100 + swaps
136     out <== final_i_plus_1 * 100 + swaps;
137 }
138
139 component main = QuicksortPartitionLogic();

```

Listing 71: Faulty translated Circom code with Functional Logic Hallucination (P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":45:11
3
4     45         pivot <==
5         ^
6     46             (high === 0 ? arr[0] :
7         ^ A semicolon is needed here
8
9 error[P1012]: illegal expression
10   "circuit.circom":45:5
11
12     45         pivot <==
13     46             (high === 0 ? arr[0] :
14         ^ here
15
16 error[P1012]: illegal expression
17   "circuit.circom":46:19
18
19     46             (high === 0 ? arr[0] :
20         ^
21     47                 (high === 1 ? arr[1] :
22     48                 (high === 2 ? arr[2] :
23     49                 (high === 3 ? arr[3] :
24     50                 (high === 4 ? arr[4] :
25     51                 (high === 5 ? arr[5] : arr[6]))))));
26         ^ here
27
28 previous errors were found

```

Listing 72: Circom compiler error log

Explanation: In this example, the pivot assignment tries to implement ‘arr[high]’ with a JavaScript-style nested ternary using ‘high === 0 ? ...’, which Circom does not accept, so parsing fails with a missing semicolon and illegal expression errors. The pivot selection should instead be built with

Circom-compatible selector logic, for example by computing indicator bits for each possible ‘high’ value and combining ‘arr[0]’ through ‘arr[6]’ with those selectors.

A-25 Illegal Context Declaration (P1012)

Description: This error happens when a Circom declaration uses syntax that is only valid in a specific context, but the parser finds it in the wrong form or place. For ‘component main’, the only supported list inside braces is typically a ‘public [...]’ declaration, so writing a bare bracketed list causes a parse failure; fix it by using the correct ‘component main {public [...]} = Template();’ syntax or by removing the braces entirely if no public inputs need to be declared.

Original Error Tag: P1012-FEAT-Illegal-Context-Declaration

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 73: Source Python iterated_game_memory_score implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template iterated_game_memory_score() {
4     signal input p1;
5     signal input p2;
6     signal input h1;
7     signal input h2;
8     signal input h3;
9     signal input r;
10
11     signal output bp;
12     signal output ts;
13     signal output pe;
14     signal output fs;
15     signal output fs_neg;
16     signal output fs_non_neg;
17     signal output result;
18
19     // Condition computations
20     signal cond1; // p1 * p2
21     signal cond2; // p1 * (1 - p2)
22     signal cond3; // (1 - p1) * p2
23     signal cond4; // (1 - p1) * (if r > 5 then 1 else 0)
24
25     // Selection signals
26     signal sel1;
27     signal sel2;
28     signal sel3;
29     signal sel4;

```

```

30
31 // Compute conditions
32 cond1 <== p1 * p2;
33 cond2 <== p1 * (1 - p2);
34 cond3 <== (1 - p1) * p2;
35
36 // For cond4: (1 - p1) * (r > 5) - need comparator for r > 5
37 component gt5 = GreaterThan(64); // assuming 64-bit values
38 gt5.in[0] <== r;
39 gt5.in[1] <== 5;
40 signal r_gt_5 <== gt5.out;
41 cond4 <== (1 - p1) * r_gt_5;
42
43 // Selection constraints (mutual exclusion)
44 sel1 <== cond1;
45 sel2 <== cond2 * (1 - cond1);
46 sel3 <== cond3 * (1 - cond1) * (1 - cond2);
47
48 // Conditional assignments for bp
49 bp <-- sel1 * 3 + sel2 * 0 + sel3 * 5 + (1 - sel1 - sel2 - sel3) * 1;
50
51 // ts computation
52 ts <== h1 + h2 + h3;
53
54 // pe computation
55 pe <== cond4;
56
57 // fs computation
58 fs <== bp * 100 + ts * 20 - pe * 50;
59
60 // For fs_neg: detect if fs < 0
61 // We'll use a range check approach assuming fs is in a known range
62 // For simplicity, assume fs fits in 16 bits and we want to check sign
63 component lt0 = LessThan(16); // assuming fs is in reasonable range
64 // To handle signed comparison, we'll add an offset
65 // Let's assume fs is in range [-2^15, 2^15-1], so we add 2^15 to make it positive
66 signal fs_offset <== fs + 32768; // 2^15
67 signal zero_offset <== 32768;
68 lt0.in[0] <== zero_offset;
69 lt0.in[1] <== fs_offset;
70 fs_neg <== lt0.out; // fs_neg = 1 if fs < 0
71
72 fs_non_neg <== 1 - fs_neg;
73
74 // Result computation (max(0, fs))
75 result <== fs_non_neg * fs;
76
77 // Add constraints
78 cond1 * (cond1 - 1) === 0;
79 cond2 * (cond2 - 1) === 0;
80 cond3 * (cond3 - 1) === 0;
81 cond4 * (cond4 - 1) === 0;
82
83 sel1 * (sel1 - 1) === 0;
84 sel2 * (sel2 - 1) === 0;
85 sel3 * (sel3 - 1) === 0;
86
87 sel1 + sel2 + sel3 <= 1;

```

```

88
89     sel1 * (bp - 3) === 0;
90     sel2 * bp === 0;
91     sel3 * (bp - 5) === 0;
92
93     fs_neg * (fs_neg - 1) === 0;
94 }
95
96 component main { [p1, p2, h1, h2, h3, r] } = iterated_game_memory_score();

```

Listing 74: Faulty translated Circom code with Illegal Context Declaration (P1012)

```

1 error[P1012]: UnrecognizedToken { token: (2563, Token(45, "["), 2564), expected: ["\"
  public\""] }
2   "circuit.circom":98:18
3
4 98  component main { [p1, p2, h1, h2, h3, r] } = iterated_game_memory_score();
5      ^ here
6
7 previous errors were found

```

Listing 75: Circom compiler error log

Explanation: In this example, the main component is declared as ‘component main { [p1, p2, h1, h2, h3, r] } = ..’, but Circom expects ‘public’ before the input list. The parser sees ‘[’ where it was expecting “public”, so changing it to ‘component main {public [p1, p2, h1, h2, h3, r]} = iterated_game_memory_score();’ resolves this specific syntax error.

A-26 Illegal Operator in Expression (P1012)

Description: This error class happens when Circom code uses operators or expression forms that are not valid inside circuit constraints, especially ordinary inequality comparisons like ‘>=’, ‘<=’, ‘>’, or ‘<’ on signals. Circom works over finite fields, where values do not have a native ordering, so these comparisons are not legal constraint expressions; instead, you must encode the property with dedicated comparator/range-check components such as ‘LessThan’, ‘Num2Bits’, or another bounded-range gadget, and keep constraints in the supported equality-style form.

Original Error Tag: P1012-FEAT-Illegal-Operator-In-Expression

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 76: Source Python ema_signal_logic implementation

```

1 pragma circom 2.1.4;
2
3 template EMASignalLogic() {
4     signal input p0;
5     signal input p1;

```

```

6  signal input p2;
7  signal input p3;
8  signal input p4;
9  signal output out;
10
11  // Intermediate EMA values
12  signal ema0;
13  signal ema1;
14  signal ema2;
15  signal ema3;
16  signal ema4;
17
18  // Temporary calculation signals
19  signal temp1;
20  signal temp2;
21  signal temp3;
22  signal temp4;
23  signal temp5;
24  signal temp6;
25  signal temp7;
26  signal temp8;
27  signal temp9;
28  signal temp10;
29  signal temp11;
30  signal temp12;
31
32  // Division results and remainders
33  signal div_result1;
34  signal div_result2;
35  signal div_result3;
36  signal div_result4;
37  signal rem1;
38  signal rem2;
39  signal rem3;
40  signal rem4;
41
42  // Range check components for remainders (assuming we have a range checker)
43  // For simplicity, we'll use basic range constraints
44
45  // Step 1: Initialize first EMA as first price * 100 (to maintain precision)
46  ema0 <== p0 * 100;
47
48  // Step 2: Calculate EMA1 = (p1 * 40 + ema0 * 60) / 100
49  temp1 <== p1 * 40;
50  temp2 <== ema0 * 60;
51  temp3 <== temp1 + temp2;
52  // Integer division: temp3 = div_result1 * 100 + rem1
53  temp3 <== div_result1 * 100 + rem1;
54  ema1 <== div_result1;
55
56  // Step 3: Calculate EMA2 = (p2 * 40 + ema1 * 60) / 100
57  temp4 <== p2 * 40;
58  temp5 <== ema1 * 60;
59  temp6 <== temp4 + temp5;
60  temp6 <== div_result2 * 100 + rem2;
61  ema2 <== div_result2;
62
63  // Step 4: Calculate EMA3 = (p3 * 40 + ema2 * 60) / 100

```

```

64 temp7 <== p3 * 40;
65 temp8 <== ema2 * 60;
66 temp9 <== temp7 + temp8;
67 temp9 <== div_result3 * 100 + rem3;
68 ema3 <== div_result3;
69
70 // Step 5: Calculate EMA4 = (p4 * 40 + ema3 * 60) / 100
71 temp10 <== p4 * 40;
72 temp11 <== ema3 * 60;
73 temp12 <== temp10 + temp11;
74 temp12 <== div_result4 * 100 + rem4;
75 ema4 <== div_result4;
76
77 // Final output
78 out <== ema4;
79
80 // Range constraints for remainders (0 <= rem < 100)
81 // These ensure the division is correct by constraining the remainder
82 // In practice, you might want to use a proper range checker component
83 rem1 * (99 - rem1) >= 0; // This ensures 0 <= rem1 <= 99
84 rem2 * (99 - rem2) >= 0; // This ensures 0 <= rem2 <= 99
85 rem3 * (99 - rem3) >= 0; // This ensures 0 <= rem3 <= 99
86 rem4 * (99 - rem4) >= 0; // This ensures 0 <= rem4 <= 99
87 }
88
89 component main { public [p0, p1, p2, p3, p4] } = EMASignalLogic();

```

Listing 77: Faulty translated Circom code with Illegal Operator in Expression (P1012)

```

1 error[P1012]: illegal expression
2   "circuit.circom":85:5
3
4 85      rem1 * (99 - rem1) >= 0; // This ensures 0 <= rem1 <= 99
5         ~~~~~ here
6
7 error[P1012]: illegal expression
8   "circuit.circom":86:5
9
10 86      rem2 * (99 - rem2) >= 0; // This ensures 0 <= rem2 <= 99
11         ~~~~~ here
12
13 error[P1012]: illegal expression
14   "circuit.circom":87:5
15
16 87      rem3 * (99 - rem3) >= 0; // This ensures 0 <= rem3 <= 99
17         ~~~~~ here
18
19 error[P1012]: illegal expression
20   "circuit.circom":88:5
21
22 88      rem4 * (99 - rem4) >= 0; // This ensures 0 <= rem4 <= 99
23         ~~~~~ here
24
25 error[P1008]: Missing semicolon
26   "circuit.circom":91:66
27
28 91      component main { public [p0, p1, p2, p3, p4] } = EMASignalLogic();
29      ^

```

```

30 92
31 93   component main = EMASignalLogic();
32     ^ A semicolon is needed here
33
34 previous errors were found

```

Listing 78: Circom compiler error log

Explanation: In this example, the circuit tries to enforce ‘ $0 \leq \text{rem1}.\text{rem4} \leq 99$ ’ with expressions like ‘ $\text{rem1} * (99 - \text{rem1}) \geq 0$ ’, but ‘ ≥ 0 ’ is not a valid Circom constraint, so P1012 is raised. There is also a separate syntax issue at the end: the file effectively defines ‘main’ twice, which triggers the missing-semicolon/parse error and should be fixed by keeping only one valid ‘component main ... = EMASignalLogic();’ declaration.

A-27 Bare Illegal Expression (P1012)

Description: This error class happens when the Circom parser encounters syntax in a place where only specific top-level declarations are allowed, such as ‘template’, ‘function’, ‘component’, or ‘bus’. It is usually caused by putting a construct in the wrong scope, using unsupported syntax, or breaking the parser earlier so that it only notices the problem at a later token like a closing brace. To fix it, check the lines just before the reported location for misplaced declarations, invalid function syntax, or statements that must be split into valid Circom forms.

Original Error Tag: P1012-FEAT-Bare-Illegal-Expression

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2   sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3   qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4   if sc != ss or qc != qs: return 0
5   iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6   sh, inv = [s0, s1, s2, s3, s4], 0
7   for i in range(5):
8       for j in range(i+1, 5):
9           if sh[i] > sh[j]: inv += 1
10  return int(inv + iv)

```

Listing 79: Source Python shuffle_integrity_logic implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template ShuffleIntegrityLogic(){
6     signal input c0;
7     signal input c1;
8     signal input c2;
9     signal input c3;
10    signal input c4;
11    signal input s0;
12    signal input s1;
13    signal input s2;
14    signal input s3;
15    signal input s4;
16    signal input sd; // unused in this implementation

```

```

17  signal output out;
18
19  // Calculate sums
20  signal sc;
21  sc <== c0 + c1 + c2 + c3 + c4;
22
23  signal ss;
24  ss <== s0 + s1 + s2 + s3 + s4;
25
26  // Calculate quadratic sums
27  signal qc;
28  qc <== c0 * c0 + c1 * c1 + c2 * c2 + c3 * c3 + c4 * c4;
29
30  signal qs;
31  qs <== s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3 + s4 * s4;
32
33  // Check if sums and quadratic sums match
34  component eq_sc_ss = IsEqual();
35  eq_sc_ss.in[0] <== sc;
36  eq_sc_ss.in[1] <== ss;
37
38  component eq_qc_qs = IsEqual();
39  eq_qc_qs.in[0] <== qc;
40  eq_qc_qs.in[1] <== qs;
41
42  signal sums_match <== eq_sc_ss.out;
43  signal quads_match <== eq_qc_qs.out;
44
45  signal integrity_check <== sums_match * quads_match;
46
47  // If integrity check fails, return 0
48  signal temp_out;
49  temp_out <== integrity_check * calculate_inv_and_iv() + (1 - integrity_check) * 0;
50
51  out <== temp_out;
52
53  // Helper function to calculate inversion count and initial value
54  function calculate_inv_and_iv() -> signal {
55      // For this implementation, we'll return a fixed value since Circom doesn't
56      // support complex functions
57      // This is a simplified version that just returns a constant
58      return 100; // Placeholder return value
59  }
60
61  component main = ShuffleIntegrityLogic();

```

Listing 80: Faulty translated Circom code with Bare Illegal Expression (P1012)

```

1  error[P1012]: UnrecognizedToken { token: (1641, Token(80, "}"), 1642), expected: ["\"
   bus\"", "\"component\"", "\"function\"", "\"template\""] }
2  "circuit.circom":59:1
3
4  59  }
5      ^ here
6
7  previous errors were found

```

Listing 81: Circom compiler error log

Explanation: In this example, the main issue is that ‘calculate_inv_and_iv’ is declared as a ‘function’ inside the ‘template’, which Circom does not allow; functions must be defined at the top level. The ‘function calculate_inv_and_iv() -> signal’ syntax is also invalid in Circom, so by the time the parser reaches the final ‘}’, it is no longer inside a valid template body and reports that it expected a new top-level declaration instead.

A-28 Illegal Index Expression (P1012)

Description: This error class happens when Circom encounters syntax that cannot form a valid expression, often because an assignment or constraint operator is malformed or split incorrectly. A common fix is to rewrite the statement using a valid Circom operator such as ‘<==’ in one piece, make sure the full expression is attached to that operator, and end the statement with a semicolon so the parser can recognize it correctly.

Original Error Tag: P1012-FEAT-Index-Illegal-Expression

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 82: Source Python aave_v3_interest_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template aave_v3_interest_logic() {
6     signal input d;
7     signal input r;
8     signal input b;
9     signal input s1;
10    signal input s2;
11    signal input opt;
12
13    signal output out;
14
15    var SCALE = 10000;
16    var RESERVE_FACTOR = 9000;
17    var KINK1 = 8000;
18    var KINK2 = 9500;
19    var CAP = 1000000;
20}
```

```

21 // r == 0
22 component rEqZero = IsEqual();
23 rEqZero.in[0] <= r;
24 rEqZero.in[1] <= 0;
25 signal isRZero <= rEqZero.out;
26
27 // Safe denominator for u = (d * 10000) / r
28 signal safeR <= r + isRZero;
29
30 signal uRaw;
31 (uRaw, _) <= IntDivide(64)(d * SCALE, safeR);
32
33 // If r == 0, Java returns b immediately.
34 signal u <= uRaw * (1 - isRZero);
35
36 // u <= opt
37 component uLeOpt = LessEqThan(64);
38 uLeOpt.in[0] <= u;
39 uLeOpt.in[1] <= opt;
40 signal useFirstSlope <= uLeOpt.out;
41 signal useSecondSlope <= 1 - useFirstSlope;
42
43 // Safe denominators for the two slope divisions
44 component optEqZero = IsEqual();
45 optEqZero.in[0] <= opt;
46 optEqZero.in[1] <= 0;
47 signal isOptZero <= optEqZero.out;
48 signal safeOpt <= opt + isOptZero;
49
50 signal tailDenBase <= SCALE - opt;
51 component tailEqZero = IsEqual();
52 tailEqZero.in[0] <= tailDenBase;
53 tailEqZero.in[1] <= 0;
54 signal isTailZero <= tailEqZero.out;
55 signal safeTailDen <= tailDenBase + isTailZero;
56
57 // br = b + (u * s1 / opt) if u <= opt
58 // br = b + s1 + ((u - opt) * s2 / (10000 - opt)) otherwise
59 signal firstSlopePart;
60 (firstSlopePart, _) <= IntDivide(64)(u * s1, safeOpt);
61
62 signal secondSlopeNum <= (u - opt) * s2;
63 signal secondSlopePart;
64 (secondSlopePart, _) <= IntDivide(64)(secondSlopeNum, safeTailDen);
65
66 signal br <
67     == b
68     + useFirstSlope * firstSlopePart
69     + useSecondSlope * (s1 + secondSlopePart);
70
71 // sr = ((br * u) / 10000 * 9000) / 10000
72 signal srBase;
73 (srBase, _) <= IntDivide(64)(br * u, SCALE);
74
75 signal sr;
76 (sr, _) <= IntDivide(64)(srBase * RESERVE_FACTOR, SCALE);
77
78 // rp = 0;

```

```

79 // if (u > 9500) rp = (u - 9500) * 100;
80 // else if (u > 8000) rp = (u - 8000) * 20;
81 component uGt9500 = GreaterThan(64);
82 uGt9500.in[0] <= u;
83 uGt9500.in[1] <= KINK2;
84 signal isGt9500 <= uGt9500.out;
85
86 component uGt8000 = GreaterThan(64);
87 uGt8000.in[0] <= u;
88 uGt8000.in[1] <= KINK1;
89 signal isGt8000 <= uGt8000.out;
90
91 signal rpHigh <= (u - KINK2) * 100;
92 signal rpMid <= (u - KINK1) * 20;
93
94 signal rp <
95     == isGt9500 * rpHigh
96     + (1 - isGt9500) * isGt8000 * rpMid;
97
98 signal res <= br + sr + rp;
99
100 // return (res > 1000000) ? 1000000 : res;
101 component resGtCap = GreaterThan(64);
102 resGtCap.in[0] <= res;
103 resGtCap.in[1] <= CAP;
104 signal isResGtCap <= resGtCap.out;
105
106 signal cappedRes <= isResGtCap * CAP + (1 - isResGtCap) * res;
107
108 // if (r == 0) return b; else return cappedRes;
109 out <= isRZero * b + (1 - isRZero) * cappedRes;
110 }
111
112 component main = aave_v3_interest_logic();

```

Listing 83: Faulty translated Circom code with Illegal Index Expression (P1012)

```

1 error[P1008]: Missing semicolon
2 "circuit.circom":66:15
3
4 66         signal br <
5         ^
6 67         == b
7         ^ A semicolon is needed here
8
9 error[P1012]: illegal expression
10 "circuit.circom":67:12
11
12 67         == b
13         ^
14 68         + useFirstSlope * firstSlopePart
15 69         + useSecondSlope * (s1 + secondSlopePart);
16         ^ here
17
18 error[P1008]: Missing semicolon
19 "circuit.circom":94:15
20
21 94         signal rp <

```

```

22      ^
23 95          == isGt9500 * rpHigh
24      ^ A semicolon is needed here
25
26 error[P1012]: illegal expression
27     "circuit.circom":95:12
28
29 95          == isGt9500 * rpHigh
30      ^
31 96          + (1 - isGt9500) * isGt8000 * rpMid;
32      ^ here
33
34 previous errors were found

```

Listing 84: Circom compiler error log

Explanation: In this example, ‘signal br <’ followed by ‘== ...’ and ‘signal rp <’ followed by ‘== ...’ accidentally split the ‘<==’ operator across lines. Circom then treats the ‘signal’ line as unfinished, reports a missing semicolon, and flags the following ‘== ...’ as an illegal expression.

A-29 Illegal Comparison Expression (P1012)

Description: This error class happens when Circom code uses comparison-based expressions in places where the language does not allow them, especially C-style ternary operators with signal comparisons such as ‘x == k ? a : b’. In Circom, comparisons over signals are not general runtime expressions, so the parser often reports ‘illegal expression’ and then cascades into misleading ‘Missing semicolon’ errors. The fix is to replace these constructs with valid circuit logic: use comparator components like ‘IsEqual’, ‘LessThan’, or ‘LessEqThan’, build boolean selector signals, and combine candidates with arithmetic mux-style expressions instead of chained ‘?:’ conditionals.

Original Error Tag: P1012-FEAT-Illegal-Comparison-Expression

Example:

```

1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)

```

Listing 85: Source Python quicksort_partition_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template quicksort_partition_logic() {
4     signal input a0;
5     signal input a1;
6     signal input a2;

```

```

7  signal input a3;
8  signal input a4;
9  signal input a5;
10 signal input a6;
11 signal input low;
12 signal input high;
13 signal output out;
14
15 // Array elements
16 signal arr[7];
17 arr[0] <== a0;
18 arr[1] <== a1;
19 arr[2] <== a2;
20 arr[3] <== a3;
21 arr[4] <== a4;
22 arr[5] <== a5;
23 arr[6] <== a6;
24
25 // Get pivot element at index high
26 signal pivot;
27 pivot <-- (high == 0) ? arr[0] :
28           (high == 1) ? arr[1] :
29           (high == 2) ? arr[2] :
30           (high == 3) ? arr[3] :
31           (high == 4) ? arr[4] :
32           (high == 5) ? arr[5] : arr[6];
33
34 // Initialize i = low - 1
35 signal i;
36 i <== low - 1;
37
38 // Initialize swap counter
39 signal swaps;
40 swaps <== 0;
41
42 // Temporary signals for swapping
43 signal temp_arr[7];
44 temp_arr[0] <== arr[0];
45 temp_arr[1] <== arr[1];
46 temp_arr[2] <== arr[2];
47 temp_arr[3] <== arr[3];
48 temp_arr[4] <== arr[4];
49 temp_arr[5] <== arr[5];
50 temp_arr[6] <== arr[6];
51
52 // Process each element from 0 to 6
53 signal processed_arr[7][7]; // [iteration][index]
54 signal processed_i[8];      // i after each iteration
55 signal processed_swaps[8];  // swaps after each iteration
56
57 // Initialize first state
58 processed_i[0] <== i;
59 processed_swaps[0] <== swaps;
60
61 // Copy initial array to first iteration
62 for (var idx = 0; idx < 7; idx++) {
63     processed_arr[0][idx] <== temp_arr[idx];
64 }

```

```

65
66 // Loop through j from 0 to 6
67 for (var j = 0; j < 7; j++) {
68     // Check if j is within range [low, high)
69     component low_check_le = LessEqThan(64);
70     component high_check_lt = LessThan(64);
71
72     low_check_le.in[0] <== low;
73     low_check_le.in[1] <== j;
74     high_check_lt.in[0] <== j;
75     high_check_lt.in[1] <== high;
76
77     signal in_range;
78     in_range <== low_check_le.out * high_check_lt.out;
79
80     // Get current arr[j]
81     signal current_j_val;
82     current_j_val <-- (j == 0) ? processed_arr[j][0] :
83                       (j == 1) ? processed_arr[j][1] :
84                       (j == 2) ? processed_arr[j][2] :
85                       (j == 3) ? processed_arr[j][3] :
86                       (j == 4) ? processed_arr[j][4] :
87                       (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
88
89     // Compare with pivot
90     component le_pivot = LessEqThan(64);
91     le_pivot.in[0] <== current_j_val;
92     le_pivot.in[1] <== pivot;
93     signal should_swap <== le_pivot.out;
94
95     // Combined condition: in_range AND should_swap
96     signal do_swap <== in_range * should_swap;
97
98     // Calculate new i value
99     signal new_i;
100    new_i <== processed_i[j] + do_swap;
101
102    // Calculate new swaps value
103    signal new_swaps;
104    new_swaps <== processed_swaps[j] + do_swap;
105
106    // Copy array values to next iteration
107    for (var idx = 0; idx < 7; idx++) {
108        processed_arr[j+1][idx] <== processed_arr[j][idx];
109    }
110
111    // Perform swap if needed
112    signal temp_i_plus_1_val;
113    temp_i_plus_1_val <-- (processed_i[j]+1 == 0) ? processed_arr[j][0] :
114                        (processed_i[j]+1 == 1) ? processed_arr[j][1] :
115                        (processed_i[j]+1 == 2) ? processed_arr[j][2] :
116                        (processed_i[j]+1 == 3) ? processed_arr[j][3] :
117                        (processed_i[j]+1 == 4) ? processed_arr[j][4] :
118                        (processed_i[j]+1 == 5) ? processed_arr[j][5] :
119                        (processed_i[j]+1 == 6) ? processed_arr[j][6] :
120
121    processed_arr[j][0];
122
123    // Conditional swap logic

```

```

122     signal new_j_val;
123     signal new_i_plus_1_val;
124
125     new_j_val <= do_swap * temp_i_plus_1_val + (1 - do_swap) * processed_arr[j][j]
];
126     new_i_plus_1_val <= do_swap * processed_arr[j][j] + (1 - do_swap) *
temp_i_plus_1_val;
127
128     // Update array values if swap happens
129     for (var idx = 0; idx < 7; idx++) {
130         signal update_j <= (idx == j) ? do_swap : 0;
131         signal update_i_plus_1 <= (idx == processed_i[j] + 1) ? do_swap : 0;
132
133         processed_arr[j+1][idx] <=
134             processed_arr[j][idx] * (1 - update_j) * (1 - update_i_plus_1) +
135             new_j_val * update_j +
136             new_i_plus_1_val * update_i_plus_1;
137     }
138
139     processed_i[j+1] <= new_i;
140     processed_swaps[j+1] <= new_swaps;
141 }
142
143 // Final swap between (i+1) and high
144 signal final_i_plus_1 <= processed_i[7] + 1;
145
146 // Get values at positions final_i_plus_1 and high
147 signal val_at_final_i_plus_1;
148 val_at_final_i_plus_1 <-- (final_i_plus_1 == 0) ? processed_arr[7][0] :
149     (final_i_plus_1 == 1) ? processed_arr[7][1] :
150     (final_i_plus_1 == 2) ? processed_arr[7][2] :
151     (final_i_plus_1 == 3) ? processed_arr[7][3] :
152     (final_i_plus_1 == 4) ? processed_arr[7][4] :
153     (final_i_plus_1 == 5) ? processed_arr[7][5] :
processed_arr[7][6];
154
155 signal val_at_high;
156 val_at_high <-- (high == 0) ? processed_arr[7][0] :
157     (high == 1) ? processed_arr[7][1] :
158     (high == 2) ? processed_arr[7][2] :
159     (high == 3) ? processed_arr[7][3] :
160     (high == 4) ? processed_arr[7][4] :
161     (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
162
163 // Create final array after last swap
164 signal final_arr[7];
165 for (var idx = 0; idx < 7; idx++) {
166     signal is_i_plus_1 <= (idx == final_i_plus_1) ? 1 : 0;
167     signal is_high <= (idx == high) ? 1 : 0;
168
169     final_arr[idx] <=
170         processed_arr[7][idx] * (1 - is_i_plus_1) * (1 - is_high) +
171         val_at_high * is_i_plus_1 +
172         val_at_final_i_plus_1 * is_high;
173 }
174
175 // Calculate result: (i + 1) * 100 + swaps
176 out <= (final_i_plus_1) * 100 + processed_swaps[7];

```

```

177 }
178
179 component main = quicksort_partition_logic();

```

Listing 86: Faulty translated Circom code with Illegal Comparison Expression (P1012)

```

1 error[P1008]: Missing semicolon
2   "circuit.circom":30:26
3
4 30           (high == 1) ? arr[1] :
5                ^ A semicolon is needed here
6
7 error[P1008]: Missing semicolon
8   "circuit.circom":30:35
9
10 30           (high == 1) ? arr[1] :
11                ^ A semicolon is needed here
12
13 error[P1012]: illegal expression
14   "circuit.circom":30:28
15
16 30           (high == 1) ? arr[1] :
17                ^^^^^^^^^ here
18
19 error[P1008]: Missing semicolon
20   "circuit.circom":32:26
21
22 32           (high == 3) ? arr[3] :
23                ^ A semicolon is needed here
24
25 error[P1012]: illegal expression
26   "circuit.circom":31:14
27
28 31           (high == 2) ? arr[2] :
29 32           (high == 3) ? arr[3] :
30   ^ here
31
32 error[P1008]: Missing semicolon
33   "circuit.circom":32:35
34
35 32           (high == 3) ? arr[3] :
36                ^ A semicolon is needed here
37
38 error[P1012]: illegal expression
39   "circuit.circom":32:28
40
41 32           (high == 3) ? arr[3] :
42                ^^^^^^^^^ here
43
44 error[P1008]: Missing semicolon
45   "circuit.circom":34:26
46
47 34           (high == 5) ? arr[5] : arr[6];
48                ^ A semicolon is needed here
49
50 error[P1012]: illegal expression
51   "circuit.circom":33:14
52

```



```

53 33          (high == 4) ? arr[4] :
54 34          (high == 5) ? arr[5] : arr[6];
55     ^ here
56
57 error[P1008]: Missing semicolon
58   "circuit.circom":34:35
59
60 34          (high == 5) ? arr[5] : arr[6];
61                                ^ A semicolon is needed here
62
63 error[P1012]: illegal expression
64   "circuit.circom":34:28
65
66 34          (high == 5) ? arr[5] : arr[6];
67                                ~~~~~ here
68
69 error[P1012]: illegal expression
70   "circuit.circom":34:37
71
72 34          (high == 5) ? arr[5] : arr[6];
73                                ~~~~~ here
74
75 error[P1008]: Missing semicolon
76   "circuit.circom":85:35
77
78 85          (j == 1) ? processed_arr[j][1] :
79                                ^ A semicolon is needed here
80
81 error[P1008]: Missing semicolon
82   "circuit.circom":85:57
83
84 85          (j == 1) ? processed_arr[j][1] :
85                                ^ A semicolon is needed
86   here
87
88 error[P1012]: illegal expression
89   "circuit.circom":85:37
90
91 85          (j == 1) ? processed_arr[j][1] :
92                                ~~~~~ here
93
94 error[P1008]: Missing semicolon
95   "circuit.circom":87:35
96
97 87          (j == 3) ? processed_arr[j][3] :
98                                ^ A semicolon is needed here
99
100 error[P1012]: illegal expression
101   "circuit.circom":86:26
102
103 86          (j == 2) ? processed_arr[j][2] :
104 87          (j == 3) ? processed_arr[j][3] :
105     ^ here
106
107 error[P1008]: Missing semicolon
108   "circuit.circom":87:57
109
110 87          (j == 3) ? processed_arr[j][3] :

```

```

110                                     ^ A semicolon is needed
111     here
112 error[P1012]: illegal expression
113     "circuit.circom":87:37
114
115 87                                     (j == 3) ? processed_arr[j][3] :
116                                     ~~~~~ here
117
118 error[P1008]: Missing semicolon
119     "circuit.circom":89:35
120
121 89                                     (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
122                                     ^ A semicolon is needed here
123
124 error[P1012]: illegal expression
125     "circuit.circom":88:26
126
127 88                                     (j == 4) ? processed_arr[j][4] :
128 89                                     (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
129     ^ here
130
131 error[P1008]: Missing semicolon
132     "circuit.circom":89:57
133
134 89                                     (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
135                                     ^ A semicolon is needed
136     here
137 error[P1012]: illegal expression
138     "circuit.circom":89:37
139
140 89                                     (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
141                                     ~~~~~ here
142
143 error[P1012]: illegal expression
144     "circuit.circom":89:59
145
146 89                                     (j == 5) ? processed_arr[j][5] : processed_arr[j][6];
147                                     ~~~~~
148     here
149 error[P1008]: Missing semicolon
150     "circuit.circom":116:54
151
152 116                                     (processed_i[j]+1 == 1) ? processed_arr[j][1] :
153                                     ^ A semicolon is needed here
154
155 error[P1008]: Missing semicolon
156     "circuit.circom":116:76
157
158 116                                     (processed_i[j]+1 == 1) ? processed_arr[j][1] :
159                                     ^ A
160     semicolon is needed here
161 error[P1012]: illegal expression
162     "circuit.circom":116:56
163

```

```

164 116 (processed_i[j]+1 == 1) ? processed_arr[j][1] :
165 ~~~~~ here
166
167 error[P1008]: Missing semicolon
168 "circuit.circom":118:54
169
170 118 (processed_i[j]+1 == 3) ? processed_arr[j][3] :
171 ~ A semicolon is needed here
172
173 error[P1012]: illegal expression
174 "circuit.circom":117:30
175
176 117 (processed_i[j]+1 == 2) ? processed_arr[j][2] :
177 118 (processed_i[j]+1 == 3) ? processed_arr[j][3] :
178 ~ here
179
180 error[P1008]: Missing semicolon
181 "circuit.circom":118:76
182
183 118 (processed_i[j]+1 == 3) ? processed_arr[j][3] :
184 ~ A
185 semicolon is needed here
186
187 error[P1012]: illegal expression
188 "circuit.circom":118:56
189
190 118 (processed_i[j]+1 == 3) ? processed_arr[j][3] :
191 ~~~~~ here
192
193 error[P1008]: Missing semicolon
194 "circuit.circom":120:54
195
196 120 (processed_i[j]+1 == 5) ? processed_arr[j][5] :
197 ~ A semicolon is needed here
198
199 error[P1012]: illegal expression
200 "circuit.circom":119:30
201
202 119 (processed_i[j]+1 == 4) ? processed_arr[j][4] :
203 120 (processed_i[j]+1 == 5) ? processed_arr[j][5] :
204 ~ here
205
206 error[P1008]: Missing semicolon
207 "circuit.circom":120:76
208
209 120 (processed_i[j]+1 == 5) ? processed_arr[j][5] :
210 ~ A
211 semicolon is needed here
212
213 error[P1012]: illegal expression
214 "circuit.circom":120:56
215
216 120 (processed_i[j]+1 == 5) ? processed_arr[j][5] :
217 ~~~~~ here
218
219 error[P1012]: illegal expression
220 "circuit.circom":121:30

```

```

220 121                                (processed_i[j]+1 == 6) ? processed_arr[j][6] :
      processed_arr[j][0];
221
      ~~~~~ here
222
223 error[P1008]: Missing semicolon
224   "circuit.circom":151:51
225
226 151                                (final_i_plus_1 == 1) ? processed_arr[7][1] :
227                                ^ A semicolon is needed here
228
229 error[P1008]: Missing semicolon
230   "circuit.circom":151:73
231
232 151                                (final_i_plus_1 == 1) ? processed_arr[7][1] :
233                                ^ A
      semicolon is needed here
234
235 error[P1012]: illegal expression
236   "circuit.circom":151:53
237
238 151                                (final_i_plus_1 == 1) ? processed_arr[7][1] :
239                                ~~~~~ here
240
241 error[P1008]: Missing semicolon
242   "circuit.circom":153:51
243
244 153                                (final_i_plus_1 == 3) ? processed_arr[7][3] :
245                                ^ A semicolon is needed here
246
247 error[P1012]: illegal expression
248   "circuit.circom":152:29
249
250 152                                (final_i_plus_1 == 2) ? processed_arr[7][2] :
251 153                                (final_i_plus_1 == 3) ? processed_arr[7][3] :
252   ^ here
253
254 error[P1008]: Missing semicolon
255   "circuit.circom":153:73
256
257 153                                (final_i_plus_1 == 3) ? processed_arr[7][3] :
258                                ^ A
      semicolon is needed here
259
260 error[P1012]: illegal expression
261   "circuit.circom":153:53
262
263 153                                (final_i_plus_1 == 3) ? processed_arr[7][3] :
264                                ~~~~~ here
265
266 error[P1008]: Missing semicolon
267   "circuit.circom":155:51
268
269 155                                (final_i_plus_1 == 5) ? processed_arr[7][5] :
      processed_arr[7][6];
270                                ^ A semicolon is needed here
271
272 error[P1012]: illegal expression

```

```

273     "circuit.circom":154:29
274
275 154                                     (final_i_plus_1 == 4) ? processed_arr[7][4] :
276 155                                     (final_i_plus_1 == 5) ? processed_arr[7][5] :
      processed_arr[7][6];
277     ^ here
278
279 error[P1008]: Missing semicolon
280     "circuit.circom":155:73
281
282 155                                     (final_i_plus_1 == 5) ? processed_arr[7][5] :
      processed_arr[7][6];
283                                     ^ A
      semicolon is needed here
284
285 error[P1012]: illegal expression
286     "circuit.circom":155:53
287
288 155                                     (final_i_plus_1 == 5) ? processed_arr[7][5] :
      processed_arr[7][6];
289                                     ~~~~~~ here
290
291 error[P1012]: illegal expression
292     "circuit.circom":155:75
293
294 155                                     (final_i_plus_1 == 5) ? processed_arr[7][5] :
      processed_arr[7][6];
295                                     ~~~~~~ here
296
297 error[P1008]: Missing semicolon
298     "circuit.circom":159:32
299
300 159                                     (high == 1) ? processed_arr[7][1] :
301                                     ^ A semicolon is needed here
302
303 error[P1008]: Missing semicolon
304     "circuit.circom":159:54
305
306 159                                     (high == 1) ? processed_arr[7][1] :
307                                     ^ A semicolon is needed here
308
309 error[P1012]: illegal expression
310     "circuit.circom":159:34
311
312 159                                     (high == 1) ? processed_arr[7][1] :
313                                     ~~~~~~ here
314
315 error[P1008]: Missing semicolon
316     "circuit.circom":161:32
317
318 161                                     (high == 3) ? processed_arr[7][3] :
319                                     ^ A semicolon is needed here
320
321 error[P1012]: illegal expression
322     "circuit.circom":160:20
323
324 160                                     (high == 2) ? processed_arr[7][2] :

```

```

325 161                (high == 3) ? processed_arr[7][3] :
326     ^ here
327
328 error[P1008]: Missing semicolon
329     "circuit.circom":161:54
330
331 161                (high == 3) ? processed_arr[7][3] :
332                                     ^ A semicolon is needed here
333
334 error[P1012]: illegal expression
335     "circuit.circom":161:34
336
337 161                (high == 3) ? processed_arr[7][3] :
338                                     ~~~~~~ here
339
340 error[P1008]: Missing semicolon
341     "circuit.circom":163:32
342
343 163                (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
344                                     ^ A semicolon is needed here
345
346 error[P1012]: illegal expression
347     "circuit.circom":162:20
348
349 162                (high == 4) ? processed_arr[7][4] :
350 163                (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
351     ^ here
352
353 error[P1008]: Missing semicolon
354     "circuit.circom":163:54
355
356 163                (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
357                                     ^ A semicolon is needed here
358
359 error[P1012]: illegal expression
360     "circuit.circom":163:34
361
362 163                (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
363                                     ~~~~~~ here
364
365 error[P1012]: illegal expression
366     "circuit.circom":163:56
367
368 163                (high == 5) ? processed_arr[7][5] : processed_arr[7][6];
369                                     ~~~~~~ here
370
371 previous errors were found

```

Listing 87: Circom compiler error log

Explanation: In this example, the circuit repeatedly tries to select array elements with chained ternaries like ‘(high == 1) ? arr[1] : ...’, ‘(j == 3) ? processed_arr[j][3] : ...’, and ‘(final_i_plus_1 == 5) ? ...’, which Circom rejects. The reported semicolon errors are secondary; the real problem is that these signal-dependent comparison/ternary expressions for pivot lookup, indexed reads, and final swaps are not valid Circom syntax and must be rewritten as selector-based circuit constraints.

B Undefined Symbol Calls

B-1 Undefined Addition/Subtraction Symbol (T2021)

Description: This error class happens when a Circom file tries to instantiate a component or template name that the compiler cannot resolve, usually because the symbol was never defined, was misspelled, is not exported by the included library, or belongs to a different Circomlib version/API. The fix is to verify that the template actually exists in the imported files, use the correct template names for your library version, and replace nonexistent arithmetic components with direct arithmetic expressions or your own helper templates when Circomlib does not provide ‘Add()’, ‘Sub()’, ‘Mul()’, or ‘Div()’.

Original Error Tag: T2021 » Calling-Symbol » Arithmetic-AddSub

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 88: Source Python aave_v3_interest_logic implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2 include "node_modules/circomlib/circuits/math.circom";
3
4 template aave_v3_interest_logic() {
5     signal input d;
6     signal input r;
7     signal input b;
8     signal input s1;
9     signal input s2;
10    signal input opt;
11    signal output out;
12
13    // Constants
14    signal ten_thousand <= 10000;
15    signal nine_thousand <= 9000;
16    signal nine_five_hundred <= 9500;
17    signal eight_thousand <= 8000;
18    signal one_million <= 1000000;
19    signal twenty <= 20;
20    signal one_hundred <= 100;
21
22    // Check if r == 0
23    component isZero = IsZero();
24    isZero.in <= r;
25    signal rIsZero <= isZero.out;
26
27    // Calculate u = (d * 10000) / r
28    component mul_d_10000 = Mul();
```

```

29 mul_d_10000.in[0] <= d;
30 mul_d_10000.in[1] <= ten_thousand;
31
32 component div_u = Div();
33 div_u.in[0] <= mul_d_10000.out;
34 div_u.in[1] <= r;
35 signal u <= div_u.out;
36
37 // Compare u <= opt
38 component lte_opt = LessEqThan(64);
39 lte_opt.in[0] <= u;
40 lte_opt.in[1] <= opt;
41 signal uLteOpt <= lte_opt.out;
42
43 // Calculate br based on condition
44 // if u <= opt: br = b + (u * s1 / opt)
45 // else: br = b + s1 + ((u - opt) * s2 / (10000 - opt))
46
47 // Calculate u * s1
48 component mul_u_s1 = Mul();
49 mul_u_s1.in[0] <= u;
50 mul_u_s1.in[1] <= s1;
51
52 // Divide (u * s1) by opt
53 component div_u_s1_opt = Div();
54 div_u_s1_opt.in[0] <= mul_u_s1.out;
55 div_u_s1_opt.in[1] <= opt;
56
57 // Calculate u - opt
58 component sub_u_opt = Sub();
59 sub_u_opt.in[0] <= u;
60 sub_u_opt.in[1] <= opt;
61
62 // Calculate 10000 - opt
63 component sub_10000_opt = Sub();
64 sub_10000_opt.in[0] <= ten_thousand;
65 sub_10000_opt.in[1] <= opt;
66
67 // Calculate (u - opt) * s2
68 component mul_u_opt_s2 = Mul();
69 mul_u_opt_s2.in[0] <= sub_u_opt.out;
70 mul_u_opt_s2.in[1] <= s2;
71
72 // Divide ((u - opt) * s2) by (10000 - opt)
73 component div_u_opt_s2_10000_opt = Div();
74 div_u_opt_s2_10000_opt.in[0] <= mul_u_opt_s2.out;
75 div_u_opt_s2_10000_opt.in[1] <= sub_10000_opt.out;
76
77 // Calculate b + s1
78 component add_b_s1 = Add();
79 add_b_s1.in[0] <= b;
80 add_b_s1.in[1] <= s1;
81
82 // Calculate b + (u * s1 / opt)
83 component add_b_u_s1_opt = Add();
84 add_b_u_s1_opt.in[0] <= b;
85 add_b_u_s1_opt.in[1] <= div_u_s1_opt.out;
86

```



```

87 // Calculate b + s1 + ((u - opt) * s2 / (10000 - opt))
88 component add_b_s1_u_opt_s2 = Add();
89 add_b_s1_u_opt_s2.in[0] <== add_b_s1.out;
90 add_b_s1_u_opt_s2.in[1] <== div_u_opt_s2_10000_opt.out;
91
92 // Select br based on condition
93 signal br <== uLteOpt * add_b_u_s1_opt.out + (1 - uLteOpt) * add_b_s1_u_opt_s2.out
94 ;
95
96 // Calculate sr = ((br * u) / 10000 * 9000) / 10000
97 component mul_br_u = Mul();
98 mul_br_u.in[0] <== br;
99 mul_br_u.in[1] <== u;
100
101 component div_br_u_10000 = Div();
102 div_br_u_10000.in[0] <== mul_br_u.out;
103 div_br_u_10000.in[1] <== ten_thousand;
104
105 component mul_div_9000 = Mul();
106 mul_div_9000.in[0] <== div_br_u_10000.out;
107 mul_div_9000.in[1] <== nine_thousand;
108
109 component div_sr = Div();
110 div_sr.in[0] <== mul_div_9000.out;
111 div_sr.in[1] <== ten_thousand;
112 signal sr <== div_sr.out;
113
114 // Calculate rp based on conditions
115 // if u > 9500: rp = (u - 9500) * 100
116 // else if u > 8000: rp = (u - 8000) * 20
117 // else: rp = 0
118
119 // Check u > 9500
120 component gt_9500 = GreaterThan(64);
121 gt_9500.in[0] <== u;
122 gt_9500.in[1] <== nine_five_hundred;
123 signal uGt9500 <== gt_9500.out;
124
125 // Check u > 8000
126 component gt_8000 = GreaterThan(64);
127 gt_8000.in[0] <== u;
128 gt_8000.in[1] <== eight_thousand;
129 signal uGt8000 <== gt_8000.out;
130
131 // Calculate u - 9500
132 component sub_u_9500 = Sub();
133 sub_u_9500.in[0] <== u;
134 sub_u_9500.in[1] <== nine_five_hundred;
135
136 // Calculate (u - 9500) * 100
137 component mul_u_9500_100 = Mul();
138 mul_u_9500_100.in[0] <== sub_u_9500.out;
139 mul_u_9500_100.in[1] <== one_hundred;
140
141 // Calculate u - 8000
142 component sub_u_8000 = Sub();
143 sub_u_8000.in[0] <== u;
144 sub_u_8000.in[1] <== eight_thousand;

```

```

144
145 // Calculate (u - 8000) * 20
146 component mul_u_8000_20 = Mul();
147 mul_u_8000_20.in[0] <= sub_u_8000.out;
148 mul_u_8000_20.in[1] <= twenty;
149
150 // Select rp based on conditions
151 signal rp <= uGt9500 * mul_u_9500_100.out + (1 - uGt9500) * uGt8000 *
mul_u_8000_20.out;
152
153 // Calculate res = br + sr + rp
154 component add_br_sr = Add();
155 add_br_sr.in[0] <= br;
156 add_br_sr.in[1] <= sr;
157
158 component add_res = Add();
159 add_res.in[0] <= add_br_sr.out;
160 add_res.in[1] <= rp;
161 signal res <= add_res.out;
162
163 // Check if res > 1000000
164 component gt_1m = GreaterThan(64);
165 gt_1m.in[0] <= res;
166 gt_1m.in[1] <= one_million;
167 signal resGt1m <= gt_1m.out;
168
169 // Final output: if r == 0 return b, else return min(res, 1000000)
170 signal finalRes <= (1 - rIsZero) * (resGt1m * one_million + (1 - resGt1m) * res)
+ rIsZero * b;
171
172 out <= finalRes;
173 }
174
175 component main = aave_v3_interest_logic();

```

Listing 89: Faulty translated Circom code with Undefined Addition/Subtraction Symbol (T2021)

```

1 error[T2021]: Calling symbol
2 "circuit.circom":29:29
3
4 29 component mul_d_10000 = Mul();
5          ~~~~~ Calling unknown symbol
6
7 error[T2021]: Calling symbol
8 "circuit.circom":33:23
9
10 33 component div_u = Div();
11          ~~~~~ Calling unknown symbol
12
13 error[T2021]: Calling symbol
14 "circuit.circom":49:26
15
16 49 component mul_u_s1 = Mul();
17          ~~~~~ Calling unknown symbol
18
19 error[T2021]: Calling symbol
20 "circuit.circom":54:30
21

```

```

22 54      component div_u_s1_opt = Div();
23      ~~~~~ Calling unknown symbol
24
25 error[T2021]: Calling symbol
26 "circuit.circom":59:27
27
28 59      component sub_u_opt = Sub();
29      ~~~~~ Calling unknown symbol
30
31 error[T2021]: Calling symbol
32 "circuit.circom":64:31
33
34 64      component sub_10000_opt = Sub();
35      ~~~~~ Calling unknown symbol
36
37 error[T2021]: Calling symbol
38 "circuit.circom":69:30
39
40 69      component mul_u_opt_s2 = Mul();
41      ~~~~~ Calling unknown symbol
42
43 error[T2021]: Calling symbol
44 "circuit.circom":74:40
45
46 74      component div_u_opt_s2_10000_opt = Div();
47      ~~~~~ Calling unknown symbol
48
49 error[T2021]: Calling symbol
50 "circuit.circom":79:26
51
52 79      component add_b_s1 = Add();
53      ~~~~~ Calling unknown symbol
54
55 error[T2021]: Calling symbol
56 "circuit.circom":84:32
57
58 84      component add_b_u_s1_opt = Add();
59      ~~~~~ Calling unknown symbol
60
61 error[T2021]: Calling symbol
62 "circuit.circom":89:35
63
64 89      component add_b_s1_u_opt_s2 = Add();
65      ~~~~~ Calling unknown symbol
66
67 error[T2021]: Calling symbol
68 "circuit.circom":97:26
69
70 97      component mul_br_u = Mul();
71      ~~~~~ Calling unknown symbol
72
73 error[T2021]: Calling symbol
74 "circuit.circom":101:32
75
76 101     component div_br_u_10000 = Div();
77      ~~~~~ Calling unknown symbol
78
79 error[T2021]: Calling symbol

```

```

80     "circuit.circom":105:30
81
82 105     component mul_div_9000 = Mul();
83                                     ~~~~~ Calling unknown symbol
84
85 error[T2021]: Calling symbol
86     "circuit.circom":109:24
87
88 109     component div_sr = Div();
89                                     ~~~~~ Calling unknown symbol
90
91 error[T2021]: Calling symbol
92     "circuit.circom":132:28
93
94 132     component sub_u_9500 = Sub();
95                                     ~~~~~ Calling unknown symbol
96
97 error[T2021]: Calling symbol
98     "circuit.circom":137:32
99
100 137     component mul_u_9500_100 = Mul();
101                                     ~~~~~ Calling unknown symbol
102
103 error[T2021]: Calling symbol
104     "circuit.circom":142:28
105
106 142     component sub_u_8000 = Sub();
107                                     ~~~~~ Calling unknown symbol
108
109 error[T2021]: Calling symbol
110     "circuit.circom":147:31
111
112 147     component mul_u_8000_20 = Mul();
113                                     ~~~~~ Calling unknown symbol
114
115 error[T2021]: Calling symbol
116     "circuit.circom":155:27
117
118 155     component add_br_sr = Add();
119                                     ~~~~~ Calling unknown symbol
120
121 error[T2021]: Calling symbol
122     "circuit.circom":159:25
123
124 159     component add_res = Add();
125                                     ~~~~~ Calling unknown symbol
126
127 previous errors were found

```

Listing 90: Circom compiler error log

Explanation: In this example, the circuit includes ‘comparators.circom’ and ‘math.circom’ but then calls ‘Mul()’, ‘Div()’, ‘Sub()’, and ‘Add()’ many times, and those symbols are not defined in the imported files, so every instantiation fails with ‘Calling unknown symbol’. You should rewrite those steps using native Circom arithmetic (‘a + b’, ‘a - b’, ‘a * b’) and only use a valid division gadget/template if you explicitly define or import one.

B-2 Undefined Bit/Data Operation Symbol (T2021)

Description: This error class happens when a Circom circuit tries to instantiate a template or function name that the compiler cannot resolve, usually because the symbol does not exist in the imported libraries, the import path/version is wrong, or the template is being called with the wrong signature. To fix it, verify that each component name actually exists in the circomlib version you are using, import the correct file that defines it, use the exact template name expected by that library, and pass any required template parameters such as comparator bit widths.

Original Error Tag: T2021 » Calling-Symbol » Bit-Data-Ops

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 91: Source Python aave_v3_interest_logic implementation

```
1 pragma circom 2.1.0;
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/arithmetic/compare.circom";
4 include "node_modules/circomlib/circuits/arithmetic/num2fixed.circom";
5 include "node_modules/circomlib/circuits/arithmetic/fixed_point.circom";
6
7 template aave_v3_interest_logic() {
8     signal input d;
9     signal input r;
10    signal input b;
11    signal input s1;
12    signal input s2;
13    signal input opt;
14    signal output out;
15
16    component is_zero = IsZero();
17    is_zero.in <== r;
18
19    component u_num = Num2Fixed();
20    u_num.in <== d * 10000;
21    component u_denom = Num2Fixed();
22    u_denom.in <== r;
23    component u_div = FixedPointDiv();
24    u_div.num <== u_num.out;
25    u_div.denom <== u_denom.out;
26    signal u <== u_div.out;
27
28    component u_leq_opt = LessEqThan();
29    u_leq_opt.in[0] <== u;
30    u_leq_opt.in[1] <== opt;
```

```

31  signal u_leq_opt_out <== u_leq_opt.out;
32  component br_if_true = FixedPointMul();
33  br_if_true.in[0] <== u;
34  br_if_true.in[1] <== s1;
35  component br_if_true_div = FixedPointDiv();
36  br_if_true_div.num <== br_if_true.out;
37  br_if_true_div.denom <== opt;
38  component br_if_true_add = FixedPointAdd();
39  br_if_true_add.in[0] <== b;
40  br_if_true_add.in[1] <== br_if_true_div.out;
41  component br_if_false = FixedPointAdd();
42  br_if_false.in[0] <== b;
43  br_if_false.in[1] <== s1;
44  component u_sub_opt = FixedPointSub();
45  u_sub_opt.in[0] <== u;
46  u_sub_opt.in[1] <== opt;
47  component u_sub_opt_mul = FixedPointMul();
48  u_sub_opt_mul.in[0] <== u_sub_opt.out;
49  u_sub_opt_mul.in[1] <== s2;
50  component u_sub_opt_mul_div = FixedPointDiv();
51  u_sub_opt_mul_div.num <== u_sub_opt_mul.out;
52  u_sub_opt_mul_div.denom <== 10000 - opt;
53  component br_if_false_add = FixedPointAdd();
54  br_if_false_add.in[0] <== br_if_false.out;
55  br_if_false_add.in[1] <== u_sub_opt_mul_div.out;
56  component br = Mux();
57  br.cond <== u_leq_opt_out;
58  br.in[0] <== br_if_true_add.out;
59  br.in[1] <== br_if_false_add.out;
60
61  component br_mul_u = FixedPointMul();
62  br_mul_u.in[0] <== br.out;
63  br_mul_u.in[1] <== u;
64  component br_mul_u_div = FixedPointDiv();
65  br_mul_u_div.num <== br_mul_u.out;
66  br_mul_u_div.denom <== 10000;
67  component sr_mul = FixedPointMul();
68  sr_mul.in[0] <== br_mul_u_div.out;
69  sr_mul.in[1] <== 9000;
70  component sr_div = FixedPointDiv();
71  sr_div.num <== sr_mul.out;
72  sr_div.denom <== 10000;
73  signal sr <== sr_div.out;
74
75  component u_gt_9500 = GreaterThan();
76  u_gt_9500.in[0] <== u;
77  u_gt_9500.in[1] <== 9500;
78  signal u_gt_9500_out <== u_gt_9500.out;
79  component rp_if_true = FixedPointMul();
80  rp_if_true.in[0] <== u - 9500;
81  rp_if_true.in[1] <== 100;
82  component u_gt_8000 = GreaterThan();
83  u_gt_8000.in[0] <== u;
84  u_gt_8000.in[1] <== 8000;
85  signal u_gt_8000_out <== u_gt_8000.out;
86  component rp_if_false = FixedPointMul();
87  rp_if_false.in[0] <== u - 8000;
88  rp_if_false.in[1] <== 20;

```

```

89     component rp = Mux();
90     rp.cond <= u_gt_9500_out;
91     rp.in[0] <= rp_if_true.out;
92     rp.in[1] <= rp_if_false.out;
93
94     component res = FixedPointAdd();
95     res.in[0] <= br.out;
96     res.in[1] <= sr;
97     component res_add = FixedPointAdd();
98     res_add.in[0] <= res.out;
99     res_add.in[1] <= rp.out;
100
101     component res_leq_1000000 = LessEqThan();
102     res_leq_1000000.in[0] <= res_add.out;
103     res_leq_1000000.in[1] <= 1000000;
104     signal res_leq_1000000_out <= res_leq_1000000.out;
105     component out1 = Mux();
106     out1.cond <= res_leq_1000000_out;
107     out1.in[0] <= res_add.out;
108     out1.in[1] <= 1000000;
109
110     component out_if_zero = Mux();
111     out_if_zero.cond <= is_zero.out;
112     out_if_zero.in[0] <= b;
113     out_if_zero.in[1] <= out1.out;
114     out <= out_if_zero.out;
115 }
116
117 component main = aave_v3_interest_logic();

```

Listing 92: Faulty translated Circom code with Undefined Bit/Data Operation Symbol (T2021)

```

1 error[T2021]: Calling symbol
2   "circuit.circom":17:23
3
4 17     component u_num = Num2Fixed();
5           ~~~~~ Calling unknown symbol
6
7 error[T2021]: Calling symbol
8   "circuit.circom":19:25
9
10 19     component u_denom = Num2Fixed();
11           ~~~~~ Calling unknown symbol
12
13 error[T2021]: Calling symbol
14   "circuit.circom":21:23
15
16 21     component u_div = FixedPointDiv();
17           ~~~~~ Calling unknown symbol
18
19 error[T2012]: Calling function with wrong number of arguments
20   "circuit.circom":26:27
21
22 26     component u_leq_opt = LessEqThan();
23           ~~~~~ Got 0 params, 1 where expected
24
25 error[T2021]: Calling symbol
26   "circuit.circom":30:28

```

```

27
28 30      component br_if_true = FixedPointMul();
29                                ~~~~~ Calling unknown symbol
30
31 error[T2021]: Calling symbol
32   "circuit.circom":33:32
33
34 33      component br_if_true_div = FixedPointDiv();
35                                ~~~~~ Calling unknown symbol
36
37 error[T2021]: Calling symbol
38   "circuit.circom":36:32
39
40 36      component br_if_true_add = FixedPointAdd();
41                                ~~~~~ Calling unknown symbol
42
43 error[T2021]: Calling symbol
44   "circuit.circom":39:29
45
46 39      component br_if_false = FixedPointAdd();
47                                ~~~~~ Calling unknown symbol
48
49 error[T2021]: Calling symbol
50   "circuit.circom":42:27
51
52 42      component u_sub_opt = FixedPointSub();
53                                ~~~~~ Calling unknown symbol
54
55 error[T2021]: Calling symbol
56   "circuit.circom":45:31
57
58 45      component u_sub_opt_mul = FixedPointMul();
59                                ~~~~~ Calling unknown symbol
60
61 error[T2021]: Calling symbol
62   "circuit.circom":48:35
63
64 48      component u_sub_opt_mul_div = FixedPointDiv();
65                                ~~~~~ Calling unknown symbol
66
67 error[T2021]: Calling symbol
68   "circuit.circom":51:33
69
70 51      component br_if_false_add = FixedPointAdd();
71                                ~~~~~ Calling unknown symbol
72
73 error[T2021]: Calling symbol
74   "circuit.circom":54:20
75
76 54      component br = Mux();
77                                ~~~~~ Calling unknown symbol
78
79 error[T2021]: Calling symbol
80   "circuit.circom":59:26
81
82 59      component br_mul_u = FixedPointMul();
83                                ~~~~~ Calling unknown symbol
84

```



```

85 error[T2021]: Calling symbol
86     "circuit.circom":62:30
87
88 62     component br_mul_u_div = FixedPointDiv();
89                                ~~~~~ Calling unknown symbol
90
91 error[T2021]: Calling symbol
92     "circuit.circom":65:24
93
94 65     component sr_mul = FixedPointMul();
95                                ~~~~~ Calling unknown symbol
96
97 error[T2021]: Calling symbol
98     "circuit.circom":68:24
99
100 68     component sr_div = FixedPointDiv();
101                                ~~~~~ Calling unknown symbol
102
103 error[T2012]: Calling function with wrong number of arguments
104     "circuit.circom":73:27
105
106 73     component u_gt_9500 = GreaterThan();
107                                ~~~~~ Got 0 params, 1 where expected
108
109 error[T2021]: Calling symbol
110     "circuit.circom":77:28
111
112 77     component rp_if_true = FixedPointMul();
113                                ~~~~~ Calling unknown symbol
114
115 error[T2012]: Calling function with wrong number of arguments
116     "circuit.circom":80:27
117
118 80     component u_gt_8000 = GreaterThan();
119                                ~~~~~ Got 0 params, 1 where expected
120
121 error[T2021]: Calling symbol
122     "circuit.circom":84:29
123
124 84     component rp_if_false = FixedPointMul();
125                                ~~~~~ Calling unknown symbol
126
127 error[T2021]: Calling symbol
128     "circuit.circom":87:20
129
130 87     component rp = Mux();
131                                ~~~~~ Calling unknown symbol
132
133 error[T2021]: Calling symbol
134     "circuit.circom":92:21
135
136 92     component res = FixedPointAdd();
137                                ~~~~~ Calling unknown symbol
138
139 error[T2021]: Calling symbol
140     "circuit.circom":95:25
141
142 95     component res_add = FixedPointAdd();

```

```

143          ~~~~~ Calling unknown symbol
144
145 error[T2012]: Calling function with wrong number of arguments
146   "circuit.circom":99:33
147
148 99      component res_leq_1000000 = LessEqThan();
149          ~~~~~ Got 0 params, 1 where expected
150
151 error[T2021]: Calling symbol
152   "circuit.circom":103:22
153
154 103      component out1 = Mux();
155          ~~~~~ Calling unknown symbol
156
157 error[T2021]: Calling symbol
158   "circuit.circom":108:29
159
160 108      component out_if_zero = Mux();
161          ~~~~~ Calling unknown symbol
162
163 previous errors were found

```

Listing 93: Circom compiler error log

Explanation: In this example, ‘Num2Fixed’, ‘FixedPointDiv’, ‘FixedPointMul’, ‘FixedPointAdd’, ‘FixedPointSub’, and ‘Mux’ are not known symbols under the current imports/version, so those component instantiations fail with ‘T2021’. Also, ‘LessEqThan()’ and ‘GreaterThan()’ are being called without their required bit-width parameter, so they should be instantiated with something like ‘LessEqThan(n)’ and ‘GreaterThan(n)’ using an appropriate number of bits for the compared values.

B-3 Undefined Comparison Symbol (T2021)

Description: This error class occurs when Circom tries to instantiate or call a symbol that is not defined in the current scope, typically because the component/function name is wrong, the file that defines it was not included correctly, or the library uses a different symbol name than expected. To fix it, verify the exact exported template name in the imported ‘circom’ file, check spelling and capitalization, confirm the include path is correct, and replace the call with the actual available component or implement the logic manually if that symbol does not exist.

Original Error Tag: T2021 » Calling-Symbol » Logic-Comparison

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 94: Source Python iterated_game_memory_score implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4 include "node_modules/circomlib/circuits/mux1.circom";
5
6 template iterated_game_memory_score() {
7     // Input signals
8     signal input p1;
9     signal input p2;
10    signal input h1;
11    signal input h2;
12    signal input h3;
13    signal input r;
14
15    // Output signal
16    signal output fs;
17
18    // Binary check for p1 and p2
19    signal p1_is_binary <== p1 * (1 - p1);
20    signal p2_is_binary <== p2 * (1 - p2);
21
22    // Base Payoff (bp) calculation using discrete conditional mapping
23    // Cases: (1,1)=>3, (1,0)=>0, (0,1)=>5, (0,0)=>1
24
25    // Create flags for each action pair
26    signal p1_eq_1 <== p1;
27    signal p1_eq_0 <== 1 - p1;
28    signal p2_eq_1 <== p2;
29    signal p2_eq_0 <== 1 - p2;
30
31    // Calculate each possible bp value weighted by its condition
32    signal bp_case_11 <== p1_eq_1 * p2_eq_1 * 3;
33    signal bp_case_10 <== p1_eq_1 * p2_eq_0 * 0;
34    signal bp_case_01 <== p1_eq_0 * p2_eq_1 * 5;
35    signal bp_case_00 <== p1_eq_0 * p2_eq_0 * 1;
36
37    // Sum all cases to get bp (only one will be non-zero due to binary constraints)
38    signal bp <== bp_case_11 + bp_case_10 + bp_case_01 + bp_case_00;
39
40    // History Bonus (ts) calculation
41    signal ts <== h1 + h2 + h3;
42
43    // Penalty (pe) calculation: 1 if p1 == 0 and r > 5, else 0
44    component gt = GreaterThan(32); // Assuming r fits in 32 bits
45    gt.in[0] <== r;
46    gt.in[1] <== 5;
47
48    signal r_gt_5 <== gt.out;
49    signal pe <== p1_eq_0 * r_gt_5;
50
51    // Final Score calculation: fs = (bp 100) + (ts 20) - (pe 50)
52    signal bp_scaled <== bp * 100;
53    signal ts_scaled <== ts * 20;
54    signal pe_scaled <== pe * 50;
55
56    signal raw_fs <== bp_scaled + ts_scaled - pe_scaled;
57

```

```

58 // Non-negative clamping (max with 0)
59 component is_negative = LessThan(32); // Assuming raw_fs fits in 32 bits
60 is_negative.in[0] <== 0;
61 is_negative.in[1] <== raw_fs;
62
63 // If raw_fs is negative (0 > raw_fs), is_negative.out = 1, else 0
64 signal negative_flag <== is_negative.out;
65
66 // Select between 0 and raw_fs based on negative_flag
67 // When negative_flag = 1 (raw_fs is negative), select 0
68 // When negative_flag = 0 (raw_fs is non-negative), select raw_fs
69 component mux = Mux1();
70 mux.sel <== negative_flag;
71 mux.in[0] <== raw_fs;
72 mux.in[1] <== 0;
73
74 fs <== mux.out;
75 }
76
77 component main = iterated_game_memory_score();

```

Listing 95: Faulty translated Circom code with Undefined Comparison Symbol (T2021)

```

1 error[T2021]: Calling symbol
2   "circuit.circom":68:21
3
4 68      component mux = Mux1();
5           ~~~~~ Calling unknown symbol
6
7 previous errors were found

```

Listing 96: Circom compiler error log

Explanation: In this example, ‘component mux = Mux1();’ fails because ‘Mux1’ is not a known template after including ‘mux1.circom’, so the symbol name likely does not match what that file actually defines. The comparison logic is fine up to that point; the immediate issue is that the final clamp uses a multiplexer component name that Circom cannot resolve.

B-4 Undefined Division/Modulo Symbol (T2021)

Description: This error class happens when a circuit tries to instantiate or call a division/modulo-related component symbol that the compiler cannot resolve. In Circom, this usually means the template name is wrong, the file defining it was not included correctly, the library version does not provide that symbol, or the component’s API/name changed; fix it by checking the imported file, confirming the exact template name exported by your circomlib version, and replacing the call with the correct division template or your own implementation.

Original Error Tag: T2021 » Calling-Symbol » Arithmetic-DivMod

Example:

```

1 def anova_f_test_logic(g1_s, g1_n, g2_s, g2_n, total_ss):
2     m1, m2 = g1_s // g1_n, g2_s // g2_n
3     gm = (g1_s + g2_s) // (g1_n + g2_n)
4     ssb = g1_n * (m1 - gm)**2 + g2_n * (m2 - gm)**2
5     ssw = total_ss - ssb

```

```

6   if ssw <= 0: return int(gm)
7   return int((ssb * 100) // ssw + gm)

```

Listing 97: Source Python anova_f_test_logic implementation

```

1 pragma circom 2.1.4;
2
3 /* Import the required components from the Circom library.
4   Adjust the include paths to match your environment.
5 */
6 include "node_modules/circomlib/circuits/comparators.circom";
7 include "node_modules/circomlib/circuits/bitify.circom";
8 include "node_modules/circomlib/circuits/div.circom";
9
10
11 template AnovaFTestLogic() {
12     /* ---- inputs ---- */
13     signal input g1s;
14     signal input g1n;
15     signal input g2s;
16     signal input g2n;
17     signal input tss;
18
19     signal output out;
20
21     /* ---- 1. means of the two groups ---- */
22     component divM1 = Div(252);
23     divM1.a <== g1s;
24     divM1.b <== g1n;
25     signal m1 <== divM1.q;           // m1 = g1s / g1n
26
27     component divM2 = Div(252);
28     divM2.a <== g2s;
29     divM2.b <== g2n;
30     signal m2 <== divM2.q;           // m2 = g2s / g2n
31
32     /* ---- 2. grand mean ---- */
33     signal sumS <== g1s + g2s;
34     signal sumN <== g1n + g2n;
35
36     component divGM = Div(252);
37     divGM.a <== sumS;
38     divGM.b <== sumN;
39     signal gm <== divGM.q;           // gm = (g1s+g2s)/(g1n+g2n)
40
41     /---- 3. between-group sum of squares ----/
42     signal diff1 <== m1 - gm; // (m1-gm)
43     signal diff2 <== m2 - gm; // (m2-gm)
44     signal diff1Sq <== diff1 * diff1; // (m1-gm)
45     signal diff2Sq <== diff2 * diff2; // (m2-gm)
46     signal ssb1 <== g1n * diff1Sq; // n1(m1-gm)
47     signal ssb2 <== g2n * diff2Sq; // n2(m2-gm)
48     signal ssb <== ssb1 + ssb2; // ssb
49
50     /* ---- 4. within-group sum of squares ---- */
51     signal ssw <== tss - ssb; // ssw = tss - ssb
52
53     /* ---- 5. decide whether ssw <= 0 (zero or negative) ---- */

```

```

54 // zero-check
55 component isZeroSSW = IsZero();
56 isZeroSSW.in <== ssw;
57 // sign-bit (most-significant bit of the 254-bit field representation)
58 component bitsSSW = Num2Bits(254);
59 bitsSSW.in <== ssw;
60 signal signBit <== bitsSSW.out[253]; // 1 => negative in the field
61 // OR of "zero" and "negative"
62 signal shouldReturnGM <== isZeroSSW.out + signBit
63 isZeroSSW.out * signBit;
64 signal sswGTZero <== 1 - shouldReturnGM; // 1 => ssw > 0
65
66 /* ---- 6. safe divisor (avoid zero / negative) ---- */
67 // When ssw <= 0 we substitute 1 so that the Div component never sees 0
68 signal sswSafe <== sswGTZero * ssw + (1 - sswGTZero);
69
70 /* ---- 7. quotient = (ssb100) / ssw ---- */
71 component divRes = Div(252);
72 divRes.a <== ssb * 100; // dividend
73 divRes.b <== sswSafe; // divisor (always > 0)
74 signal quotient <== divRes.q; // integer division result
75
76 /* ---- 8. final mux (if-then-else) ---- */
77 // out = shouldReturnGM ? gm : (quotient + gm)
78 out <== shouldReturnGM * gm
79 sswGTZero * (quotient + gm);
80 }
81
82 /* ---- main component ---- */
83 component main = AnovaFTestLogic();

```

Listing 98: Faulty translated Circom code with Undefined Division/Modulo Symbol (T2021)

```

1 error[T2021]: Calling symbol
2 "circuit.circom":32:23
3
4 32 component divM1 = Div(252);
5          ~~~~~ Calling unknown symbol
6
7 error[T2021]: Calling symbol
8 "circuit.circom":37:23
9
10 37 component divM2 = Div(252);
11          ~~~~~ Calling unknown symbol
12
13 error[T2021]: Calling symbol
14 "circuit.circom":46:23
15
16 46 component divGM = Div(252);
17          ~~~~~ Calling unknown symbol
18
19 error[T2021]: Calling symbol
20 "circuit.circom":85:24
21
22 85 component divRes = Div(252);
23          ~~~~~ Calling unknown symbol
24
25 previous errors were found

```

Listing 99: Circom compiler error log

Explanation: In this example, ‘Div(252)’ is instantiated four times, but the compiler cannot find any symbol named ‘Div’ even though ‘div.circom’ is included. So the issue is not the arithmetic itself, but that this library/version/path does not define a callable ‘Div’ template with that name, and those component declarations must be updated to the correct available divider component.

B-5 Undefined Multiplication Symbol (T2021)

Description: This error class happens when a Circom file tries to instantiate a template or function that is either not defined, not imported, or called with the wrong constructor signature. It is commonly caused by assuming helper components like ‘Mul’, ‘Div’, ‘Add’, ‘Sub’, or ‘Mux’ exist in circomlib when they do not, or by passing parameters to a template that expects none. The fix is to check the actual templates exported by the included libraries, use the correct number of template parameters, and replace nonexistent arithmetic "components" with native Circom expressions or with valid library templates that provide the intended behavior.

Original Error Tag: T2021 » Calling-Symbol » Arithmetic-Mul

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 100: Source Python aave_v3_interest_logic implementation

```
1 // Import necessary templates
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/arithmetic.circom";
4
5 // Define the Aave V3 interest logic template
6 template AaveV3InterestLogic() {
7     // Define input signals
8     signal input d;
9     signal input r;
10    signal input b;
11    signal input s1;
12    signal input s2;
13    signal input opt;
14
15    // Define output signal
16    signal output out;
17
18    // Define internal signals
```

```

19  signal u;
20  signal br;
21  signal sr;
22  signal rp;
23  signal res;
24
25  // Check if r is zero
26  component isZero = IsZero(64);
27  isZero.in <== r;
28
29  // Calculate u
30  component uCalc = Div(64);
31  uCalc.in1 <== d * 10000;
32  uCalc.in2 <== r;
33  u <== uCalc.out;
34
35  // Calculate br
36  component brCalc = Mul(64);
37  brCalc.in1 <== u;
38  brCalc.in2 <== s1;
39  component brOpt = Div(64);
40  brOpt.in1 <== brCalc.out;
41  brOpt.in2 <== opt;
42  component brAdd = Add(64);
43  brAdd.in1 <== b;
44  brAdd.in2 <== brOpt.out;
45  component brOptCheck = GreaterEqThan(64);
46  brOptCheck.in[0] <== u;
47  brOptCheck.in[1] <== opt;
48  component brSel = Mux(64);
49  brSel.in1 <== brAdd.out;
50  brSel.in2 <== b + s1 + ((u - opt) * s2 / (10000 - opt));
51  brSel.sel <== brOptCheck.out;
52  br <== brSel.out;
53
54  // Calculate sr
55  component srCalc = Mul(64);
56  srCalc.in1 <== br;
57  srCalc.in2 <== u;
58  component srDiv = Div(64);
59  srDiv.in1 <== srCalc.out;
60  srDiv.in2 <== 10000;
61  component srMul = Mul(64);
62  srMul.in1 <== srDiv.out;
63  srMul.in2 <== 9000;
64  component srDiv2 = Div(64);
65  srDiv2.in1 <== srMul.out;
66  srDiv2.in2 <== 10000;
67  sr <== srDiv2.out;
68
69  // Calculate rp
70  component rpCheck1 = GreaterEqThan(64);
71  rpCheck1.in[0] <== u;
72  rpCheck1.in[1] <== 9500;
73  component rpCalc1 = Sub(64);
74  rpCalc1.in1 <== u;
75  rpCalc1.in2 <== 9500;
76  component rpMul1 = Mul(64);

```



```

77  rpMul1.in1 <== rpCalc1.out;
78  rpMul1.in2 <== 100;
79  component rpCheck2 = GreaterEqThan(64);
80  rpCheck2.in[0] <== u;
81  rpCheck2.in[1] <== 8000;
82  component rpCalc2 = Sub(64);
83  rpCalc2.in1 <== u;
84  rpCalc2.in2 <== 8000;
85  component rpMul2 = Mul(64);
86  rpMul2.in1 <== rpCalc2.out;
87  rpMul2.in2 <== 20;
88  component rpSel = Mux(64);
89  rpSel.in1 <== rpMul1.out;
90  rpSel.in2 <== rpMul2.out;
91  rpSel.sel <== rpCheck2.out;
92  component rpSel2 = Mux(64);
93  rpSel2.in1 <== rpSel.out;
94  rpSel2.in2 <== 0;
95  rpSel2.sel <== rpCheck1.out;
96  rp <== rpSel2.out;
97
98  // Calculate res
99  component resCalc = Add(64);
100  resCalc.in1 <== br;
101  resCalc.in2 <== sr;
102  resCalc.in3 <== rp;
103  res <== resCalc.out;
104
105  // Check if res is greater than 1000000
106  component resCheck = GreaterEqThan(64);
107  resCheck.in[0] <== res;
108  resCheck.in[1] <== 1000000;
109  component resSel = Mux(64);
110  resSel.in1 <== res;
111  resSel.in2 <== 1000000;
112  resSel.sel <== resCheck.out;
113  out <== resSel.out;
114
115  // If r is zero, return b
116  component outSel = Mux(64);
117  outSel.in1 <== resSel.out;
118  outSel.in2 <== b;
119  outSel.sel <== isZero.out;
120  out <== outSel.out;
121 }
122
123 // Define the main component
124 component main = AaveV3InterestLogic();

```

Listing 101: Faulty translated Circom code with Undefined Multiplication Symbol (T2021)

```

1  error[T2012]: Calling function with wrong number of arguments
2  "circuit.circom":30:24
3
4  30      component isZero = IsZero(64);
5          ~~~~~~ Got 1 params, 0 where expected
6
7  error[T2021]: Calling symbol

```

```

8      "circuit.circom":34:23
9
10     34      component uCalc = Div(64);
11              ~~~~~ Calling unknown symbol
12
13     error[T2021]: Calling symbol
14         "circuit.circom":40:24
15
16     40      component brCalc = Mul(64);
17              ~~~~~ Calling unknown symbol
18
19     error[T2021]: Calling symbol
20         "circuit.circom":43:23
21
22     43      component brOpt = Div(64);
23              ~~~~~ Calling unknown symbol
24
25     error[T2021]: Calling symbol
26         "circuit.circom":46:23
27
28     46      component brAdd = Add(64);
29              ~~~~~ Calling unknown symbol
30
31     error[T2021]: Calling symbol
32         "circuit.circom":52:23
33
34     52      component brSel = Mux(64);
35              ~~~~~ Calling unknown symbol
36
37     error[T2021]: Calling symbol
38         "circuit.circom":59:24
39
40     59      component srCalc = Mul(64);
41              ~~~~~ Calling unknown symbol
42
43     error[T2021]: Calling symbol
44         "circuit.circom":62:23
45
46     62      component srDiv = Div(64);
47              ~~~~~ Calling unknown symbol
48
49     error[T2021]: Calling symbol
50         "circuit.circom":65:23
51
52     65      component srMul = Mul(64);
53              ~~~~~ Calling unknown symbol
54
55     error[T2021]: Calling symbol
56         "circuit.circom":68:24
57
58     68      component srDiv2 = Div(64);
59              ~~~~~ Calling unknown symbol
60
61     error[T2021]: Calling symbol
62         "circuit.circom":77:25
63
64     77      component rpCalc1 = Sub(64);
65              ~~~~~ Calling unknown symbol

```

```

66
67 error[T2021]: Calling symbol
68     "circuit.circom":80:24
69
70 80     component rpMul1 = Mul(64);
71           ~~~~~ Calling unknown symbol
72
73 error[T2021]: Calling symbol
74     "circuit.circom":86:25
75
76 86     component rpCalc2 = Sub(64);
77           ~~~~~ Calling unknown symbol
78
79 error[T2021]: Calling symbol
80     "circuit.circom":89:24
81
82 89     component rpMul2 = Mul(64);
83           ~~~~~ Calling unknown symbol
84
85 error[T2021]: Calling symbol
86     "circuit.circom":92:23
87
88 92     component rpSel = Mux(64);
89           ~~~~~ Calling unknown symbol
90
91 error[T2021]: Calling symbol
92     "circuit.circom":96:24
93
94 96     component rpSel2 = Mux(64);
95           ~~~~~ Calling unknown symbol
96
97 error[T2021]: Calling symbol
98     "circuit.circom":103:25
99
100 103    component resCalc = Add(64);
101           ~~~~~ Calling unknown symbol
102
103 error[T2021]: Calling symbol
104     "circuit.circom":113:24
105
106 113    component resSel = Mux(64);
107           ~~~~~ Calling unknown symbol
108
109 error[T2021]: Calling symbol
110     "circuit.circom":120:24
111
112 120    component outSel = Mux(64);
113           ~~~~~ Calling unknown symbol
114
115 previous errors were found

```

Listing 102: Circom compiler error log

Explanation: In this example, ‘IsZero(64)’ is invalid because ‘IsZero’ expects 0 template parameters, and nearly all of ‘Div(64)’, ‘Mul(64)’, ‘Add(64)’, ‘Sub(64)’, and ‘Mux(64)’ are failing because those symbols are not defined in the imported files. The circuit is mixing valid comparator templates like ‘GreaterEqThan(64)’ with nonexistent arithmetic/mux component constructors, so those

computations must be rewritten using normal arithmetic constraints or real imported templates.

B-6 Undeclared Symbol (T2021)

Description: This error class happens when Circom code refers to a name that was never declared in the current scope, or uses a scalar signal as if it were an array/object. Common causes are typos, missing declarations, loop-scope mistakes, or trying to index individually declared signals like ‘x0, x1, x2’ with ‘x[i]’. To fix it, make sure every referenced symbol is declared before use and that its shape matches how you access it: either declare true arrays such as ‘signal input c[5]; signal input s[5];’ or rewrite the code to use the explicit names ‘c0...c4’ and ‘s0...s4’.

Original Error Tag: T2021 » Undeclared-Symbol

Example:

```
1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)
```

Listing 103: Source Python shuffle_integrity_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "circomlib/comparators.circom";
4 include "circomlib/aliascheck.circom";
5
6 template ShuffleIntegrity() {
7     signal input c0, c1, c2, c3, c4;
8     signal input s0, s1, s2, s3, s4;
9     signal input sd; // unused but kept for interface
10    signal output out;
11
12    // Sum check
13    signal sum_c;
14    signal sum_s;
15    sum_c <== c0 + c1 + c2 + c3 + c4;
16    sum_s <== s0 + s1 + s2 + s3 + s4;
17    sum_c - sum_s == 0;
18
19    // Sum of squares check
20    signal sum_sq_c;
21    signal sum_sq_s;
22    sum_sq_c <== c0*c0 + c1*c1 + c2*c2 + c3*c3 + c4*c4;
23    sum_sq_s <== s0*s0 + s1*s1 + s2*s2 + s3*s3 + s4*s4;
24    sum_sq_c - sum_sq_s == 0;
25
26    // Equality checks s0 vs c[i] for iv calculation
27    signal eq[5];
28    component isZero[5];
29    for (var i = 0; i < 5; i++) {
```

```

30     isZero[i] = IsZero();
31     isZero[i].in <== s0 - c[i];
32     eq[i] <== isZero[i].out;
33 }
34
35 // OR tree for found_flag (if any c[i] equals s0)
36 signal or01, or23, or0123, found_flag;
37 or01 <== eq[0] + eq[1] - eq[0]*eq[1];
38 or23 <== eq[2] + eq[3] - eq[2]*eq[3];
39 or0123 <== or01 + or23 - or01*or23;
40 found_flag <== or0123 + eq[4] - or0123*eq[4];
41
42 // iv = 100 if found_flag = 1, else 0
43 signal iv;
44 iv <== 100 * found_flag;
45
46 // Inversion count: count pairs where s[i] > s[j] for i < j
47 signal inv;
48 inv <== 0;
49
50 // We need 10 comparisons for all i<j pairs
51 var pair_index = 0;
52 for (var i = 0; i < 5; i++) {
53     for (var j = i+1; j < 5; j++) {
54         // Use GreaterThan for strict comparison (s[i] > s[j])
55         component gt = GreaterThan(64);
56         gt.in[0] <== s[i];
57         gt.in[1] <== s[j];
58
59         // Add to inversion count
60         inv <== inv + gt.out;
61         pair_index++;
62     }
63 }
64
65 // Output = inv + iv
66 out <== inv + iv;
67 }
68
69 component main = ShuffleIntegrity();

```

Listing 104: Faulty translated Circom code with Undeclared Symbol (T2021)

```

1 error[T2021]: Undeclared symbol
2   "circuit.circom":30:31
3
4 30         isZero[i].in <== s0 - c[i];
5           ~~~~~ Using unknown symbol
6
7 error[T2021]: Undeclared symbol
8   "circuit.circom":55:26
9
10 55         gt.in[0] <== s[i];
11           ~~~~~ Using unknown symbol
12
13 error[T2021]: Undeclared symbol
14   "circuit.circom":56:26
15

```

```

16 56          gt.in[1] <== s[j];
17          ^^^^^ Using unknown symbol
18
19 previous errors were found

```

Listing 105: Circom compiler error log

Explanation: In this example, ‘c’ and ‘s’ are never declared as arrays, but the code uses ‘c[i]’, ‘s[i]’, and ‘s[j]’ inside loops. The circuit only declares ‘c0...c4’ and ‘s0...s4’, so the compiler reports ‘c’ and ‘s’ as unknown symbols; changing the inputs to arrays or replacing the indexed accesses with the explicit signal names will resolve it.

B-7 T3001 Other (T3001)

Description: This error happens when a Circom ‘function’ is used with circuit-level objects such as ‘signal’, ‘component’, or template-related operators. In Circom, functions are purely compile-time helpers for ‘var’ values, so they cannot iterate over, inspect, or compute with signal arrays the way templates can. To fix it, move the logic into a ‘template’, or rewrite it inline in the circuit using explicit signal constraints and fixed compile-time sizes instead of passing signals into a function.

Original Error Tag: T2001

Example:

```

1 def xor_cipher_block(p1, p2, k1, k2):
2     return [(p1 ^ k1) * 100 + (p2 ^ k2)]

```

Listing 106: Source Python xor_cipher_block implementation

```

1 include "circomlib/circuits/bitify.circom";
2
3 template XorCipherBlock() {
4     signal input p1;
5     signal input p2;
6     signal input k1;
7     signal input k2;
8     signal output out;
9
10    // Decompose inputs into bits (64-bit assumed)
11    component p1_bits = Num2Bits(64);
12    component k1_bits = Num2Bits(64);
13    component p2_bits = Num2Bits(64);
14    component k2_bits = Num2Bits(64);
15
16    p1_bits.in <== p1;
17    k1_bits.in <== k1;
18    p2_bits.in <== p2;
19    k2_bits.in <== k2;
20
21    // Compute p1 ^ k1
22    signal xor1[64];
23    signal x1; // reconstructed value of p1 ^ k1
24
25    for (var i = 0; i < 64; i++) {
26        // XOR: a + b - 2*a*b
27        xor1[i] <== p1_bits.out[i] + k1_bits.out[i] - 2 * p1_bits.out[i] * k1_bits.out[i];

```

```

28 }
29
30 // Reconstruct x1 from bits
31 x1 <== sumBits(xor1);
32
33 // Compute p2 ^ k2
34 signal xor2[64];
35 signal x2;
36
37 for (var i = 0; i < 64; i++) {
38     xor2[i] <== p2_bits.out[i] + k2_bits.out[i] - 2 * p2_bits.out[i] * k2_bits.out
39     [i];
40 }
41
42 x2 <== sumBits(xor2);
43
44 // Final output: (p1 ^ k1) * 100 + (p2 ^ k2)
45 out <== x1 * 100 + x2;
46
47 // Helper function to convert bit array to integer
48 function sumBits(bits) {
49     var res = 0;
50     var power = 1;
51     for (var i = 0; i < bits.length; i++) {
52         res += bits[i] * power;
53         power *= 2;
54     }
55     return res;
56 }
57
58 component main = XorCipherBlock();

```

Listing 107: Faulty translated Circom code with T3001 Other (T3001)

```

1 error[T2001]: Function uses component operators
2     "circuit.circom":53:25
3
4 53     for (var i = 0; i < bits.length; i++) {
5         ~~~~~~ Template operator found
6
7 previous errors were found

```

Listing 108: Circom compiler error log

Explanation: In this example, ‘sumBits(xor1)’ passes a signal array into the ‘sumBits’ function, and inside that function ‘bits.length’ is used as if ‘bits’ were a normal compile-time array. Since ‘xor1’ is a signal array, Circom reports ‘Template operator found’; the reconstruction of the integer from bits should be done directly in the template or in a dedicated template parameterized by the fixed bit width ‘64’.

C Non-Quadratic Constraints

C-1 Topological Sequence Violation (T3001)

Description: This error class happens when a circuit uses a signal before the compiler has seen any constraint or assignment that defines it. In Circom, signals must be introduced in a valid dependency order so that every read comes after initialization; otherwise the compiler reports an invalid access / topological order violation. The usual fix is to assign or constrain the signal earlier, or restructure the logic by adding the missing computation so every consumer depends only on already-defined signals.

Original Error Tag: Flow: Sequence Violation (Topological Order)

Example:

```
1 def validator_slashing_weight(balance, total_slashed, total_active):
2     if total_active == 0: return [0]
3     penalty = (balance * 3 * total_slashed) // total_active
4     return [penalty if penalty < balance else balance]
```

Listing 109: Source Python validator_slashing_weight implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template validator_slashing_weight() {
4     signal input bal;
5     signal input ts;
6     signal input ta;
7     signal output out[1];
8
9     signal num;
10    signal q;
11    signal r;
12    signal nonzero;
13    signal taOrOne;
14    signal capped;
15
16    component isTaZero = IsZero();
17    isTaZero.in <== ta;
18
19    num <== bal * 3 * ts;
20    nonzero <== 1 - isTaZero.out;
21    taOrOne <== ta + isTaZero.out;
22
23    component remLtDen = LessThan(64);
24    remLtDen.in[0] <== r;
25    remLtDen.in[1] <== taOrOne;
26
27    nonzero * (num - ta * q - r) == 0;
28    isTaZero.out * q == 0;
29
30    component qLtBal = LessThan(64);
31    qLtBal.in[0] <== q;
32    qLtBal.in[1] <== bal;
33
34    capped <== qLtBal.out * q + (1 - qLtBal.out) * bal;
35    out[0] <== nonzero * capped;
36 }
37
```



```
38 component main = validator_slashing_weight();
```

Listing 110: Faulty translated Circom code with Topological Sequence Violation (T3001)

```
1 error[T3001]: Exception caused by invalid access: trying to access to a signal that is
  not initialized
2 "circuit.circom":26:24
3
4 26      remLtDen.in[0] <= r;
5           ^ found here
6
7      = call trace:
8        ->validator_slashing_weight
9
10 previous errors were found
```

Listing 111: Circom compiler error log

Explanation: In this example, ‘r’ is connected into ‘remLtDen.in[0]’ before ‘r’ has ever been assigned or constrained anywhere in the circuit. The code intends ‘q’ and ‘r’ to represent quotient and remainder from dividing ‘num’ by ‘ta’, but neither is actually initialized before use, so you need to define them first and then apply the comparison and arithmetic constraints.

C-2 Forbidden Bitwise Operation (T3001)

Description: This error happens when a circuit uses an operation that cannot be expressed as a quadratic constraint in Circom, such as bitwise operators on unconstrained field elements. To fix it, rewrite the logic using arithmetic constraints that are R1CS-friendly, typically by decomposing the value into bits with a bitification gadget (for example, ‘Num2Bits’) or by modeling the desired property directly with algebraic relations instead of ‘&’, ‘|’, ‘{ }’, shifts, or similar forbidden operations.

Original Error Tag: Alg: Forbidden Bitwise Op

Example:

```
1 def is_even(n):
2     return [1 if n % 2 == 0 else 0]
```

Listing 112: Source Python is_even implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template MyIsEven() {
4     signal input n;
5     signal output out;
6
7     signal mod2 <= n & 1;
8
9     component eq = IsEqual();
10    eq.in[0] <= mod2;
11    eq.in[1] <= 0;
12
13    out <= eq.out;
14 }
15
16 component main = MyIsEven();
```

Listing 113: Faulty translated Circom code with Forbidden Bitwise Operation (T3001)

```
1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":9:5
3
4 9      signal mod2 <= n & 1;
5      ~~~~~ found here
6
7 = call trace:
8   ->MyIsEven
9
10 previous errors were found
```

Listing 114: Circom compiler error log

Explanation: In this example, ‘ $n \& 1$ ’ is trying to read the least significant bit of ‘ n ’ using a bitwise AND, which Circom rejects as non-quadratic. To check whether ‘ n ’ is even, decompose ‘ n ’ into bits and constrain the lowest bit to be ‘0’, or express ‘ $n = 2k$ ’ for some witness ‘ k ’.

C-3 Constant/Linear (T3001)

Description: This error class happens when a Circom constraint contains a product of more than two non-constant signal expressions, making it non-quadratic. Circom only allows constraints that can be written in quadratic form, so the fix is to break the expression into smaller steps with intermediate signals, so each individual constraint has at most one signal-by-signal multiplication.

Original Error Tag: Alg: Constant/Linear

Example:

```
1 def validator_slashing_weight(balance, total_slashed, total_active):
2     if total_active == 0: return [0]
3     penalty = (balance * 3 * total_slashed) // total_active
4     return [penalty if penalty < balance else balance]
```

Listing 115: Source Python validator_slashing_weight implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4 include "node_modules/circomlib/circuits/bitify.circom";
5
6 template validator_slashing_weight(nBits) {
7     signal input bal;
8     signal input ts;
9     signal input ta;
10
11     signal output out;
12
13     // Zero-check
14     signal isZeroTa;
15     signal nonZeroTa;
16     signal taSafe;
17
18     // Numerator
```

```

19  signal t1;
20  signal num;
21
22  // Division witness
23  signal q;
24  signal r;
25
26  // Min selection
27  signal lt_bal;
28  signal min_q_bal;
29
30  // ta == 0 ?
31  component iz = IsZero();
32  iz.in <== ta;
33  isZeroTa <== iz.out;
34  nonZeroTa <== 1 - isZeroTa;
35
36  // Safe denominator:
37  // - if ta != 0, taSafe = ta
38  // - if ta == 0, taSafe = 1
39  taSafe <== ta + isZeroTa;
40
41  // num = 3 * bal * ts
42  t1 <== bal * ts;
43  num <== 3 * t1;
44
45  // Witness hints for quotient and remainder
46  // If ta == 0, force q = 0 and r = 0 via the multiplier nonZeroTa.
47  q <-- nonZeroTa * (num \ taSafe);
48  r <-- nonZeroTa * (num % taSafe);
49
50  // Range constraints
51  component balBits = Num2Bits(nBits);
52  balBits.in <== bal;
53
54  component tsBits = Num2Bits(nBits);
55  tsBits.in <== ts;
56
57  component taBits = Num2Bits(nBits);
58  taBits.in <== ta;
59
60  component qBits = Num2Bits(nBits);
61  qBits.in <== q;
62
63  component rBits = Num2Bits(nBits);
64  rBits.in <== r;
65
66  // Enforce q = 0 and r = 0 when ta == 0
67  isZeroTa * q === 0;
68  isZeroTa * r === 0;
69
70  // Enforce division relation when ta != 0:
71  // num = ta * q + r
72  nonZeroTa * (num - ta * q - r) === 0;
73
74  // Enforce remainder bound:
75  // if ta != 0 => r < ta
76  // if ta == 0 => r < 1, which holds since r = 0

```

```

77 component rLtTaSafe = LessThan(nBits);
78 rLtTaSafe.in[0] <= r;
79 rLtTaSafe.in[1] <= taSafe;
80 rLtTaSafe.out == 1;
81
82 // lt_bal = 1 iff q < bal
83 component qLtBal = LessThan(nBits);
84 qLtBal.in[0] <= q;
85 qLtBal.in[1] <= bal;
86 lt_bal <= qLtBal.out;
87
88 // min(q, bal)
89 min_q_bal <= bal + lt_bal * (q - bal);
90
91 // out = 0 if ta == 0, else min(q, bal)
92 out <= nonZeroTa * min_q_bal;
93 }
94
95 component main = validator_slashing_weight(64);

```

Listing 116: Faulty translated Circom code with Constant/Linear (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":71:5
3
4 71      nonZeroTa * (num - ta * q - r) == 0;
5      ~~~~~ found here
6
7      = call trace:
8        ->validator_slashing_weight
9
10 previous errors were found

```

Listing 117: Circom compiler error log

Explanation: In this example, ‘nonZeroTa * (num - ta * q - r) == 0’ is invalid because ‘ta * q’ is already a multiplication of signals, and multiplying that whole term again by ‘nonZeroTa’ creates a cubic expression. A typical fix is to first define an intermediate like ‘prod <= ta * q’, then enforce ‘nonZeroTa * (num - prod - r) == 0’, which keeps each constraint quadratic.

C-4 Parallel Sum (Needs Intermediate Signal) (T3001)

Description: This error class happens when a Circom constraint contains more than one multiplication in the same expression, making it non-quadratic. Circom only accepts constraints that can be written with at most one product of linear terms, so the usual fix is to split the computation into intermediate signals and enforce each multiplication separately before doing additions in a final linear step.

Original Error Tag: Alg: Parallel Sum - Need Intermediate Signal

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2   bp = 0
3   if p1 == 1 and p2 == 1: bp = 3
4   elif p1 == 1 and p2 == 0: bp = 0
5   elif p1 == 0 and p2 == 1: bp = 5

```

```

6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 118: Source Python iterated_game_memory_score implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template iterated_game_memory_score() {
6     signal input p1;
7     signal input p2;
8     signal input h1;
9     signal input h2;
10    signal input h3;
11    signal input r;
12
13    signal output out;
14
15    // Equality checks for current actions
16    component p1_is_0 = IsEqual();
17    component p1_is_1 = IsEqual();
18    component p2_is_0 = IsEqual();
19    component p2_is_1 = IsEqual();
20
21    p1_is_0.in[0] <== p1;
22    p1_is_0.in[1] <== 0;
23
24    p1_is_1.in[0] <== p1;
25    p1_is_1.in[1] <== 1;
26
27    p2_is_0.in[0] <== p2;
28    p2_is_0.in[1] <== 0;
29
30    p2_is_1.in[0] <== p2;
31    p2_is_1.in[1] <== 1;
32
33    // Payoff table selection
34    signal case11 <== p1_is_1.out * p2_is_1.out; // (1,1) -> 3
35    signal case10 <== p1_is_1.out * p2_is_0.out; // (1,0) -> 0
36    signal case01 <== p1_is_0.out * p2_is_1.out; // (0,1) -> 5
37    signal case00 <== p1_is_0.out * p2_is_0.out; // (0,0) -> 1
38
39    signal bp <== 3 * case11 + 0 * case10 + 5 * case01 + 1 * case00;
40
41    // History total
42    signal ts <== h1 + h2 + h3;
43
44    // Penalty indicator: p1 = 0 and r > 5
45    component r_gt_5 = GreaterThan(64);
46    r_gt_5.in[0] <== r;
47    r_gt_5.in[1] <== 5;
48
49    signal pe <== p1_is_0.out * r_gt_5.out;
50

```

```

51 // Raw score parts
52 signal baseScore <== 100 * bp + 20 * ts;
53
54 // Apply penalty with clamp at zero:
55 // if pe = 0 -> out = baseScore
56 // if pe = 1 -> out = max(baseScore - 50, 0)
57 component base_ge_50 = GreaterEqThan(64);
58 base_ge_50.in[0] <== baseScore;
59 base_ge_50.in[1] <== 50;
60
61 signal penalizedClamped <== base_ge_50.out * (baseScore - 50);
62 signal noPenaltySel <== 1 - pe;
63
64 out <== noPenaltySel * baseScore + pe * penalizedClamped;
65 }
66
67 component main = iterated_game_memory_score();

```

Listing 119: Faulty translated Circom code with Parallel Sum (Needs Intermediate Signal) (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":64:5
3
4 64      out <== noPenaltySel * baseScore + pe * penalizedClamped;
5         ~~~~~ found here
6
7 = call trace:
8   ->iterated_game_memory_score
9
10 previous errors were found

```

Listing 120: Circom compiler error log

Explanation: In this example, ‘out <== noPenaltySel * baseScore + pe * penalizedClamped;’ contains two products added together, so it violates the quadratic-form rule. Define intermediate signals for ‘noPenaltySel * baseScore’ and ‘pe * penalizedClamped’, then set ‘out’ equal to their sum.

C-5 Comparison Logic (Non-Algebraic) (T3001)

Description: This error happens when Circom encounters non-quadratic logic, such as direct use of comparison operators like ‘<’, ‘>’, ‘<=’, or ‘>=’ inside signal assignments or constraints. Circom constraints must be expressed as quadratic equations over field elements, so comparisons must be implemented with dedicated comparator gadgets from libraries like ‘circomlib’, and their boolean outputs should then be used in arithmetic form to select values or enforce conditions.

Original Error Tag: Alg: Comparison Logic (Non-Algebraic)

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2   bp = 0
3   if p1 == 1 and p2 == 1: bp = 3
4   elif p1 == 1 and p2 == 0: bp = 0
5   elif p1 == 0 and p2 == 1: bp = 5
6   else: bp = 1
7   ts = h1 + h2 + h3
8   pe = 1 if (p1 == 0 and r > 5) else 0

```

```

9   fs = (bp * 100) + (ts * 20) - (pe * 50)
10  return int(fs if fs > 0 else 0)

```

Listing 121: Source Python iterated_game_memory_score implementation

```

1  include "node_modules/circomlib/circuits/comparators.circom";
2
3  template iterated_game_memory_score() {
4      signal input p1;
5      signal input p2;
6      signal input h1;
7      signal input h2;
8      signal input h3;
9      signal input r;
10     signal output out;
11     component gte1 = GreaterEqThan(64);
12     component gte2 = GreaterEqThan(64);
13     component gte3 = GreaterEqThan(64);
14     component gte4 = GreaterEqThan(64);
15     gte1.in[0] <== p1;
16     gte1.in[1] <== 1;
17     gte2.in[0] <== p2;
18     gte2.in[1] <== 1;
19     gte3.in[0] <== p1;
20     gte3.in[1] <== 0;
21     gte4.in[0] <== p2;
22     gte4.in[1] <== 0;
23     signal cond1 <== gte1.out;
24     signal cond2 <== gte2.out;
25     signal cond3 <== gte3.out;
26     signal cond4 <== gte4.out;
27     signal bp <== cond1 * 3 + (1 - cond1) * (1 - cond2 - cond3);
28     signal ts <== h1 + h2 + h3;
29     signal pe <== cond4 * 1 + (1 - cond4) * 0;
30     signal fs <== (bp * 100) + (ts * 20) - (pe * 50);
31     signal cond5 <== fs < 0;
32     out <== cond5 * 0 + (1 - cond5) * fs;
33 }
34 component main = iterated_game_memory_score();

```

Listing 122: Faulty translated Circom code with Comparison Logic (Non-Algebraic) (T3001)

```

1  error[T3001]: Non quadratic constraints are not allowed!
2      "circuit.circom":33:5
3
4  33      signal cond5 <== fs < 0;
5          ~~~~~ found here
6
7      = call trace:
8          ->iterated_game_memory_score
9
10 previous errors were found

```

Listing 123: Circom compiler error log

Explanation: In this example, ‘signal cond5 <== fs < 0;’ is trying to compare ‘fs’ to zero directly, which Circom cannot translate into a valid quadratic constraint. To fix it, compute whether ‘fs’ is

negative using a comparator component or redesign the score so negativity is represented through a valid boolean signal, then use that signal to choose between '0' and 'fs'.

C-6 High-Order Polynomial (T3001)

Description: This error happens when a Circom constraint multiplies too many non-constant terms together, producing a polynomial of degree 3 or higher. Circom only allows quadratic constraints, so you must refactor the expression into a sequence of intermediate signals where each constraint contains at most one multiplication of two signal expressions; in practice, break chained products into separate assignments and reuse boolean selectors or helper signals.

Original Error Tag: Alg: High-Order Polynomial (Degree 3+)

Example:

```

1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)

```

Listing 124: Source Python iterated_game_memory_score implementation

```

1 include "circomlib/circuits/comparators.circom";
2
3 template iterated_game_memory_score() {
4     // Public inputs
5     signal input p1;
6     signal input p2;
7     signal input h1;
8     signal input h2;
9     signal input h3;
10    signal input r;
11
12    // Output
13    signal output out;
14
15    // Enforce p1, p2 are bits
16    p1 * (p1 - 1) === 0;
17    p2 * (p2 - 1) === 0;
18
19    // Equality indicators (direct assignment since p1,p2 are bits)
20    signal eq_p1_1;
21    signal eq_p2_1;
22    signal eq_p1_0;
23    signal eq_p2_0;
24
25    eq_p1_1 <== p1;
26    eq_p1_0 <== 1 - p1;
27    eq_p2_1 <== p2;
28    eq_p2_0 <== 1 - p2;
29

```



```

30 // Boolean constraints (redundant due to above, but included for clarity)
31 eq_p1_1 * (eq_p1_1 - 1) === 0;
32 eq_p2_1 * (eq_p2_1 - 1) === 0;
33 eq_p1_0 * (eq_p1_0 - 1) === 0;
34 eq_p2_0 * (eq_p2_0 - 1) === 0;
35
36 // AND conditions
37 signal and1;
38 signal and2;
39 signal and3;
40
41 and1 <== eq_p1_1 * eq_p2_1; // p1=1, p2=1
42 and2 <== eq_p1_1 * eq_p2_0; // p1=1, p2=0
43 and3 <== eq_p1_0 * eq_p2_1; // p1=0, p2=1
44
45 // Selection logic (mutually exclusive)
46 signal sel1;
47 signal sel2;
48 signal sel3;
49
50 sel1 <== and1;
51 sel2 <== and2 * (1 - sel1);
52 sel3 <== and3 * (1 - sel1) * (1 - sel2);
53
54 // Base points
55 signal bp;
56 bp <== 3*sel1 + 0*sel2 + 5*sel3 + 1*(1 - sel1 - sel2 - sel3);
57
58 // Total history score
59 signal ts;
60 ts <== h1 + h2 + h3;
61
62 // Penalty condition: p1 == 0 && r > 5
63 // Compute gt_r_5 = (r > 5) ? 1 : 0 using LessThan
64 component gtComp = LessThan(16);
65 gtComp.in[0] <== 5;
66 gtComp.in[1] <== r;
67 signal gt_r_5 <== gtComp.out; // 1 if 5 < r, i.e., r > 5
68
69 // Boolean constraint for gt_r_5
70 gt_r_5 * (gt_r_5 - 1) === 0;
71
72 signal pe;
73 pe <== eq_p1_0 * gt_r_5;
74
75 // Final score before clamp
76 signal fs;
77 fs <== bp * 100 + ts * 20 - pe * 50;
78
79 // Clamp negative scores to zero: out = max(0, fs)
80 // Since fs can be negative, we use offset method for comparison
81 // Assume fs >= -100, so fs + 100 >= 0
82 signal fs_offset;
83 fs_offset <== fs + 100;
84
85 component ltComp = LessThan(16);
86 ltComp.in[0] <== fs_offset;
87 ltComp.in[1] <== 100; // fs + 100 < 100 => fs < 0

```

```

88     signal lt_fs_0 <== ltComp.out;
89
90     // Boolean constraint
91     lt_fs_0 * (lt_fs_0 - 1) === 0;
92
93     // Output
94     out <== (1 - lt_fs_0) * fs;
95
96     // Additional constraint: if lt_fs_0=1, out=0
97     out * lt_fs_0 === 0;
98 }
99
100 component main = iterated_game_memory_score();

```

Listing 125: Faulty translated Circom code with High-Order Polynomial (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":54:5
3
4 54     sel3 <== and3 * (1 - sel1) * (1 - sel2);
5      ~~~~~ found here
6
7     = call trace:
8       ->iterated_game_memory_score
9
10 previous errors were found

```

Listing 126: Circom compiler error log

Explanation: In this example, ‘sel3 <== and3 * (1 - sel1) * (1 - sel2);’ is cubic because it multiplies three signal-dependent factors. Even if these values are boolean, Circom still treats the expression as degree 3, so ‘sel3’ must be computed through one or more intermediate signals, such as first multiplying two factors and then multiplying the result by the third in a separate quadratic step.

C-7 Forbidden Division (T3001)

Description: This error class happens when a Circom constraint is not quadratic, meaning it cannot be expressed in the allowed form using only additions and multiplications of signals. Direct division by a signal is forbidden because it introduces a non-quadratic relation; the usual fix is to rewrite division as multiplication by an inverse witness, e.g. ‘a / b’ becomes ‘q <- a * inv’ together with constraints like ‘b * inv === 1’ and ‘q * b === a’, plus any needed zero-denominator handling.

Original Error Tag: Alg: Forbidden Division

Example:

```

1 def anova_f_test_logic(g1_s, g1_n, g2_s, g2_n, total_ss):
2     m1, m2 = g1_s // g1_n, g2_s // g2_n
3     gm = (g1_s + g2_s) // (g1_n + g2_n)
4     ssb = g1_n * (m1 - gm)**2 + g2_n * (m2 - gm)**2
5     ssw = total_ss - ssb
6     if ssw <= 0: return int(gm)
7     return int((ssb * 100) // ssw + gm)

```

Listing 127: Source Python anova_f_test_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template anova_f_test_logic() {
4     signal input g1s;
5     signal input g1n;
6     signal input g2s;
7     signal input g2n;
8     signal input tss;
9     signal output out;
10
11     signal m1;
12     signal m2;
13     signal gm;
14     signal ssb;
15     signal ssw;
16
17     m1 <== g1s / g1n;
18     m2 <== g2s / g2n;
19     gm <== (g1s + g2s) / (g1n + g2n);
20
21     ssb <== g1n * (m1 - gm) * (m1 - gm) + g2n * (m2 - gm) * (m2 - gm);
22     ssw <== tss - ssb;
23
24     component lte = LessEqThan(64);
25     lte.in[0] <== ssw;
26     lte.in[1] <== 0;
27
28     signal sel;
29     sel <== lte.out;
30
31     signal inv;
32     signal frac;
33     signal expr;
34
35     ssw * inv == 1 - sel;
36     sel * ssw == 0;
37
38     frac <== ssb * 100 * inv;
39     expr <== frac + gm;
40
41     out <== sel * gm + (1 - sel) * expr;
42 }
43
44 component main = anova_f_test_logic();

```

Listing 128: Faulty translated Circom code with Forbidden Division (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2     "circuit.circom":19:5
3
4 19     m1 <== g1s / g1n;
5     ~~~~~ found here
6
7     = call trace:
8     ->anova_f_test_logic
9
10 previous errors were found

```

Listing 129: Circom compiler error log

Explanation: In this example, ‘ $m1 \leq g1s / g1n$ ’, ‘ $m2 \leq g2s / g2n$ ’, and ‘ $gm \leq (g1s + g2s) / (g1n + g2n)$ ’ all perform direct signal division, so Circom rejects them as non-quadratic. You need to introduce inverse signals for ‘ $g1n$ ’, ‘ $g2n$ ’, and ‘ $g1n + g2n$ ’ and constrain those products instead, similar to how ‘ ssw ’ is later handled with ‘ inv ’ and ‘ sel ’.

C-8 Multiple Definition/Re-assignment (T3001)

Description: This error class happens when a Circom signal is assigned more than once. In Circom, signals are single-assignment constraints, so once a signal has been defined with ‘ $\leq$ ’, ‘ $\geq$ ’, or equivalent constrained assignment, you cannot later overwrite it like mutable state in an imperative language. The usual fix is to introduce a new signal for each stage of a computation, such as ‘ $ema0$ ’, ‘ $ema1$ ’, ‘ $ema2$ ’, etc., or restructure the logic so each signal is defined exactly once and later values depend on earlier ones rather than reassigning them.

Original Error Tag: Assign: Multiple Definition/Re-assign

Example:

```
1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)
```

Listing 130: Source Python ema_signal_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4 include "node_modules/circomlib/circuits/bitify.circom";
5
6 template SafeDiv() {
7     signal input numerator;
8     signal input denominator;
9     signal output quotient;
10    signal output remainder;
11
12    // Handle zero division
13    component isZero = IsZero();
14    isZero.in <== denominator;
15
16    // For non-zero denominator: numerator = denominator * quotient + remainder
17    // For zero denominator: quotient = 0, remainder = numerator
18    signal safeDenominator <== denominator + isZero.out;
19
20    quotient <-- numerator / safeDenominator;
21    remainder <-- numerator % safeDenominator;
22
23    // Verify the division
24    numerator == denominator * quotient + remainder;
25 }
```

```

26
27 template EMA_Signal_Logic() {
28     signal input p0;
29     signal input p1;
30     signal input p2;
31     signal input p3;
32     signal input p4;
33     signal output out;
34
35     // Constants
36     var alpha = 40;
37     var scale = 100;
38
39     // Initialize EMA with first price scaled by 100
40     signal ema <== p0 * scale;
41
42     // Process remaining prices
43     component div1 = SafeDiv();
44     component div2 = SafeDiv();
45     component div3 = SafeDiv();
46     component div4 = SafeDiv();
47
48     // EMA calculation for each step
49     // ema = (price * alpha) + (ema * (100 - alpha) / 100)
50
51     // Step 1: p1
52     signal term1_1 <== p1 * alpha;
53     signal term2_1 <== ema * (scale - alpha);
54     div1.numerator <== term2_1;
55     div1.denominator <== scale;
56     ema <== term1_1 + div1.quotient;
57
58     // Step 2: p2
59     signal term1_2 <== p2 * alpha;
60     signal term2_2 <== ema * (scale - alpha);
61     div2.numerator <== term2_2;
62     div2.denominator <== scale;
63     ema <== term1_2 + div2.quotient;
64
65     // Step 3: p3
66     signal term1_3 <== p3 * alpha;
67     signal term2_3 <== ema * (scale - alpha);
68     div3.numerator <== term2_3;
69     div3.denominator <== scale;
70     ema <== term1_3 + div3.quotient;
71
72     // Step 4: p4
73     signal term1_4 <== p4 * alpha;
74     signal term2_4 <== ema * (scale - alpha);
75     div4.numerator <== term2_4;
76     div4.denominator <== scale;
77     ema <== term1_4 + div4.quotient;
78
79     out <== ema;
80 }
81
82 component main = EMA_Signal_Logic();

```

Listing 131: Faulty translated Circom code with Multiple Definition/Re-assignment (T3001)

```
1 error[T3001]: Exception caused by invalid assignment
2 Signal has been already assigned
3   "circuit.circom":39:5
4
5 39     signal ema <== p0 * scale;
6     ----- This is the previous assignment to the variable
7
8 55     ema <== term1_1 + div1.quotient;
9     ~~~~~ found here
10
11     = call trace:
12       ->EMA_Signal_Logic
13
14 previous errors were found
```

Listing 132: Circom compiler error log

Explanation: In this example, ‘ema’ is first assigned at initialization with ‘p0 * scale’, and then it is assigned again at each EMA update step, starting at line 55. That makes Circom reject the circuit because ‘ema’ is being used like a mutable variable; instead, each EMA iteration should write into a fresh signal and the final one should be connected to ‘out’.

C-9 Parallel Sum (Needs Simplification) (T3001)

Description: This error happens when a Circom constraint becomes higher than quadratic, usually because several multiplicative terms are expanded and then added together in one expression. Circom only allows constraints of the form constant/linear terms or a single product of two linear expressions, so the fix is to factor the expression or break it into intermediate signals so each ‘<==’ introduces at most one multiplication layer.

Original Error Tag: Alg: Parallel Sum - Need Factorization

Example:

```
1 def iterated_game_memory_score(p1, p2, h1, h2, h3, r):
2     bp = 0
3     if p1 == 1 and p2 == 1: bp = 3
4     elif p1 == 1 and p2 == 0: bp = 0
5     elif p1 == 0 and p2 == 1: bp = 5
6     else: bp = 1
7     ts = h1 + h2 + h3
8     pe = 1 if (p1 == 0 and r > 5) else 0
9     fs = (bp * 100) + (ts * 20) - (pe * 50)
10    return int(fs if fs > 0 else 0)
```

Listing 133: Source Python iterated_game_memory_score implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template iterated_game_memory_score() {
4     signal input p1; // player 1 strategy bit
5     signal input p2; // player 2 strategy bit
```

```

6  signal input h1; // historical state 1
7  signal input h2; // historical state 2
8  signal input h3; // historical state 3
9  signal input r;  // reference threshold
10
11  signal output out;
12
13  // Calculate base payoff based on strategy matrix
14  // Payoff table: [0,1,3,5] for combinations (0,0), (0,1), (1,0), (1,1)
15  signal base_payoff;
16  base_payoff <== p1 * p2 * 5 + p1 * (1 - p2) * 3 + (1 - p1) * p2 * 1 + (1 - p1) *
    (1 - p2) * 0;
17
18  // Calculate historical sum
19  signal hist_sum;
20  hist_sum <== h1 + h2 + h3;
21
22  // Determine penalty condition: player 1 defects (p1=1) and r > threshold
23  // For simplicity, assume threshold is 0, so we just check if r > 0 and p1 = 1
24  component r_gt_0 = GreaterThan(64);
25  r_gt_0.in[0] <== r;
26  r_gt_0.in[1] <== 0;
27  signal r_positive <== r_gt_0.out;
28
29  signal penalty_condition;
30  penalty_condition <== p1 * r_positive; // Both p1=1 and r>0 must be true
31
32  // Calculate penalty: 1 if condition met, 0 otherwise
33  signal penalty;
34  penalty <== penalty_condition;
35
36  // Calculate weighted components
37  signal base_weighted;
38  base_weighted <== base_payoff * 100;
39
40  signal hist_weighted;
41  hist_weighted <== hist_sum * 20;
42
43  signal penalty_weighted;
44  penalty_weighted <== penalty * 50;
45
46  // Calculate raw score: 100*base + 20*hist - 50*penalty
47  signal raw_score;
48  raw_score <== base_weighted + hist_weighted - penalty_weighted;
49
50  // Ensure non-negativity: max(0, raw_score)
51  component non_neg_check = GreaterEqThan(64);
52  non_neg_check.in[0] <== raw_score;
53  non_neg_check.in[1] <== 0;
54  signal is_non_negative <== non_neg_check.out;
55
56  out <== is_non_negative * raw_score;
57 }
58
59 component main = iterated_game_memory_score();

```

Listing 134: Faulty translated Circom code with Parallel Sum (Needs Simplification) (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":18:5
3
4 18      base_payoff <== p1 * p2 * 5 + p1 * (1 - p2) * 3 + (1 - p1) * p2 * 1 + (1 - p1)
      * (1 - p2) * 0;
5
6 .....
7      found here
8
9 = call trace:
10  ->iterated_game_memory_score
previous errors were found

```

Listing 135: Circom compiler error log

Explanation: In this example, the payoff lookup is written as a sum of multiple products like ‘ $p1 * p2 * 5$ ’ and ‘ $p1 * (1 - p2) * 3$ ’, which expands into a non-quadratic constraint for ‘base_payoff’. You should rewrite the payoff computation in factored form or via step-by-step intermediate signals, such as simplifying the boolean payoff formula before assigning it.

C-10 Forbidden Non-Polynomial Operator (T3001)

Description: This error class happens when a Circom constraint uses an operation that cannot be represented as a quadratic polynomial over the field, such as division, integer division, modulo, or other non-polynomial operators. In Circom, constraints must be rewritten using multiplication-based relations and explicit witnesses, so the usual fix is to introduce intermediate signals and enforce equations like ‘ $a = b * c$ ’ instead of writing ‘ $a = b / c$ ’, and if needed add range or nonzero checks to make the intended division semantics valid.

Original Error Tag: Alg: Forbidden Operator (Non-Polynomial)

Example:

```

1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2     fee = (amt_in * tax_bips) // 10000
3     amt_eff = amt_in - fee
4     # x * y = k
5     amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6     for i in range(exp):
7         amt_out = (amt_out * 9995) // 10000
8     if amt_out < 0: return 0
9     return int(amt_out)

```

Listing 136: Source Python fixed_point_swap_slippage implementation

```

1 pragma circom 2.1.0;
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/sign.circom";
4
5 template FixedPointMulDiv() {
6     signal input in[3];
7     signal output out;
8     out <== (in[0] * in[1]) \ in[2];
9 }
10

```



```

11 template fixed_point_swap_slippage(e) {
12     signal input ai;
13     signal input ri;
14     signal input ro;
15     signal input tb;
16     signal output out;
17
18     signal f;
19     f <== (ai * tb) \ 10000;
20
21     signal ae;
22     ae <== ai - f;
23
24     signal ao;
25     ao <== (ae * ro) \ (ri + ae);
26
27     component for_loop[e];
28     for (var i = 0; i < e; i++) {
29         for_loop[i] = FixedPointMulDiv();
30         if (i == 0) {
31             for_loop[i].in[0] <== ao;
32         } else {
33             for_loop[i].in[0] <== for_loop[i-1].out;
34         }
35         for_loop[i].in[1] <== 9995;
36         for_loop[i].in[2] <== 10000;
37     }
38
39     signal final_ao;
40     if (e > 0) {
41         final_ao <== for_loop[e-1].out;
42     } else {
43         final_ao <== ao;
44     }
45
46     component lt = LessThan(64);
47     lt.in[0] <== final_ao;
48     lt.in[1] <== 0;
49
50     out <== (1 - lt.out) * final_ao;
51 }
52
53 component main = fixed_point_swap_slippage(10);

```

Listing 137: Faulty translated Circom code with Forbidden Non-Polynomial Operator (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2 "circuit.circom":19:5
3
4 19     f <== (ai * tb) \ 10000;
5         ~~~~~ found here
6
7     = call trace:
8         ->fixed_point_swap_slippage
9
10 previous errors were found

```

Listing 138: Circom compiler error log

Explanation: In this example, the circuit uses ‘\{\}’ in expressions like ‘f <== (ai * tb) \{\} 10000;’ and ‘ao <== (ae * ro) \{\} (ri + ae);’, and Circom rejects these because integer division is not allowed inside constraints. Even the ‘FixedPointMulDiv’ helper still contains the same forbidden operator, so the fixed-point arithmetic must be redesigned using witness signals plus multiplicative constraints rather than direct division.

C-11 Other Initialization Gap (T3001)

Description: This error class occurs when a Circom signal is used in an expression before it has been given any value through an assignment or otherwise fully defined by the circuit’s construction. In practice, this usually means a witness value is being read too early, so the fix is to ensure every signal is initialized before any access to it, either by assigning it first, restructuring the computation order, or introducing proper intermediate values so all dependencies are defined before use.

Original Error Tag: Flow: General Initialization Gap

Example:

```
1 def cournot_competition_equilibrium(a, c, n, qo):
2     q = (a - c - qo) // 2
3     dummy = n // 1000
4     return int((q if q > 0 else 0) + dummy)
```

Listing 139: Source Python cournot_competition_equilibrium implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4 include "node_modules/circomlib/circuits/bitify.circom";
5
6 template CournotEquilibrium() {
7     signal input a;
8     signal input c;
9     signal input n_val;
10    signal input qo;
11    signal output out;
12
13    // num = a - c - qo
14    signal num;
15    num <== a - c - qo;
16
17    // q = floor(num / 2) (integer division)
18    signal q;
19    signal r_q;
20    r_q <-- num - 2*q;
21    // Constraint: num = 2*q + r_q
22    2*q + r_q === num;
23    // Constraint: 0 <= r_q < 2 (binary)
24    r_q * (r_q - 1) === 0;
25
26    // d = floor(n_val / 1000)
27    signal d;
28    signal r_d;
29    r_d <-- n_val - 1000*d;
30    // Constraint: n_val = 1000*d + r_d
31    1000*d + r_d === n_val;
```

```

32 // Constraint: 0 <= r_d < 1000
33 component rangeCheck = Num2Bits(10); // 2^10 = 1024 > 999
34 rangeCheck.in <= r_d;
35
36 // is_q_negative = 1 if q < 0 else 0
37 component ltZero = LessThan(64);
38 ltZero.in[0] <= q;
39 ltZero.in[1] <= 0;
40 signal is_q_negative;
41 is_q_negative <= ltZero.out;
42
43 // Binary constraint for is_q_negative
44 is_q_negative * (is_q_negative - 1) === 0;
45
46 // out = d + (1 - is_q_negative)*q
47 // which simplifies to: out = q + d - is_q_negative*q
48 out <= q + d - is_q_negative * q;
49 }
50
51 component main = CournotEquilibrium();

```

Listing 140: Faulty translated Circom code with Other Initialization Gap (T3001)

```

1 error[T3001]: Exception caused by invalid access: trying to access to a signal that is
  not initialized
2 "circuit.circom":19:21
3
4 19      r_q <-- num - 2*q;
5           ^ found here
6
7      = call trace:
8        ->CournotEquilibrium
9
10 previous errors were found

```

Listing 141: Circom compiler error log

Explanation: In this example, ‘q’ is declared but never assigned before it is read in ‘r_q <-- num - 2*q’; so Circom raises an invalid access error at that line. The circuit tries to define ‘q’ indirectly from the remainder relation, but Circom still requires ‘q’ to have an initialized value before it can appear on the right-hand side of an assignment.

C-12 Mixed Quadratic-Linear (T3001)

Description: This error class happens when a Circom constraint is not purely quadratic, meaning it contains too many multiplicative factors or mixes a product with additional linear structure in a single ‘===’ relation. Circom only allows constraints that can be expressed as ‘A * B + C = 0’ with each side linear in signals, so the usual fix is to break the computation into intermediate signals and enforce it through multiple quadratic constraints, often by isolating one multiplication per constraint and handling selector/zero-check logic separately.

Original Error Tag: Alg: Mixed Quadratic-Linear (a*b + c)

Example:

```

1 def validator_slashing_weight(balance, total_slashed, total_active):

```

```

2  if total_active == 0: return [0]
3  penalty = (balance * 3 * total_slashed) // total_active
4  return [penalty if penalty < balance else balance]

```

Listing 142: Source Python validator_slashing_weight implementation

```

1  pragma circom 2.1.0;
2  include "node_modules/circomlib/circuits/comparators.circom";
3  include "node_modules/circomlib/circuits/bitify.circom";
4
5  template ValidatorSlashingWeight(balance_bits, total_bits) {
6      signal input balance;
7      signal input total_slashed;
8      signal input total_active;
9      signal output penalty;
10
11      component isZero = IsZero();
12      isZero.in <== total_active;
13
14      signal balance_times_3 <== balance * 3;
15      signal numerator <== balance_times_3 * total_slashed;
16
17      component div = SafeDiv(total_bits);
18      div.numerator <== numerator;
19      div.denominator <== total_active;
20      div.isZero <== isZero.out;
21
22      signal calculated_penalty <== div.out;
23
24      component lt = LessThan(balance_bits);
25      lt.in[0] <== calculated_penalty;
26      lt.in[1] <== balance;
27
28      signal diff <== balance - calculated_penalty;
29      penalty <== calculated_penalty + (1 - lt.out) * diff;
30  }
31
32  template SafeDiv(bits) {
33      signal input numerator;
34      signal input denominator;
35      signal input isZero;
36      signal output out;
37
38      signal inverse;
39      inverse <-- 1 / denominator;
40
41      out <-- numerator * inverse * (1 - isZero);
42      out * denominator == numerator * (1 - isZero);
43
44      component num2bits = Num2Bits(bits);
45      num2bits.in <== out;
46  }
47
48  component main = ValidatorSlashingWeight(64, 128);

```

Listing 143: Faulty translated Circom code with Mixed Quadratic-Linear (T3001)

```

1 error[T3001]: Non quadratic constraints are not allowed!
2   "circuit.circom":42:5
3
4 42      out * denominator === numerator * (1 - isZero);
5      ~~~~~ found here
6
7      = call trace:
8        ->ValidatorSlashingWeight
9        ->SafeDiv
10
11 previous errors were found

```

Listing 144: Circom compiler error log

Explanation: In this example, the failing constraint ‘out * denominator === numerator * (1 - isZero)’ is non-quadratic because the right-hand side multiplies ‘numerator’ by ‘(1 - isZero)’ while the left-hand side already contains another product, creating a mixed quadratic-linear form across the equality. To fix it, introduce an intermediate signal such as ‘maskedNumerator <== numerator * (1 - isZero)’ and then constrain ‘out * denominator === maskedNumerator’, or otherwise restructure the zero-division logic into separate quadratic constraints.

C-13 Imperative Self-Update (T3001)

Description: This error happens when Circom code treats a ‘signal’ like a mutable variable and tries to assign to it more than once, such as writing ‘a <== ...’ and later ‘a <== a + 1’. In Circom, signals are single-assignment wires in a constraint system, not imperative program variables, so each signal must be defined exactly once. The fix is to introduce new intermediate signals for each step of the computation, or use arrays of signals to represent successive states, so every value in the update chain has its own unique signal.

Original Error Tag: Assign: Imperative Self-Update (a = a+1)

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 145: Source Python ema_signal_logic implementation

```

1 template emaSignalLogic() {
2     signal input p0;
3     signal input p1;
4     signal input p2;
5     signal input p3;
6     signal input p4;
7     signal output out;
8
9     signal prices[5];
10    prices[0] <== p0;
11    prices[1] <== p1;

```

```

12 prices[2] <== p2;
13 prices[3] <== p3;
14 prices[4] <== p4;
15
16 signal alpha <== 40;
17 signal ema <== prices[0] * 100;
18
19 // i = 1
20 ema <== (prices[1] * alpha) + (ema * (100 - alpha)) / 100;
21 // i = 2
22 ema <== (prices[2] * alpha) + (ema * (100 - alpha)) / 100;
23 // i = 3
24 ema <== (prices[3] * alpha) + (ema * (100 - alpha)) / 100;
25 // i = 4
26 ema <== (prices[4] * alpha) + (ema * (100 - alpha)) / 100;
27
28 out <== ema;
29 }
30
31 component main = emaSignalLogic();

```

Listing 146: Faulty translated Circom code with Imperative Self-Update (T3001)

```

1 error[T3001]: Exception caused by invalid assignment
2 Signal has been already assigned
3 "circuit.circom":21:5
4
5 21      signal ema <== prices[0] * 100;
6      ----- This is the previous assignment to the variable
7
8 24      ema <== (prices[1] * alpha) + (ema * (100 - alpha)) / 100;
9      ~~~~~ found here
10
11 = call trace:
12   ->emaSignalLogic
13
14 previous errors were found

```

Listing 147: Circom compiler error log

Explanation: In this example, ‘ema’ is first assigned ‘prices[0] * 100’, then reassigned several times as if it were being updated in a loop. That is invalid in Circom, so you should create separate signals like ‘ema0’, ‘ema1’, ‘ema2’, ‘ema3’, and ‘ema4’, where each new EMA value depends on the previous one, and finally connect ‘out’ to the last signal.

C-14 Constraint-Only Missing Assignment (T3001)

Description: This error class happens when a signal is used only inside constraints (‘===’) but is never actually assigned a witness value with ‘<==’ or ‘<-’. In Circom, constraints define relationships, but they do not initialize signals for witness generation on their own, so any later read of such a signal is treated as access to an uninitialized value. To fix it, make sure every intermediate signal that will be read later is explicitly assigned, or rewrite the computation so the value is derived with ‘<==’ before being reused.

Original Error Tag: Flow: Constraint-Only (Missing Assignment)

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 148: Source Python ema_signal_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template ema_signal_logic() {
4     signal input p0;
5     signal input p1;
6     signal input p2;
7     signal input p3;
8     signal input p4;
9     signal output out;
10
11     signal alpha <== 40;
12     signal hundred <== 100;
13     signal hundred_minus_alpha <== hundred - alpha;
14
15     signal ema0 <== p0 * hundred;
16
17     signal term1_1 <== p1 * alpha;
18     signal term2_1 <== ema0 * hundred_minus_alpha;
19     signal ema1;
20     ema1 * hundred == term1_1 * hundred + term2_1;
21
22     signal term1_2 <== p2 * alpha;
23     signal term2_2 <== ema1 * hundred_minus_alpha;
24     signal ema2;
25     ema2 * hundred == term1_2 * hundred + term2_2;
26
27     signal term1_3 <== p3 * alpha;
28     signal term2_3 <== ema2 * hundred_minus_alpha;
29     signal ema3;
30     ema3 * hundred == term1_3 * hundred + term2_3;
31
32     signal term1_4 <== p4 * alpha;
33     signal term2_4 <== ema3 * hundred_minus_alpha;
34     signal ema4;
35     ema4 * hundred == term1_4 * hundred + term2_4;
36
37     out <== ema4;
38 }
39
40 component main = ema_signal_logic();

```

Listing 149: Faulty translated Circom code with Constraint-Only Missing Assignment (T3001)

```

1 error[T3001]: Exception caused by invalid access: trying to access to a signal that is
2     not initialized
3     "circuit.circom":22:5

```

```

4 22      ema1 * hundred === term1_1 * hundred + term2_1;
5      ^^^^ found here
6
7      = call trace:
8      ->ema_signal_logic
9
10 previous errors were found

```

Listing 150: Circom compiler error log

Explanation: In this example, ‘ema1’, ‘ema2’, ‘ema3’, and ‘ema4’ are declared and constrained by equations, but ‘ema1’ is never assigned before it is used in ‘term2_2 <== ema1 * hundred_minus_alpha’. The compiler reports the first problematic access around the ‘ema1 * hundred === ...’ chain because ‘ema1’ is constraint-only, so the EMA intermediates need explicit assignments or a formulation that avoids reading unconstrained intermediate signals.

D Dynamic Signal in Static Context

D-1 Signal-Dependent Declaration (T2050)

Description: This error class happens when a Circom ‘if’/‘else’ condition depends on a signal value that is not known at compile/constraint-generation time, while the branches create different constraints or declare signals/components. Circom must know the full circuit structure statically, so control flow that changes declarations or constraints based on unknown witness values is not allowed. The fix is to rewrite the logic into unconditional constraints using selectors, boolean indicators, or arithmetic gadgets, and ensure all signals/components are declared outside signal-dependent branches.

Original Error Tag: T2050

Example:

```

1 def validator_slashing_weight(balance, total_slashed, total_active):
2     if total_active == 0: return [0]
3     penalty = (balance * 3 * total_slashed) // total_active
4     return [penalty if penalty < balance else balance]

```

Listing 151: Source Python validator_slashing_weight implementation

```

1 template validator_slashing_weight() {
2     signal input balance;
3     signal input total_slashed;
4     signal input total_active;
5     signal output out;
6
7     if (total_active == 0) {
8         out <== 0;
9     } else {
10         signal penalty <== (balance * 3 * total_slashed) / total_active;
11         out <== penalty < balance ? penalty : balance;
12     }
13 }
14 component main = validator_slashing_weight();

```

Listing 152: Faulty translated Circom code with Signal-Dependent Declaration (T2050)


```

1 error[T2005]: Typing error found
2   "circuit.circom":11:9
3
4 11     if (total_active == 0) {
5         ~~~~~ There are constraints depending on the value of the
6         condition and it can be unknown during the constraint generation phase
7
8 error[T2050]: Typing error found
9   "circuit.circom":11:9
10
11 11     if (total_active == 0) {
12         ~~~~~ There are signal, bus or component declarations
13         depending on the value of the condition and it can be unknown during the constraint
14         generation phase
15
16 previous errors were found

```

Listing 153: Circom compiler error log

Explanation: In this example, ‘total_active’ is an input signal, so the compiler cannot know whether ‘total_active == 0’ when generating the circuit. The ‘if’ changes both the constraint set and declares ‘penalty’ only in one branch, so the code must be refactored to compute both cases in a static way and then select the correct result with constraints instead of branching on ‘total_active’.

D-2 Dynamic Array Length (T20460)

Description: This error class happens when a Circom circuit tries to use a signal-dependent value to control compile-time structure, such as array sizes, loop bounds, conditional branches that emit constraints, or array indexing. In Circom, all constraints must be fully determined during the constraint generation phase, so these structural values must come from constants or template parameters, not runtime signals. The fix is to make such values static, unroll the logic with a fixed maximum size, and use selectors/masks to enable or disable behavior inside already-generated constraints instead of branching on unknown values.

Original Error Tag: T20460

Example:

```

1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2     fee = (amt_in * tax_bips) // 10000
3     amt_eff = amt_in - fee
4     # x * y = k
5     amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6     for i in range(exp):
7         amt_out = (amt_out * 9995) // 10000
8     if amt_out < 0: return 0
9     return int(amt_out)

```

Listing 154: Source Python fixed_point_swap_slippage implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2 include "node_modules/circomlib/circuits/sign.circom";
3
4 template fixed_point_swap_slippage() {
5     signal input ai;

```

```

6  signal input ri;
7  signal input ro;
8  signal input tb;
9  signal input e;
10 signal output out;
11
12 // Calculate f = (ai * tb) / 10000
13 signal f <-- (ai * tb) \ 10000;
14
15 // Calculate ae = ai - f
16 signal ae <-- ai - f;
17
18 // Calculate ao = (ae * ro) / (ri + ae)
19 signal denominator <-- ri + ae;
20 signal ao_temp <-- (ae * ro) \ denominator;
21 signal ao <== ao_temp;
22
23 // Apply exponential decay: ao = (ao * 9995) / 10000 for e iterations
24 signal decay_factor <-- 9995;
25 signal decay_divisor <-- 10000;
26
27 // Create loop using component instantiation
28 component decay[e];
29 for (var i = 0; i < e; i++) {
30     decay[i] = FixedPointMulDiv();
31     if (i == 0) {
32         decay[i].in[0] <== ao;
33     } else {
34         decay[i].in[0] <== decay[i-1].out;
35     }
36     decay[i].in[1] <== decay_factor;
37     decay[i].in[2] <== decay_divisor;
38 }
39
40 signal final_ao;
41 if (e > 0) {
42     final_ao <== decay[e-1].out;
43 } else {
44     final_ao <== ao;
45 }
46
47 // Check if final_ao < 0 and return 0 if true, else return final_ao
48 component lt = LessThan(64);
49 lt.in[0] <== final_ao;
50 lt.in[1] <== 0;
51
52 signal is_negative <== lt.out;
53
54 // out = is_negative ? 0 : final_ao
55 out <== (1 - is_negative) * final_ao;
56 }
57
58 template FixedPointMulDiv() {
59     signal input in[3]; // [numerator, multiplier, divisor]
60     signal output out;
61
62     // out = (in[0] * in[1]) / in[2]
63     signal product <-- in[0] * in[1];

```

```

64     out <-- product \ in[2];
65 }
66
67 component main = fixed_point_swap_slippage();

```

Listing 155: Faulty translated Circom code with Dynamic Array Length (T20460)

```

1 error[T20460]: Typing error found
2   "circuit.circom":29:21
3
4 29     component decay[e];
5           ^ The length of every array must be known during the
6     constraint generation phase
7 error[T2005]: Typing error found
8   "circuit.circom":30:21
9
10 30     for (var i = 0; i < e; i++) {
11           ^^^^^ There are constraints depending on the value of the
12     condition and it can be unknown during the constraint generation phase
13 error[T20462]: Typing error found
14   "circuit.circom":43:22
15
16 43     final_ao <== decay[e-1].out;
17           ^^^^^^^^^^^^^^^^^ Non-quadratic constraint was detected
18     statically, using unknown index will cause the constraint to be non-quadratic
19 error[T2005]: Typing error found
20   "circuit.circom":42:9
21
22 42     if (e > 0) {
23           ^^^^^ There are constraints depending on the value of the condition and it
24     can be unknown during the constraint generation phase
25 previous errors were found

```

Listing 156: Circom compiler error log

Explanation: In this example, ‘e’ is declared as an input signal but is used as the length of ‘decay[e]’, the bound of ‘for (var i = 0; i < e; i++)’, the condition ‘if (e > 0)’, and the index ‘decay[e-1]’. Because ‘e’ is not known at constraint generation time, Circom cannot determine how many components to create or which constraints to emit, so ‘e’ must be replaced with a template parameter or the decay logic must be rewritten with a fixed-size loop and signal-based selection.

D-3 Dynamic Index (T20462)

Description: This error class happens when a Circom constraint stops being statically quadratic, most commonly because an array is accessed or assigned with a signal-valued index like ‘arr[i]’ where ‘i’ is not a compile-time constant. Circom must know the exact wiring of constraints at compile time, so dynamic memory-style reads and writes are not allowed; the usual fix is to replace them with explicit fixed-index logic using selectors, muxes, one-hot index flags, or by unrolling the state transition so every possible stack slot is handled with constant indices.

Original Error Tag: T20462

Example:

```
1 def stack_rpn_calculator(o0, o1, o2, o3, o4):
2     ops = [o0, o1, o2, o3, o4]
3     stack = [0] * 5
4     top = -1
5     for i in range(5):
6         v = ops[i]
7         if v > 0:
8             top += 1
9             stack[top] = v
10        elif v < 0:
11            v2 = stack[top]; top -= 1
12            v1 = stack[top]; top -= 1
13            res = 0
14            if v == -1: res = v1 + v2
15            if v == -2: res = v1 - v2
16            if v == -3: res = v1 * v2
17            top += 1
18            stack[top] = res
19    return int(stack[0] + top)
```

Listing 157: Source Python stack_rpn_calculator implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template StackRpnCalculator() {
4     signal input o0;
5     signal input o1;
6     signal input o2;
7     signal input o3;
8     signal input o4;
9
10    signal output out;
11
12    // Initialize stack and top
13    signal stack[5];
14    signal top;
15
16    // We'll simulate the loop step-by-step for each of the 5 operations
17    // Initial state
18    stack[0] <== 0;
19    stack[1] <== 0;
20    stack[2] <== 0;
21    stack[3] <== 0;
22    stack[4] <== 0;
23    top <== -1;
24
25    // Process o0
26    component isPositive0 = IsPositive();
27    isPositive0.in <== o0;
28    component isNegative0 = IsPositive();
29    isNegative0.in <== -o0;
30
31    signal newTop0;
32    signal newVal0;
33    signal res0;
34}
```

```

35 // If o0 > 0
36 newTop0 <== top + isPositive0.out;
37 newVal0 <== isPositive0.out * o0;
38 stack[0] <== (1 - isPositive0.out) * stack[0] + newVal0;
39
40 // If o0 < 0, we can't do anything because stack is empty, so this case should not
41 // happen in valid inputs
42 // We assume valid RPN input
43
44 top <== newTop0;
45
46 // Process o1
47 component isPositive1 = IsPositive();
48 isPositive1.in <== o1;
49 component isNegative1 = IsPositive();
50 isNegative1.in <== -o1;
51
52 signal currentTop1;
53 currentTop1 <== top;
54
55 signal newTop1;
56 signal newVal1;
57
58 // Push if positive
59 newTop1 <== currentTop1 + isPositive1.out;
60 newVal1 <== isPositive1.out * o1;
61 stack[newTop1] <== (1 - isPositive1.out) * stack[newTop1] + newVal1;
62
63 // Handle negative (operation)
64 component op1Add = Eq();
65 op1Add.a <== o1;
66 op1Add.b <== -1;
67
68 component op1Sub = Eq();
69 op1Sub.a <== o1;
70 op1Sub.b <== -2;
71
72 component op1Mul = Eq();
73 op1Mul.a <== o1;
74 op1Mul.b <== -3;
75
76 signal v2_1;
77 signal v1_1;
78 signal res1;
79
80 v2_1 <== stack[currentTop1];
81 v1_1 <== stack[currentTop1 - 1];
82
83 res1 <==
84     op1Add.out * (v1_1 + v2_1) +
85     op1Sub.out * (v1_1 - v2_1) +
86     op1Mul.out * (v1_1 * v2_1);
87
88 // If it's an operation, update stack
89 signal isOp1;
90 isOp1 <== isNegative1.out;
91
92 signal updatedTop1;

```

```

92   updatedTop1 <== currentTop1 - 2 + isOp1;
93
94   // Update stack for operation case
95   stack[updatedTop1] <== (1 - isOp1) * stack[newTop1] + isOp1 * res1;
96
97   // For positions that are not written, they keep their previous value
98   // We need to handle all stack positions explicitly
99   signal tempStack1_0;
100  signal tempStack1_1;
101  signal tempStack1_2;
102  signal tempStack1_3;
103  signal tempStack1_4;
104
105  tempStack1_0 <== (1 - isPositive1.out) * (1 - isOp1) * stack[0] +
106                  isPositive1.out * (newTop1 == 0 ? newVal1 : stack[0]) +
107                  isOp1 * (updatedTop1 == 0 ? res1 : stack[0]);
108
109  tempStack1_1 <== (1 - isPositive1.out) * (1 - isOp1) * stack[1] +
110                  isPositive1.out * (newTop1 == 1 ? newVal1 : stack[1]) +
111                  isOp1 * (updatedTop1 == 1 ? res1 : stack[1]);
112
113  tempStack1_2 <== (1 - isPositive1.out) * (1 - isOp1) * stack[2] +
114                  isPositive1.out * (newTop1 == 2 ? newVal1 : stack[2]) +
115                  isOp1 * (updatedTop1 == 2 ? res1 : stack[2]);
116
117  tempStack1_3 <== (1 - isPositive1.out) * (1 - isOp1) * stack[3] +
118                  isPositive1.out * (newTop1 == 3 ? newVal1 : stack[3]) +
119                  isOp1 * (updatedTop1 == 3 ? res1 : stack[3]);
120
121  tempStack1_4 <== (1 - isPositive1.out) * (1 - isOp1) * stack[4] +
122                  isPositive1.out * (newTop1 == 4 ? newVal1 : stack[4]) +
123                  isOp1 * (updatedTop1 == 4 ? res1 : stack[4]);
124
125  stack[0] <== tempStack1_0;
126  stack[1] <== tempStack1_1;
127  stack[2] <== tempStack1_2;
128  stack[3] <== tempStack1_3;
129  stack[4] <== tempStack1_4;
130
131  top <== (1 - isOp1) * newTop1 + isOp1 * updatedTop1;
132
133  // Process o2
134  component isPositive2 = IsPositive();
135  isPositive2.in <== o2;
136  component isNegative2 = IsPositive();
137  isNegative2.in <== -o2;
138
139  signal currentTop2;
140  currentTop2 <== top;
141
142  component op2Add = Eq();
143  op2Add.a <== o2;
144  op2Add.b <== -1;
145
146  component op2Sub = Eq();
147  op2Sub.a <== o2;
148  op2Sub.b <== -2;
149

```

```

150     component op2Mul = Eq();
151     op2Mul.a <== o2;
152     op2Mul.b <== -3;
153
154     signal v2_2;
155     signal v1_2;
156     signal res2;
157
158     v2_2 <== stack[currentTop2];
159     v1_2 <== stack[currentTop2 - 1];
160
161     res2 <==
162         op2Add.out * (v1_2 + v2_2) +
163         op2Sub.out * (v1_2 - v2_2) +
164         op2Mul.out * (v1_2 * v2_2);
165
166     signal isOp2;
167     isOp2 <== isNegative2.out;
168
169     signal newTop2;
170     newTop2 <== currentTop2 + isPositive2.out;
171     signal newVal2;
172     newVal2 <== isPositive2.out * o2;
173
174     signal updatedTop2;
175     updatedTop2 <== currentTop2 - 2 + isOp2;
176
177     signal tempStack2_0;
178     signal tempStack2_1;
179     signal tempStack2_2;
180     signal tempStack2_3;
181     signal tempStack2_4;
182
183     tempStack2_0 <== (1 - isPositive2.out) * (1 - isOp2) * stack[0] +
184         isPositive2.out * (newTop2 == 0 ? newVal2 : stack[0]) +
185         isOp2 * (updatedTop2 == 0 ? res2 : stack[0]);
186
187     tempStack2_1 <== (1 - isPositive2.out) * (1 - isOp2) * stack[1] +
188         isPositive2.out * (newTop2 == 1 ? newVal2 : stack[1]) +
189         isOp2 * (updatedTop2 == 1 ? res2 : stack[1]);
190
191     tempStack2_2 <== (1 - isPositive2.out) * (1 - isOp2) * stack[2] +
192         isPositive2.out * (newTop2 == 2 ? newVal2 : stack[2]) +
193         isOp2 * (updatedTop2 == 2 ? res2 : stack[2]);
194
195     tempStack2_3 <== (1 - isPositive2.out) * (1 - isOp2) * stack[3] +
196         isPositive2.out * (newTop2 == 3 ? newVal2 : stack[3]) +
197         isOp2 * (updatedTop2 == 3 ? res2 : stack[3]);
198
199     tempStack2_4 <== (1 - isPositive2.out) * (1 - isOp2) * stack[4] +
200         isPositive2.out * (newTop2 == 4 ? newVal2 : stack[4]) +
201         isOp2 * (updatedTop2 == 4 ? res2 : stack[4]);
202
203     stack[0] <== tempStack2_0;
204     stack[1] <== tempStack2_1;
205     stack[2] <== tempStack2_2;
206     stack[3] <== tempStack2_3;
207     stack[4] <== tempStack2_4;

```

```

208     top <== (1 - isOp2) * newTop2 + isOp2 * updatedTop2;
209
210
211     // Process o3
212     component isPositive3 = IsPositive();
213     isPositive3.in <== o3;
214     component isNegative3 = IsPositive();
215     isNegative3.in <== -o3;
216
217     signal currentTop3;
218     currentTop3 <== top;
219
220     component op3Add = Eq();
221     op3Add.a <== o3;
222     op3Add.b <== -1;
223
224     component op3Sub = Eq();
225     op3Sub.a <== o3;
226     op3Sub.b <== -2;
227
228     component op3Mul = Eq();
229     op3Mul.a <== o3;
230     op3Mul.b <== -3;
231
232     signal v2_3;
233     signal v1_3;
234     signal res3;
235
236     v2_3 <== stack[currentTop3];
237     v1_3 <== stack[currentTop3 - 1];
238
239     res3 <==
240         op3Add.out * (v1_3 + v2_3) +
241         op3Sub.out * (v1_3 - v2_3) +
242         op3Mul.out * (v1_3 * v2_3);
243
244     signal isOp3;
245     isOp3 <== isNegative3.out;
246
247     signal newTop3;
248     newTop3 <== currentTop3 + isPositive3.out;
249     signal newVal3;
250     newVal3 <== isPositive3.out * o3;
251
252     signal updatedTop3;
253     updatedTop3 <== currentTop3 - 2 + isOp3;
254
255     signal tempStack3_0;
256     signal tempStack3_1;
257     signal tempStack3_2;
258     signal tempStack3_3;
259     signal tempStack3_4;
260
261     tempStack3_0 <== (1 - isPositive3.out) * (1 - isOp3) * stack[0] +
262         isPositive3.out * (newTop3 == 0 ? newVal3 : stack[0]) +
263         isOp3 * (updatedTop3 == 0 ? res3 : stack[0]);
264
265     tempStack3_1 <== (1 - isPositive3.out) * (1 - isOp3) * stack[1] +

```



```

266         isPositive3.out * (newTop3 == 1 ? newVal3 : stack[1]) +
267         isOp3 * (updatedTop3 == 1 ? res3 : stack[1]);
268
269     tempStack3_2 <== (1 - isPositive3.out) * (1 - isOp3) * stack[2] +
270     isPositive3.out * (newTop3 == 2 ? newVal3 : stack[2]) +
271     isOp3 * (updatedTop3 == 2 ? res3 : stack[2]);
272
273     tempStack3_3 <== (1 - isPositive3.out) * (1 - isOp3) * stack[3] +
274     isPositive3.out * (newTop3 == 3 ? newVal3 : stack[3]) +
275     isOp3 * (updatedTop3 == 3 ? res3 : stack[3]);
276
277     tempStack3_4 <== (1 - isPositive3.out) * (1 - isOp3) * stack[4] +
278     isPositive3.out * (newTop3 == 4 ? newVal3 : stack[4]) +
279     isOp3 * (updatedTop3 == 4 ? res3 : stack[4]);
280
281     stack[0] <== tempStack3_0;
282     stack[1] <== tempStack3_1;
283     stack[2] <== tempStack3_2;
284     stack[3] <== tempStack3_3;
285     stack[4] <== tempStack3_4;
286
287     top <== (1 - isOp3) * newTop3 + isOp3 * updatedTop3;
288
289     // Process o4
290     component isPositive4 = IsPositive();
291     isPositive4.in <== o4;
292     component isNegative4 = IsPositive();
293     isNegative4.in <== -o4;
294
295     signal currentTop4;
296     currentTop4 <== top;
297
298     component op4Add = Eq();
299     op4Add.a <== o4;
300     op4Add.b <== -1;
301
302     component op4Sub = Eq();
303     op4Sub.a <== o4;
304     op4Sub.b <== -2;
305
306     component op4Mul = Eq();
307     op4Mul.a <== o4;
308     op4Mul.b <== -3;
309
310     signal v2_4;
311     signal v1_4;
312     signal res4;
313
314     v2_4 <== stack[currentTop4];
315     v1_4 <== stack[currentTop4 - 1];
316
317     res4 <==
318         op4Add.out * (v1_4 + v2_4) +
319         op4Sub.out * (v1_4 - v2_4) +
320         op4Mul.out * (v1_4 * v2_4);
321
322     signal isOp4;
323     isOp4 <== isNegative4.out;

```

```

324
325     signal newTop4;
326     newTop4 <== currentTop4 + isPositive4.out;
327     signal newVal4;
328     newVal4 <== isPositive4.out * o4;
329
330     signal updatedTop4;
331     updatedTop4 <== currentTop4 - 2 + isOp4;
332
333     signal tempStack4_0;
334     signal tempStack4_1;
335     signal tempStack4_2;
336     signal tempStack4_3;
337     signal tempStack4_4;
338
339     tempStack4_0 <== (1 - isPositive4.out) * (1 - isOp4) * stack[0] +
340                     isPositive4.out * (newTop4 == 0 ? newVal4 : stack[0]) +
341                     isOp4 * (updatedTop4 == 0 ? res4 : stack[0]);
342
343     tempStack4_1 <== (1 - isPositive4.out) * (1 - isOp4) * stack[1] +
344                     isPositive4.out * (newTop4 == 1 ? newVal4 : stack[1]) +
345                     isOp4 * (updatedTop4 == 1 ? res4 : stack[1]);
346
347     tempStack4_2 <== (1 - isPositive4.out) * (1 - isOp4) * stack[2] +
348                     isPositive4.out * (newTop4 == 2 ? newVal4 : stack[2]) +
349                     isOp4 * (updatedTop4 == 2 ? res4 : stack[2]);
350
351     tempStack4_3 <== (1 - isPositive4.out) * (1 - isOp4) * stack[3] +
352                     isPositive4.out * (newTop4 == 3 ? newVal4 : stack[3]) +
353                     isOp4 * (updatedTop4 == 3 ? res4 : stack[3]);
354
355     tempStack4_4 <== (1 - isPositive4.out) * (1 - isOp4) * stack[4] +
356                     isPositive4.out * (newTop4 == 4 ? newVal4 : stack[4]) +
357                     isOp4 * (updatedTop4 == 4 ? res4 : stack[4]);
358
359     stack[0] <== tempStack4_0;
360     stack[1] <== tempStack4_1;
361     stack[2] <== tempStack4_2;
362     stack[3] <== tempStack4_3;
363     stack[4] <== tempStack4_4;
364
365     top <== (1 - isOp4) * newTop4 + isOp4 * updatedTop4;
366
367     // Final output: stack[0] + top
368     out <== stack[0] + top;
369 }
370
371 // Helper template for checking if a number is positive
372 template IsPositive() {
373     signal input in;
374     signal output out;
375
376     component gte = GreaterThan(64);
377     gte.in[0] <== in;
378     gte.in[1] <== 0;
379     out <== gte.out;
380 }
381

```

```

382 // Helper template for equality check
383 template Eq() {
384     signal input a;
385     signal input b;
386     signal output out;
387
388     signal diff;
389     diff <== a - b;
390     component isZero = IsZero();
391     isZero.in <== diff;
392     out <== 1 - isZero.out;
393 }
394
395 component main = StackRpnCalculator();

```

Listing 158: Faulty translated Circom code with Dynamic Index (T20462)

```

1 error[T20462]: Typing error found
2   "circuit.circom":62:24
3
4 62     stack[newTop1] <== (1 - isPositive1.out) * stack[newTop1] + newVal1;
5           ~~~~~ Non-
        quadratic constraint was detected statically, using unknown index will cause the
        constraint to be non-quadratic
6
7 error[T20462]: Typing error found
8   "circuit.circom":62:5
9
10 62     stack[newTop1] <== (1 - isPositive1.out) * stack[newTop1] + newVal1;
11           ~~~~~ Non-
        quadratic constraint was detected statically, using unknown index will cause the
        constraint to be non-quadratic
12
13 error[T20462]: Typing error found
14   "circuit.circom":81:14
15
16 81     v2_1 <== stack[currentTop1];
17           ~~~~~ Non-quadratic constraint was detected statically,
        using unknown index will cause the constraint to be non-quadratic
18
19 error[T20462]: Typing error found
20   "circuit.circom":82:14
21
22 82     v1_1 <== stack[currentTop1 - 1];
23           ~~~~~ Non-quadratic constraint was detected
        statically, using unknown index will cause the constraint to be non-quadratic
24
25 error[T20462]: Typing error found
26   "circuit.circom":97:28
27
28 97     stack[updatedTop1] <== (1 - isOp1) * stack[newTop1] + isOp1 * res1;
29           ~~~~~ Non-
        quadratic constraint was detected statically, using unknown index will cause the
        constraint to be non-quadratic
30
31 error[T20462]: Typing error found
32   "circuit.circom":97:5
33

```

```

34 97      stack[updatedTop1] <== (1 - isOp1) * stack[newTop1] + isOp1 * res1;
35      ~~~~~ Non-
        quadratic constraint was detected statically, using unknown index will cause the
        constraint to be non-quadratic
36
37 error[T20462]: Typing error found
38     "circuit.circom":160:14
39
40 160      v2_2 <== stack[currentTop2];
41      ~~~~~ Non-quadratic constraint was detected statically,
        using unknown index will cause the constraint to be non-quadratic
42
43 error[T20462]: Typing error found
44     "circuit.circom":161:14
45
46 161      v1_2 <== stack[currentTop2 - 1];
47      ~~~~~ Non-quadratic constraint was detected
        statically, using unknown index will cause the constraint to be non-quadratic
48
49 error[T20462]: Typing error found
50     "circuit.circom":238:14
51
52 238      v2_3 <== stack[currentTop3];
53      ~~~~~ Non-quadratic constraint was detected statically,
        using unknown index will cause the constraint to be non-quadratic
54
55 error[T20462]: Typing error found
56     "circuit.circom":239:14
57
58 239      v1_3 <== stack[currentTop3 - 1];
59      ~~~~~ Non-quadratic constraint was detected
        statically, using unknown index will cause the constraint to be non-quadratic
60
61 error[T20462]: Typing error found
62     "circuit.circom":316:14
63
64 316      v2_4 <== stack[currentTop4];
65      ~~~~~ Non-quadratic constraint was detected statically,
        using unknown index will cause the constraint to be non-quadratic
66
67 error[T20462]: Typing error found
68     "circuit.circom":317:14
69
70 317      v1_4 <== stack[currentTop4 - 1];
71      ~~~~~ Non-quadratic constraint was detected
        statically, using unknown index will cause the constraint to be non-quadratic
72
73 previous errors were found

```

Listing 159: Circom compiler error log

Explanation: In this example, the circuit tries to implement a stack with signal-dependent positions such as ‘stack[newTop1]’, ‘stack[currentTop1]’, and ‘stack[currentTop1 - 1]’, where ‘newTop1’ and ‘currentTop1’ depend on inputs and earlier signals. Because those indices are not constants, Circom cannot compile them into quadratic constraints, so the stack read/write logic must be rewritten as fixed-slot conditional selection instead of direct indexed access.

D-4 Dynamic Template Parameter (T20461)

Description: This error happens when a Circom component is instantiated with a template parameter that is not known at compile/constraint-generation time. In Circom, template parameters such as sizes, bounds, and subcomponent configuration values must be constants or otherwise statically resolvable, not runtime signals. To fix it, replace the dynamic parameter with a compile-time constant, move the value into a signal input instead of a template parameter if the subcircuit supports that pattern, or redesign the circuit so the component structure does not depend on witness-time values.

Original Error Tag: T20461

Example:

```
1 def mac_poly1305_lite(m1, m2, r, s, p):
2     return [(m1 * r + m2) * r + s) % p]
```

Listing 160: Source Python mac_poly1305_lite implementation

```
1 pragma circom 2.1.4;
2
3 include "circomlib/poseidon.circom";
4
5 // Helper template for range checking (0 <= x < bound)
6 template RangeCheck(bound) {
7     signal input in;
8
9     // Check that in is non-negative
10    component lt1 = LessThan(bound);
11    lt1.in[0] <== in;
12    lt1.in[1] <== bound;
13    lt1.out == 1; // in < bound
14
15    // Note: LessThan template from circomlib ensures in >= 0
16 }
17
18 // Modular addition template
19 template ModAdd() {
20     signal input a;
21     signal input b;
22     signal input p;
23     signal output out;
24
25     signal q; // quotient (non-negative integer)
26     signal diff;
27
28     // Constraint: a + b = q * p + out
29     diff <== a + b - q * p - out;
30     diff == 0;
31
32     // Range check: 0 <= out < p
33     component rangeCheck = RangeCheck(p);
34     rangeCheck.in <== out;
35
36     // Ensure q is non-negative (implicitly enforced by the constraint)
37     // For completeness, we could add a range check on q if needed
38 }
39
40 // Modular multiplication template
```

```

41 template ModMul() {
42     signal input a;
43     signal input b;
44     signal input p;
45     signal output out;
46
47     signal q; // quotient (non-negative integer)
48     signal diff;
49
50     // Constraint:  $a * b = q * p + out$ 
51     diff <== a * b - q * p - out;
52     diff == 0;
53
54     // Range check:  $0 \leq out < p$ 
55     component rangeCheck = RangeCheck(p);
56     rangeCheck.in <== out;
57
58     // Ensure q is non-negative (implicitly enforced by the constraint)
59 }
60
61 // Main Poly1305 MAC template
62 template MacPoly1305Lite() {
63     signal input m1;
64     signal input m2;
65     signal input r;
66     signal input s;
67     signal input p;
68     signal output result;
69
70     // Step 1:  $t1 = m1 * r \bmod p$ 
71     component modMul1 = ModMul();
72     modMul1.a <== m1;
73     modMul1.b <== r;
74     modMul1.p <== p;
75     signal t1;
76     t1 <== modMul1.out;
77
78     // Step 2:  $t2 = (t1 + m2) \bmod p$ 
79     component modAdd1 = ModAdd();
80     modAdd1.a <== t1;
81     modAdd1.b <== m2;
82     modAdd1.p <== p;
83     signal t2;
84     t2 <== modAdd1.out;
85
86     // Step 3:  $t3 = t2 * r \bmod p$ 
87     component modMul2 = ModMul();
88     modMul2.a <== t2;
89     modMul2.b <== r;
90     modMul2.p <== p;
91     signal t3;
92     t3 <== modMul2.out;
93
94     // Step 4:  $t4 = (t3 + s) \bmod p$ 
95     component modAdd2 = ModAdd();
96     modAdd2.a <== t3;
97     modAdd2.b <== s;
98     modAdd2.p <== p;

```

```

99     signal t4;
100     t4 <= modAdd2.out;
101
102     // Output: result = t4 mod p (already reduced)
103     result <= t4;
104 }
105
106 // Main circuit
107 component main = MacPoly1305Lite();

```

Listing 161: Faulty translated Circom code with Dynamic Template Parameter (T20461)

```

1 error[T20461]: Typing error found
2   "circuit.circom":55:28
3
4 55     component rangeCheck = RangeCheck(p);
5           ~~~~~ Every component instantiation must be
           resolved during the constraint generation phase. This component declaration uses a
           value that can be unknown during the constraint generation phase.
6
7 error[T20461]: Typing error found
8   "circuit.circom":33:28
9
10 33     component rangeCheck = RangeCheck(p);
11           ~~~~~ Every component instantiation must be
           resolved during the constraint generation phase. This component declaration uses a
           value that can be unknown during the constraint generation phase.
12
13 previous errors were found

```

Listing 162: Circom compiler error log

Explanation: In this example, ‘RangeCheck(p)’ is used inside ‘ModAdd’ and ‘ModMul’, but ‘p’ is declared as a ‘signal input’, so its value is only known when the witness is generated. Because ‘RangeCheck’ expects ‘bound’ as a template parameter, Circom rejects the instantiation since it cannot determine the component structure from a runtime signal.

D-5 Signal-Dependent Condition (T2005)

Description: This error class happens when a Circom circuit tries to use a signal or other runtime-dependent value to control circuit structure during constraint generation. In Circom, array sizes, loop bounds, conditionals that add constraints, and component indexing must all be determined at compile time, because the compiler must know the full set of constraints before proving. To fix it, replace dynamic control flow with compile-time constants, template parameters, or a fixed maximum size plus selector logic/masking so the circuit shape does not depend on unknown inputs.

Original Error Tag: T2005

Example:

```

1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2     fee = (amt_in * tax_bips) // 10000
3     amt_eff = amt_in - fee
4     # x * y = k
5     amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6     for i in range(exp):

```

```

7     amt_out = (amt_out * 9995) // 10000
8     if amt_out < 0: return 0
9     return int(amt_out)

```

Listing 163: Source Python fixed_point_swap_slippage implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";
2 include "node_modules/circomlib/circuits/sign.circom";
3
4 template fixed_point_swap_slippage() {
5     signal input ai;
6     signal input ri;
7     signal input ro;
8     signal input tb;
9     signal input e;
10    signal output out;
11
12    // Calculate f = (ai * tb) / 10000
13    signal f <-- (ai * tb) \ 10000;
14
15    // Calculate ae = ai - f
16    signal ae <-- ai - f;
17
18    // Calculate ao = (ae * ro) / (ri + ae)
19    signal denominator <-- ri + ae;
20    signal ao_temp <-- (ae * ro) \ denominator;
21    signal ao <== ao_temp;
22
23    // Apply exponential decay: ao = (ao * 9995) / 10000 for e iterations
24    signal decay_factor <-- 9995;
25    signal decay_divisor <-- 10000;
26
27    // Create loop using component instantiation
28    component decay[e];
29    for (var i = 0; i < e; i++) {
30        decay[i] = FixedPointMulDiv();
31        if (i == 0) {
32            decay[i].in[0] <== ao;
33        } else {
34            decay[i].in[0] <== decay[i-1].out;
35        }
36        decay[i].in[1] <== decay_factor;
37        decay[i].in[2] <== decay_divisor;
38    }
39
40    signal final_ao;
41    if (e > 0) {
42        final_ao <== decay[e-1].out;
43    } else {
44        final_ao <== ao;
45    }
46
47    // Check if final_ao < 0 and return 0 if true, else return final_ao
48    component lt = LessThan(64);
49    lt.in[0] <== final_ao;
50    lt.in[1] <== 0;
51
52    signal is_negative <== lt.out;

```



```

53 // out = is_negative ? 0 : final_ao
54 out <== (1 - is_negative) * final_ao;
55 }
56
57
58 template FixedPointMulDiv() {
59     signal input in[3]; // [numerator, multiplier, divisor]
60     signal output out;
61
62     // out = (in[0] * in[1]) / in[2]
63     signal product <-- in[0] * in[1];
64     out <-- product \ in[2];
65 }
66
67 component main = fixed_point_swap_slippage();

```

Listing 164: Faulty translated Circom code with Signal-Dependent Condition (T2005)

```

1 error[T20460]: Typing error found
2   "circuit.circom":29:21
3
4 29     component decay[e];
5         ^ The length of every array must be known during the
6     constraint generation phase
7
8 error[T2005]: Typing error found
9   "circuit.circom":30:21
10
11 30     for (var i = 0; i < e; i++) {
12         ^^^^^ There are constraints depending on the value of the
13     condition and it can be unknown during the constraint generation phase
14
15 error[T20462]: Typing error found
16   "circuit.circom":43:22
17
18 43     final_ao <== decay[e-1].out;
19         ^^^^^^^^^^^^^^^^^ Non-quadratic constraint was detected
20     statically, using unknown index will cause the constraint to be non-quadratic
21
22 error[T2005]: Typing error found
23   "circuit.circom":42:9
24
25 42     if (e > 0) {
26         ^^^^^ There are constraints depending on the value of the condition and it
27     can be unknown during the constraint generation phase
28
29 previous errors were found

```

Listing 165: Circom compiler error log

Explanation: In this example, ‘e’ is declared as an input signal, but it is used as the length of ‘decay[e]’, as the bound in ‘for (var i = 0; i < e; i++)’, in ‘if (e > 0)’, and as the index ‘decay[e-1]’. Since ‘e’ is not known during constraint generation, the compiler cannot determine how many components to instantiate or which constraints to emit, so ‘e’ must be made a template parameter or the decay logic must be rewritten with a fixed number of iterations and conditional selection.

D-6 Declaration in While Scope (T2011)

Description: This error happens when a Circom template tries to declare a new signal, bus, or component inside a scope where the compiler cannot treat the declaration as static circuit structure, such as inside a ‘while’ loop or another dynamic scope. In Circom, declarations must be made in the initial template scope, or inside an ‘if’ whose condition is known at compile time; the fix is to move those declarations outside the loop/scope and reuse predeclared arrays or variables so the circuit shape is fully determined during compilation.

Original Error Tag: T2011

Example:

```
1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2     fee = (amt_in * tax_bips) // 10000
3     amt_eff = amt_in - fee
4     # x * y = k
5     amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6     for i in range(exp):
7         amt_out = (amt_out * 9995) // 10000
8     if amt_out < 0: return 0
9     return int(amt_out)
```

Listing 166: Source Python fixed_point_swap_slippage implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template FixedPointSwapSlippage() {
4     signal input amt_in;
5     signal input res_in;
6     signal input res_out;
7     signal input tax_bips;
8     signal input exp;
9     signal output amt_out;
10
11     // Calculate fee: (amt_in * tax_bips) // 10000
12     signal fee <-- amt_in * tax_bips / 10000;
13     fee == amt_in * tax_bips - (amt_in * tax_bips % 10000);
14
15     // Calculate effective amount: amt_in - fee
16     signal amt_eff <== amt_in - fee;
17
18     // Calculate initial amount out: (amt_eff * res_out) // (res_in + amt_eff)
19     signal numerator <== amt_eff * res_out;
20     signal denominator <== res_in + amt_eff;
21     signal amt_out_temp <-- numerator / denominator;
22     amt_out_temp == numerator - (numerator % denominator);
23
24     // Apply slippage reduction for exp iterations: (amt_out * 9995) // 10000
25     signal intermediate[exp+1];
26     intermediate[0] <== amt_out_temp;
27
28     for (var i = 0; i < exp; i++) {
29         signal step_numerator <== intermediate[i] * 9995;
30         intermediate[i+1] <-- step_numerator / 10000;
31         intermediate[i+1] == step_numerator - (step_numerator % 10000);
32     }
33 }
```

```

34 // Ensure non-negative output using comparison
35 component lt = LessThan(64);
36 lt.in[0] <= 0;
37 lt.in[1] <= intermediate[exp];
38
39 // Select between 0 and intermediate[exp] based on comparison
40 amt_out <= (1 - lt.out) * intermediate[exp];
41 }
42
43 component main = FixedPointSwapSlippage();

```

Listing 167: Faulty translated Circom code with Declaration in While Scope (T2011)

```

1 error[T2011]: Signal, bus or component declaration inside While scope. Signals, buses
  and components can only be defined in the initial scope or in If scopes with known
  condition
2 "circuit.circom":31:9
3
4 31          signal step_numerator <= intermediate[i] * 9995;
5          ~~~~~ Is outside the initial
6          scope
7 previous errors were found

```

Listing 168: Circom compiler error log

Explanation: In this example, ‘signal step_numerator <= intermediate[i] * 9995;’ is declared inside the ‘for’ loop, so Circom rejects it as a scoped signal declaration outside the allowed initial scope. You should predeclare storage for ‘step_numerator’ (for example, as an array) before the loop, or compute the expression directly into already-declared signals inside the loop.

E Project/Module Errors

E-1 Duplicate Symbol (T2008)

Description: This error class occurs when the compiler finds two callables with the same name in the same scope, such as templates, functions, or components introduced by your file and an included library. The fix is to ensure each callable symbol is defined only once: remove the duplicate local definition, rename your version, or stop including the file that already provides it if you do not need both.

Original Error Tag: T2008

Example:

```

1 def validator_slashing_weight(balance, total_slashed, total_active):
2     if total_active == 0: return [0]
3     penalty = (balance * 3 * total_slashed) // total_active
4     return [penalty if penalty < balance else balance]

```

Listing 169: Source Python validator_slashing_weight implementation

```

1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";

```

```

4 include "node_modules/circomlib/circuits/bitify.circom";
5
6 // Standard zero-check gadget:
7 // out = 1 iff in == 0, else 0
8 template IsZero() {
9     signal input in;
10    signal output out;
11
12    signal inv;
13
14    inv <-- in != 0 ? 1 / in : 0;
15    out <== 1 - in * inv;
16    in * out == 0;
17 }
18
19 /*
20 Assumptions:
21 - bal, ts, ta are nonnegative integers fitting in inBits bits
22 - qBits must be large enough for q = floor((3 * bal * ts) / ta)
23   A safe choice for 64-bit inputs is qBits = 130
24
25 Semantics proved:
26 if ta == 0:
27     out = 0
28 else:
29     p = floor((3 * bal * ts) / ta)
30     out = min(p, bal)
31 */
32 template validator_slashing_weight(inBits, qBits) {
33     signal input bal;
34     signal input ts;
35     signal input ta;
36     signal output out;
37
38     signal is_ta_zero;
39     signal is_ta_nonzero;
40
41     signal num;
42     signal q;
43     signal r;
44     signal p;
45
46     signal lt_out;
47     signal uncapped_out;
48
49     // Range constraints for inputs
50     component bal_bits = Num2Bits(inBits);
51     bal_bits.in <== bal;
52
53     component ts_bits = Num2Bits(inBits);
54     ts_bits.in <== ts;
55
56     component ta_bits = Num2Bits(inBits);
57     ta_bits.in <== ta;
58
59     // ta == 0 check
60     component z = IsZero();
61     z.in <== ta;

```

```

62 is_ta_zero <== z.out;
63 is_ta_nonzero <== 1 - is_ta_zero;
64
65 // numerator = 3 * bal * ts
66 num <== 3 * bal * ts;
67
68 // Witness for quotient and remainder
69 // When ta == 0, force q = 0 and r = 0
70 q <-- ta != 0 ? (3 * bal * ts) \ ta : 0;
71 r <-- ta != 0 ? (3 * bal * ts) % ta : 0;
72
73 p <== q;
74
75 // Range constraints for q and r
76 component q_bits = Num2Bits(qBits);
77 q_bits.in <== q;
78
79 component r_bits = Num2Bits(inBits);
80 r_bits.in <== r;
81
82 // Enforce division relation only when ta != 0:
83 // num = ta * q + r
84 is_ta_nonzero * (num - ta * q - r) === 0;
85
86 // Force q = 0 and r = 0 when ta == 0
87 is_ta_zero * q === 0;
88 is_ta_zero * r === 0;
89
90 // Enforce r < ta when ta != 0
91 component r_lt_ta = LessThan(inBits);
92 r_lt_ta.in[0] <== r;
93 r_lt_ta.in[1] <== ta;
94 is_ta_nonzero * (r_lt_ta.out - 1) === 0;
95
96 // lt_out = 1 iff p < bal
97 // Compare in qBits domain, so bal is also lifted to qBits
98 component bal_qbits = Num2Bits(qBits);
99 bal_qbits.in <== bal;
100
101 component p_lt_bal = LessThan(qBits);
102 p_lt_bal.in[0] <== p;
103 p_lt_bal.in[1] <== bal;
104 lt_out <== p_lt_bal.out;
105
106 // uncapped_out = min(p, bal)
107 uncapped_out <== lt_out * p + (1 - lt_out) * bal;
108
109 // if ta == 0 -> out = 0
110 // else out = uncapped_out
111 out <== is_ta_nonzero * uncapped_out;
112 }
113
114 // Example instantiation for 64-bit nonnegative inputs.
115 // qBits = 130 is safe because 3 * bal * ts can be up to about 2^130.
116 component main = validator_slashing_weight(64, 130);

```

Listing 170: Faulty translated Circom code with Duplicate Symbol (T2008)

```

1 error[T2008]: Duplicated callable symbol
2   "/home/peng/TransZKEval/circuits/node_modules/circomlib/circuits/comparators.
   circom":24:1
3
4   24   template IsZero() {
5       25       signal input in;
6       26       signal output out;
7       27
8       28       signal inv;
9       29
10      30       inv <-- in!=0 ? 1/in : 0;
11      31
12      32       out <== -in*inv +1;
13      33       in*out === 0;
14      34   }
15      ^ IsZero is already in use
16
17 previous errors were found

```

Listing 171: Circom compiler error log

Explanation: In this example, ‘comparators.circom’ already defines ‘template IsZero()’, and the target code defines another ‘template IsZero()’ after including that library. Circom therefore rejects the second declaration; use the library’s ‘IsZero’ directly or rename your custom zero-check template.

E-2 Bad Include/Import (P1009)

Description: This error happens when a Circom import/include statement uses syntax or arguments the compiler does not support, such as named imports, extra tokens, or module-style paths borrowed from JavaScript. To fix it, use Circom’s valid file-inclusion syntax for your compiler version, typically by including the actual ‘circom’ source file path and then using the templates defined in that file, rather than trying to import specific symbols with ‘{ ... }’.

Original Error Tag: P1009

Example:

```

1 def validator_slashing_weight(balance, total_slashed, total_active):
2     if total_active == 0: return [0]
3     penalty = (balance * 3 * total_slashed) // total_active
4     return [penalty if penalty < balance else balance]

```

Listing 172: Source Python validator_slashing_weight implementation

```

1 // Import necessary components
2 import { IsZero } from "circomlib";
3 import { LessThan } from "circomlib";
4 import { Div } from "circomlib";
5
6 // Define the template
7 template validator_slashing_weight() {
8     // Define signals
9     signal input bal;
10    signal input ts;
11    signal input ta;
12    signal output out;

```

```

13
14 // Define intermediate signals
15 signal ta_zero;
16 signal p;
17 signal p_lt_bal;
18
19 // Zero check constraint
20 component isZero = IsZero(64);
21 isZero.in <= ta;
22 ta_zero <= isZero.out;
23
24 // Slashing weight calculation constraint
25 component div = Div(64, 64);
26 div.in[0] <= bal * 3 * ts;
27 div.in[1] <= ta;
28 div.in[2] <= ta_zero;
29 p <= div.out;
30
31 // Less-than constraint
32 component lessThan = LessThan(64);
33 lessThan.in[0] <= p;
34 lessThan.in[1] <= bal;
35 p_lt_bal <= lessThan.out;
36
37 // Output selection constraint
38 out <= ta_zero * 0 + (1 - ta_zero) * (p_lt_bal * p + (1 - p_lt_bal) * bal);
39 }
40
41 // Define the main component
42 component main = validator_slashing_weight();

```

Listing 173: Faulty translated Circom code with Bad Include/Import (P1009)

```

1 error[P1009]: unrecognized argument in include directive
2   "circuit.circom":8:1
3
4 8 import { Div } from "circomlib";
5   ~~~~~~ this argument
6
7 previous errors were found

```

Listing 174: Circom compiler error log

Explanation: In this example, ‘import { Div } from "circomlib";’ is invalid because Circom does not accept the ‘{ Div }’ argument in an import directive. The code should instead include the relevant Circom library files that define ‘IsZero’, ‘LessThan’, and ‘Div’ using proper Circom file paths, then instantiate those templates normally.

E-3 Template Not Exist (TAC01)

Description: This error class happens when a Circom template or component name is used but is not actually defined or imported in the current compilation unit. It is usually caused by a missing ‘include’, a wrong template name, a library/version mismatch, or assuming a helper template exists when it does not; the fix is to define that template yourself or import the correct file/library that provides it, then instantiate it using the real interface it exposes.

Original Error Tag: TAC01

Example:

```
1 def aave_v3_interest_logic(d, r, b, s1, s2, opt):
2     if r == 0: return int(b)
3     u = int((d * 10000) // r)
4     br = 0
5     if u <= opt: br = int(b + (u * s1 // opt))
6     else: br = int(b + s1 + ((u - opt) * s2 // (10000 - opt)))
7     sr = int((br * u * 9000 // 10000) // 10000)
8     rp = 0
9     if u > 9500: rp = int((u - 9500) * 100)
10    elif u > 8000: rp = int((u - 8000) * 20)
11    res = int(br + sr + rp)
12    return res if res < 1000000 else 1000000
```

Listing 175: Source Python aave_v3_interest_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 // Assumes IntDivide(n) is available, following the style of the retrieved templates.
6 template aave_v3_interest_logic() {
7     signal input d;
8     signal input r;
9     signal input b;
10    signal input s1;
11    signal input s2;
12    signal input opt;
13
14    signal output out;
15
16    // if r == 0 { return b }
17    component rIsZero = IsZero();
18    rIsZero.in <== r;
19
20    signal safeR <== r + rIsZero.out;
21
22    // u := (d * 10000) / r
23    signal u;
24    (u, _) <== IntDivide(64)(d * 10000, safeR);
25
26    // if u <= opt { br = b + (u * s1 / opt) } else { br = b + s1 + ((u-opt)*s2
27    // (10000-opt)) }
28    component uLeOpt = LessEqThan(64);
29    uLeOpt.in[0] <== u;
30    uLeOpt.in[1] <== opt;
31
32    component uGtOpt = GreaterThan(64);
33    uGtOpt.in[0] <== u;
34    uGtOpt.in[1] <== opt;
35
36    component optIsZero = IsZero();
37    optIsZero.in <== opt;
38    signal safeOpt <== opt + optIsZero.out;
39
40    signal brLowPart;
```



```

40 (brLowPart, _) <== IntDivide(64)(u * s1, safeOpt);
41 signal brLow <== b + brLowPart;
42
43 signal denomHigh <== 10000 - opt;
44 component denomHighIsZero = IsZero();
45 denomHighIsZero.in <== denomHigh;
46 signal safeDenomHigh <== denomHigh + denomHighIsZero.out;
47
48 signal diffOpt <== uGtOpt.out * (u - opt);
49 signal brHighPart;
50 (brHighPart, _) <== IntDivide(64)(diffOpt * s2, safeDenomHigh);
51 signal brHigh <== b + s1 + brHighPart;
52
53 signal br <== uLeOpt.out * brLow + uGtOpt.out * brHigh;
54
55 // sr := ((br * u) / 10000 * 9000) / 10000
56 signal srBase;
57 (srBase, _) <== IntDivide(64)(br * u, 10000);
58
59 signal sr;
60 (sr, _) <== IntDivide(64)(srBase * 9000, 10000);
61
62 // rp := 0;
63 // if u > 9500 { rp = (u-9500)*100 } else if u > 8000 { rp = (u-8000)*20 }
64 component uGt9500 = GreaterThan(64);
65 uGt9500.in[0] <== u;
66 uGt9500.in[1] <== 9500;
67
68 component uGt8000 = GreaterThan(64);
69 uGt8000.in[0] <== u;
70 uGt8000.in[1] <== 8000;
71
72 signal rpHigh <== uGt9500.out * (u - 9500) * 100;
73 signal rpMidSel <== uGt8000.out * (1 - uGt9500.out);
74 signal rpMid <== rpMidSel * (u - 8000) * 20;
75 signal rp <== rpHigh + rpMid;
76
77 // res := br + sr + rp
78 signal res <== br + sr + rp;
79
80 // if res > 1000000 { return 1000000 }
81 component resGtCap = GreaterThan(64);
82 resGtCap.in[0] <== res;
83 resGtCap.in[1] <== 1000000;
84
85 signal cappedRes <== resGtCap.out * 1000000 + (1 - resGtCap.out) * res;
86
87 // if r == 0 return b else return cappedRes
88 out <== rIsZero.out * b + (1 - rIsZero.out) * cappedRes;
89 }
90
91 component main = aave_v3_interest_logic();

```

Listing 176: Faulty translated Circom code with Template Not Exist (TAC01)

```

1 error[TAC01]: The template IntDivide does not exist
2   "circuit.circom":24:16
3

```

```

4 24      (u, _) <== IntDivide(64)(d * 10000, safeR);
5          ~~~~~ This is the anonymous component
6      whose use is not allowed
7 previous errors were found

```

Listing 177: Circom compiler error log

Explanation: In this example, the circuit calls ‘IntDivide(64)’ and expects it to return ‘(quotient, remainder)’, but only ‘comparators.circom’ is included, and no ‘IntDivide’ template is available in scope. As a result, the compiler flags the ‘IntDivide(64)(...)’ call at line 24, and the same issue would affect the other division calls too until a valid division template is added or the code is rewritten without it.

E-4 Bad Assign Target (TAC02)

Description: This error happens when a Circom assignment or wiring operator such as ‘<==’ is used with an invalid left-hand side. In Circom, the left side must be something assignable like a signal, variable, component, or tuple of those, not an arithmetic expression. To fix it, move all computation to the right-hand side and keep only a plain assignable target on the left, or rewrite the relation as an explicit equality-style constraint using intermediate signals if needed.

Original Error Tag: TAC02

Example:

```

1 def ema_signal_logic(p0, p1, p2, p3, p4):
2     prices = [p0, p1, p2, p3, p4]
3     alpha = 40
4     ema = prices[0] * 100
5     for i in range(1, 5):
6         ema = (prices[i] * alpha) + (ema * (100 - alpha) // 100)
7     return int(ema)

```

Listing 178: Source Python ema_signal_logic implementation

```

1 pragma circom 2.1.4;
2
3 template ema_signal_logic() {
4     // Public inputs
5     signal input p0;
6     signal input p1;
7     signal input p2;
8     signal input p3;
9     signal input p4;
10
11     // Output
12     signal output out;
13
14     // Intermediate signals
15     signal ema0;
16     signal ema1;
17     signal ema2;
18     signal ema3;
19     signal ema4;
20

```

```

21 // Constant alpha = 40
22 var alpha = 40;
23 var hundred = 100;
24 var hundred_minus_alpha = 100 - alpha; // = 60
25
26 // Constraint: ema0 == p0 * 100
27 ema0 <== p0 * hundred;
28
29 // Constraint: 100 * ema1 == 100 * p1 * alpha + ema0 * (100 - alpha)
30 // Rearranged: ema1 == (p1 * alpha) + (ema0 * (100 - alpha) / 100)
31 hundred * ema1 <== hundred * p1 * alpha + ema0 * hundred_minus_alpha;
32
33 // Constraint: 100 * ema2 == 100 * p2 * alpha + ema1 * (100 - alpha)
34 hundred * ema2 <== hundred * p2 * alpha + ema1 * hundred_minus_alpha;
35
36 // Constraint: 100 * ema3 == 100 * p3 * alpha + ema2 * (100 - alpha)
37 hundred * ema3 <== hundred * p3 * alpha + ema2 * hundred_minus_alpha;
38
39 // Constraint: 100 * ema4 == 100 * p4 * alpha + ema3 * (100 - alpha)
40 hundred * ema4 <== hundred * p4 * alpha + ema3 * hundred_minus_alpha;
41
42 // Output constraint
43 out <== ema4;
44 }
45
46 component main = ema_signal_logic();

```

Listing 179: Faulty translated Circom code with Bad Assign Target (TAC02)

```

1 error[TAC02]: This expression must be a tuple, a component, a signal or a variable
2   "circuit.circom":33:5
3
4 33      hundred * ema1 <== hundred * p1 * alpha + ema0 * hundred_minus_alpha;
5         ~~~~~ This is the tuple whose use is not allowed
6
7 previous errors were found

```

Listing 180: Circom compiler error log

Explanation: In this example, ‘hundred * ema1 <== ...’ is invalid because ‘hundred * ema1’ is an expression, not a signal. The same issue appears for ‘ema2’, ‘ema3’, and ‘ema4’; you should assign to ‘ema1’, ‘ema2’, etc. directly and rearrange the formula so the multiplication by ‘hundred’ is handled on the right side or through extra intermediate constraints.

E-5 Missing Main (P1001)

Description: This error happens when a Circom project defines templates and helper logic but does not declare a top-level ‘main’ component for the compiler to build. Circom needs exactly one concrete entry point, usually something like ‘component main = SomeTemplate(...)’; so the fix is to instantiate the intended top-level template in ‘main’ and ensure it has all required parameters.

Original Error Tag: P1001

Example:

```

1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2   fee = (amt_in * tax_bips) // 10000

```

```

3  amt_eff = amt_in - fee
4  # x * y = k
5  amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6  for i in range(exp):
7      amt_out = (amt_out * 9995) // 10000
8  if amt_out < 0: return 0
9  return int(amt_out)

```

Listing 181: Source Python fixed_point_swap_slippage implementation

```

1  include "circomlib/circuits/comparators.circom";
2  include "circomlib/circuits/dividemod.circom";
3
4  // Template for division by a variable denominator
5  template DivModVariable() {
6      signal input num;
7      signal input den;
8      signal output quotient;
9      signal output remainder;
10
11     // num = den * quotient + remainder
12     // 0 <= remainder < den
13
14     quotient <== (num - remainder) / den;
15
16     // Constraint: num == den * quotient + remainder
17     component check = IsEqual();
18     check.in[0] <== den * quotient + remainder;
19     check.in[1] <== num;
20     check.out === 1;
21
22     // Constraint: remainder < den (and remainder >= 0)
23     component lt = LessThan(253); // assuming 253-bit field safety
24     lt.in[0] <== remainder;
25     lt.in[1] <== den;
26     lt.out === 1;
27
28     // Ensure remainder >= 0 (implicit in field, but we assume non-negative inputs)
29 }
30
31 template FixedPointSwapSlippage(e) {
32     signal input ai;
33     signal input ri;
34     signal input ro;
35     signal input tb;
36     signal output out;
37
38     // Step 1: Calculate fee f = floor((ai * tb) / 10000)
39     component feeDiv = DivMod(10000);
40     feeDiv.in <== ai * tb;
41     signal f <== feeDiv.quotient;
42     // rem1 is feeDiv.remainder (automatically constrained)
43
44     // Step 2: Effective amount after fee: ae = ai - f
45     signal ae <== ai - f;
46
47     // Step 3: Denominator: denom = ri + ae
48     signal denom <== ri + ae;

```

```

49 // Ensure denom > 0 (pool invariant)
50 component gtZero = GreaterThan(253);
51 gtZero.in[0] <= denom;
52 gtZero.in[1] <= 0;
53 // We'll use this constraint implicitly by ensuring valid pool state
54
55 // Step 4: Initial output: ao0 = floor((ae * ro) / denom)
56 component initDiv = DivModVariable();
57 initDiv.num <= ae * ro;
58 initDiv.den <= denom;
59 signal ao0 <= initDiv.quotient;
60 // rem2 is initDiv.remainder (automatically constrained)
61
62 // Step 5: Slippage loop - unrolled for e iterations
63 signal ao[e + 1];
64 ao[0] <= ao0;
65
66 for (var i = 0; i < e; i++) {
67     component slipDiv = DivMod(10000);
68     slipDiv.in <= ao[i] * 9995;
69     ao[i + 1] <= slipDiv.quotient;
70     // Each slipDiv automatically constrains remainder < 10000
71 }
72
73 // Step 6: Final output (clamp to non-negative, though should be non-negative
74 // already)
75 out <= ao[e];
76
77 // Additional safety: ensure out >= 0
78 // Since all operations preserve non-negativity with non-negative inputs,
79 // this is theoretically redundant but good practice
80 component nonNeg = GreaterEqThan(253);
81 nonNeg.in[0] <= out;
82 nonNeg.in[1] <= 0;
83 // We don't need to explicitly constrain this as it should always be true
84 }
85
86 // Example instantiation - you would set e based on your specific use case
87 // component main = FixedPointSwapSlippage(3); // for 3 slippage rounds

```

Listing 182: Faulty translated Circom code with Missing Main (P1001)

```

1 error[P1001]: No main specified in the project structure
2
3 previous errors were found

```

Listing 183: Circom compiler error log

Explanation: In this example, the circuit defines ‘DivModVariable’ and ‘FixedPointSwapSlippage(e)’, but the only ‘main’ instantiation is commented out. Because ‘// component main = FixedPointSwapSlippage(3);’ is not active, the compiler finds no entry point and raises ‘P1001’.

F Wrong Argument Count

F-1 Wrong Argument Count (T2012)

Description: This error happens when a Circom template or function is called with an argument count that does not match its definition. The fix is to check the template/function signature, then instantiate it with exactly the required number of parameters in the correct order, or change the definition if it should take a different number of compile-time arguments.

Original Error Tag: T2012

Example:

```
1 def fixed_point_swap_slippage(amt_in, res_in, res_out, tax_bips, exp):
2     fee = (amt_in * tax_bips) // 10000
3     amt_eff = amt_in - fee
4     # x * y = k
5     amt_out = (amt_eff * res_out) // (res_in + amt_eff)
6     for i in range(exp):
7         amt_out = (amt_out * 9995) // 10000
8     if amt_out < 0: return 0
9     return int(amt_out)
```

Listing 184: Source Python fixed_point_swap_slippage implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template fixed_point_swap_slippage(exp) {
6     signal input amt_in;
7     signal input res_in;
8     signal input res_out;
9     signal input tax_bips;
10    signal output out;
11
12    // fee = (amt_in * tax_bips) // 10000
13    signal fee;
14    signal fee_rem;
15    fee <-- (amt_in * tax_bips) \ 10000;
16    fee_rem <-- (amt_in * tax_bips) % 10000;
17    amt_in * tax_bips === fee * 10000 + fee_rem;
18
19    component fee_rem_lt = LessThan(64);
20    fee_rem_lt.in[0] <== fee_rem;
21    fee_rem_lt.in[1] <== 10000;
22    fee_rem_lt.out === 1;
23
24    // amt_eff = amt_in - fee
25    signal amt_eff <== amt_in - fee;
26
27    // amt_out = (amt_eff * res_out) // (res_in + amt_eff)
28    signal amt_out_0;
29    signal amt_out_0_rem;
30    amt_out_0 <-- (amt_eff * res_out) \ (res_in + amt_eff);
31    amt_out_0_rem <-- (amt_eff * res_out) % (res_in + amt_eff);
32    amt_eff * res_out === amt_out_0 * (res_in + amt_eff) + amt_out_0_rem;
33}
```

```

34 component amt_out_0_rem_lt = LessThan(252);
35 amt_out_0_rem_lt.in[0] <== amt_out_0_rem;
36 amt_out_0_rem_lt.in[1] <== res_in + amt_eff;
37 amt_out_0_rem_lt.out === 1;
38
39 // for i in range(exp):
40 //     amt_out = (amt_out * 9995) // 10000
41 signal amt_out_arr[exp + 1];
42 amt_out_arr[0] <== amt_out_0;
43
44 signal amt_out_rem[exp > 0 ? exp : 1];
45 component amt_out_rem_lt[exp > 0 ? exp : 1];
46
47 for (var i = 0; i < exp; i++) {
48     amt_out_arr[i+1] <-- (amt_out_arr[i] * 9995) \ 10000;
49     amt_out_rem[i] <-- (amt_out_arr[i] * 9995) % 10000;
50     amt_out_arr[i] * 9995 === amt_out_arr[i+1] * 10000 + amt_out_rem[i];
51
52     amt_out_rem_lt[i] = LessThan(64);
53     amt_out_rem_lt[i].in[0] <== amt_out_rem[i];
54     amt_out_rem_lt[i].in[1] <== 10000;
55     amt_out_rem_lt[i].out === 1;
56 }
57
58 // if amt_out < 0: return 0
59 // (In field arithmetic with valid positive inputs, amt_out is always >= 0)
60 out <== amt_out_arr[exp];
61 }

```

Listing 185: Faulty translated Circom code with Wrong Argument Count (T2012)

```

1 error[T2012]: Calling function with wrong number of arguments
2   "circuit.circom":63:18
3
4 63 component main = fixed_point_swap_slippage();
5   ~~~~~~ Got 0 params, 1 where expected
6
7 previous errors were found

```

Listing 186: Circom compiler error log

Explanation: In this example, ‘fixed_point_swap_slippage’ is defined as ‘template fixed_point_swap_slippage(exp)’ so it requires one template parameter, ‘exp’. But ‘component main = fixed_point_swap_slippage();’ passes zero parameters, so Circom reports that it got 0 arguments where 1 was expected.

G Type Mismatch

G-1 Dimension Mismatch (T2061)

Description: This error class happens when a Circom assignment connects signals with incompatible shapes, such as assigning a scalar value to an array signal or vice versa. To fix it, make sure the left-hand and right-hand sides have the same dimensions: assign to a single indexed element if you want one value, or declare a separate scalar signal if the result is meant to be a single accumulated value.

Original Error Tag: T2061

Example:

```
1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)
```

Listing 187: Source Python quicksort_partition_logic implementation

```
1 pragma circom 2.1.4;
2
3 include "node_modules/circomlib/circuits/comparators.circom";
4
5 template quicksort_partition_logic() {
6     signal input a0;
7     signal input a1;
8     signal input a2;
9     signal input a3;
10    signal input a4;
11    signal input a5;
12    signal input a6;
13    signal input low;
14    signal input high;
15
16    signal output out;
17
18    signal arr[8][7];
19    signal iVal[8];
20    signal swaps[8];
21    signal pivot;
22    signal highSelSum;
23
24    component eqHigh[7];
25    signal highSel[7];
26    signal highSelSumAcc[8];
27    signal pivotAcc[8];
28
29    arr[0][0] <== a0;
30    arr[0][1] <== a1;
31    arr[0][2] <== a2;
32    arr[0][3] <== a3;
33    arr[0][4] <== a4;
34    arr[0][5] <== a5;
35    arr[0][6] <== a6;
36
37    highSelSumAcc[0] <== 0;
38    pivotAcc[0] <== 0;
39}
```



```

40  for (var k = 0; k < 7; k++) {
41      eqHigh[k] = IsEqual();
42      eqHigh[k].in[0] <= high;
43      eqHigh[k].in[1] <= k;
44      highSel[k] <= eqHigh[k].out;
45      highSelSumAcc[k + 1] <= highSelSumAcc[k] + highSel[k];
46      pivotAcc[k + 1] <= pivotAcc[k] + highSel[k] * arr[0][k];
47  }
48
49  highSel <= highSelSumAcc[7];
50  pivot <= pivotAcc[7];
51
52  highSelSum === 1;
53
54  iVal[0] <= low - 1;
55  swaps[0] <= 0;
56
57  component geLow[7];
58  component ltHigh[7];
59  component lePivot[7];
60  component eqTarget[7][7];
61
62  signal inRange[7];
63  signal doSwap[7];
64  signal target[7];
65  signal arrAtTarget[7];
66  signal arrAtTargetAcc[7][8];
67  signal finalPos;
68
69  for (var j = 0; j < 7; j++) {
70      geLow[j] = GreaterEqThan(64);
71      geLow[j].in[0] <= j;
72      geLow[j].in[1] <= low;
73
74      ltHigh[j] = LessThan(64);
75      ltHigh[j].in[0] <= j;
76      ltHigh[j].in[1] <= high;
77
78      inRange[j] <= geLow[j].out * ltHigh[j].out;
79
80      lePivot[j] = LessEqThan(64);
81      lePivot[j].in[0] <= arr[j][j];
82      lePivot[j].in[1] <= pivot;
83
84      doSwap[j] <= inRange[j] * lePivot[j].out;
85
86      target[j] <= iVal[j] + 1;
87      arrAtTargetAcc[j][0] <= 0;
88
89      for (var p = 0; p < 7; p++) {
90          eqTarget[j][p] = IsEqual();
91          eqTarget[j][p].in[0] <= target[j];
92          eqTarget[j][p].in[1] <= p;
93
94          arrAtTargetAcc[j][p + 1] <= arrAtTargetAcc[j][p] + eqTarget[j][p].out *
95          arr[j][p];
96      }

```

```

97     arrAtTarget[j] <== arrAtTargetAcc[j][7];
98
99     iVal[j + 1] <== iVal[j] + doSwap[j];
100    swaps[j + 1] <== swaps[j] + doSwap[j];
101
102    for (var p = 0; p < 7; p++) {
103        if (p == j) {
104            arr[j + 1][p] <== doSwap[j] * (
105                eqTarget[j][p].out * arr[j][j] +
106                (1 - eqTarget[j][p].out) * arrAtTarget[j]
107            ) + (1 - doSwap[j]) * arr[j][p];
108        } else {
109            arr[j + 1][p] <== doSwap[j] * (
110                eqTarget[j][p].out * arr[j][j] +
111                (1 - eqTarget[j][p].out) * arr[j][p]
112            ) + (1 - doSwap[j]) * arr[j][p];
113        }
114    }
115 }
116
117 finalPos <== iVal[7] + 1;
118 out <== finalPos * 100 + swaps[7];
119 }
120
121 component main = quicksort_partition_logic();

```

Listing 188: Faulty translated Circom code with Dimension Mismatch (T2061)

```

1 error[T2061]: Typing error found
2     "circuit.circom":49:5
3
4 49     highSel <== highSelSumAcc[7];
5     ~~~~~ Assignee and assigned types do not match.
6 Expected dimensions: 1, found 0
7
8 previous errors were found

```

Listing 189: Circom compiler error log

Explanation: In this example, ‘highSel’ is declared as an array ‘signal highSel[7]’, but ‘highSelSumAcc[7]’ is a scalar, so ‘highSel <== highSelSumAcc[7];’ causes the dimension mismatch. The code likely meant ‘highSelSum <== highSelSumAcc[7];’, especially since ‘highSelSum == 1;’ is checked right after.

G-2 Tag on Signal (T2048)

Description: This error class happens when Circom’s type system sees you using the wrong kind of object for member access, usually by assigning a template instance to a ‘signal’ instead of declaring a ‘component’. Tags like ‘in’ and ‘out’ only exist on component instances whose interface is defined by the template, so the fix is to instantiate the template with ‘component x = MyTemplate();’ and reserve ‘signal’ for field values and expressions.

Original Error Tag: T2048

Example:

```

1 def liquidity_mining_decay(base_r, time_p, halving_p, boost_m):
2     if halving_p == 0: return 0
3     current_r = base_r
4     stages = time_p // halving_p
5     for _ in range(int(stages)):
6         current_r = (current_r * 50) // 100
7         if current_r < 10: break
8     rem = time_p % halving_p
9     decay = (current_r * rem * 50) // (halving_p * 100)
10    final_r = (current_r - decay) * boost_m // 100
11    return int(final_r)

```

Listing 190: Source Python liquidity_mining_decay implementation

```

1 pragma circom 2.1.0;
2 include "node_modules/circomlib/circuits/comparators.circom";
3
4 template CustomIsZero() {
5     signal input in;
6     signal output out;
7
8     signal inv <-- 1 / (in + (in == 0));
9     out <== 1 - (in * inv);
10    in * (1 - out) == 0;
11 }
12
13 template DecayLoop() {
14     signal input in_cr;
15     signal input in_st;
16     signal output out_cr;
17
18     signal isStZero <== CustomIsZero();
19     isStZero.in <== in_st;
20
21     signal new_cr <== in_cr * 50 \ 100;
22
23     component lt10 = LessThan(64);
24     lt10.in[0] <== new_cr;
25     lt10.in[1] <== 10;
26
27     signal continue_cr <== (1 - lt10.out) * new_cr + lt10.out * in_cr;
28
29     component next = DecayLoop();
30     next.in_cr <== continue_cr;
31     next.in_st <== in_st - 1;
32
33     out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
34 }
35
36 template LiquidityMiningDecay() {
37     signal input br;
38     signal input tp;
39     signal input hp;
40     signal input bm;
41     signal output fr;
42
43     signal isHpZero <== CustomIsZero();

```

```

44  isHpZero.in <== hp;
45
46  signal st <== tp \ hp;
47
48  signal cr_init <== br;
49
50  component loop = DecayLoop();
51  loop.in_cr <== cr_init;
52  loop.in_st <== st;
53
54  signal rem <== tp - (st * hp);
55
56  signal cr_after_loop <== loop.out_cr;
57  signal numerator <== cr_after_loop * rem * 50;
58  signal denominator <== hp * 100;
59  signal d <== numerator \ denominator;
60
61  signal cr_minus_d <== cr_after_loop - d;
62  signal fr_raw <== cr_minus_d * bm;
63  signal fr_calc <== fr_raw \ 100;
64
65  fr <== (1 - isHpZero.out) * fr_calc;
66 }
67
68 component main = LiquidityMiningDecay();

```

Listing 191: Faulty translated Circom code with Tag on Signal (T2048)

```

1  error[T2060]: Typing error found
2  "circuit.circom":44:5
3
4  44      signal isHpZero <== CustomIsZero();
5  ~~~~~ Assignee and assigned types do not match.
6  Expected expression found template.
7
8  error[T2048]: Typing error found
9  "circuit.circom":45:5
10
11  45      isHpZero.in <== hp;
12  ~~~~~ Tag not found in signal: only accesses to tags that appear
13  in the definition of the signal are allowed
14
15  error[T2060]: Typing error found
16  "circuit.circom":19:5
17
18  19      signal isStZero <== CustomIsZero();
19  ~~~~~ Assignee and assigned types do not match.
20  Expected expression found template.
21
22  error[T2048]: Typing error found
23  "circuit.circom":20:5
24
25  20      isStZero.in <== in_st;
26  ~~~~~ Tag not found in signal: only accesses to tags that
27  appear in the definition of the signal are allowed
28
29  error[T2048]: Typing error found
30  "circuit.circom":34:16

```

```

29
30 34      out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
31      ~~~~~ Tag not found in signal: only accesses to tags that
      appear in the definition of the signal are allowed
32
33 error[T2048]: Typing error found
34      "circuit.circom":34:44
35
36 34      out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
37      ~~~~~ Tag not found in signal:
      only accesses to tags that appear in the definition of the signal are allowed
38
39 error[T2048]: Typing error found
40      "circuit.circom":66:17
41
42 66      fr <== (1 - isHpZero.out) * fr_calc;
43      ~~~~~ Tag not found in signal: only accesses to tags that
      appear in the definition of the signal are allowed
44
45 previous errors were found

```

Listing 192: Circom compiler error log

Explanation: In this example, ‘CustomIsZero()’ is being instantiated as ‘signal isStZero <== CustomIsZero();’ and ‘signal isHpZero <== CustomIsZero();’, which makes Circom treat them as signals rather than components. Because of that, accesses like ‘isStZero.in’, ‘isStZero.out’, and ‘isHpZero.out’ fail; they should instead be declared as ‘component isStZero = CustomIsZero();’ and ‘component isHpZero = CustomIsZero();’.

G-3 = on Signal (T2054)

Description: This error class happens when Circom code uses the wrong assignment operator for a signal. In Circom, ‘=’ is not used to assign values to signals; signal constraints must be written with ‘<==’ for constrained assignments or ‘<-’ for witness-only assignments. A common fix is to replace ‘=’ with the appropriate signal operator, and if you are trying to update a value repeatedly, restructure the code into intermediate signals like ‘inv0’, ‘inv1’, ‘inv2’, etc., because signals are not mutable variables you can reassign imperatively.

Original Error Tag: T2054

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1
10    return int(inv + iv)

```

Listing 193: Source Python shuffle_integrity_logic implementation

```

1 include "node_modules/circomlib/circuits/comparators.circom";

```

```

2 include "node_modules/circomlib/circuits/multiplexer.circom";
3
4 template ShuffleIntegrityLogic() {
5     signal input c0;
6     signal input c1;
7     signal input c2;
8     signal input c3;
9     signal input c4;
10    signal input s0;
11    signal input s1;
12    signal input s2;
13    signal input s3;
14    signal input s4;
15    signal input sd;
16    signal output out;
17
18    // Check sum equality:  $c_0+c_1+c_2+c_3+c_4 == s_0+s_1+s_2+s_3+s_4$ 
19    signal c_sum <== c0 + c1 + c2 + c3 + c4;
20    signal s_sum <== s0 + s1 + s2 + s3 + s4;
21    signal sum_check <== (c_sum == s_sum) ? 1 : 0;
22
23    // If sum check fails, return 0
24    signal sum_fail <== 1 - sum_check;
25    signal sum_fail_out <== sum_fail * 0;
26
27    // Check sum of squares equality:  $c_0^2+c_1^2+c_2^2+c_3^2+c_4^2 == s_0^2+s_1^2+s_2^2+s_3^2+s_4^2$ 
28    signal c0_sq <== c0 * c0;
29    signal c1_sq <== c1 * c1;
30    signal c2_sq <== c2 * c2;
31    signal c3_sq <== c3 * c3;
32    signal c4_sq <== c4 * c4;
33    signal c_sq_sum <== c0_sq + c1_sq + c2_sq + c3_sq + c4_sq;
34
35    signal s0_sq <== s0 * s0;
36    signal s1_sq <== s1 * s1;
37    signal s2_sq <== s2 * s2;
38    signal s3_sq <== s3 * s3;
39    signal s4_sq <== s4 * s4;
40    signal s_sq_sum <== s0_sq + s1_sq + s2_sq + s3_sq + s4_sq;
41
42    signal sq_sum_check <== (c_sq_sum == s_sq_sum) ? 1 : 0;
43    signal sq_sum_fail <== 1 - sq_sum_check;
44    signal sq_sum_fail_out <== sq_sum_fail * 0;
45
46    // Combined failure check
47    signal both_checks_pass <== sum_check * sq_sum_check;
48    signal any_fail <== 1 - both_checks_pass;
49
50    // If any check fails, return 0
51    signal failure_out <== any_fail * 0;
52
53    // If checks pass, continue with the logic
54    signal checks_pass <== both_checks_pass;
55
56    // Check if any  $c_i$  equals  $s_0$ 
57    signal c0_eq_s0 <== (c0 == s0) ? 1 : 0;
58    signal c1_eq_s0 <== (c1 == s0) ? 1 : 0;

```

```

59 signal c2_eq_s0 <== (c2 == s0) ? 1 : 0;
60 signal c3_eq_s0 <== (c3 == s0) ? 1 : 0;
61 signal c4_eq_s0 <== (c4 == s0) ? 1 : 0;
62 signal any_eq_s0 <== c0_eq_s0 + c1_eq_s0 + c2_eq_s0 + c3_eq_s0 + c4_eq_s0;
63 signal iv <== (any_eq_s0 > 0) ? 100 : 0;
64
65 // Calculate inversions in s array
66 signal s_array[5];
67 s_array[0] <== s0;
68 s_array[1] <== s1;
69 s_array[2] <== s2;
70 s_array[3] <== s3;
71 s_array[4] <== s4;
72
73 signal inv <== 0;
74
75 // Manual unrolling of nested loops
76 component cmp_0_1 = LessThan(64);
77 cmp_0_1.in[0] <== s_array[1];
78 cmp_0_1.in[1] <== s_array[0];
79 inv = inv + cmp_0_1.out;
80
81 component cmp_0_2 = LessThan(64);
82 cmp_0_2.in[0] <== s_array[2];
83 cmp_0_2.in[1] <== s_array[0];
84 inv = inv + cmp_0_2.out;
85
86 component cmp_0_3 = LessThan(64);
87 cmp_0_3.in[0] <== s_array[3];
88 cmp_0_3.in[1] <== s_array[0];
89 inv = inv + cmp_0_3.out;
90
91 component cmp_0_4 = LessThan(64);
92 cmp_0_4.in[0] <== s_array[4];
93 cmp_0_4.in[1] <== s_array[0];
94 inv = inv + cmp_0_4.out;
95
96 component cmp_1_2 = LessThan(64);
97 cmp_1_2.in[0] <== s_array[2];
98 cmp_1_2.in[1] <== s_array[1];
99 inv = inv + cmp_1_2.out;
100
101 component cmp_1_3 = LessThan(64);
102 cmp_1_3.in[0] <== s_array[3];
103 cmp_1_3.in[1] <== s_array[1];
104 inv = inv + cmp_1_3.out;
105
106 component cmp_1_4 = LessThan(64);
107 cmp_1_4.in[0] <== s_array[4];
108 cmp_1_4.in[1] <== s_array[1];
109 inv = inv + cmp_1_4.out;
110
111 component cmp_2_3 = LessThan(64);
112 cmp_2_3.in[0] <== s_array[3];
113 cmp_2_3.in[1] <== s_array[2];
114 inv = inv + cmp_2_3.out;
115
116 component cmp_2_4 = LessThan(64);

```

```

117     cmp_2_4.in[0] <= s_array[4];
118     cmp_2_4.in[1] <= s_array[2];
119     inv = inv + cmp_2_4.out;
120
121     component cmp_3_4 = LessThan(64);
122     cmp_3_4.in[0] <= s_array[4];
123     cmp_3_4.in[1] <= s_array[3];
124     inv = inv + cmp_3_4.out;
125
126     // Final result: inv + iv if checks pass, 0 otherwise
127     signal success_result <= inv + iv;
128     out <= checks_pass * success_result + failure_out;
129 }
130
131 component main = ShuffleIntegrityLogic();

```

Listing 194: Faulty translated Circom code with

```

1 error[T2054]: Typing error found
2   "circuit.circom":80:5
3
4 80     inv = inv + cmp_0_1.out;
5     ~~~~~ The operator does not match the types of the assigned
6     elements.
7     Assignments to signals do not allow the operator =, try using <= or <-- instead
8
9 error[T2054]: Typing error found
10  "circuit.circom":85:5
11
12 85     inv = inv + cmp_0_2.out;
13     ~~~~~ The operator does not match the types of the assigned
14     elements.
15     Assignments to signals do not allow the operator =, try using <= or <-- instead
16
17 error[T2054]: Typing error found
18  "circuit.circom":90:5
19
20 90     inv = inv + cmp_0_3.out;
21     ~~~~~ The operator does not match the types of the assigned
22     elements.
23     Assignments to signals do not allow the operator =, try using <= or <-- instead
24
25 error[T2054]: Typing error found
26  "circuit.circom":95:5
27
28 95     inv = inv + cmp_0_4.out;
29     ~~~~~ The operator does not match the types of the assigned
30     elements.
31     Assignments to signals do not allow the operator =, try using <= or <-- instead
32
33 error[T2054]: Typing error found
34  "circuit.circom":100:5
35
36 100    inv = inv + cmp_1_2.out;
37     ~~~~~ The operator does not match the types of the assigned
38     elements.
39     Assignments to signals do not allow the operator =, try using <= or <-- instead

```



```

36 error[T2054]: Typing error found
37     "circuit.circom":105:5
38
39 105     inv = inv + cmp_1_3.out;
40     ~~~~~~ The operator does not match the types of the assigned
41     elements.
42     Assignments to signals do not allow the operator =, try using <== or <-- instead
43 error[T2054]: Typing error found
44     "circuit.circom":110:5
45
46 110     inv = inv + cmp_1_4.out;
47     ~~~~~~ The operator does not match the types of the assigned
48     elements.
49     Assignments to signals do not allow the operator =, try using <== or <-- instead
50 error[T2054]: Typing error found
51     "circuit.circom":115:5
52
53 115     inv = inv + cmp_2_3.out;
54     ~~~~~~ The operator does not match the types of the assigned
55     elements.
56     Assignments to signals do not allow the operator =, try using <== or <-- instead
57 error[T2054]: Typing error found
58     "circuit.circom":120:5
59
60 120     inv = inv + cmp_2_4.out;
61     ~~~~~~ The operator does not match the types of the assigned
62     elements.
63     Assignments to signals do not allow the operator =, try using <== or <-- instead
64 error[T2054]: Typing error found
65     "circuit.circom":125:5
66
67 125     inv = inv + cmp_3_4.out;
68     ~~~~~~ The operator does not match the types of the assigned
69     elements.
70     Assignments to signals do not allow the operator =, try using <== or <-- instead
71 previous errors were found

```

Listing 195: Circom compiler error log

Explanation: In this example, ‘inv’ is declared as a signal with ‘signal inv <== 0;’ and then later treated like a mutable accumulator with lines such as ‘inv = inv + cmp_0_1.out;’. That is invalid in Circom: ‘inv’ cannot be updated with ‘=’, and it also should not be repeatedly reassigned, so the inversion count must be built using a chain of new signals or a single summed expression over all comparator outputs.

G-4 Operator Type Error (T2028)

Description: This error class is caused by using signals with incompatible types or dimensions in expressions and assignments, most commonly mixing a scalar signal with an array signal. In Circom, some component outputs such as ‘Num2Bits(n).out’ are arrays of bits, not a single field element, so they cannot be added, multiplied, or assigned where a scalar is expected; the fix is to use the correct

scalar signal from the component, reconstruct the value from the bit array if needed, or replace the component with one whose output type matches the intended arithmetic.

Original Error Tag: T2028

Example:

```

1 def liquidity_mining_decay(base_r, time_p, halving_p, boost_m):
2     if halving_p == 0: return 0
3     current_r = base_r
4     stages = time_p // halving_p
5     for _ in range(int(stages)):
6         current_r = (current_r * 50) // 100
7         if current_r < 10: break
8     rem = time_p % halving_p
9     decay = (current_r * rem * 50) // (halving_p * 100)
10    final_r = (current_r - decay) * boost_m // 100
11    return int(final_r)

```

Listing 196: Source Python liquidity_mining_decay implementation

```

1 pragma circom 2.1.0;
2 include "node_modules/circomlib/circuits/comparators.circom";
3 include "node_modules/circomlib/circuits/bitify.circom";
4
5 template Divide(n) {
6     signal input in[2];
7     signal output out;
8
9     component rangeCheck = Num2Bits(n);
10    rangeCheck.in <== in[0] - out * in[1];
11    out * in[1] + rangeCheck.out == in[0];
12    component lt = LessThan(n);
13    lt.in[0] <== rangeCheck.out;
14    lt.in[1] <== in[1];
15    lt.out == 1;
16 }
17
18 template Modulo(n) {
19     signal input in[2];
20     signal output out;
21
22     component div = Divide(n);
23     div.in[0] <== in[0];
24     div.in[1] <== in[1];
25     out <== in[0] - div.out * in[1];
26 }
27
28 template MultDiv(n) {
29     signal input a;
30     signal input b;
31     signal input c;
32     signal input d;
33     signal output out;
34
35     component rangeCheck = Num2Bits(n);
36     rangeCheck.in <== a * b * c - out * d;
37     out * d + rangeCheck.out == a * b * c;
38     component lt = LessThan(n);

```

```

39     lt.in[0] <== rangeCheck.out;
40     lt.in[1] <== d;
41     lt.out == 1;
42 }
43
44 template SingleDecayStep() {
45     signal input prev_cr;
46     signal input step_index;
47     signal input total_steps;
48     signal output next_cr;
49
50     component lt = LessThan(64);
51     lt.in[0] <== step_index;
52     lt.in[1] <== total_steps;
53     signal should_execute <== lt.out;
54
55     component halve = Divide(128);
56     halve.in[0] <== prev_cr * 50;
57     halve.in[1] <== 100;
58     signal halved_cr <== halve.out;
59
60     component lt10 = LessThan(64);
61     lt10.in[0] <== halved_cr;
62     lt10.in[1] <== 10;
63     signal break_condition <== lt10.out;
64
65     signal continue_execution <== should_execute * (1 - break_condition);
66     next_cr <== continue_execution * halved_cr + (1 - continue_execution) * prev_cr;
67 }
68
69 template liquidity_mining_decay() {
70     signal input br;
71     signal input tp;
72     signal input hp;
73     signal input bm;
74     signal output fr;
75
76     component isZero = IsZero();
77     isZero.in <== hp;
78
79     signal nonZeroHP <== 1 - isZero.out;
80
81     component div = Divide(128);
82     div.in[0] <== tp;
83     div.in[1] <== hp;
84     signal st <== div.out;
85
86     signal cr_init <== br;
87
88     var n = 10;
89     component decay[n];
90     for (var i = 0; i < n; i++) {
91         decay[i] = SingleDecayStep();
92         if (i == 0) {
93             decay[i].prev_cr <== cr_init;
94         } else {
95             decay[i].prev_cr <== decay[i-1].next_cr;
96         }
97     }
98 }

```

```

97     decay[i].step_index <== i;
98     decay[i].total_steps <== st;
99 }
100
101 signal cr_after_loop <== decay[n-1].next_cr;
102
103 component mod = Modulo(128);
104 mod.in[0] <== tp;
105 mod.in[1] <== hp;
106 signal rem <== mod.out;
107
108 component numerator = MultDiv(256);
109 numerator.a <== cr_after_loop;
110 numerator.b <== rem;
111 numerator.c <== 50;
112 numerator.d <== hp * 100;
113 signal d <== numerator.out;
114
115 signal cr_minus_d <== cr_after_loop - d;
116
117 component finalMultDiv = MultDiv(256);
118 finalMultDiv.a <== cr_minus_d;
119 finalMultDiv.b <== bm;
120 finalMultDiv.c <== 1;
121 finalMultDiv.d <== 100;
122 signal raw_fr <== finalMultDiv.out;
123
124 fr <== nonZeroHP * raw_fr;
125 }
126
127 component main = liquidity_mining_decay();

```

Listing 197: Faulty translated Circom code with Operator Type Error (T2028)

```

1 error[T2028]: Typing error found
2   "circuit.circom":11:19
3
4 11     out * in[1] + rangeCheck.out === in[0];
5           ~~~~~ Type not allowed by the operator
6
7 error[T2061]: Typing error found
8   "circuit.circom":13:5
9
10 13     lt.in[0] <== rangeCheck.out;
11           ~~~~~ Assignee and assigned types do not match.
12 Expected dimensions: 0, found 1
13
14 error[T2028]: Typing error found
15   "circuit.circom":37:15
16
17 37     out * d + rangeCheck.out === a * b * c;
18           ~~~~~ Type not allowed by the operator
19
20 error[T2061]: Typing error found
21   "circuit.circom":39:5
22
23 39     lt.in[0] <== rangeCheck.out;
24           ~~~~~ Assignee and assigned types do not match.

```

```

25 Expected dimensions: 0, found 1
26
27 previous errors were found

```

Listing 198: Circom compiler error log

Explanation: In this example, ‘rangeCheck’ is a ‘Num2Bits(n)’ component, and ‘rangeCheck.out’ is a 1-dimensional array of bits, but the code uses it like a single number in ‘out * in[1] + rangeCheck.out’ and assigns it to ‘lt.in[0]’, which expects a scalar. The intended remainder/value is actually ‘rangeCheck.in <= in[0] - out * in[1]’ (and similarly ‘a * b * c - out * d’), while ‘rangeCheck.out’ should only be used as the bit decomposition of that scalar.

G-5 Component vs Value (T2057)

Description: This error class happens in Circom when a symbol is declared with the wrong kind of type for how it is later used, especially when ‘component’ objects are treated like normal values. A ‘component’ must hold an instance of a template, while arithmetic values, inputs, and intermediate results should be stored in ‘signal’ or ‘var’ declarations depending on whether they are part of the circuit or compile-time logic. To fix it, declare data containers with the correct type, instantiate components only from templates, and avoid assigning plain expressions or signals into ‘component’ variables.

Original Error Tag: T2057

Example:

```

1 def stack_rpn_calculator(o0, o1, o2, o3, o4):
2     ops = [o0, o1, o2, o3, o4]
3     stack = [0] * 5
4     top = -1
5     for i in range(5):
6         v = ops[i]
7         if v > 0:
8             top += 1
9             stack[top] = v
10        elif v < 0:
11            v2 = stack[top]; top -= 1
12            v1 = stack[top]; top -= 1
13            res = 0
14            if v == -1: res = v1 + v2
15            if v == -2: res = v1 - v2
16            if v == -3: res = v1 * v2
17            top += 1
18            stack[top] = res
19    return int(stack[0] + top)

```

Listing 199: Source Python stack_rpn_calculator implementation

```

1 template stack_rpn_calculator() {
2     // Declare input signals
3     signal input o0;
4     signal input o1;
5     signal input o2;
6     signal input o3;
7     signal input o4;
8

```

```

9 // Declare output signals
10 signal output out;
11
12 // Declare a component to represent the stack
13 component stack[5];
14
15 // Initialize the stack with zeros
16 for (var i = 0; i < 5; i++) {
17     stack[i] = 0;
18 }
19
20 // Initialize the top of the stack to -1
21 var top = -1;
22
23 // Define a component to represent the operations
24 component ops[5];
25
26 // Initialize the operations with the input values
27 ops[0] = o0;
28 ops[1] = o1;
29 ops[2] = o2;
30 ops[3] = o3;
31 ops[4] = o4;
32
33 // Define a component to represent the result
34 component res;
35
36 // Iterate over the operations
37 for (var i = 0; i < 5; i++) {
38     // Get the current operation
39     var v = ops[i];
40
41     // If the operation is positive, push it onto the stack
42     if (v > 0) {
43         top++;
44         stack[top] = v;
45     }
46     // If the operation is negative, pop two values from the stack, perform the
47     // operation, and push the result back onto the stack
48     else if (v < 0) {
49         // Get the top two values from the stack
50         var v2 = stack[top];
51         top--;
52         var v1 = stack[top];
53         top--;
54
55         // Perform the operation
56         if (v == -1) {
57             res = v1 + v2;
58         } else if (v == -2) {
59             res = v1 - v2;
60         } else if (v == -3) {
61             res = v1 * v2;
62         }
63
64         // Push the result back onto the stack
65         top++;
66         stack[top] = res;

```

```

66     }
67 }
68
69 // The final result is the top value on the stack plus the number of operations
70 // that were not pushed onto the stack
71 out <== stack[0] + top;
72 }

```

Listing 200: Faulty translated Circom code with Component vs Value (T2057)

```

1 error[T2057]: Typing error found
2   "circuit.circom":21:9
3
4 21         stack[i] = 0;
5           ~~~~~ Assignee and assigned types do not match.
6 Expected template but found expression.
7
8 error[T2057]: Typing error found
9   "circuit.circom":31:5
10
11 31         ops[0] = o0;
12           ~~~~~ Assignee and assigned types do not match.
13 Expected template but found expression.
14
15 error[T2057]: Typing error found
16   "circuit.circom":32:5
17
18 32         ops[1] = o1;
19           ~~~~~ Assignee and assigned types do not match.
20 Expected template but found expression.
21
22 error[T2057]: Typing error found
23   "circuit.circom":33:5
24
25 33         ops[2] = o2;
26           ~~~~~ Assignee and assigned types do not match.
27 Expected template but found expression.
28
29 error[T2057]: Typing error found
30   "circuit.circom":34:5
31
32 34         ops[3] = o3;
33           ~~~~~ Assignee and assigned types do not match.
34 Expected template but found expression.
35
36 error[T2057]: Typing error found
37   "circuit.circom":35:5
38
39 35         ops[4] = o4;
40           ~~~~~ Assignee and assigned types do not match.
41 Expected template but found expression.
42
43 error[T2003]: Typing error found
44   "circuit.circom":43:17
45
46 43         var v = ops[i];
47           ~~~~~ The type of this symbol is not known
48

```

```

49 error[T2057]: Typing error found
50     "circuit.circom":48:13
51
52 48             stack[top] = v;
53             ~~~~~ Assignee and assigned types do not match.
54 Expected template but found expression.
55
56 error[T2003]: Typing error found
57     "circuit.circom":53:22
58
59 53             var v2 = stack[top];
60             ~~~~~ The type of this symbol is not known
61
62 error[T2003]: Typing error found
63     "circuit.circom":55:22
64
65 55             var v1 = stack[top];
66             ~~~~~ The type of this symbol is not known
67
68 error[T2057]: Typing error found
69     "circuit.circom":60:17
70
71 60             res = v1 + v2;
72             ~~~~~ Assignee and assigned types do not match.
73 Expected template but found expression.
74
75 error[T2057]: Typing error found
76     "circuit.circom":62:17
77
78 62             res = v1 - v2;
79             ~~~~~ Assignee and assigned types do not match.
80 Expected template but found expression.
81
82 error[T2057]: Typing error found
83     "circuit.circom":64:17
84
85 64             res = v1 * v2;
86             ~~~~~ Assignee and assigned types do not match.
87 Expected template but found expression.
88
89 error[T2003]: Typing error found
90     "circuit.circom":69:26
91
92 69             stack[top] = res;
93             ~~~ The type of this symbol is not known
94
95 error[T2003]: Typing error found
96     "circuit.circom":74:13
97
98 74     out <== stack[0] + top;
99     ~~~~~ The type of this symbol is not known
100
101 previous errors were found

```

Listing 201: Circom compiler error log

Explanation: In this example, ‘stack’, ‘ops’, and ‘res’ are declared as ‘component’, but the code uses them as if they were numeric storage locations for signals and arithmetic expressions like ‘0’, ‘o0’,

or `v1 + v2`. Because of that, Circom expects template instances in those assignments, cannot infer the later types of `ops[i]`, `stack[top]`, and `res`, and reports both the mismatch and unknown-type follow-up errors.

G-6 Template as Signal (T2060)

Description: This error class happens when Circom code mixes up different kinds of objects in the type system, especially treating a template instantiation like a normal signal expression. A template such as `CustomIsZero()` must be assigned to a `component`, not to a `signal`; otherwise the compiler sees a template where it expected a field expression, and any later access like `.in` or `.out` fails because signals do not expose component ports unless they were declared that way. The fix is to declare template instances with `component`, then wire their input/output signals through the component's defined ports.

Original Error Tag: T2060

Example:

```

1 def liquidity_mining_decay(base_r, time_p, halving_p, boost_m):
2     if halving_p == 0: return 0
3     current_r = base_r
4     stages = time_p // halving_p
5     for _ in range(int(stages)):
6         current_r = (current_r * 50) // 100
7         if current_r < 10: break
8     rem = time_p % halving_p
9     decay = (current_r * rem * 50) // (halving_p * 100)
10    final_r = (current_r - decay) * boost_m // 100
11    return int(final_r)

```

Listing 202: Source Python `liquidity_mining_decay` implementation

```

1 pragma circom 2.1.0;
2 include "node_modules/circomlib/circuits/comparators.circom";
3
4 template CustomIsZero() {
5     signal input in;
6     signal output out;
7
8     signal inv <-- 1 / (in + (in == 0));
9     out <== 1 - (in * inv);
10    in * (1 - out) === 0;
11 }
12
13 template DecayLoop() {
14     signal input in_cr;
15     signal input in_st;
16     signal output out_cr;
17
18     signal isStZero <== CustomIsZero();
19     isStZero.in <== in_st;
20
21     signal new_cr <== in_cr * 50 \ 100;
22
23     component lt10 = LessThan(64);
24     lt10.in[0] <== new_cr;

```

```

25 lt10.in[1] <== 10;
26
27 signal continue_cr <== (1 - lt10.out) * new_cr + lt10.out * in_cr;
28
29 component next = DecayLoop();
30 next.in_cr <== continue_cr;
31 next.in_st <== in_st - 1;
32
33 out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
34 }
35
36 template LiquidityMiningDecay() {
37     signal input br;
38     signal input tp;
39     signal input hp;
40     signal input bm;
41     signal output fr;
42
43     signal isHpZero <== CustomIsZero();
44     isHpZero.in <== hp;
45
46     signal st <== tp \ hp;
47
48     signal cr_init <== br;
49
50     component loop = DecayLoop();
51     loop.in_cr <== cr_init;
52     loop.in_st <== st;
53
54     signal rem <== tp - (st * hp);
55
56     signal cr_after_loop <== loop.out_cr;
57     signal numerator <== cr_after_loop * rem * 50;
58     signal denominator <== hp * 100;
59     signal d <== numerator \ denominator;
60
61     signal cr_minus_d <== cr_after_loop - d;
62     signal fr_raw <== cr_minus_d * bm;
63     signal fr_calc <== fr_raw \ 100;
64
65     fr <== (1 - isHpZero.out) * fr_calc;
66 }
67
68 component main = LiquidityMiningDecay();

```

Listing 203: Faulty translated Circom code with Template as Signal (T2060)

```

1 error[T2060]: Typing error found
2   "circuit.circom":44:5
3
4 44     signal isHpZero <== CustomIsZero();
5      ~~~~~ Assignee and assigned types do not match.
6 Expected expression found template.
7
8 error[T2048]: Typing error found
9   "circuit.circom":45:5
10
11 45     isHpZero.in <== hp;

```

```

12      ~~~~~ Tag not found in signal: only accesses to tags that appear
13      in the definition of the signal are allowed
14
15 error[T2060]: Typing error found
16      "circuit.circom":19:5
17
18 19      signal isStZero <== CustomIsZero();
19      ~~~~~ Assignee and assigned types do not match.
20      Expected expression found template.
21
22 error[T2048]: Typing error found
23      "circuit.circom":20:5
24
25 20      isStZero.in <== in_st;
26      ~~~~~ Tag not found in signal: only accesses to tags that
27      appear in the definition of the signal are allowed
28
29 error[T2048]: Typing error found
30      "circuit.circom":34:16
31
32 34      out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
33      ~~~~~ Tag not found in signal: only accesses to tags that
34      appear in the definition of the signal are allowed
35
36 error[T2048]: Typing error found
37      "circuit.circom":34:44
38
39 34      out_cr <== isStZero.out * in_cr + (1 - isStZero.out) * next.out_cr;
40      ~~~~~ Tag not found in signal:
41      only accesses to tags that appear in the definition of the signal are allowed
42
43 error[T2048]: Typing error found
44      "circuit.circom":66:17
45
46 66      fr <== (1 - isHpZero.out) * fr_calc;
47      ~~~~~ Tag not found in signal: only accesses to tags that
48      appear in the definition of the signal are allowed
49
50 previous errors were found

```

Listing 204: Circom compiler error log

Explanation: In this example, ‘isStZero’ and ‘isHpZero’ are declared as ‘signal’ but assigned ‘CustomIsZero()’, which is a template instance. They should be written as ‘component isStZero = CustomIsZero();’ and ‘component isHpZero = CustomIsZero();’; after that, accesses like ‘isStZero.in’, ‘isStZero.out’, and ‘isHpZero.out’ will type-check correctly.

G-7 Unknown Type (T2003)

Description: This error class happens when Circom cannot determine a symbol’s type because declarations and assignments mix incompatible kinds of objects, especially confusing ‘component’/template instances with numeric values, signals, or local ‘var’ expressions. To fix it, declare each value with the correct kind (‘signal’, ‘signal input/output’, or ‘var’), instantiate ‘component’s only from templates, and use the proper assignment form: ‘=’ for compile-time variables and ‘<==’/‘==>’ for signal wiring and constraints.

Original Error Tag: T2003

Example:

```
1 def stack_rpn_calculator(o0, o1, o2, o3, o4):
2     ops = [o0, o1, o2, o3, o4]
3     stack = [0] * 5
4     top = -1
5     for i in range(5):
6         v = ops[i]
7         if v > 0:
8             top += 1
9             stack[top] = v
10        elif v < 0:
11            v2 = stack[top]; top -= 1
12            v1 = stack[top]; top -= 1
13            res = 0
14            if v == -1: res = v1 + v2
15            if v == -2: res = v1 - v2
16            if v == -3: res = v1 * v2
17            top += 1
18            stack[top] = res
19    return int(stack[0] + top)
```

Listing 205: Source Python stack_rpn_calculator implementation

```
1 template stack_rpn_calculator() {
2     // Declare input signals
3     signal input o0;
4     signal input o1;
5     signal input o2;
6     signal input o3;
7     signal input o4;
8
9     // Declare output signals
10    signal output out;
11
12    // Declare a component to represent the stack
13    component stack[5];
14
15    // Initialize the stack with zeros
16    for (var i = 0; i < 5; i++) {
17        stack[i] = 0;
18    }
19
20    // Initialize the top of the stack to -1
21    var top = -1;
22
23    // Define a component to represent the operations
24    component ops[5];
25
26    // Initialize the operations with the input values
27    ops[0] = o0;
28    ops[1] = o1;
29    ops[2] = o2;
30    ops[3] = o3;
31    ops[4] = o4;
32
33    // Define a component to represent the result
```

```

34 component res;
35
36 // Iterate over the operations
37 for (var i = 0; i < 5; i++) {
38     // Get the current operation
39     var v = ops[i];
40
41     // If the operation is positive, push it onto the stack
42     if (v > 0) {
43         top++;
44         stack[top] = v;
45     }
46     // If the operation is negative, pop two values from the stack, perform the
47     // operation, and push the result back onto the stack
48     else if (v < 0) {
49         // Get the top two values from the stack
50         var v2 = stack[top];
51         top--;
52         var v1 = stack[top];
53         top--;
54
55         // Perform the operation
56         if (v == -1) {
57             res = v1 + v2;
58         } else if (v == -2) {
59             res = v1 - v2;
60         } else if (v == -3) {
61             res = v1 * v2;
62         }
63
64         // Push the result back onto the stack
65         top++;
66         stack[top] = res;
67     }
68 }
69
70 // The final result is the top value on the stack plus the number of operations
71 // that were not pushed onto the stack
72 out <== stack[0] + top;
73 }

```

Listing 206: Faulty translated Circom code with Unknown Type (T2003)

```

1 error[T2057]: Typing error found
2   "circuit.circom":21:9
3
4 21         stack[i] = 0;
5   ~~~~~ Assignee and assigned types do not match.
6   Expected template but found expression.
7
8 error[T2057]: Typing error found
9   "circuit.circom":31:5
10
11 31         ops[0] = o0;
12   ~~~~~ Assignee and assigned types do not match.
13   Expected template but found expression.
14
15 error[T2057]: Typing error found

```

```

16 "circuit.circom":32:5
17
18 32 ops[1] = o1;
19      ~~~~~ Assignee and assigned types do not match.
20 Expected template but found expression.
21
22 error[T2057]: Typing error found
23 "circuit.circom":33:5
24
25 33 ops[2] = o2;
26      ~~~~~ Assignee and assigned types do not match.
27 Expected template but found expression.
28
29 error[T2057]: Typing error found
30 "circuit.circom":34:5
31
32 34 ops[3] = o3;
33      ~~~~~ Assignee and assigned types do not match.
34 Expected template but found expression.
35
36 error[T2057]: Typing error found
37 "circuit.circom":35:5
38
39 35 ops[4] = o4;
40      ~~~~~ Assignee and assigned types do not match.
41 Expected template but found expression.
42
43 error[T2003]: Typing error found
44 "circuit.circom":43:17
45
46 43 var v = ops[i];
47      ~~~~~ The type of this symbol is not known
48
49 error[T2057]: Typing error found
50 "circuit.circom":48:13
51
52 48 stack[top] = v;
53      ~~~~~ Assignee and assigned types do not match.
54 Expected template but found expression.
55
56 error[T2003]: Typing error found
57 "circuit.circom":53:22
58
59 53 var v2 = stack[top];
60      ~~~~~ The type of this symbol is not known
61
62 error[T2003]: Typing error found
63 "circuit.circom":55:22
64
65 55 var v1 = stack[top];
66      ~~~~~ The type of this symbol is not known
67
68 error[T2057]: Typing error found
69 "circuit.circom":60:17
70
71 60 res = v1 + v2;
72      ~~~~~ Assignee and assigned types do not match.
73 Expected template but found expression.

```

```

74 error[T2057]: Typing error found
75     "circuit.circom":62:17
76
77
78 62             res = v1 - v2;
79             ~~~~~ Assignee and assigned types do not match.
80 Expected template but found expression.
81
82 error[T2057]: Typing error found
83     "circuit.circom":64:17
84
85 64             res = v1 * v2;
86             ~~~~~ Assignee and assigned types do not match.
87 Expected template but found expression.
88
89 error[T2003]: Typing error found
90     "circuit.circom":69:26
91
92 69             stack[top] = res;
93             ~~~ The type of this symbol is not known
94
95 error[T2003]: Typing error found
96     "circuit.circom":74:13
97
98 74             out <= stack[0] + top;
99             ~~~~~ The type of this symbol is not known
100
101 previous errors were found

```

Listing 207: Circom compiler error log

Explanation: In this example, ‘stack’, ‘ops’, and ‘res’ are declared as ‘component’s, but the code uses them like ordinary numeric storage by assigning expressions such as ‘0’, ‘o0’, ‘v1 + v2’, and reading them into ‘var’s. Because those names were declared as components instead of signals or variables, Circom expects template instances there, so later accesses like ‘ops[i]’, ‘stack[top]’, and ‘res’ have no usable numeric type.

G-8 Array Access Mismatch (T2032)

Description: This error class happens when a Circom expression’s shape does not match the signal or access pattern you use with it, especially when confusing scalars, arrays, and component outputs. A common fix is to make sure declarations match the actual dimensions of the value being assigned: use scalar signals for single field elements, signal arrays for vectors, and components when instantiating templates like ‘Num2Bits’; then access the correct output field such as ‘out[i]’ instead of treating the whole component as a plain signal.

Original Error Tag: T2032

Example:

```

1 def is_even(n):
2     return [1 if n % 2 == 0 else 0]

```

Listing 208: Source Python is_even implementation

```

1 template is_even() {

```

```

2  signal input n;
3  signal output out;
4
5  // Use Num2Bits to convert the input to a bit vector
6  signal bits <== Num2Bits(254)(n);
7
8  // Check the sign bit (bit 0) to determine if the number is even
9  signal is_even <== bits[0];
10
11 // Use the Mux Formula to select the output
12 out <== is_even * (1 - 2 * is_even) + 1;
13 }

```

Listing 209: Faulty translated Circom code with Array Access Mismatch (T2032)

```

1 error[T2061]: Typing error found
2   "circuit.circom":10:5
3
4 10      signal bits <== Num2Bits(254)(n);
5      ~~~~~ Assignee and assigned types do not match.
6      Expected dimensions: 0, found 1
7
8 error[T2032]: Typing error found
9   "circuit.circom":13:24
10
11 13      signal is_even <== bits[0];
12      ~~~~~ Array access does not match the dimensions of the
13      expression.
14      Expected 0 dimensions, given 1.
15 previous errors were found

```

Listing 210: Circom compiler error log

Explanation: In this example, ‘Num2Bits(254)(n)’ is not a scalar value you can assign directly to ‘signal bits’; it produces a component whose ‘out’ is an array of 254 bits. Because ‘bits’ was declared as a single signal, ‘bits[0]’ is also invalid-declare a ‘component’ for ‘Num2Bits’ and read ‘componentName.out[0]’ to get the least significant bit used for the evenness check.

G-9 <== on Non-Signal (T2055)

Description: This error class is caused by using the wrong assignment operator for the kind of value being assigned in Circom. Signals must be assigned with ‘<==’ or ‘<-’, while compile-time variables declared with ‘var’ must be assigned with ‘=’; to fix it, check whether the left-hand side is a signal or a ‘var’/array of ‘var’s and use the operator that matches that type.

Original Error Tag: T2055

Example:

```

1 def stack_rpn_calculator(o0, o1, o2, o3, o4):
2     ops = [o0, o1, o2, o3, o4]
3     stack = [0] * 5
4     top = -1
5     for i in range(5):
6         v = ops[i]
7         if v > 0:

```



```

8         top += 1
9         stack[top] = v
10    elif v < 0:
11        v2 = stack[top]; top -= 1
12        v1 = stack[top]; top -= 1
13        res = 0
14        if v == -1: res = v1 + v2
15        if v == -2: res = v1 - v2
16        if v == -3: res = v1 * v2
17        top += 1
18        stack[top] = res
19    return int(stack[0] + top)

```

Listing 211: Source Python stack_rpn_calculator implementation

```

1 template stack_rpn_calculator() {
2     signal input o0;
3     signal input o1;
4     signal input o2;
5     signal input o3;
6     signal input o4;
7     signal output out;
8
9     var ops[5];
10    ops[0] <== o0;
11    ops[1] <== o1;
12    ops[2] <== o2;
13    ops[3] <== o3;
14    ops[4] <== o4;
15
16    var top = 0;
17    var stack_0 = 0;
18    var stack_1 = 0;
19    var stack_2 = 0;
20    var stack_3 = 0;
21    var stack_4 = 0;
22
23    for (var i = 0; i < 5; i++) {
24        var v = ops[i];
25        var cond_pos = v > 0;
26        var cond_neg = v < 0;
27        var cond_add = v == -1;
28        var cond_sub = v == -2;
29        var cond_mul = v == -3;
30
31        var sel_pos = cond_pos;
32        var sel_neg = cond_neg;
33
34        var new_top = sel_pos * (top + 1) + sel_neg * (top - 1) + (1 - sel_pos -
35        sel_neg) * top;
36        var new_stack_0 = sel_pos * v + (1 - sel_pos) * stack_0;
37        var new_stack_1 = sel_pos * stack_0 + (1 - sel_pos) * stack_1;
38        var new_stack_2 = sel_pos * stack_1 + (1 - sel_pos) * stack_2;
39        var new_stack_3 = sel_pos * stack_2 + (1 - sel_pos) * stack_3;
40        var new_stack_4 = sel_pos * stack_3 + (1 - sel_pos) * stack_4;
41
42        var v1 = stack_1;
43        var v2 = stack_2;

```

```

43     var res = cond_add * (v1 + v2) + cond_sub * (v1 - v2) + cond_mul * (v1 * v2);
44
45     new_stack_0 = sel_neg * res + (1 - sel_neg) * stack_0;
46     new_stack_1 = sel_neg * v1 + (1 - sel_neg) * stack_1;
47     new_stack_2 = sel_neg * stack_2 + (1 - sel_neg) * stack_2;
48     new_stack_3 = sel_neg * stack_3 + (1 - sel_neg) * stack_3;
49     new_stack_4 = sel_neg * stack_4 + (1 - sel_neg) * stack_4;
50
51     top = new_top;
52     stack_0 = new_stack_0;
53     stack_1 = new_stack_1;
54     stack_2 = new_stack_2;
55     stack_3 = new_stack_3;
56     stack_4 = new_stack_4;
57 }
58
59 out = stack_0 + top;
60 }
61
62 component main = stack_rpn_calculator();

```

Listing 212: Faulty translated Circom code with <

```

1 error[T2055]: Typing error found
2   "circuit.circom":14:5
3
4 14     ops[0] <== o0;
5     ~~~~~ The operator does not match the types of the assigned elements.
6   Only assignments to signals allow the operators <== and <--, try using = instead
7
8 error[T2055]: Typing error found
9   "circuit.circom":15:5
10
11 15     ops[1] <== o1;
12     ~~~~~ The operator does not match the types of the assigned elements.
13   Only assignments to signals allow the operators <== and <--, try using = instead
14
15 error[T2055]: Typing error found
16   "circuit.circom":16:5
17
18 16     ops[2] <== o2;
19     ~~~~~ The operator does not match the types of the assigned elements.
20   Only assignments to signals allow the operators <== and <--, try using = instead
21
22 error[T2055]: Typing error found
23   "circuit.circom":17:5
24
25 17     ops[3] <== o3;
26     ~~~~~ The operator does not match the types of the assigned elements.
27   Only assignments to signals allow the operators <== and <--, try using = instead
28
29 error[T2055]: Typing error found
30   "circuit.circom":18:5
31
32 18     ops[4] <== o4;
33     ~~~~~ The operator does not match the types of the assigned elements.
34   Only assignments to signals allow the operators <== and <--, try using = instead
35

```

```

36 error[T2054]: Typing error found
37     "circuit.circom":63:5
38
39 63     out = stack_0 + top;
40     ~~~~~~ The operator does not match the types of the assigned
41     elements.
42     Assignments to signals do not allow the operator =, try using <== or <-- instead
43 previous errors were found

```

Listing 213: Circom compiler error log

Explanation: In this example, ‘ops’ is declared as ‘var ops[5]’, so lines like ‘ops[0] <== o0;’ are invalid because ‘<==’ can only assign to signals; these should use ‘=’ if you want to store expressions in variables, or ‘ops’ should be made a signal array if signal assignment is intended. Conversely, ‘out’ is a ‘signal output’, so ‘out = stack_0 + top;’ is invalid and must use ‘<==’ instead.

H Component Initialization/Interface

H-1 Uninitialized Component (T20466)

Description: This error class happens when Circom code tries to read or write a component’s signals before that component has been instantiated with a concrete template. Declaring ‘component x;’ or ‘component arr[n];’ only reserves the name; you must still initialize each one, for example with ‘x = SomeTemplate(...);’ or ‘arr[i] = SomeTemplate(...);’, before using ‘x.in’, ‘x.out’, or similar fields. To fix it, instantiate every component first, make sure the chosen template actually defines the signals you access, and then connect signals with matching dimensions and types.

Original Error Tag: T20466

Example:

```

1 def stack_rpn_calculator(o0, o1, o2, o3, o4):
2     ops = [o0, o1, o2, o3, o4]
3     stack = [0] * 5
4     top = -1
5     for i in range(5):
6         v = ops[i]
7         if v > 0:
8             top += 1
9             stack[top] = v
10        elif v < 0:
11            v2 = stack[top]; top -= 1
12            v1 = stack[top]; top -= 1
13            res = 0
14            if v == -1: res = v1 + v2
15            if v == -2: res = v1 - v2
16            if v == -3: res = v1 * v2
17            top += 1
18            stack[top] = res
19        return int(stack[0] + top)

```

Listing 214: Source Python stack_rpn_calculator implementation

```

1 template stack_rpn_calculator() {

```

```

2  // Declaration of signals and components
3  signal input o0;
4  signal input o1;
5  signal input o2;
6  signal input o3;
7  signal input o4;
8  signal output out;
9  component stack[5];
10 component top;
11
12 // Statements
13 for (var i = 0; i < 5; i++) {
14     stack[i].in[0] <== o0;
15     stack[i].in[1] <== o1;
16     stack[i].in[2] <== o2;
17     stack[i].in[3] <== o3;
18     stack[i].in[4] <== o4;
19     stack[i].out <== stack[i].in;
20 }
21
22 // Create a component for the top of the stack
23 top.in[0] <== stack[0].out;
24 top.in[1] <== stack[1].out;
25 top.in[2] <== stack[2].out;
26 top.in[3] <== stack[3].out;
27 top.in[4] <== stack[4].out;
28
29 // Define the operations
30 component add = GreaterEqThan(64);
31 component sub = GreaterEqThan(64);
32 component mul = GreaterEqThan(64);
33
34 // Assign the operations
35 add.in[0] <== top.out[0];
36 add.in[1] <== top.out[1];
37 add.out <== add.in;
38 sub.in[0] <== top.out[0];
39 sub.in[1] <== top.out[1];
40 sub.out <== sub.in;
41 mul.in[0] <== top.out[0];
42 mul.in[1] <== top.out[1];
43 mul.out <== mul.in;
44
45 // Define the output
46 out <== 0;
47 for (var i = 0; i < 5; i++) {
48     if (o0 == i + 1) {
49         out <== top.out[i];
50     }
51     if (o1 == i + 1) {
52         out <== top.out[i];
53     }
54     if (o2 == i + 1) {
55         out <== top.out[i];
56     }
57     if (o3 == i + 1) {
58         out <== top.out[i];
59     }

```

```

60         if (o4 == i + 1) {
61             out <= top.out[i];
62         }
63     }
64     if (o0 == -1) {
65         out <= add.out;
66     }
67     if (o0 == -2) {
68         out <= sub.out;
69     }
70     if (o0 == -3) {
71         out <= mul.out;
72     }
73 }
74 component main = stack_rpn_calculator();

```

Listing 215: Faulty translated Circom code with Uninitialized Component (T20466)

```

1 error[T20466]: Typing error found
2   "circuit.circom":18:9
3
4 18         stack[i].in[0] <= o0;
5           ~~~~~ Trying to access to a signal of a component that has
6           not been initialized
7
8 error[T20466]: Typing error found
9   "circuit.circom":19:9
10
11 19         stack[i].in[1] <= o1;
12           ~~~~~ Trying to access to a signal of a component that has
13           not been initialized
14
15 error[T20466]: Typing error found
16   "circuit.circom":20:9
17
18 20         stack[i].in[2] <= o2;
19           ~~~~~ Trying to access to a signal of a component that has
20           not been initialized
21
22 error[T20466]: Typing error found
23   "circuit.circom":21:9
24
25 21         stack[i].in[3] <= o3;
26           ~~~~~ Trying to access to a signal of a component that has
27           not been initialized
28
29 error[T20466]: Typing error found
30   "circuit.circom":22:9
31
32 22         stack[i].in[4] <= o4;
33           ~~~~~ Trying to access to a signal of a component that has
34           not been initialized
35
36 error[T20466]: Typing error found
37   "circuit.circom":23:26
38
39 23         stack[i].out <= stack[i].in;

```

```

35         ~~~~~ Trying to access to a signal of a component
      that has not been initialized
36
37 error[T20466]: Typing error found
38   "circuit.circom":27:19
39
40 27     top.in[0] <== stack[0].out;
41     ~~~~~ Trying to access to a signal of a component that
      has not been initialized
42
43 error[T20466]: Typing error found
44   "circuit.circom":28:19
45
46 28     top.in[1] <== stack[1].out;
47     ~~~~~ Trying to access to a signal of a component that
      has not been initialized
48
49 error[T20466]: Typing error found
50   "circuit.circom":29:19
51
52 29     top.in[2] <== stack[2].out;
53     ~~~~~ Trying to access to a signal of a component that
      has not been initialized
54
55 error[T20466]: Typing error found
56   "circuit.circom":30:19
57
58 30     top.in[3] <== stack[3].out;
59     ~~~~~ Trying to access to a signal of a component that
      has not been initialized
60
61 error[T20466]: Typing error found
62   "circuit.circom":31:19
63
64 31     top.in[4] <== stack[4].out;
65     ~~~~~ Trying to access to a signal of a component that
      has not been initialized
66
67 error[T20466]: Typing error found
68   "circuit.circom":39:19
69
70 39     add.in[0] <== top.out[0];
71     ~~~~~ Trying to access to a signal of a component that has
      not been initialized
72
73 error[T20466]: Typing error found
74   "circuit.circom":40:19
75
76 40     add.in[1] <== top.out[1];
77     ~~~~~ Trying to access to a signal of a component that has
      not been initialized
78
79 error[T2061]: Typing error found
80   "circuit.circom":41:5
81
82 41     add.out <== add.in;
83     ~~~~~ Assignee and assigned types do not match.
84 Expected dimensions: 0, found 1

```

```

85
86 error[T20466]: Typing error found
87   "circuit.circom":42:19
88
89 42      sub.in[0] <== top.out[0];
90           ~~~~~ Trying to access to a signal of a component that has
           not been initialized
91
92 error[T20466]: Typing error found
93   "circuit.circom":43:19
94
95 43      sub.in[1] <== top.out[1];
96           ~~~~~ Trying to access to a signal of a component that has
           not been initialized
97
98 error[T2061]: Typing error found
99   "circuit.circom":44:5
100
101 44      sub.out <== sub.in;
102           ~~~~~ Assignee and assigned types do not match.
103   Expected dimensions: 0, found 1
104
105 error[T20466]: Typing error found
106   "circuit.circom":45:19
107
108 45      mul.in[0] <== top.out[0];
109           ~~~~~ Trying to access to a signal of a component that has
           not been initialized
110
111 error[T20466]: Typing error found
112   "circuit.circom":46:19
113
114 46      mul.in[1] <== top.out[1];
115           ~~~~~ Trying to access to a signal of a component that has
           not been initialized
116
117 error[T2061]: Typing error found
118   "circuit.circom":47:5
119
120 47      mul.out <== mul.in;
121           ~~~~~ Assignee and assigned types do not match.
122   Expected dimensions: 0, found 1
123
124 error[T20466]: Typing error found
125   "circuit.circom":53:21
126
127 53      out <== top.out[i];
128           ~~~~~ Trying to access to a signal of a component that
           has not been initialized
129
130 error[T20466]: Typing error found
131   "circuit.circom":56:21
132
133 56      out <== top.out[i];
134           ~~~~~ Trying to access to a signal of a component that
           has not been initialized
135
136 error[T20466]: Typing error found

```

```

137 "circuit.circom":59:21
138
139 59          out <== top.out[i];
140          ~~~~~ Trying to access to a signal of a component that
          has not been initialized
141
142 error[T20466]: Typing error found
143 "circuit.circom":62:21
144
145 62          out <== top.out[i];
146          ~~~~~ Trying to access to a signal of a component that
          has not been initialized
147
148 error[T20466]: Typing error found
149 "circuit.circom":65:21
150
151 65          out <== top.out[i];
152          ~~~~~ Trying to access to a signal of a component that
          has not been initialized
153
154 previous errors were found

```

Listing 216: Circom compiler error log

Explanation: In this example, ‘stack[5]’ and ‘top’ are declared but never initialized, so every use of ‘stack[i].in’, ‘stack[i].out’, and ‘top.out’ triggers T20466. There is also a separate typing problem because ‘add’, ‘sub’, and ‘mul’ are instantiated as ‘GreaterEqThan(64)’ components, but the code assigns ‘add.out <== add.in’ (and similarly for the others), which mismatches a scalar output with an array input.

H-2 Mixed Templates in Array (T2056)

Description: This error class happens when Circom’s type system detects that a component, signal, or array element is being used with an incompatible type or declaration pattern. A very common cause is declaring an array of components and then trying to fill different positions with different templates, even if their interfaces look similar; in Circom, every element of a component array must come from the same template instantiation. The fix is to use separate component variables or separate arrays for each template kind, and to make sure signal assignments use Circom operators like ‘<==’ or ‘<-’ instead of plain ‘=’.

Original Error Tag: T2056

Example:

```

1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2   arr = [a0, a1, a2, a3, a4, a5, a6]
3   pivot = arr[high]
4   i = low - 1
5   swaps = 0
6   for j in range(7):
7     if j >= low and j < high:
8       if arr[j] <= pivot:
9         i += 1
10        arr[i], arr[j] = arr[j], arr[i]
11        swaps += 1
12   arr[i + 1], arr[high] = arr[high], arr[i + 1]

```



```
13 return int((i + 1) * 100 + swaps)
```

Listing 217: Source Python quicksort_partition_logic implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template quicksort_partition_logic() {
4     signal input a0;
5     signal input a1;
6     signal input a2;
7     signal input a3;
8     signal input a4;
9     signal input a5;
10    signal input a6;
11    signal input low;
12    signal input high;
13    signal output result;
14
15    // Pivot and initial i value
16    signal pivot;
17    signal i_val;
18
19    // Range check components
20    component in_range_comp_0[2];
21    in_range_comp_0[0] = LessEqThan(64); // 0 >= low
22    in_range_comp_0[1] = LessThan(64);   // 0 < high
23
24    component in_range_comp_1[2];
25    in_range_comp_1[0] = LessEqThan(64); // 1 >= low
26    in_range_comp_1[1] = LessThan(64);   // 1 < high
27
28    component in_range_comp_2[2];
29    in_range_comp_2[0] = LessEqThan(64); // 2 >= low
30    in_range_comp_2[1] = LessThan(64);   // 2 < high
31
32    component in_range_comp_3[2];
33    in_range_comp_3[0] = LessEqThan(64); // 3 >= low
34    in_range_comp_3[1] = LessThan(64);   // 3 < high
35
36    component in_range_comp_4[2];
37    in_range_comp_4[0] = LessEqThan(64); // 4 >= low
38    in_range_comp_4[1] = LessThan(64);   // 4 < high
39
40    component in_range_comp_5[2];
41    in_range_comp_5[0] = LessEqThan(64); // 5 >= low
42    in_range_comp_5[1] = LessThan(64);   // 5 < high
43
44    component in_range_comp_6[2];
45    in_range_comp_6[0] = LessEqThan(64); // 6 >= low
46    in_range_comp_6[1] = LessThan(64);   // 6 < high
47
48    // Less than or equal to pivot components
49    component less_equal_comp_0 = LessEqThan(64);
50    component less_equal_comp_1 = LessEqThan(64);
51    component less_equal_comp_2 = LessEqThan(64);
52    component less_equal_comp_3 = LessEqThan(64);
53    component less_equal_comp_4 = LessEqThan(64);
54    component less_equal_comp_5 = LessEqThan(64);
```

```

55     component less_equal_comp_6 = LessEqThan(64);
56
57     // Connect inputs for range checks
58     in_range_comp_0[0].in[0] <== low;
59     in_range_comp_0[0].in[1] <== 0;
60     in_range_comp_0[1].in[0] <== 0;
61     in_range_comp_0[1].in[1] <== high;
62
63     in_range_comp_1[0].in[0] <== low;
64     in_range_comp_1[0].in[1] <== 1;
65     in_range_comp_1[1].in[0] <== 1;
66     in_range_comp_1[1].in[1] <== high;
67
68     in_range_comp_2[0].in[0] <== low;
69     in_range_comp_2[0].in[1] <== 2;
70     in_range_comp_2[1].in[0] <== 2;
71     in_range_comp_2[1].in[1] <== high;
72
73     in_range_comp_3[0].in[0] <== low;
74     in_range_comp_3[0].in[1] <== 3;
75     in_range_comp_3[1].in[0] <== 3;
76     in_range_comp_3[1].in[1] <== high;
77
78     in_range_comp_4[0].in[0] <== low;
79     in_range_comp_4[0].in[1] <== 4;
80     in_range_comp_4[1].in[0] <== 4;
81     in_range_comp_4[1].in[1] <== high;
82
83     in_range_comp_5[0].in[0] <== low;
84     in_range_comp_5[0].in[1] <== 5;
85     in_range_comp_5[1].in[0] <== 5;
86     in_range_comp_5[1].in[1] <== high;
87
88     in_range_comp_6[0].in[0] <== low;
89     in_range_comp_6[0].in[1] <== 6;
90     in_range_comp_6[1].in[0] <== 6;
91     in_range_comp_6[1].in[1] <== high;
92
93     // Connect inputs for less than or equal to pivot checks
94     less_equal_comp_0.in[0] <== a0;
95     less_equal_comp_0.in[1] <== a4; // pivot = a4
96
97     less_equal_comp_1.in[0] <== a1;
98     less_equal_comp_1.in[1] <== a4;
99
100    less_equal_comp_2.in[0] <== a2;
101    less_equal_comp_2.in[1] <== a4;
102
103    less_equal_comp_3.in[0] <== a3;
104    less_equal_comp_3.in[1] <== a4;
105
106    less_equal_comp_4.in[0] <== a4;
107    less_equal_comp_4.in[1] <== a4;
108
109    less_equal_comp_5.in[0] <== a5;
110    less_equal_comp_5.in[1] <== a4;
111
112    less_equal_comp_6.in[0] <== a6;

```

```

113 less_equal_comp_6.in[1] <= a4;
114
115 // Define all signals
116 signal in_range_flags_0;
117 signal in_range_flags_1;
118 signal in_range_flags_2;
119 signal in_range_flags_3;
120 signal in_range_flags_4;
121 signal in_range_flags_5;
122 signal in_range_flags_6;
123
124 signal less_equal_flags_0;
125 signal less_equal_flags_1;
126 signal less_equal_flags_2;
127 signal less_equal_flags_3;
128 signal less_equal_flags_4;
129 signal less_equal_flags_5;
130 signal less_equal_flags_6;
131
132 signal swap_flags_0;
133 signal swap_flags_1;
134 signal swap_flags_2;
135 signal swap_flags_3;
136 signal swap_flags_4;
137 signal swap_flags_5;
138 signal swap_flags_6;
139
140 signal update_i_flags_0;
141 signal update_i_flags_1;
142 signal update_i_flags_2;
143 signal update_i_flags_3;
144 signal update_i_flags_4;
145 signal update_i_flags_5;
146 signal update_i_flags_6;
147
148 signal arr_0;
149 signal arr_1;
150 signal arr_2;
151 signal arr_3;
152 signal arr_4;
153 signal arr_5;
154 signal arr_6;
155
156 signal temp_i_0;
157 signal temp_i_1;
158 signal temp_i_2;
159 signal temp_i_3;
160 signal temp_i_4;
161 signal temp_i_5;
162 signal temp_i_6;
163
164 signal temp_swaps_0;
165 signal temp_swaps_1;
166 signal temp_swaps_2;
167 signal temp_swaps_3;
168 signal temp_swaps_4;
169 signal temp_swaps_5;
170 signal temp_swaps_6;

```

```

171
172     signal final_i;
173     signal total_swaps;
174
175     signal new_arr_0;
176     signal new_arr_1;
177     signal new_arr_2;
178     signal new_arr_3;
179     signal new_arr_4;
180     signal new_arr_5;
181     signal new_arr_6;
182
183     // Assign outputs from comparators
184     in_range_flags_0 = in_range_comp_0[0].out * in_range_comp_0[1].out;
185     in_range_flags_1 = in_range_comp_1[0].out * in_range_comp_1[1].out;
186     in_range_flags_2 = in_range_comp_2[0].out * in_range_comp_2[1].out;
187     in_range_flags_3 = in_range_comp_3[0].out * in_range_comp_3[1].out;
188     in_range_flags_4 = in_range_comp_4[0].out * in_range_comp_4[1].out;
189     in_range_flags_5 = in_range_comp_5[0].out * in_range_comp_5[1].out;
190     in_range_flags_6 = in_range_comp_6[0].out * in_range_comp_6[1].out;
191
192     less_equal_flags_0 <= less_equal_comp_0.out;
193     less_equal_flags_1 <= less_equal_comp_1.out;
194     less_equal_flags_2 <= less_equal_comp_2.out;
195     less_equal_flags_3 <= less_equal_comp_3.out;
196     less_equal_flags_4 <= less_equal_comp_4.out;
197     less_equal_flags_5 <= less_equal_comp_5.out;
198     less_equal_flags_6 <= less_equal_comp_6.out;
199
200     // Compute pivot and initial i value
201     pivot <-- a4;
202     i_val <-- low - 1;
203
204     // Compute swap flags
205     swap_flags_0 <-- in_range_flags_0 * less_equal_flags_0;
206     swap_flags_1 <-- in_range_flags_1 * less_equal_flags_1;
207     swap_flags_2 <-- in_range_flags_2 * less_equal_flags_2;
208     swap_flags_3 <-- in_range_flags_3 * less_equal_flags_3;
209     swap_flags_4 <-- in_range_flags_4 * less_equal_flags_4;
210     swap_flags_5 <-- in_range_flags_5 * less_equal_flags_5;
211     swap_flags_6 <-- in_range_flags_6 * less_equal_flags_6;
212
213     // Update i flags are same as swap flags
214     update_i_flags_0 <-- swap_flags_0;
215     update_i_flags_1 <-- swap_flags_1;
216     update_i_flags_2 <-- swap_flags_2;
217     update_i_flags_3 <-- swap_flags_3;
218     update_i_flags_4 <-- swap_flags_4;
219     update_i_flags_5 <-- swap_flags_5;
220     update_i_flags_6 <-- swap_flags_6;
221
222     // Initialize arrays
223     arr_0 <-- a0;
224     arr_1 <-- a1;
225     arr_2 <-- a2;
226     arr_3 <-- a3;
227     arr_4 <-- a4;
228     arr_5 <-- a5;

```

```

229 arr_6 <-- a6;
230
231 // Update i values step by step
232 temp_i_0 <-- i_val + update_i_flags_0;
233 temp_i_1 <-- temp_i_0 + update_i_flags_1;
234 temp_i_2 <-- temp_i_1 + update_i_flags_2;
235 temp_i_3 <-- temp_i_2 + update_i_flags_3;
236 temp_i_4 <-- temp_i_3 + update_i_flags_4;
237 temp_i_5 <-- temp_i_4 + update_i_flags_5;
238 temp_i_6 <-- temp_i_5 + update_i_flags_6;
239
240 // Update swap counts
241 temp_swaps_0 <-- 0 + swap_flags_0;
242 temp_swaps_1 <-- temp_swaps_0 + swap_flags_1;
243 temp_swaps_2 <-- temp_swaps_1 + swap_flags_2;
244 temp_swaps_3 <-- temp_swaps_2 + swap_flags_3;
245 temp_swaps_4 <-- temp_swaps_3 + swap_flags_4;
246 temp_swaps_5 <-- temp_swaps_4 + swap_flags_5;
247 temp_swaps_6 <-- temp_swaps_5 + swap_flags_6;
248
249 // Final results
250 final_i <-- temp_i_6 + 1;
251 total_swaps <-- temp_swaps_6;
252
253 // Simplified array assignments
254 new_arr_0 <-- arr_0;
255 new_arr_1 <-- arr_1;
256 new_arr_2 <-- arr_2;
257 new_arr_3 <-- arr_3;
258 new_arr_4 <-- arr_4;
259 new_arr_5 <-- arr_5;
260 new_arr_6 <-- arr_6;
261
262 // Result calculation
263 result <-- final_i * 100 + total_swaps;
264
265 // Add binary constraints for flags
266 in_range_flags_0 * (in_range_flags_0 - 1) === 0;
267 in_range_flags_1 * (in_range_flags_1 - 1) === 0;
268 in_range_flags_2 * (in_range_flags_2 - 1) === 0;
269 in_range_flags_3 * (in_range_flags_3 - 1) === 0;
270 in_range_flags_4 * (in_range_flags_4 - 1) === 0;
271 in_range_flags_5 * (in_range_flags_5 - 1) === 0;
272 in_range_flags_6 * (in_range_flags_6 - 1) === 0;
273
274 less_equal_flags_0 * (less_equal_flags_0 - 1) === 0;
275 less_equal_flags_1 * (less_equal_flags_1 - 1) === 0;
276 less_equal_flags_2 * (less_equal_flags_2 - 1) === 0;
277 less_equal_flags_3 * (less_equal_flags_3 - 1) === 0;
278 less_equal_flags_4 * (less_equal_flags_4 - 1) === 0;
279 less_equal_flags_5 * (less_equal_flags_5 - 1) === 0;
280 less_equal_flags_6 * (less_equal_flags_6 - 1) === 0;
281
282 swap_flags_0 * (swap_flags_0 - 1) === 0;
283 swap_flags_1 * (swap_flags_1 - 1) === 0;
284 swap_flags_2 * (swap_flags_2 - 1) === 0;
285 swap_flags_3 * (swap_flags_3 - 1) === 0;
286 swap_flags_4 * (swap_flags_4 - 1) === 0;

```

```

287 swap_flags_5 * (swap_flags_5 - 1) == 0;
288 swap_flags_6 * (swap_flags_6 - 1) == 0;
289
290 update_i_flags_0 * (update_i_flags_0 - 1) == 0;
291 update_i_flags_1 * (update_i_flags_1 - 1) == 0;
292 update_i_flags_2 * (update_i_flags_2 - 1) == 0;
293 update_i_flags_3 * (update_i_flags_3 - 1) == 0;
294 update_i_flags_4 * (update_i_flags_4 - 1) == 0;
295 update_i_flags_5 * (update_i_flags_5 - 1) == 0;
296 update_i_flags_6 * (update_i_flags_6 - 1) == 0;
297
298 // Swap flag computation constraints
299 swap_flags_0 == in_range_flags_0 * less_equal_flags_0;
300 swap_flags_1 == in_range_flags_1 * less_equal_flags_1;
301 swap_flags_2 == in_range_flags_2 * less_equal_flags_2;
302 swap_flags_3 == in_range_flags_3 * less_equal_flags_3;
303 swap_flags_4 == in_range_flags_4 * less_equal_flags_4;
304 swap_flags_5 == in_range_flags_5 * less_equal_flags_5;
305 swap_flags_6 == in_range_flags_6 * less_equal_flags_6;
306
307 // Update flag constraints
308 update_i_flags_0 == swap_flags_0;
309 update_i_flags_1 == swap_flags_1;
310 update_i_flags_2 == swap_flags_2;
311 update_i_flags_3 == swap_flags_3;
312 update_i_flags_4 == swap_flags_4;
313 update_i_flags_5 == swap_flags_5;
314 update_i_flags_6 == swap_flags_6;
315
316 // Final result constraint
317 result == final_i * 100 + total_swaps;
318 }
319
320 component main = quicksort_partition_logic();

```

Listing 218: Faulty translated Circom code with Mixed Templates in Array (T2056)

```

1 error[T2056]: Typing error found
2   "circuit.circom":23:5
3
4 23     in_range_comp_0[0] = LessEqThan(64); // 0 >= low
5     ~~~~~ Assignee and assigned types do not match.
6 All components of an array must be instances of the same template.
7
8 error[T2056]: Typing error found
9   "circuit.circom":27:5
10
11 27     in_range_comp_1[0] = LessEqThan(64); // 1 >= low
12     ~~~~~ Assignee and assigned types do not match.
13 All components of an array must be instances of the same template.
14
15 error[T2056]: Typing error found
16   "circuit.circom":31:5
17
18 31     in_range_comp_2[0] = LessEqThan(64); // 2 >= low
19     ~~~~~ Assignee and assigned types do not match.
20 All components of an array must be instances of the same template.
21

```

```

22 error[T2056]: Typing error found
23     "circuit.circom":35:5
24
25 35     in_range_comp_3[0] = LessEqThan(64); // 3 >= low
26     ~~~~~ Assignee and assigned types do not match.
27 All components of an array must be instances of the same template.
28
29 error[T2056]: Typing error found
30     "circuit.circom":39:5
31
32 39     in_range_comp_4[0] = LessEqThan(64); // 4 >= low
33     ~~~~~ Assignee and assigned types do not match.
34 All components of an array must be instances of the same template.
35
36 error[T2056]: Typing error found
37     "circuit.circom":43:5
38
39 43     in_range_comp_5[0] = LessEqThan(64); // 5 >= low
40     ~~~~~ Assignee and assigned types do not match.
41 All components of an array must be instances of the same template.
42
43 error[T2056]: Typing error found
44     "circuit.circom":47:5
45
46 47     in_range_comp_6[0] = LessEqThan(64); // 6 >= low
47     ~~~~~ Assignee and assigned types do not match.
48 All components of an array must be instances of the same template.
49
50 error[T2054]: Typing error found
51     "circuit.circom":186:5
52
53 186     in_range_flags_0 = in_range_comp_0[0].out * in_range_comp_0[1].out;
54     ~~~~~ The
55     operator does not match the types of the assigned elements.
56 Assignments to signals do not allow the operator =, try using <== or <-- instead
57
58 error[T2054]: Typing error found
59     "circuit.circom":187:5
60
61 187     in_range_flags_1 = in_range_comp_1[0].out * in_range_comp_1[1].out;
62     ~~~~~ The
63     operator does not match the types of the assigned elements.
64 Assignments to signals do not allow the operator =, try using <== or <-- instead
65
66 error[T2054]: Typing error found
67     "circuit.circom":188:5
68
69 188     in_range_flags_2 = in_range_comp_2[0].out * in_range_comp_2[1].out;
70     ~~~~~ The
71     operator does not match the types of the assigned elements.
72 Assignments to signals do not allow the operator =, try using <== or <-- instead
73
74 error[T2054]: Typing error found
75     "circuit.circom":189:5
76
77 189     in_range_flags_3 = in_range_comp_3[0].out * in_range_comp_3[1].out;
78     ~~~~~ The
79     operator does not match the types of the assigned elements.

```

```

76 Assignments to signals do not allow the operator =, try using <== or <-- instead
77
78 error[T2054]: Typing error found
79 "circuit.circom":190:5
80
81 190         in_range_flags_4 = in_range_comp_4[0].out * in_range_comp_4[1].out;
82         ~~~~~~ The
83         operator does not match the types of the assigned elements.
84 Assignments to signals do not allow the operator =, try using <== or <-- instead
85
86 error[T2054]: Typing error found
87 "circuit.circom":191:5
88
89 191         in_range_flags_5 = in_range_comp_5[0].out * in_range_comp_5[1].out;
90         ~~~~~~ The
91         operator does not match the types of the assigned elements.
92 Assignments to signals do not allow the operator =, try using <== or <-- instead
93
94 error[T2054]: Typing error found
95 "circuit.circom":192:5
96
97 192         in_range_flags_6 = in_range_comp_6[0].out * in_range_comp_6[1].out;
98         ~~~~~~ The
99         operator does not match the types of the assigned elements.
100 Assignments to signals do not allow the operator =, try using <== or <-- instead
101
102 previous errors were found

```

Listing 219: Circom compiler error log

Explanation: In this example, each ‘in_range_comp_X[2]’ array is declared once but then populated with both ‘LessEqThan(64)’ and ‘LessThan(64)’, which violates the rule that all elements in a component array must be instances of the same template. The later ‘in_range_flags_X = ...’ lines also fail because ‘in_range_flags_X’ are signals, so they must be assigned with ‘<==’ or ‘<--’, not ‘=’.

H-3 Undefined Port (T2046)

Description: This error happens when you try to connect to a signal or bus field that does not exist on a component’s declared interface. In Circom, each component only exposes the exact input/output names defined in its template, so you must check the library definition and wire signals using the correct field names, indices, or bus structure. To fix it, inspect the component in the imported ‘circom’ file and replace the invalid references with the actual declared ports.

Original Error Tag: T2046

Example:

```

1 def shuffle_integrity_logic(c0, c1, c2, c3, c4, s0, s1, s2, s3, s4, sd):
2     sc, ss = (c0+c1+c2+c3+c4), (s0+s1+s2+s3+s4)
3     qc, qs = (c0*c0+c1*c1+c2*c2+c3*c3+c4*c4), (s0*s0+s1*s1+s2*s2+s3*s3+s4*s4)
4     if sc != ss or qc != qs: return 0
5     iv = 100 if s0 in [c0, c1, c2, c3, c4] else 0
6     sh, inv = [s0, s1, s2, s3, s4], 0
7     for i in range(5):
8         for j in range(i+1, 5):
9             if sh[i] > sh[j]: inv += 1

```



```
10 return int(inv + iv)
```

Listing 220: Source Python shuffle_integrity_logic implementation

```
1 include "node_modules/circomlib/circuits/comparators.circom";
2
3 template shuffle_integrity_logic() {
4     signal input c0, c1, c2, c3, c4;
5     signal input s0, s1, s2, s3, s4;
6     signal input sd; // unused in logic but present in function signature
7
8     signal output out;
9
10    // Compute sc = c0 + c1 + c2 + c3 + c4
11    signal sc;
12    sc <== c0 + c1 + c2 + c3 + c4;
13
14    // Compute ss = s0 + s1 + s2 + s3 + s4
15    signal ss;
16    ss <== s0 + s1 + s2 + s3 + s4;
17
18    // Compute qc = c0^2 + ... + c4^2
19    signal qc;
20    qc <== c0*c0 + c1*c1 + c2*c2 + c3*c3 + c4*c4;
21
22    // Compute qs = s0^2 + ... + s4^2
23    signal qs;
24    qs <== s0*s0 + s1*s1 + s2*s2 + s3*s3 + s4*s4;
25
26    // Check if sc == ss and qc == qs
27    component eq_sc_ss = IsZero();
28    eq_sc_ss.in <== sc - ss;
29
30    component eq_qc_qs = IsZero();
31    eq_qc_qs.in <== qc - qs;
32
33    signal valid_sums;
34    valid_sums <== eq_sc_ss.out * eq_qc_qs.out; // 1 if both equalities hold, else 0
35
36    // Check if s0 is in [c0, c1, c2, c3, c4]
37    component cmp0 = IsEqual();
38    cmp0.a <== s0;
39    cmp0.b <== c0;
40
41    component cmp1 = IsEqual();
42    cmp1.a <== s0;
43    cmp1.b <== c1;
44
45    component cmp2 = IsEqual();
46    cmp2.a <== s0;
47    cmp2.b <== c2;
48
49    component cmp3 = IsEqual();
50    cmp3.a <== s0;
51    cmp3.b <== c3;
52
53    component cmp4 = IsEqual();
54    cmp4.a <== s0;
```

```

55     cmp4.b <== c4;
56
57     signal s0_in_c;
58     s0_in_c <== 1 - (1 - cmp0.out) * (1 - cmp1.out) * (1 - cmp2.out) * (1 - cmp3.out)
        * (1 - cmp4.out);
59
60     signal iv;
61     iv <== s0_in_c * 100;
62
63     // Count inversions in [s0, s1, s2, s3, s4]
64     signal sh[5];
65     sh[0] <== s0;
66     sh[1] <== s1;
67     sh[2] <== s2;
68     sh[3] <== s3;
69     sh[4] <== s4;
70
71     signal inv;
72     inv <== 0;
73
74     component gt[10];
75     var idx = 0;
76     for (var i = 0; i < 5; i++) {
77         for (var j = i + 1; j < 5; j++) {
78             gt[idx] = GreaterThan(64);
79             gt[idx].in[0] <== sh[i];
80             gt[idx].in[1] <== sh[j];
81             inv <== inv + gt[idx].out;
82             idx++;
83         }
84     }
85
86     signal total;
87     total <== inv + iv;
88
89     // If sums or squares don't match, output 0; else output total
90     out <== valid_sums * total;
91 }
92
93 component main = shuffle_integrity_logic();

```

Listing 221: Faulty translated Circom code with Undefined Port (T2046)

```

1 error[T2046]: Bus or signal not defined in component
2     "circuit.circom":40:5
3
4 40     cmp0.a <== s0;
5     ~~~~~ a is not defined in cmp0
6
7 error[T2046]: Bus or signal not defined in component
8     "circuit.circom":41:5
9
10 41     cmp0.b <== c0;
11     ~~~~~ b is not defined in cmp0
12
13 error[T2046]: Bus or signal not defined in component
14     "circuit.circom":44:5
15

```

```

16 44      cmp1.a <== s0;
17      ~~~~~ a is not defined in cmp1
18
19 error[T2046]: Bus or signal not defined in component
20      "circuit.circom":45:5
21
22 45      cmp1.b <== c1;
23      ~~~~~ b is not defined in cmp1
24
25 error[T2046]: Bus or signal not defined in component
26      "circuit.circom":48:5
27
28 48      cmp2.a <== s0;
29      ~~~~~ a is not defined in cmp2
30
31 error[T2046]: Bus or signal not defined in component
32      "circuit.circom":49:5
33
34 49      cmp2.b <== c2;
35      ~~~~~ b is not defined in cmp2
36
37 error[T2046]: Bus or signal not defined in component
38      "circuit.circom":52:5
39
40 52      cmp3.a <== s0;
41      ~~~~~ a is not defined in cmp3
42
43 error[T2046]: Bus or signal not defined in component
44      "circuit.circom":53:5
45
46 53      cmp3.b <== c3;
47      ~~~~~ b is not defined in cmp3
48
49 error[T2046]: Bus or signal not defined in component
50      "circuit.circom":56:5
51
52 56      cmp4.a <== s0;
53      ~~~~~ a is not defined in cmp4
54
55 error[T2046]: Bus or signal not defined in component
56      "circuit.circom":57:5
57
58 57      cmp4.b <== c4;
59      ~~~~~ b is not defined in cmp4
60
61 previous errors were found

```

Listing 222: Circom compiler error log

Explanation: In this example, ‘cmp0’ through ‘cmp4’ are instances of ‘IsEqual()’, but ‘IsEqual’ does not define inputs named ‘a’ and ‘b’, so assignments like ‘cmp0.a <== s0;’ and ‘cmp0.b <== c0;’ fail. You likely need to connect through the real interface of ‘IsEqual’, such as its ‘in’ bus (for example ‘cmp0.in[0]’ and ‘cmp0.in[1]’) instead of ‘a’ and ‘b’.

H-4 Partial Component Access (T2043)

Description: This error class happens when Circom sees an array access pattern that is only legal for variable signal arrays, but the code is doing it on something like a component array or another structure that must be fully resolved statically. In practice, Circom requires component accesses to be fully specified at compile time, so the fix is usually to avoid partial indexing on component internals, expand the wiring with fully known indices, or refactor through intermediate signals so each access path is explicit.

Original Error Tag: T2043

Example:

```
1 def quicksort_partition_logic(a0, a1, a2, a3, a4, a5, a6, low, high):
2     arr = [a0, a1, a2, a3, a4, a5, a6]
3     pivot = arr[high]
4     i = low - 1
5     swaps = 0
6     for j in range(7):
7         if j >= low and j < high:
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11                swaps += 1
12    arr[i + 1], arr[high] = arr[high], arr[i + 1]
13    return int((i + 1) * 100 + swaps)
```

Listing 223: Source Python quicksort_partition_logic implementation

```
1 template quicksort_partition_logic() {
2     signal input a0;
3     signal input a1;
4     signal input a2;
5     signal input a3;
6     signal input a4;
7     signal input a5;
8     signal input a6;
9     signal input low;
10    signal input high;
11    signal output out;
12
13    signal arr[7];
14    arr[0] <== a0;
15    arr[1] <== a1;
16    arr[2] <== a2;
17    arr[3] <== a3;
18    arr[4] <== a4;
19    arr[5] <== a5;
20    arr[6] <== a6;
21
22    signal pivot;
23    pivot <== arr[high];
24
25    signal i;
26    signal swaps;
27    i <== low - 1;
28    swaps <== 0;
29}
```

```

30 signal in_bounds[7];
31 signal less_than_pivot[7];
32 component less_than_pivot_cmp[7] = LessEqThan(64);
33 signal new_i[7];
34 signal new_swaps[7];
35 signal new_arr[7][7];
36
37 for (var j = 0; j < 7; j++) {
38     in_bounds[j] <= j >= low && j < high;
39     less_than_pivot_cmp[j].in[0] <= arr[j];
40     less_than_pivot_cmp[j].in[1] <= pivot;
41     less_than_pivot[j] <= less_than_pivot_cmp[j].out;
42     new_i[j] <= i + less_than_pivot[j];
43     new_swaps[j] <= swaps + less_than_pivot[j];
44     for (var k = 0; k < 7; k++) {
45         if (k == j) {
46             new_arr[j][k] <= arr[i];
47         } else if (k == i) {
48             new_arr[j][k] <= arr[j];
49         } else {
50             new_arr[j][k] <= arr[k];
51         }
52     }
53 }
54
55 signal final_i;
56 signal final_swaps;
57 signal final_arr[7];
58 final_i <= new_i[6];
59 final_swaps <= new_swaps[6];
60 for (var k = 0; k < 7; k++) {
61     final_arr[k] <= new_arr[6][k];
62 }
63
64 signal final_new_arr[7];
65 for (var k = 0; k < 7; k++) {
66     if (k == high) {
67         final_new_arr[k] <= final_arr[final_i + 1];
68     } else if (k == final_i + 1) {
69         final_new_arr[k] <= final_arr[high];
70     } else {
71         final_new_arr[k] <= final_arr[k];
72     }
73 }
74
75 out <= (final_i + 1) * 100 + final_swaps;
76 }
77
78 component main = quicksort_partition_logic();

```

Listing 224: Faulty translated Circom code with Partial Component Access (T2043)

```

1 error[T2043]: Typing error found
2     "circuit.circom":36:3
3
4 36     component less_than_pivot_cmp[7] = LessEqThan(64);
5     ~~~~~ Only variable arrays can be
        accessed partially

```

```

6
7 previous errors were found

```

Listing 225: Circom compiler error log

Explanation: In this example, ‘less_than_pivot_cmp’ is an array of ‘LessEqThan’ components, and the code later connects ‘less_than_pivot_cmp[j].in[0]’ and ‘less_than_pivot_cmp[j].in[1]’. That pattern is being treated as a partial access on a component array, which Circom rejects, so the comparator connections need to be rewritten in a form where each component/input access is fully explicit.

H-5 Inline Component Array (T2034)

Description: This error class happens when Circom component arrays are declared or initialized using an unsupported syntax, especially inline array literals of component instances. In Circom, components must be instantiated individually or inside a loop after declaring the component array, rather than assigned as ‘[Comp(), Comp(), ...]’; otherwise the array is not treated as properly initialized, which then triggers follow-up errors when you try to access ‘in’ or ‘out’ on its elements.

Original Error Tag: T2034

Example:

```

1 def merkle_mountain_range_peak(size):
2     peak_map = 0
3     curr = size
4     for i in range(5):
5         if curr >= (1 << (4 - i)):
6             peak_map |= (1 << (4 - i))
7             curr -= (1 << (4 - i))
8     return [peak_map]

```

Listing 226: Source Python merkle_mountain_range_peak implementation

```

1 template merkle_mountain_range_peak() {
2     signal input size;
3     signal output out;
4
5     signal pm[5];
6     signal c[5];
7     signal b[5];
8     signal bit_set[5];
9
10    component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan(32),
11    GreaterEqThan(32), GreaterEqThan(32)];
12
13    // Initialize first values
14    pm[0] <== 0;
15    c[0] <== size;
16
17    for (var i = 0; i < 5; i++) {
18        // Calculate b = 1 << (4 - i)
19        b[i] <== 1 << (4 - i);
20
21        // Check if c >= b using GreaterEqThan comparator
22        ge_check[i].in[0] <== c[i];
23        ge_check[i].in[1] <== b[i];
24        bit_set[i] <== ge_check[i].out;
25    }
26 }

```

```

24
25     // Calculate pm[i+1] = pm[i] | (bit_set[i] ? b[i] : 0)
26     if (i < 4) {
27         pm[i+1] <= pm[i] + bit_set[i] * b[i];
28         // Calculate c[i+1] = c[i] - (bit_set[i] ? b[i] : 0)
29         c[i+1] <= c[i] - bit_set[i] * b[i];
30     }
31 }
32
33 out <= pm[4];
34 }
35
36 component main = merkle_mountain_range_peak();

```

Listing 227: Faulty translated Circom code with Inline Component Array (T2034)

```

1 error[T2034]: Typing error found
2   "circuit.circom":14:30
3
4 14     component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan
5      (32), GreaterEqThan(32), GreaterEqThan(32)];
6      ~~~~~ Components can not be declared
7      inside inline arrays
8
9 error[T2034]: Typing error found
10  "circuit.circom":14:49
11
12 14     component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan
13      (32), GreaterEqThan(32), GreaterEqThan(32)];
14      ~~~~~ Components can
15      not be declared inside inline arrays
16
17 error[T2034]: Typing error found
18  "circuit.circom":14:68
19
20 14     component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan
21      (32), GreaterEqThan(32), GreaterEqThan(32)];
22      ~~~~~ Components can not be declared inside inline arrays
23
24 error[T2034]: Typing error found
25  "circuit.circom":14:87
26
27 14     component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan
28      (32), GreaterEqThan(32), GreaterEqThan(32)];
29      ~~~~~ Components can not be declared inside
30      inline arrays
31
32 error[T2043]: Typing error found

```

```

32 "circuit.circom":14:5
33
34 14      component ge_check[5] = [GreaterEqThan(32), GreaterEqThan(32), GreaterEqThan
35      (32), GreaterEqThan(32), GreaterEqThan(32)];
36
37      ~~~~~
38      Only variable arrays can be accessed partially
39
40 error[T20466]: Typing error found
41 "circuit.circom":25:9
42
43 25      ge_check[i].in[0] <== c[i];
44      ~~~~~
45      Trying to access to a signal of a component
46      that has not been initialized
47
48 error[T20466]: Typing error found
49 "circuit.circom":26:9
50
51 26      ge_check[i].in[1] <== b[i];
52      ~~~~~
53      Trying to access to a signal of a component
54      that has not been initialized
55
56 error[T20466]: Typing error found
57 "circuit.circom":27:24
58
59 27      bit_set[i] <== ge_check[i].out;
60      ~~~~~
61      Trying to access to a signal of a component
62      that has not been initialized
63
64 previous errors were found

```

Listing 228: Circom compiler error log

Explanation: In this example, ‘ge_check[5]’ is declared with an inline array of ‘GreaterEqThan(32)’ instances, but Circom does not allow components inside inline arrays. Because that declaration fails, each ‘ge_check[i]’ remains uninitialized, so the later lines connecting ‘ge_check[i].in[0]’, ‘ge_check[i].in[1]’, and ‘ge_check[i].out’ also fail.